Microsoft Agent 365 PoC Runbook

> **Goal of this PoC:** Validate Microsoft Agent 365 controls across **Observe**, **Govern**, and **Secure** using documented setup steps, measurable tests, and reviewable evidence.

Chapters

#	Pillar	Chapter	Prerequisites	What it proves
1	Observe	Agent discovery and inventory	Preparation	Every agent appears in one registry with owner, status and risk, in the portal and through Microsoft Graph.
2	Observe	Connected platforms · Agent 365 SDK	Preparation	Agents on non-Microsoft platforms (Amazon Bedrock, Databricks Genie, Google Vertex AI and others) and custom SDK agents land in the same registry.
3	Govern	Agent identity and ownership	Preparation	Every agent has a Microsoft Entra Agent ID with accountable owners and sponsors.
4	Govern	Tools and MCP server governance	Preparation	Plugins, skills, MCP servers and connectors are inventoried, approved and centrally blockable.
5	Govern	Agent lifecycle and audit	Preparation	An agent can be installed, blocked, reassigned, deleted, restored and permanently deleted, with an audit trail.
6	Secure	Conditional Access and least privilege	Preparation	Only approved, low-risk agent identities get tokens, and access to resources is time-bound.
7	Secure	Sensitive data protection (Purview)	Preparation	Sensitivity labels and DLP stop Copilot and agents from processing or emailing regulated content, and Communication Compliance and Insider Risk Management supervise agents.
8	Secure	Threat detection and runtime protection (Defender)	Preparation	Risky agent actions are detected or blocked at runtime and reach the SOC as incidents.

#	Pillar	Chapter	Prerequisites	What it proves
9	Secure	Shadow AI and local agents	Preparation	Unapproved AI apps and developer agent tools on managed devices are visible and can be blocked.

Prerequisites and reading order

Open the chapter you want to validate and start with its **Before you start** checklist and linked prerequisite page. Each prerequisite page covers the applicable tenant checks, accounts, role assignments, test assets, and preparation steps.

Chapters 1 and 3 provide inventory and identity foundations where those are required. Follow the specific dependency links in your selected chapter; you do not need to run every chapter. Agree optional paths and success criteria before setup.

Before configuration, agree the approved PoC tenant, test window, selected outcomes, and preparation owners with the change owner and reviewer. Use synthetic data and controlled test accounts and recipients. Choose a restricted evidence location and verify existing assets before reusing them.

Tenant readiness

1. The setup administrator signs in to the Microsoft 365 admin center and opens **Agents** > **Overview** and **Agents** > **All agents** > **Registry**.
2. The reviewer verifies the pages needed for their read-only tasks with their own account.
3. Check the Microsoft Learn prerequisites linked from the selected chapter, including licensing and supported tenant or cloud availability. Record any unmet requirement before continuing.

Check result

- The required pages open for the assigned accounts, and the applicable feature prerequisites have been verified.

How to use this runbook

Use each chapter's **Topics and time guide** to jump to a task. Estimates cover hands-on work on one agent or a small test set after preparation; service propagation, log ingestion, approval queues, and unattended installation or deployment waits are separate. New integrations may need more time.

Follow the numbered tasks using the stated role and **Documentation** links. Verify **Check result** after setup and **Expected result** during tests, then collect the listed **Evidence**. The **What you proved** checklist is the chapter's success criteria: agree the selected outcomes before testing and tick only those supported by evidence. Record optional paths that were not run as out of scope.

Each page ends with consolidated **References**, in addition to the links beside the relevant instructions.

Least-privilege model

- **Test admin / setup account:** holds only the roles needed for the selected configuration tasks.
- **Reviewer / auditor:** holds read-only roles for configuration, evidence, and logs; does not hold the setup roles.

Grant read-only access first and separate the change maker from the reviewer. Use time-bound **Active** assignments for the PoC window, with just-in-time activation for Global Administrator, and record assignment expiry. Each chapter's **Required permissions** table lists its task-specific roles.

A **User Administrator** creates missing PoC accounts. A **Privileged Role Administrator** assigns Microsoft Entra roles; a member of **Organization Management** assigns applicable Microsoft Purview role groups. Use the documented role-management permissions for other services.

Documentation: Assign Microsoft Entra roles · Microsoft Entra built-in roles · Roles and role groups in Microsoft Purview

---

Portals used

Portal	URL	Purpose
Microsoft 365 admin center	https://admin.cloud.microsoft	Agent Registry, Agent Map, requests, lifecycle, tools and MCP servers, connected platforms, Shadow AI and local agents
Microsoft Entra admin center	https://entra.microsoft.com	Agent ID, owners and sponsors, Conditional Access, ID Protection, access packages, Lifecycle Workflows, sign-in and audit logs
Microsoft Purview portal	https://purview.microsoft.com	Audit, sensitivity labels, DLP, Communication Compliance, Insider Risk Management, DSPM for AI
Microsoft Defender portal	https://security.microsoft.com	Security for AI, AI agent inventory, alerts and incidents, Advanced Hunting
Power Platform admin center	https://admin.powerplatform.microsoft.com	Copilot Studio environments, threat detection integration, connector governance
Copilot Studio	https://copilotstudio.microsoft.com	Build and publish Copilot Studio test agents
Microsoft Intune admin center	https://intune.microsoft.com	Block policies for Shadow AI and local agents, device compliance
Azure portal	https://portal.azure.com	Custom agent hosting, Tools Gateway (API Management)

Repository content

- `chapter-NN-*/README.md`: one chapter per folder.
- `chapter-NN-*/prerequisites.md`: detailed preparation for that chapter.
- Microsoft-native samples: Copilot Studio declarative agent and Microsoft Foundry agent samples.
- Custom agent samples: Agent 365 SDK agent samples for Node.js, Python and .NET.
- Agent-365-Security-PoC-Runbook.pdf: combined reading copy with all chapters, prerequisites, and sample guides. Use the Markdown pages to edit the runbook.

Several features covered in this runbook are in preview. Preview features might change before general availability.

Key references

- Overview of Microsoft Agent 365
- Get started with Microsoft Agent 365
- Agent management in the Microsoft 365 admin center
- Assign Microsoft Entra roles
- Microsoft Entra built-in roles
- Roles and role groups in Microsoft Purview

> **Ready?** Start with Chapter 1 - Agent discovery and inventory →

Prerequisites - Chapter 1: Agent discovery and inventory

> **Goal:** Prepare the accounts, access, and test assets needed for this chapter and its selected optional paths.

Apply the shared PoC preparation guidance and agree the chapter outcomes. Finish dependent build steps before their tests.

Table of contents

- Tenant readiness
- Prepare accounts and role assignments
- Test assets and dependencies
- Plan propagation and test timing
- References

Tenant readiness

Documentation: Get started with Microsoft Agent 365 · Agent management in the Microsoft 365 admin center · Frontier program

Use the task-specific roles in 1.1 Required permissions. Confirm availability rather than assuming the tenant is ready.

Follow the shared tenant readiness check using the chapter permissions and feature prerequisites linked above.

Prepare accounts and role assignments

Alongside the shared setup and reviewer accounts, prepare the following scenario accounts:

Account	Purpose
Standard test user	Discovers or installs the PoC agents made available for the test. No admin role.
Maker account	Builds the Copilot Studio or Foundry test agents and prepares the test knowledge source.
Developer account	Creates and publishes the Foundry test agent when that platform is in scope.

Assign the roles in 1.1 Required permissions using the shared model. Verify each account can access the pages needed for its tasks.

Test assets and dependencies

Documentation: Connect existing agents to Microsoft Agent 365 · Agent registry integration for Copilot Studio · Agent identity integration for Copilot Studio · Microsoft Agent 365 integration with Foundry · Publish your agent as an Agent Application · Agent overview in Microsoft 365 admin center · Get started with agents in SharePoint

- Prepare at least one Microsoft-native test agent per platform in scope, as described in 1.2. Use the samples in `samples/`.
- Prepare a test SharePoint site to use as a knowledge source for the Copilot Studio sample.
- Chapter 2 has separate guides for Connected platforms (Amazon Bedrock, Google Vertex AI, and others) and Agent 365 SDK agents. Local agents and Shadow AI are covered in Chapter 9 – Shadow AI and local agents.
- If you plan to use Microsoft Graph PowerShell (1.7.3), install PowerShell 7, which is the recommended version for the Microsoft Graph PowerShell SDK.

Check result

- Each asset needed for the selected paths is prepared or has a named build task that will finish before its dependent tests.

Plan propagation and test timing

Documentation: Agent Registry

Configuration	Propagation window to plan for
Risk signals in the Agent Registry compared with the source security portals	Up to 1 hour

> ⚠️ **Watch out:** Allow for the documented propagation window and verify the actual configuration or signal before testing. Elapsed time alone is not proof that a change has applied.

References

- Get started with Microsoft Agent 365
- Agent management in the Microsoft 365 admin center
- Frontier program
- Connect existing agents to Microsoft Agent 365
- Agent registry integration for Copilot Studio
- Agent identity integration for Copilot Studio
- Microsoft Agent 365 integration with Foundry
- Publish your agent as an Agent Application
- Get started with agents in SharePoint
- Agent Registry

Ready? Continue to the chapter. →

Chapter 1 - Agent discovery and inventory

Pillar: Observe **What it proves:** You can't govern what you can't see. Every agent available to the organization appears in one inventory, the Agent Registry in the Microsoft 365 admin center, with its type, platform, owner, status and risk signals. You can also read the same inventory programmatically with Microsoft Graph.

Success criteria: Agree the selected outcomes in What you proved before testing.

Topics and time guide

Approximate hands-on estimates for the selected scope, with prerequisites prepared. Service waiting time is separate.

Topic	Estimated hands-on time
1.1 Required permissions	~5 min
1.2 Discover Microsoft-native agents	~20-30 min
1.3 Review the Agent Registry	~10-15 min
1.4 Visualize agents with Agent Map	~5 min
1.5 Review agent requests	~5-10 min
1.6 Inspect agent details and tag agents	~10 min
1.7 Programmatic discovery with Microsoft Graph	~15-20 min
1.8 Review agents across multiple tenants (optional, preview)	~10-15 min
1.9 Evidence	~5-10 min
1.10 Troubleshooting	As needed

Outcome checklist · References

1.1 Required permissions

Use the shared account and role-assignment model with the task-specific permissions below.

Task	Least-privilege role	Section
Create the Copilot Studio and Foundry test agents	**Maker** or **developer** account from the chapter prerequisites	1.2.1, 1.2.2
Publish a Foundry agent as an Agent Application	**Foundry Project Manager** on the Foundry resource scope	1.2.2
View the Agent Registry, Overview, filters, risk counts and export	**AI Reader** or **Global Reader**	1.3
Open risk source deep links into Microsoft Entra and Microsoft Defender	**Global Reader**, **Security Reader**, **Security Administrator**, **AI Administrator** or **Global Administrator**	1.3.4

Task	Least-privilege role	Section
Open risk source deep links for Microsoft Purview Insider Risk Management alerts	**IRM Analyst** or **IRM Investigator**	1.3.4
Open Agent Map	**AI Administrator**	1.4
Approve, publish or reject agent requests	**AI Administrator**	1.5
View **Data & tools**, tag an agent and take actions from the details pane	**AI Administrator**	1.6
Read the Agent 365 catalog through Graph (`/copilot/admin/catalog/packages`)	**AI Administrator**	1.7.2, 1.7.3
Read agent identities, blueprints and blueprint principals through Graph (delegated)	**Agent ID Administrator**	1.7.2
Read risky agents and agent risk detections through Graph (delegated)	**Security Reader**	1.7.2
Read agent sign-in logs through Graph (delegated)	**Reports Reader**	1.7.2
Run Advanced Hunting queries (`AgentsInfo`)	**Security Reader**	1.7.2, 1.7.5
Register the app for app-only access	**Application Developer**	1.7.4
Grant admin consent to Microsoft Graph application permissions	**Privileged Role Administrator**	1.7.4
View agents in governed tenants (optional, preview)	**Global Reader** (delegated, in each governed tenant)	1.8
Validation and read-only review	**AI Reader**	1.3

Notes on roles:

- Only **AI Administrator** and **Global Administrator** can perform governance actions such as approving agent requests or assigning ownership. Other roles can monitor governance gaps but can't take administrative action.
- Agent Map requires the **Global Administrator** or **AI Administrator** role.
- Global Administrator is a highly privileged role. Limit its use to scenarios where you can't use an existing role.

Before you start:

- [ ] Confirm the chapter prerequisites for the selected topics.

1.2 Discover Microsoft-native agents

Documentation: Connect existing agents to Microsoft Agent 365 · Agent registry integration for Copilot Studio · Agent identity integration for Copilot Studio · Microsoft Agent 365 integration with Foundry · Publish your agent as an Agent Application · Agent overview in Microsoft 365 admin center · Get started with agents in SharePoint

Agents built with Microsoft Foundry, Microsoft Copilot Studio and Agent Builder in Microsoft Copilot integrate with Agent 365 automatically.

Platform	How the agent reaches the Registry	Identity behavior	Sample
Copilot Studio	Appears in the Registry immediately when the agent is created. Metadata updates are synchronized automatically, and the entry is removed automatically when the agent is deleted. Draft agents are visible only for Copilot Studio.	The agent ID is created at agent creation time, not during publishing. All Copilot Studio app-based agents share a single blueprint, and an agent ID is created for each agent.	`samples/copilot-studio/`
Agent Builder in Microsoft Copilot	Integrated automatically. Draft Agent Builder agents aren't currently visible.	—	—
Microsoft Foundry	Every Foundry agent that you publish appears in the Agent 365 registry automatically. Registry sync is supported for prompt agents and hosted agents. Overview analytics currently support only Microsoft Foundry V2 agents.	An unpublished agent uses the project's shared agent identity. A published agent receives its own Entra agent identity and Entra agent blueprint.	`samples/foundry/`
SharePoint	Listed as a supported platform. Draft SharePoint agents aren't currently visible.	—	—
Microsoft 365 Agents Toolkit	Agents submitted for admin approval appear in the **Requests** tab (1.5).	—	—

1.2.1 Create a Copilot Studio test agent

Performed by the **maker** (see chapter prerequisites).

1. Open `samples/copilot-studio/declarative-agent.json` and copy the name, description and instructions.
2. Sign in to Copilot Studio (`https://copilotstudio.microsoft.com`) and create a new agent named `poc-cs-trail-guide`. Use the description and instructions from the sample.
3. Add your PoC SharePoint site as a knowledge source.
4. Publish the agent.
5. Open the configuration panel for the **Teams and Microsoft Copilot** channels, select **Availability options**, and share the agent with the organization by submitting it for admin approval. You use this request in 1.5.

Check result

- The agent appears in the Registry (1.3) immediately after it was created.
- The submitted agent appears in **Agents** > **All agents** > **Requests**.

1.2.2 Create and publish a Foundry test agent

Performed by the **developer** (see chapter prerequisites). Publishing requires the **Foundry Project Manager** role on the Foundry resource scope.

1. Follow `samples/foundry/README.md` to create the prompt agent `poc-foundry-trail-guide`.
2. Publish the agent as an Agent Application, as described in Publish your agent as an Agent Application.
3. If the agent's tools access other Azure resources, reassign the required RBAC permissions to the new agent identity. Permissions don't transfer automatically when you publish.

Check result

- The published agent appears in the Registry.
- In the Microsoft Entra admin center (`https://entra.microsoft.com`), **Entra ID** > **Agents** > **Agent identities** lists the agent identity of the published agent.

1.3 Review the Agent Registry

Documentation: Agent Registry in Microsoft 365 admin center · Agent management roles and permissions · Agent overview in Microsoft 365 admin center

The Agent Registry provides a centralized view of all agents available for your organization.

1.3.1 Open the Registry and read the summary

Performed by **AI Reader**.

1. Sign in to the Microsoft 365 admin center (`https://admin.cloud.microsoft`).
2. Select **Agents** > **Overview**. Note the **Agent registry** count and the cards **Pending requests**, **Agents without owners**, **Agents at risk** and **Agents with exceptions**. The Overview shows only the top five most used platforms. To see all platforms, go to the **Registry** tab.
3. Select **Agents** > **All agents** > **Registry**.
4. Review the summary tiles:

Tile	Meaning
Total agents	The number of agents available in your organization's tenant. Applying filters doesn't change this count.
Agents without owners	The number of agents that no longer have owners at your organization.
Unmanaged agents	The number of agents created or managed outside of Agent 365, without its risk protection and observability.
Agents at risk	The total number of agents in the tenant that have one or more risk signals (see 1.3.4).

5. Review the agent types:

Agent type	Description
Microsoft agents	Agents built and maintained by Microsoft.
External partner-built agents	Agents built by trusted non-Microsoft developers and published for broader or public availability.
Published by your org	Custom agents approved and published by your organization for broader use (line-of-business agents).
Shared by creator	Agents created and shared by individual users or developers at your organization (shared agents).

Check result

- The test agents from 1.2 are listed.

- You recorded the **Total agents**, **Agents without owners**, **Unmanaged agents** and **Agents at risk** values as a baseline.

1.3.2 Filter, sort and search

Performed by **AI Reader**.

1. On the **Registry** tab, filter the list by:
   - **Status**
   - **Publisher Type**: distinguishes Microsoft agents, external partner-built agents and agents published by your organization
   - **Channel**: **Copilot**, **Teams**, **Outlook**, **Microsoft 365 apps** or **SharePoint**
   - **Platform**: the platform or product used to create the agent
   - **Data source**: **Embedded knowledge** or **Fine-tuned models**
2. To sort the list, select a column title. Only certain columns support sorting.
3. Use **Search** to find a test agent by name.
4. Select the list icon next to the **Search** box to switch between **Normal list** and **Compact list**.

Check result

- Filtering by **Platform** isolates each test agent.
- If you don't see the agents that you expect, check that no filter is still applied.

1.3.3 Find agents without owners

Performed by **AI Reader** (review) and **AI Administrator** (remediation).

Shared agents can become ownerless when you delete the user who created them from the organization. The ownerless agent count updates automatically when you hard delete a user.

1. On **Agents** > **All agents** > **Registry**, select the **Agents without owners** card. The list is filtered by **Publisher type** and **Owner**.
2. For each agent, decide on an action, such as blocking or deleting the agent, and record the decision.
3. Lifecycle actions (assign a new owner, block, delete) are covered in Chapter 5 – Agent lifecycle and audit. Owners and sponsors of agent identities are covered in Chapter 3 – Agent identity and ownership.

Check result

- The ownerless list was captured, and each entry has a recorded decision.

1.3.4 Review risk signals

Performed by **AI Reader** (Registry view) and **Security Reader** (source deep links).

Risk signals in the Agent Registry are a consolidated set of agent-related security detections from Microsoft security platforms, such as Microsoft Purview, Microsoft Entra and Microsoft Defender.

1. On the **All agents** page, review the **Risks** column. It shows the aggregated count of risk signals for each agent.
2. Select the count for an agent. The **Risk details** pane opens and shows:
   - Risk signals grouped into expandable high, medium and low severity sections, with a description of each signal.
   - An **Occurrences** count for each signal.
   - **Risk signals are sourced from**: the Microsoft security platforms that contribute signals for the agent. Each platform name is a deep link to that portal.
   - For an agent with multiple instances, risk signals grouped by instance.

3. Select the **Agents at risk** tile. It opens a prefiltered view of agents with detected risk signals.
4. Open a source deep link. Access depends on your role in the target portal:

Security portal	Required role (any one)
Microsoft Entra	Global Administrator, Global Reader, Security Reader, Security Administrator or AI Administrator
Microsoft Defender	Global Administrator, Global Reader, Security Reader, Security Administrator or AI Administrator
Microsoft Purview, Insider Risk Management alerts	IRM Analyst or IRM Investigator. Global Administrator alone is insufficient.

5. Record each agent that has a high-severity signal, and name a follow-up owner for it.

The risk signal counts in the Microsoft 365 admin center might be up to an hour behind the security portals. A count of zero indicates that no active signals are currently detected for that agent across the connected platforms.

Check result

- Every agent with a high-severity signal has a named follow-up owner. Investigation and response are covered in Chapter 8 – Threat detection and runtime protection (Defender).

1.3.5 Customize the view and export

Performed by **AI Reader**.

1. On the **Registry** toolbar, select **Customize view** and choose which columns to display.
2. Select **Export** and set the export scope to **All agents** or **Filtered agents**. The export can include over 30 items for each agent, such as name, status, channel, date created, last modified, publisher, publisher type, version, owner, description, platform and instructions. The export can take some time, depending on the number of agents in the tenant.
3. Optionally, export the list of active users for the last 30 days. It includes user principal name, total agents used, total sessions and last activity date.

Check result

- An export file exists that contains the test agents with their owner and platform values.

1.4 Visualize agents with Agent Map

Documentation: Use Agent Map in the Microsoft 365 admin center

Agent Map groups agents by the platform they were created on. It complements the Registry with a visual view for environments with large numbers of agents.

1.4.1 Open Agent Map and review clusters

Performed by **AI Administrator**.

1. In the Microsoft 365 admin center, select **Agents** > **All agents** > **Map**.
2. Identify the clusters: **Microsoft Copilot Agent Builder**, **Copilot Studio**, **Copilot Studio Legacy**, **Microsoft 365 Agents Toolkit**, **SharePoint**, **Azure AI Foundry**, **Amazon Bedrock**, **Google Vertex AI**, **Microsoft**, **External Partners** and **Others**.

3. Use **Zoom In/Out**, **Keyboard shortcuts** and **Settings** (**Max agents per platform**) to navigate the map.

Check result

- Each test agent from 1.2 appears in its platform cluster.

1.4.2 Use summary cards, filters and export

Performed by **AI Administrator**.

1. Review the summary cards: **Total agents**, **Agents at risk** (agents with at least one high security risk), **Agents without owners** and **Unmanaged agents**.
2. Filter the map by:
   - **Status**: **Available**, **Blocked**, **Draft**, **Not activated**
   - **Publisher type**: **Your org**, **Your users**, **Microsoft**, **Third party**
   - **Platform**
   - **Channel**
   - **Usage**: **Active users**, **Total sessions**, **Exception rate**, **Assisted hours**, **Security alerts**
3. Use the search bar to find an agent by name.
4. Select **Export** to download the agents displayed in the Agent Map to an Excel file.

Usage and observability filters are currently available only for tenants with fewer than 4,000 users. Usage is based on agents that report activity through Agent 365.

Check result

- The four summary cards are displayed, and the map can be filtered by platform.

1.5 Review agent requests

Documentation: Manage agent requests in Microsoft 365 admin center · Governance and lifecycle actions for agents

When a member of your organization publishes an agent to your tenant, the agent requires administrator approval before it becomes available tenant-wide. Agents created with Copilot Studio, Microsoft Foundry or Microsoft 365 Agents Toolkit can be submitted for admin approval. For the Copilot Studio test agent from 1.2.1, complete either 1.5.2 or 1.5.3.

1.5.1 Review pending requests

Performed by **AI Administrator**.

1. In the Microsoft 365 admin center, select **Agents** > **All agents** > **Requests**.
2. Filter by **State**:
   - **Pending review**
   - **Pending update**: a new version of an existing agent. Until you approve it, users can access the previous version.
   - **Pending activate**: a member of your organization asks to activate an agent so that they can create agent instances
   - **Allow user to install**: a user requests a Microsoft-built agent that admin policy makes unavailable
3. Filter by **Channel** if needed: **Microsoft Teams**, **Copilot**, **Office**, **Outlook**, **Word**, **Excel** or **PowerPoint**.
4. Select the Copilot Studio test agent submitted in 1.2.1. Confirm the capabilities, data sources, security and permissions, and custom actions that the agent can invoke.

Check result

- The test agent is listed in the **Requests** tab.

1.5.2 Approve and publish a request

Performed by **AI Administrator**.

1. In the agent details pane, select **Publish to store** to open the publishing wizard.
2. Select the **standard test user** as the audience that can install the agent.
3. Optionally, select the users or groups who will have the agent preinstalled.
4. Select **Next** to view template options. Apply an existing template, the default template or a custom template. See Policy templates.
5. Select **Next** to review permissions. In the **Review permissions** step, view the permissions that the agent requests and grant admin consent if appropriate.
6. Select **Next**, then select **Publish**.

> 💡 **Optional group audience:** To test availability for several users, select an existing approved test group instead of the individual test user. A group is not required for this chapter.

For a request in the **Pending update** state, select the agent and then select **Update in store**.

Check result

- The agent is available for installation to the selected audience.

1.5.3 Reject a submission

Performed by **AI Administrator**.

1. In the **Requests** list, select the ellipses to the right of the agent name.
2. Select **Reject submission**.

Check result

- The agent isn't made available to your organization.

1.6 Inspect agent details and tag agents

Documentation: Understand agent details in Microsoft 365 admin center · Agent settings in Microsoft 365 admin center

1.6.1 Review the agent details pane

Performed by **AI Administrator**.

1. Go to **Agents** > **All agents**. The **Registry** tab is selected.
2. Select a test agent. The details pane shows the common actions (**Install**, **Uninstall**, **Block**, **Update in store**, **Pin for users**) and a set of tabs. The tabs shown depend on the agent's capabilities.

Tab	What to check during the PoC
Details	Description, instructions, publish status, availability, publisher, deployment, agent type, channel, platform, last updated, sensitivity and version.
Users	**Available to** (which users can install and use the agent) and **Installed for** (which users automatically have the agent preinstalled). Availability and installation are independent actions.
Data & tools	Read-only. Capabilities, knowledge sources and tools available to the agent, plus Microsoft Entra details such as Agent ID and Agent User ID.
Security	Monitoring agent activity, protecting sensitive data and evaluating compliance gaps.
Permissions	Permissions to data that the agent can access and perform actions on.
Certification	Trust and attestation signals available for the agent.
Activity	How the agent is used and how it performs across the tenant.
Agent instances	Shown for agents tagged as **AI teammate**.
Connected Agents	Agents connected to the selected agent.
Computer use	Shown for agents that support computer-use capabilities, such as Researcher.

Check result

- For each test agent, you recorded the platform, owner, knowledge sources, tools and Agent ID, where shown.

1.6.2 Tag an agent

Performed by **AI Administrator**.

Tags are labels that admins and users apply to agents to organize and find them, such as by team, function or project. Tags don't change what an agent can do or access.

1. If the tag you need doesn't exist yet, select **Agents** > **Settings** > **Tags**, select **Add** and enter a name such as `Agent365-PoC`. An organization can have up to 50 tags.
2. Go to **Agents** > **All agents** and select a test agent from the **Registry** list.
3. In the agent details pane, select **Add tag**.
4. Select up to five tags from the tags available in your organization.
5. In the Registry, filter the list by the tag.

Check result

- The tag appears on the agent's page and in the **Tags** column of the agents list, and filtering by the tag returns the PoC test agents.

1.7 Programmatic discovery with Microsoft Graph

Documentation: Graph API for agent registry and agent details · Package Management API overview · List Copilot packages · List agentIdentity objects · List riskyAgents · Microsoft Entra Agent ID logs · agentSignIn resource type (agentType values) · security: runHuntingQuery

Two data sources answer different questions:

- **The Agent 365 catalog** (Package Management API, `/copilot/admin/catalog/packages`). A package represents an agent in the organization catalog. Use the List packages API to get all agents in your inventory, and the Get package details API to get detailed metadata for one agent. These APIs require the AI Administrator or Global Administrator role.
- **Microsoft Entra Agent ID objects** (agent identities, blueprints and blueprint principals). The Microsoft Entra admin center shows agents that have a Microsoft Entra Agent ID. The comprehensive agent inventory, including agents without a Microsoft Entra agent identity, is available in Agent 365.

1.7.1 Choose the right API

All APIs in this table are read operations. **Permission** is the least-privileged Microsoft Graph permission documented on the API page.

Question you want to answer	API	Version	Permission	Role (delegated)
Which agents are in the Agent 365 registry?	`GET /copilot/admin/catalog/packages`	v1.0 and beta (preview)	`CopilotPackages.Read.All`	AI Administrator or Global Administrator
What are the details of one agent?	`GET /copilot/admin/catalog/packages/{id}`	v1.0 and beta (preview)	`CopilotPackages.Read.All`	AI Administrator or Global Administrator
Which agent requests are open?	`GET /copilot/admin/catalog/packages?$filter=requestStatus eq 'pending'`	v1.0 and beta (preview)	`CopilotPackages.Read.All`	AI Administrator or Global Administrator
Which agent identities exist in Microsoft Entra?	`GET /servicePrincipals/microsoft.graph.agentIdentity`	v1.0	`AgentIdentity.Read.All`	Agent ID Administrator
Which agent identity blueprints exist?	`GET /applications/microsoft.graph.agentIdentityBlueprint`	v1.0	`AgentIdentityBlueprint.Read.All`	Agent ID Administrator
Which blueprint principals exist in my tenant?	`GET /servicePrincipals/microsoft.graph.agentIdentityBlueprintPrincipal`	v1.0	`AgentIdentityBlueprintPrincipal.Read.All`	Agent ID Administrator
Who owns an agent identity?	`GET /servicePrincipals/{id}/microsoft.graph.agentIdentity/owners`	v1.0	`AgentIdentity.Read.All`	Directory Readers or Global Reader

Question you want to answer	API	Version	Permission	Role (delegated)
Who sponsors an agent identity?	`GET /servicePrincipals/{id}/microsoft.graph.agentIdentity/sponsors`	v1.0	`AgentIdentity.Read.All` (delegated, validated in a lab tenant). The API page lists the application permission `AgentIdentity.ReadWrite.All`.	Validated with Global Administrator
Which agents does ID Protection consider risky?	`GET /identityProtection/riskyAgents`	**beta**	`IdentityRiskyAgent.Read.All`	Security Reader, Security Operator, Global Reader or Security Administrator
Which risk detections were raised for agents?	`GET /identityProtection/agentRiskDetections`	**beta**	`IdentityRiskEvent.Read.All`	Security Reader, Security Operator, Global Reader or Security Administrator
Which agent identities signed in?	`GET /auditLogs/signIns` with the filter `agent/agentType eq 'agenticAppInstance'`	**beta**	`AuditLog.Read.All`	Reports Reader
What configuration does Defender record for each agent?	`POST /security/runHuntingQuery` with the `AgentsInfo` table (preview)	v1.0	`ThreatHunting.Read.All`	Security Reader

Notes:

- APIs under `/beta` are subject to change. Use of these APIs in production applications isn't supported.
- Sponsors: you can also view the owners and sponsors of each agent identity in the Microsoft Entra admin center under **Entra ID** > **Agents** > **Agent identities** (**Owners and Sponsors** column). Sponsors are covered in Chapter 3 – Agent identity and ownership.
- `riskyAgents` supports the `$select`, `$count` and `$filter` query parameters.
- To receive the `agentIdentityBlueprintPrincipal` and `agentIDuser` enum values in responses, include the request header `Prefer: include-unknown-enum-members`.
- Paging: treat `@odata.nextLink` as an opaque URL. Request it exactly as returned, and continue until it's absent.

The queries used in this chapter:

```
GET https://graph.microsoft.com/v1.0/copilot/admin/catalog/packages
GET https://graph.microsoft.com/v1.0/copilot/admin/catalog/packages/{id}
GET https://graph.microsoft.com/v1.0/copilot/admin/catalog/packages?$filter=requestStatus eq 'pending'
GET https://graph.microsoft.com/v1.0/servicePrincipals/microsoft.graph.agentIdentity
GET https://graph.microsoft.com/v1.0/servicePrincipals/{id}/microsoft.graph.agentIdentity/owners
GET https://graph.microsoft.com/v1.0/applications/microsoft.graph.agentIdentityBlueprint
GET https://graph.microsoft.com/v1.0/servicePrincipals/microsoft.graph.agentIdentityBlueprintPrincipal
GET https://graph.microsoft.com/beta/identityProtection/riskyAgents
GET https://graph.microsoft.com/beta/identityProtection/agentRiskDetections
GET https://graph.microsoft.com/beta/auditLogs/signIns?$filter=signInEventTypes/any(t: t eq 'servicePrincipal')
and agent/agentType eq 'agenticAppInstance'
```

1.7.2 Run queries in Graph Explorer

Performed by **AI Administrator** (catalog queries), **Agent ID Administrator** (agent identity queries), **Security Reader** (risk and hunting queries) and **Reports Reader** (sign-in query).

1. Open Graph Explorer.

2. Select the profile avatar and sign in with your PoC account. Check the tenant that is shown in the top navigation.
3. Select **GET** as the HTTP method and **v1.0** as the API version.
4. Enter `https://graph.microsoft.com/v1.0/copilot/admin/catalog/packages` in the query box.
5. Select the **Modify permissions** tab, then select **Consent** next to `CopilotPackages.Read.All`. You can also use **Consent to permissions** under the profile avatar. Tenant-wide admin consent for delegated permissions can be granted by a Cloud Application Administrator, AI Administrator or Application Administrator.
6. Select **Run query**.
7. Review the **Response preview**:
    - `value` contains the packages. The documented properties include `id`, `displayName`, `type` (`microsoft`, `external`, `shared`, `custom`), `publisher`, `isBlocked`, `availableTo`, `deployedTo`, `supportedHosts`, `elementTypes` and `lastModifiedDateTime`.
    - If the response contains `@odata.nextLink`, request that URL exactly as returned, and continue until no `@odata.nextLink` is returned.
8. Repeat for the agent identity query, and consent to `AgentIdentity.Read.All`.
9. For the beta queries, select **beta** as the API version. For `riskyAgents`, add the request header `Prefer` with the value `include-unknown-enum-members` under **Request headers**, and consent to `IdentityRiskyAgent.Read.All`. For the sign-in query, consent to `AuditLog.Read.All`.
10. For Advanced Hunting, select **POST**, enter `https://graph.microsoft.com/v1.0/security/runHuntingQuery`, consent to `ThreatHunting.Read.All`, and use this request body:

    ```
    {
      "Query": "AgentsInfo | summarize arg_max(Timestamp, *) by AgentId | project AgentId, Name, Platform, PublishedStatus, LifecycleStatus, Owners, EntraAgentID, EntraBlueprintID"
    }
    ```

    The response contains a `schema` array and a `results` array. The optional `Timespan` parameter defaults to 30 days.

Check result

- The catalog query returns the PoC test agents, and you recorded the number of items after following every `@odata.nextLink`.

1.7.3 Export the inventory with Microsoft Graph PowerShell

Performed by **AI Administrator**.

1. Install the Microsoft Graph PowerShell SDK:

    ```
    Install-Module Microsoft.Graph -Scope CurrentUser -Repository PSGallery -Force
    ```

2. Run the example from the Agent Registry article. It lists all agents in your tenant and follows `@odata.nextLink` until all pages are read:

```
Connect-MgGraph -Scopes 'CopilotPackages.Read.All'

$uri = "https://graph.microsoft.com/v1.0/copilot/admin/catalog/packages"
$agentCount = 0

do {
    $response = Invoke-MgGraphRequest -Method GET -Uri $uri
    $agentCount += @($response.value).Count
    $response.value | ForEach-Object { Write-Host $_.displayName }
    $uri = $response.'@odata.nextLink'
} while ($uri)

Write-Host "Total agents: $agentCount"
```

3. Compare the **Total agents** value that the script prints with the **Total agents** tile in the Registry (1.3.1) and with the Registry export (1.3.5). Record any difference in counts.

Check result

- The script lists the display name of every agent in the catalog, including the PoC test agents, and prints the total count.
- The comparison with the Registry is recorded.

1.7.4 Run the export unattended with an app registration

Performed by **Application Developer** (app registration) and **Privileged Role Administrator** (admin consent).

Use application permissions when the call runs as a background service without a signed-in user. Microsoft Graph application permissions always require administrator consent.

1. Sign in to the Microsoft Entra admin center (`https://entra.microsoft.com`). Browse to **Entra ID** > **App registrations** and select **New registration**. Enter a name, for example `agent365-poc-inventory`. Under **Supported account types**, select **Single tenant only**.

2. In the app registration, select **Certificates & secrets** > **Certificates** > **Upload certificate**, and upload the public key of your certificate.

3. On the **API permissions** page, select **Add a permission** > **Microsoft Graph** > **Application permissions**. Add only the permissions you need, for example:

    - `CopilotPackages.Read.All`
    - `AgentIdentity.Read.All`
    - `IdentityRiskyAgent.Read.All`
    - `ThreatHunting.Read.All`

4. A **Privileged Role Administrator** grants tenant-wide admin consent: on **API permissions**, select **Grant admin consent**. Cloud Application Administrator, AI Administrator and Application Administrator can't grant consent to Microsoft Graph application permissions.

5. Make sure the certificate is present in `Cert:\CurrentUser\My\` or `Cert:\LocalMachine\My\` on the machine that runs the script. Then connect with app-only access and run the loop from 1.7.3 without the `Connect-MgGraph -Scopes` line:

```
Connect-MgGraph -ClientId "YOUR_APP_ID" -TenantId "YOUR_TENANT_ID" -CertificateThumbprint
"YOUR_CERT_THUMBPRINT"
```

Check result

- The app-only connection succeeds, and the catalog list from 1.7.3 returns the same agents as the delegated run.

1.7.5 Query the Defender agent inventory with Advanced Hunting

Performed by **Security Reader**.

The `AgentsInfo` table (preview) contains information about AI agents and their properties from various platforms. The `AIAgentsInfo` table is transitioning to `AgentsInfo`. Migrate queries from `AIAgentsInfo` to `AgentsInfo`.

1. In the Microsoft Defender portal (`https://security.microsoft.com`), select **Hunting** > **Advanced hunting**.
2. Run this query:

   ```
   AgentsInfo
   | summarize arg_max(Timestamp, *) by AgentId
   | project AgentId, Name, Platform, PublishedStatus, LifecycleStatus, Availability,
             Owners, EntraAgentID, EntraBlueprintID, InstanceCount, LastUpdatedDateTime
   | order by Platform asc, Name asc
   ```

3. To run the same query from automation, call `POST https://graph.microsoft.com/v1.0/security/runHuntingQuery` with the request body shown in 1.7.2.

Check result

- The query returns the agents in the `AgentsInfo` table, with their platform, status and owners.

1.8 Review agents across multiple tenants (optional, preview)

Documentation: Manage agents across multiple tenants in the Microsoft 365 admin center (preview)

Multi-tenant agent management is in public preview. It's intended for Microsoft partners who manage customer tenants by using Partner Center (GDAP), and for enterprises whose tenants are connected through Microsoft Entra Tenant Governance. Microsoft Entra Tenant Governance isn't the same as Multitenant Collaboration (MTO) in Microsoft 365.

Delegated role in the governed tenant	View agent inventory	Add, install, block or change availability
AI Administrator	Yes	Yes
Global Administrator	Yes	Yes
Global Reader	Yes	No

1.8.1 Open the consolidated inventory

Performed by **Global Reader** (delegated, in each governed tenant).

1. Sign in to the Microsoft 365 admin center with an account in the governing tenant.
2. Select **All tenants** > **Agents**. The **All tenants** scope is available only when the governing tenant has at least one supported tenant relationship and the signed-in administrator has a supported delegated role.
3. Review **Total agents**, **Risky agents** and **Assigned tenants**.
4. Select an agent and a governed tenant to review that tenant's details. Agent status is tenant-specific.

Check result

- The consolidated inventory lists agents from the governed tenants in scope.

1.8.2 Switch into a governed tenant

Performed by **Global Reader**.

1. In the Microsoft 365 admin center command bar, select the tenant switcher next to the current tenant name.
2. Search for or select the governed tenant. The admin center opens in the selected governed tenant by using your delegated access.
3. Use the tenant switcher again to return to the governing tenant.

Check result

- The selected governed tenant opens in the Microsoft 365 admin center, and the tenant switcher returns you to the governing tenant. The PoC doesn't run cross-tenant **Install** or **Block** unless the customer explicitly asks for it.

1.9 Evidence

- Screenshot of the Registry listing the Copilot Studio and Foundry test agents (1.2), and of **Entra ID** > **Agents** > **Agent identities** in the Microsoft Entra admin center listing the agent identity of the published Foundry agent (1.2.2).
- Agent overview screenshot showing the **Agent registry** count and the **Pending requests**, **Agents without owners** and **Agents at risk** cards.
- Registry export of all agents, and a filtered export of the PoC test agents with the owner column populated.
- Screenshots of the **Agents without owners** list and the **Agents at risk** list, with the triage decisions.
- Screenshot of one **Risk details** pane, if any agent has risk signals, and the list of agents with high-severity signals and their follow-up owners (1.3.4).
- Agent Map screenshot showing the platform clusters.
- Screenshots of the **Requests** tab before and after the approval or rejection in 1.5.
- Screenshot of a test agent's details pane (**Details** and **Data & tools** tabs) showing the applied tag.
- Microsoft Graph results: the Graph Explorer responses (1.7.2) and the output of the PowerShell example (1.7.3), plus the output of the app-only run if you performed 1.7.4. Mark results from beta APIs (`riskyAgents`, `agentRiskDetections`, `signIns`) as beta.
- `AgentsInfo` query results from Advanced Hunting.
- A note that records any difference between the Registry export and the Graph results (1.7.3).
- If you performed 1.8, a screenshot of **All tenants** > **Agents** in the governing tenant.

1.10 Troubleshooting

Symptom	Likely cause	Fix
Expected agent missing from the Registry list	A filter is set	Clear the filters. Applying filters doesn't change the **Total agents** count.
Foundry agent missing from the Registry	The agent isn't published	Publish the agent as an Agent Application (1.2.2). Every Foundry agent that you publish appears in the registry.

Symptom	Likely cause	Fix
Agent Builder, Foundry or SharePoint draft agent missing	Draft agents are visible only for Copilot Studio	Publish the agent.
Non-Microsoft or Agent 365 SDK agent missing	Connected platform or SDK onboarding isn't done	Follow the Connected platforms or Agent 365 SDK guide for the agent's integration path.
Map isn't available	Agent Map requires the Global Administrator or AI Administrator role	Use an AI Administrator account.
Usage filters missing on the map	Usage and observability filters are available only for tenants with fewer than 4,000 users, and usage is based on agents that report activity through Agent 365	Use the other map filters.
Risk source deep link doesn't open the source data	Deep links are subject to the user's roles and permissions in the target portal	Use a role listed in 1.3.4.
Risk count differs from the security portal	Counts in the admin center might be up to an hour behind the security portals	Refresh the Registry later.
`400 Bad Request` from `/copilot/admin/catalog/packages` with a filter	Unsupported filter, such as a request property combined with `supportedHosts`, `elementTypes` or `platform`; `eq`, `gt` or `lt` on `lastModifiedDateTime`; `or`, `ne` or `contains`; or `$count` with a request filter	Use one of the supported filter expressions listed in List Copilot packages.
`agentIdentityBlueprintPrincipal` or `agentIDuser` values aren't returned	The `Prefer` header is missing	Add `Prefer: include-unknown-enum-members`.
An admin action on an agent fails in the Microsoft 365 admin center	The agent is in a Power Platform environment with IP firewall in active enforcement mode	In the Power Platform admin center, go to **Security** > **Identity and access** > **IP firewall**, select the environment, and check the **Advanced** tab. See Governance and lifecycle actions.

✅ What you proved

- [ ] Each in-scope Microsoft-native test agent (Copilot Studio and Microsoft Foundry, where in scope) is listed in **Agents** > **All agents** > **Registry**.
- [ ] Every in-scope agent has a named owner, or is listed under **Agents without owners** and has a recorded remediation decision.
- [ ] The **Agents at risk** tile and **Risks** column were reviewed, and every agent with a high-severity risk signal has a named follow-up owner.
- [ ] Agent Map shows the test agents in their platform clusters.
- [ ] At least one agent request was reviewed in the **Requests** tab and either published or rejected.
- [ ] A registry export and the Microsoft Graph list of catalog packages were captured and compared, and any difference in counts is recorded.

References

- Connect existing agents to Microsoft Agent 365
- Agent registry integration for Copilot Studio
- Agent identity integration for Copilot Studio

- Microsoft Agent 365 integration with Foundry
- Publish your agent as an Agent Application
- Agent overview in Microsoft 365 admin center
- Get started with agents in SharePoint
- Agent Registry in Microsoft 365 admin center
- Agent management roles and permissions
- Use Agent Map in the Microsoft 365 admin center
- Manage agent requests in Microsoft 365 admin center
- Governance and lifecycle actions for agents
- Policy templates
- Understand agent details in Microsoft 365 admin center
- Agent settings in Microsoft 365 admin center
- Graph API for agent registry and agent details
- Package Management API overview
- List Copilot packages
- List agentIdentity objects
- List riskyAgents
- Microsoft Entra Agent ID logs
- agentSignIn resource type (agentType values)
- security: runHuntingQuery
- Manage agents across multiple tenants in the Microsoft 365 admin center (preview)

Runbook index · Next: Chapter 2 – Third-party and custom agents

Copilot Studio sample agent

> **Before you start:** Verify the chapter prerequisites and the sample-specific requirements below.

A minimal declarative agent definition that you can use as the reference configuration for a Copilot Studio test agent.

Table of contents

- Files
- How to use it
- Discovery path in the Agent Registry
- References

Files

- `declarative-agent.json`: an agent definition that uses the declarative agent schema 1.2. It contains a name, a description, instructions, conversation starters, web search and one SharePoint knowledge source.

How to use it

1. Replace `https://contoso.sharepoint.com/sites/trailguide-poc` with your PoC SharePoint site.
2. Create a new agent in Copilot Studio (`https://copilotstudio.microsoft.com`). Use the name, description and instructions from the JSON file, and add the SharePoint site as a knowledge source.
3. Publish the agent. Then open the configuration panel for the **Teams and Microsoft Copilot** channels, select **Availability options**, and share the agent with the organization by submitting it for admin approval.

The file contains no credentials or secrets.

Discovery path in the Agent Registry

Step	What happens
1. Create	When you create a Copilot Studio agent, an Agent ID is created and the agent appears in the agent registry immediately. Metadata changes are synchronized automatically.
2. Submit for approval	An agent submitted for admin approval appears in the Microsoft 365 admin center under **Agents** > **All agents** > **Requests**.
3. Approve	An AI Administrator selects **Publish to store**, selects the users or groups that can install the agent, applies a policy template, reviews permissions and publishes the agent.

All Copilot Studio app-based agents share a single blueprint, and an Agent ID is created for each agent.

References

- Agent registry integration for Copilot Studio
- Agent identity integration for Copilot Studio
- Declarative agent schema 1.2 for Microsoft 365 Copilot
- Connect and configure an agent for Teams and Microsoft 365
- Manage agent requests in Microsoft 365 admin center

Return to the chapter for registration, validation, and evidence.

Microsoft Foundry sample agent (prompt agent)

> **Before you start:** Verify the chapter prerequisites and the sample-specific requirements below.

A minimal example that creates a Microsoft Foundry prompt agent. After you publish it, it appears in the Agent 365 agent registry.

Table of contents

- Files
- Prerequisites
- Run
- Discovery path in the Agent Registry
- References

Files

- `create_foundry_agent.py`: creates the prompt agent `poc-foundry-trail-guide`. The code follows Quickstart: Create a prompt agent and uses the Foundry projects (new) API.

Prerequisites

- A model deployed in Microsoft Foundry.
- The **Foundry Project Manager** role on the Foundry resource scope to publish the agent.

Run

1. Install the packages and sign in:

```
pip install "azure-ai-projects>=2.3.0" azure-identity
az login
```

2. Copy your project endpoint from the welcome screen in the Foundry portal. In `create_foundry_agent.py`, replace `your_project_endpoint` with it, and replace `gpt-5-mini` with your model deployment if needed.
3. Run `python create_foundry_agent.py`.

Discovery path in the Agent Registry

Step	What happens
1. Create	The script creates an agent version. An unpublished agent uses the project's shared agent identity.
2. Publish	Publish the agent as an Agent Application. The published agent receives its own Entra agent identity and Entra agent blueprint. Permissions don't transfer automatically, so reassign the RBAC permissions that the agent's tools need to the new agent identity.
3. Registry sync	Every Foundry agent that you publish appears in the Agent 365 registry automatically. Registry sync is supported for prompt agents and hosted agents.
4. Activity	Activity data collection is supported for prompt agents. For hosted agents, it's supported by using the Agent 365 SDK. Logging is controlled per Foundry resource through the `agent365Config` configuration.

Verify the result in the Microsoft 365 admin center under **Agents** > **All agents** > **Registry**, and in the Microsoft Entra admin center under **Entra ID** > **Agents** > **Agent identities**.

Microsoft Foundry data residency follows the Azure region of the Foundry resource. Agent 365 data residency follows the storage location of the Microsoft Entra tenant. You can opt out individual Foundry resources from Agent 365 data collection.

References

- Quickstart: Create a prompt agent
- Microsoft Agent 365 integration with Foundry
- Publish your agent as an Agent Application
- Configure Agent 365 data collection for Microsoft Foundry

Return to the chapter for registration, validation, and evidence.

Chapter 1 – Agent samples

> **Before you start:** Verify the chapter prerequisites and the sample-specific requirements below.

Minimal examples of Microsoft-native agents that appear in the Agent Registry automatically. Use them to create test agents in Chapter 1 – Agent discovery and inventory.

Table of contents

- Samples
- References

Samples

Folder	Agent type	What it demonstrates
`copilot-studio/`	Copilot Studio agent	A reference agent definition, and how a Copilot Studio agent reaches the Registry and the **Requests** tab.
`foundry/`	Microsoft Foundry prompt agent	A Python script based on the Foundry prompt agent quickstart, and the publish step that brings the agent into the Registry.

Custom agents built with the Agent 365 SDK are covered in Chapter 2 – Agent 365 SDK. Their samples are in `chapter-02-third-party-custom-agents/samples/sdk`.

References

See the section-level Microsoft Learn links in the chapter.

Return to the chapter for registration, validation, and evidence.

Prerequisites - Chapter 2: Third-party and custom agents

> **Goal:** Prepare the accounts, access, and test assets needed for this chapter and its selected optional paths.

Choose Connected platforms or Agent 365 SDK; the paths are independent. Apply the shared PoC preparation guidance and agree the selected guide's outcome checklist before testing.

Table of contents

- Tenant readiness
- Prepare accounts and role assignments
- Test assets and dependencies
- Connected-platform preparation
- Agent 365 SDK preparation
- References

Tenant readiness

Documentation: Get started with Microsoft Agent 365 · Agent management in the Microsoft 365 admin center · Frontier program

Use the task-specific roles for Connected platforms or Agent 365 SDK. Confirm availability rather than assuming the tenant is ready.

Follow the shared tenant readiness check using the chapter permissions and feature prerequisites linked above.

Frontier opt-in, only for selected preview features

Performed by **Global Administrator**. Skip this task if no selected feature requires Frontier or opt-in is already verified.

1. In **Agents** > **Overview**, use the **Try now** banner and select **I agree** to accept the terms.
2. Record the date and time, then verify the selected preview feature is available.

Check result

- The Frontier feature needed for the selected test is available. Preview terms apply.

Prepare accounts and role assignments

Alongside the shared setup and reviewer accounts, prepare the following scenario accounts:

Account	Purpose
Platform administrator (connected-platform path only)	Prepares source-platform credentials, permissions, and any selected observability configuration.
Developer account (SDK path only)	Builds and registers custom agents with the Agent 365 CLI.

Assign the roles for the selected guide using the shared model. Verify each account can access the pages needed for its tasks.

Test assets and dependencies

Documentation: Connected platforms in Microsoft Agent 365 · Third-party agent observability (Frontier) · Choose an Agent 365 integration option · Connect Amazon Bedrock · Connect Google Vertex AI · Connect Salesforce Agentforce · Connect Databricks Genie · Connect Anthropic Claude Managed Agents · Connect Oracle Generative AI Agents · Connect Snowflake Cortex · Install and use the Agent 365 CLI · Agent 365 Skills for guided agent setup · Quickstart: Connect an existing agent to Agent 365 · Custom client app registration for Agent 365 CLI · Onboard a single agent with the Agent 365 CLI

- Verify the required inventory using Chapter 1 – Agent discovery and inventory and export the registry before you start, so you can show what this chapter adds.
- Never commit `.env` files or other files with sensitive information to source control.

Connected-platform preparation

Prepare a non-production account on the selected source platform with at least one test agent, and a platform administrator. Choose one supported platform; you don't need the SDK toolchain for this path.

Continue with 2.2 Connect a third-party agent platform and its provider-specific instructions.

Agent 365 SDK preparation

Prepare .NET 8.0 or later, the Azure CLI, an Azure subscription in the same tenant, a test agent project (you can start from `samples/sdk/`), and a Global Administrator available to complete consent. A connected-platform connection isn't required.

Continue with 2.7 Prepare the custom-agent toolchain.

Check result

- Each asset needed for the selected paths is prepared or has a named build task that will finish before its dependent tests.

References

- Get started with Microsoft Agent 365
- Agent management in the Microsoft 365 admin center
- Frontier program
- Connected platforms in Microsoft Agent 365
- Third-party agent observability (Frontier)
- Choose an Agent 365 integration option

- Connect Amazon Bedrock
- Connect Google Vertex AI
- Connect Salesforce Agentforce
- Connect Databricks Genie
- Connect Anthropic Claude Managed Agents
- Connect Oracle Generative AI Agents
- Connect Snowflake Cortex
- Install and use the Agent 365 CLI
- Agent 365 Skills for guided agent setup
- Quickstart: Connect an existing agent to Agent 365
- Custom client app registration for Agent 365 CLI
- Onboard a single agent with the Agent 365 CLI

> **Ready?** Continue to Connected platforms or Agent 365 SDK. →

Chapter 2 - Connected platforms

Pillar: Observe **What it proves:** Agents on supported third-party platforms are discovered through Connected platforms without code and appear in the Microsoft 365 agent registry with their metadata.

For agents you integrate in code, use the separate Agent 365 SDK guide. These are independent paths.

Success criteria: Agree the selected outcomes in What you proved before testing.

Topics and time guide

Approximate hands-on estimates for the selected scope, with prerequisites prepared. Service waiting time is separate.

Topic	Estimated hands-on time
2.1 Required permissions	~5 min
2.2 Connect a third-party agent platform	~10-15 min
2.3 Example: connect Amazon Bedrock	~20-30 min
2.4 Example: connect Databricks Genie	~15-20 min
2.5 Example: connect Google Vertex AI	~20-30 min
2.6 Connect other supported platforms	~15-25 min
2.12 Test and validation	~15-20 min
2.13 Evidence	~5-10 min
2.14 Troubleshooting	As needed

Outcome checklist · References

Choose the platform example for your environment; the examples are alternative paths.

2.1 Required permissions

Use the shared account and role-assignment model with the task-specific permissions below.

Task	Least-privilege role	Section
Create, validate, sync, edit, and delete connected-platform connections	AI Administrator	2.2 to 2.6
Amazon Bedrock: provide the AWS access key ID and secret access key, grant the IAM permissions, configure Bedrock activity	Platform administrator (AWS)	2.3
Databricks Genie: create the service principal and client secret, grant the permissions required to read the Genie agents in the workspace, rotate or revoke the secret	Platform administrator (Databricks)	2.4

Task	Least-privilege role	Section
Google Vertex AI: create the service account and JSON key, grant the Vertex AI permissions and observability roles, configure Google Cloud telemetry	Platform administrator (Google Cloud)	2.5
Other supported platforms: create source-platform credentials, grant source-platform permissions, enable source telemetry	Platform administrator (source platform)	2.6
Validation / read-only review	AI Reader (registry and agent activity)	2.12.1 to 2.12.4

- For connected platforms, the AI Administrator creates and manages the connection in the Microsoft 365 admin center; the platform administrator creates the source-platform credentials, grants the required permissions, and enables platform-side telemetry when applicable.

Before you start:

- [ ] Confirm the shared tenant checks and the connected-platform preparation.

2.2 Connect a third-party agent platform

Documentation: Connected platforms in Microsoft Agent 365 · Third-party agent observability (Frontier) · Choose an Agent 365 integration option

2.2.1 Understand connected platforms

Use the **Connected platforms** page to connect supported third-party agent platforms to Agent 365. After you create a connection, Agent 365 can discover supported agents and add them to the Microsoft 365 agent registry. Synchronization, management, and observability capabilities vary by platform.

Supported platforms, as listed on the Connected platforms page:

Platform	Supported environment	Agent synchronization	Observability	Configuration guide
Amazon Bedrock	Agents Classic and AgentCore	Yes	Yes	Connect Amazon Bedrock (worked example in 2.3)
Google Vertex AI	Vertex AI agents	Yes	Yes	Connect Google Vertex AI (worked example in 2.5)
Salesforce Agentforce	Agentforce	Yes	Coming soon	Connect Salesforce Agentforce
Databricks Genie	Genie	Yes	Coming soon	Connect Databricks Genie (worked example in 2.4)
Anthropic Claude	Managed Agents	Yes	Yes	Connect Anthropic Claude Managed Agents
Oracle Generative AI Agents	Generative AI Agents	Yes	Coming soon	Connect Oracle Generative AI Agents
Snowflake Cortex	Cortex Agents	Yes	Coming soon	Connect Snowflake Cortex

Choose the right integration mechanism for each agent:

	Registry sync (preview)	Agent 365 SDK
Applies to	Connected agent platforms, such as Google Vertex AI and Amazon Bedrock	Agents you build with an agent SDK or framework, such as the Microsoft 365 Agents SDK, Microsoft Agent Framework, OpenAI Agents SDK, LangChain, CrewAI, or LlamaIndex
How it's applied	Configured per platform. Imports agents for visibility and governance.	Developer integrated in your agent's code for identity, tooling, and notifications. Use Microsoft OpenTelemetry for observability.
SDK required	No	Yes
Covered in	This guide, 2.2 to 2.6	Agent 365 SDK guide, 2.7 to 2.11

2.2.2 Plan the connection with the platform administrator

Every connection follows the same lifecycle:

1. **Prepare**: Work with the platform administrator to select the account, project, workspace, compartment, or region and prepare the required credentials and permissions.
2. **Connect**: Create and validate the connection in the Microsoft 365 admin center.
3. **Synchronize**: Start synchronization so that Agent 365 discovers supported agents and adds them to the agent registry.
4. **Verify**: Confirm the expected agents, metadata, and capabilities with the platform administrator.
5. **Operate**: Monitor synchronization, rotate credentials, review errors, and remove connections when they're no longer needed.

Responsibilities:

Task	Agent 365 AI administrator	Platform administrator
Confirm Agent 365 access and role	Owns	Informed
Select the source account, project, workspace, or region	Consulted	Owns
Create credentials	Informed	Owns
Grant source-platform permissions	Informed	Owns
Create and validate the Agent 365 connection	Owns	Supports
Verify synchronized agents	Owns	Supports
Enable source telemetry	Consulted	Owns
Rotate or revoke credentials	Coordinates	Owns

2.2.3 Create a platform connection

Performed by **AI Administrator**, after the platform administrator completes the provider-specific preparation (2.3, 2.4, 2.5, or 2.6).

1. Open the Microsoft 365 admin center at https://admin.cloud.microsoft.

2. In the navigation pane, select **Agents** > **All agents**.
3. In the **Connected platforms** web part, select **Manage**.
4. Select **+ Connect a platform**.
5. Enter a connection name and description.
6. Select the platform.
7. Enter the provider-specific values described in the provider guide (see 2.3.4 for Amazon Bedrock, 2.4.3 for Databricks Genie, 2.5.4 for Google Vertex AI, and 2.6 for the other platforms).
8. Validate the credentials.
9. Save the connection.

Check result

- The credentials validate and the connection is saved. Synchronization starts only when you select **Sync agents** (2.2.4).

2.2.4 Synchronize, verify, and manage the connection

Performed by **AI Administrator**.

1. After you save the connection, select **Sync agents** to start synchronization. Manual synchronization is required.
2. Select the connection and review:
   - The platform provider.
   - The connected account, project, workspace, compartment, or region.
   - The last run date.
   - The latest synchronization status.
   - The total number of synchronized agents.
   - The synchronization results.
   - Any synchronization errors.
3. Confirm the expected agents, metadata, and capabilities with the platform administrator.
4. To rotate credentials, change the source scope, revoke access, or remove the connection, follow the relevant provider guide.

Check result

- The total number of synchronized agents matches what the platform administrator expects, and no synchronization errors are listed.

2.2.5 Turn on observability data collection (Frontier preview)

Third-party agent observability is part of the Frontier preview program. Supported observability platforms:

Platform	Environment
Amazon Bedrock	Agents Classic and AgentCore
Google Vertex AI	Vertex AI agents
Anthropic Claude	Managed Agents
Salesforce Agentforce	Agent Builder

Performed by **AI Administrator**.

1. Make sure the platform administrator configured the source telemetry described in the provider guide (for Amazon Bedrock, see 2.3.3; for Google Vertex AI, see 2.5.3). Enabling collection doesn't create source telemetry.

2. Edit the connection and confirm that **Collect agent observability data** is selected. Data collection is enabled by default for new connections; for connections created before this capability was introduced, it's disabled by default.

Check result

- **Collect agent observability data** is selected for the connection. Activity is tested in 2.12.4.

2.3 Example: connect Amazon Bedrock

Documentation: Connect Amazon Bedrock to Microsoft Agent 365 · Troubleshoot connected platforms

2.3.1 Gather the prerequisites

Performed by **AI Administrator** with the **platform administrator**.

1. Confirm that the AI Administrator can create connections in the Microsoft 365 admin center.
2. Ask the platform administrator for an AWS access key ID and secret access key.
3. Identify the AWS Regions that contain the agents you want to synchronize.
4. Grant only the permissions required for the environments you use (Agents Classic, AgentCore, or both).

2.3.2 Grant the AWS synchronization permissions

Performed by **platform administrator**.

1. For **Agents Classic**, grant:

   ```
   bedrock:ListAgents
   bedrock:GetAgent
   ec2:DescribeRegions
   ```

2. For **AgentCore**, grant:

   ```
   bedrock-agentcore:ListHarnesses
   bedrock-agentcore:GetHarness
   bedrock-agentcore:ListAgentRuntimes
   bedrock-agentcore:GetAgentRuntime
   ec2:DescribeRegions
   ```

   `bedrock:ListAgents` and the AgentCore list operations discover agents in the selected Regions. The get operations read agent details. AgentCore runtimes that belong to a harness are skipped so that an agent isn't listed twice.

3. Grant optional permissions only when the connection needs the corresponding details:

Capability	Permissions
Show Classic tools, collaborators, and knowledge sources	`bedrock:ListAgentActionGroups`, `bedrock:GetAgentActionGroup`, `bedrock:ListAgentCollaborators`, `bedrock:ListAgentKnowledgeBases`, `bedrock:ListDataSources`, `bedrock:GetDataSource`

Capability	Permissions
Show AgentCore tools, MCP servers, and connected agents	`bedrock-agentcore:GetGateway`, `bedrock-agentcore:ListGatewayTargets`, `bedrock-agentcore:GetGatewayTarget`
Connect every account in an AWS organization	`organizations:ListAccounts`, `sts:AssumeRole`

4. Don't grant delete permissions for a read-only connection. If you grant `bedrock:DeleteAgent`, `bedrock-agentcore:DeleteHarness`, or `bedrock-agentcore:DeleteAgentRuntime`, Agent 365 can remove the corresponding agent when an administrator explicitly performs that action.

Check result

- The synchronization permissions for each environment in use are granted, and no delete permissions are granted.

2.3.3 Configure Bedrock activity for observability

Performed by **platform administrator**. Skip this task if you don't test third-party observability (2.2.5). Agent 365 reads activity. It doesn't invoke agents, write activity records, or delete activity records.

For **Agents Classic**:

1. Enable trace capture for each agent in the Amazon Bedrock console.
2. Select an S3 activity bucket.
3. Store activity under `observability-store/<agent-id>/`, where `<agent-id>` is the Bedrock agent ID.
4. Grant `s3:ListBucket` on the activity bucket.
5. Grant `s3:GetObject` on objects in the activity bucket.
6. If the bucket uses a customer-managed key, also grant `kms:Decrypt` on that key.

For **AgentCore** (activity is stored in Amazon CloudWatch Logs):

1. Grant `logs:StartQuery` and `logs:GetQueryResults`.
2. Grant `logs:DescribeLogGroups` to allow Agent 365 to find custom agent endpoints. Without it, only the default endpoint is read.
3. Enable **Transaction Search** in the Amazon CloudWatch console. AgentCore activity is queryable only after Transaction Search is enabled for the AWS account.

Check result

- Classic: trace capture is enabled, activity is stored under `observability-store/<agent-id>/`, and the S3 (and, if applicable, KMS) permissions are granted.
- AgentCore: the CloudWatch Logs permissions are granted and Transaction Search is enabled.

2.3.4 Create the Amazon Bedrock connection

Performed by **AI Administrator**.

1. Open the Microsoft 365 admin center at https://admin.cloud.microsoft.
2. Select **Agents** > **All agents**.
3. In **Connected platforms**, select **Manage**.
4. Select **+ Connect a platform** and choose **Amazon Bedrock**.
5. Enter the connection name, AWS access key ID, secret access key, and selected Regions.
6. Validate the credentials.
7. Save the connection.

8. Select **Sync agents**.

Check result

- The credentials validate. If validation fails, see 2.14.

2.3.5 Verify the Amazon Bedrock connection

Performed by **AI Administrator**.

1. Open the connection and check the synchronization status and the number of agents.
2. Open an imported agent to verify its metadata.
3. For observability, open the agent and select **Activity** after you configure the source platform and activity is available.

Check result

- The expected Agents Classic and AgentCore agents are imported with their metadata.

2.4 Example: connect Databricks Genie

Documentation: Connect Databricks Genie to Microsoft Agent 365 · Connected platforms in Microsoft Agent 365 · Troubleshoot connected platforms

Databricks Genie supports agent synchronization. Its observability is listed as **Coming soon** on the Connected platforms page, so this example covers synchronization only.

2.4.1 Gather the prerequisites

Performed by **AI Administrator** with the **platform administrator**.

1. Confirm that the AI Administrator can create connections in the Microsoft 365 admin center (2.1).
2. Obtain the Databricks workspace URL.

2.4.2 Create the service principal and grant permissions

Performed by **platform administrator**.

1. Create a service principal.
2. Obtain the service principal client ID.
3. Obtain the service principal client secret.
4. Grant the permissions required to read the Genie agents in the workspace.

Check result

- You have the workspace URL, the service principal client ID, and the client secret, and the service principal can read the Genie agents in the workspace.

2.4.3 Create the Databricks Genie connection

Performed by **AI Administrator**.

1. Open the Microsoft 365 admin center at https://admin.cloud.microsoft.

2. Select **Agents** > **All agents**.
3. In **Connected platforms**, select **Manage**.
4. Select **+ Connect a platform** and choose **Databricks Genie**.
5. Enter the workspace URL, client ID, and client secret.
6. Validate the credentials.
7. Save the connection.
8. Select **Sync agents**.

Check result

- The credentials validate. If validation fails, see 2.14.

2.4.4 Verify the Databricks Genie connection

Performed by **AI Administrator**.

1. Open the connection and review the synchronization status.
2. Confirm that the expected Genie agents appear in the Microsoft 365 agent registry.

Check result

- The expected Genie agents are listed in the registry.

2.4.5 Rotate or revoke Databricks access

Performed by **platform administrator** (Databricks) and **AI Administrator** (connection).

1. To rotate access, the platform administrator rotates the service principal secret in Databricks. The AI Administrator then updates the Databricks Genie connection with the new secret and validates the credentials.
2. To revoke access, the platform administrator disables the service principal or removes its workspace permissions.

Check result

- After rotation, the connection validates with the new secret. After revocation, the service principal is disabled or has no workspace permissions.

2.5 Example: connect Google Vertex AI

Documentation: Connect Google Vertex AI to Microsoft Agent 365 · Third-party agent observability (Frontier) · Troubleshoot connected platforms

2.5.1 Gather the prerequisites

Performed by **AI Administrator** with the **platform administrator**.

1. Confirm that the AI Administrator can create connections in the Microsoft 365 admin center.
2. Ask the platform administrator for the Google Cloud project ID and region. Use the project and region where the agents are deployed.
3. Ask the platform administrator to create a Google Cloud service account and a JSON key for that account (2.5.2).

2.5.2 Create the service account and grant the Vertex AI permissions

Performed by **platform administrator**.

1. Create a Google Cloud service account.
2. Grant the service account the Vertex AI permissions required for agent discovery and management:

```
aiplatform.reasoningEngines.list
aiplatform.reasoningEngines.get
aiplatform.reasoningEngines.delete
```

3. Create a JSON key for the service account. The JSON key must contain the complete service-account key document, including the opening and closing braces. Don't enter only the `private_key` value.

Check result

- The service account has the three Vertex AI permissions, and you have its complete JSON key document.

2.5.3 Configure Vertex AI observability (Frontier preview)

Performed by **platform administrator**. Skip this task if you don't test third-party observability (2.2.5).

1. To collect observability data, grant the same service account these roles:

Role	Role ID	Purpose
BigQuery Data Viewer	`roles/bigquery.dataViewer`	Read exported trace tables
BigQuery Job User	`roles/bigquery.jobUser`	Run queries to retrieve spans
Monitoring Viewer	`roles/monitoring.viewer`	Read request-count and latency metrics
Cloud Trace User	`roles/cloudtrace.user`	Read Cloud Trace spans

   You need both BigQuery roles. Data Viewer permits reading the tables, while Job User permits running the queries that retrieve the results.

2. On the project where you deployed the agent:

    1. Enable the Vertex AI API.
    2. Enable the BigQuery API.
    3. Enable the Cloud Trace API.
    4. Enable the Cloud Monitoring API.
    5. Enable telemetry collection on the deployed agent.
    6. Enable Trace Analytics BigQuery export.
    7. Confirm that the `trace_analytics_link` dataset and its `_AllSpans` view exist.

Agent 365 can't enable telemetry collection or trace export for you.

Check result

- The four roles are granted to the service account, the four APIs are enabled, telemetry collection and Trace Analytics BigQuery export are enabled, and the `trace_analytics_link` dataset and its `_AllSpans` view exist.

2.5.4 Create the Google Vertex AI connection

Performed by **AI Administrator**.

1. Open the Microsoft 365 admin center at https://admin.cloud.microsoft.
2. Select **Agents** > **All agents**.
3. In **Connected platforms**, select **Manage**.
4. Select **+ Connect a platform** and choose **Google Vertex AI**.
5. Enter the project ID, region, and complete service-account JSON key.
6. Validate the credentials.
7. Save the connection.
8. Select **Sync agents**.

Check result

- The credentials validate. If validation fails, see 2.14.

2.5.5 Verify the Google Vertex AI connection

Performed by **AI Administrator**.

1. Open the connection and review the synchronization status and the number of synchronized agents.
2. Confirm that the expected agents are imported into the Microsoft 365 agent registry.
3. For observability, open an imported agent and select the **Activity** tab after the agent runs.

A connection can validate and import agents while returning no telemetry if the project, region, roles, APIs, or trace export aren't configured correctly (see 2.14).

Check result

- The expected Vertex AI agents are imported into the registry.

2.6 Connect other supported platforms

Documentation: Connect Salesforce Agentforce · Connect Anthropic Claude Managed Agents · Connect Oracle Generative AI Agents · Connect Snowflake Cortex

2.6.1 Prepare and connect another platform

Each platform has its own configuration guide. Follow the platform article for the source-side steps, then create, sync, and verify the connection with the same wizard as in 2.2.3 and 2.2.4.

Platform	Platform administrator prepares	Values entered in the wizard
Salesforce Agentforce	Einstein Generative AI and Agentforce turned on; an external client app with the client credentials flow and a **Run As** user with API access. For activity: Data 360 and Agentforce Session Tracing.	My Domain URL without a path, consumer key, consumer secret
Anthropic Claude (Managed Agents)	A workspace other than the default workspace, an environment and a managed agent in the workspace, and a workspace-scoped API key (requires at least the **Developer** role)	Workspace ID, workspace-scoped API key

Platform	Platform administrator prepares	Values entered in the wizard
Oracle Generative AI Agents	An API-only service user in a group, an API key pair, and a least-privilege policy that grants read access to the agent resources in the selected compartment	Private key, user OCID, tenancy OCID, fingerprint, region, compartment OCID
Snowflake Cortex	A Snowflake user with RSA key-pair authentication, and the database and schema where the agents are created	Account identifier, database, schema, Snowflake user, private key

Performed by **platform administrator** (source side) and **AI Administrator** (connection).

1. Ask the platform administrator to complete the source-side steps in the platform article.
2. Create the connection as in 2.2.3, selecting the platform and entering the values from the table.
3. Sync and verify as in 2.2.4.

Check result

- The expected agents are imported.

2.12 Test and validation

2.12.1 Test: third-party agents appear in the registry

Performed by **AI Administrator** (connection status) and **AI Reader** (registry).

1. In the Microsoft 365 admin center, select **Agents** > **All agents**, and in the **Connected platforms** web part select **Manage**. Select each PoC connection and confirm the latest synchronization status, the total number of synchronized agents, and that no synchronization errors are listed.
2. Open **Agents** > **All agents** > **Registry**.
3. Use the **Platform** filter to show the agents of the connected platform.
4. Confirm that every expected test agent is listed.
5. Open each agent and review its details.
6. Select **Export** to export the agents to a CSV file.

Expected result

- Each connection shows the expected number of synchronized agents and no synchronization errors.
- Every expected third-party agent is listed with its metadata.

2.12.2 Test: Databricks Genie agents appear in the registry after sync

Performed by **AI Administrator** (sync) and **AI Reader** (registry).

1. On the **Connected platforms** page, select **Sync agents** for the Databricks Genie connection (2.4.3).
2. Open the connection and review the synchronization status.
3. Open **Agents** > **All agents** > **Registry** and confirm that the expected Genie agents appear.

Expected result

- The expected Genie agents from the workspace appear in the Microsoft 365 agent registry.

2.12.3 Test: Google Vertex AI agents appear in the registry after sync

Performed by **AI Administrator** (sync) and **AI Reader** (registry).

1. On the **Connected platforms** page, select **Sync agents** for the Google Vertex AI connection (2.5.4).
2. Open the connection and review the synchronization status and the number of synchronized agents.
3. Open **Agents** > **All agents** > **Registry** and confirm that the expected Vertex AI agents are imported.

Expected result

- The expected Vertex AI agents from the selected project and region appear in the Microsoft 365 agent registry.

2.12.4 Test: third-party agent activity is observable (Frontier preview)

Performed by **platform administrator** (run the agent) and **AI Reader** (review).

1. On the source platform, run the test agent (for Amazon Bedrock, in a Region that the connection covers; for Google Vertex AI, in the project and region that the connection covers).
2. Wait for the platform-specific processing interval. Anthropic Claude activity can take 5 to 10 minutes after an interaction.
3. In the registry, select the synchronized agent and select the **Activity** tab.
4. Review **Sessions**, **Exceptions**, and **Runtime**. **Users** isn't available for third-party agents when the source platform doesn't provide Microsoft Entra user identities.

Expected result

- The sessions from step 1 appear on the **Activity** tab.

2.13 Evidence

- Screenshot of the **Connected platforms** page with the details of each PoC connection (Amazon Bedrock, Databricks Genie, Google Vertex AI, or other): provider, scope, last run date, latest synchronization status, number of synchronized agents.
- Screenshot of the synchronization results and errors, if any.
- The list of source-platform permissions granted to the connection credentials.
- Registry export (CSV) and a screenshot of the registry filtered by platform, showing the synchronized agents.
- (Optional) Screenshot of the **Activity** tab of a synchronized third-party agent.

2.14 Troubleshooting

Symptom	Likely cause	Fix
Connection validation fails (any platform)	Credentials not current, missing discovery permissions, connection fields that don't match the provider guide, or the source platform doesn't allow the connection	Confirm credentials, permissions, and fields against the provider guide. If credentials expired or were revoked, rotate them and update the connection.
Amazon Bedrock validation fails	Missing `ec2:DescribeRegions`, or invalid or disabled credentials	Confirm the credentials and grant `ec2:DescribeRegions`.

Symptom	Likely cause	Fix
Agents don't appear after synchronization	Agents outside the selected account, project, workspace, compartment, or region; credentials can't list the agents	Review the synchronization status and errors, confirm the scope and the discovery permissions, and select **Sync agents** again.
Agents appear with limited metadata	Missing get or optional metadata permissions (for Amazon Bedrock, `bedrock:GetAgent` or the AgentCore get operation)	Grant the relevant get permission and the optional metadata permissions from the provider guide.
Duplicate agents appear	The source platform exposes the same agent through multiple resources or environments	Verify the connection scope and synchronization results before you delete or change an agent.
Agent appears but no activity is shown	Source telemetry isn't enabled or **Collect agent observability data** is disabled	Enable source telemetry and **Collect agent observability data**, then run the agent.
Activity is delayed	Source processing or ingestion is still running	Wait for the platform-specific processing interval and check again.
Amazon Bedrock Classic agents have no activity	Incorrect trace capture, bucket location, bucket path, or S3 permissions	Enable trace capture and verify the bucket and object permissions.
Amazon Bedrock AgentCore agents have no activity	Missing Transaction Search or CloudWatch Logs permissions	Enable Transaction Search and grant `logs:StartQuery` and `logs:GetQueryResults`.
Amazon Bedrock AgentCore activity is incomplete	Missing `logs:DescribeLogGroups`	Grant the permission when custom endpoints are used.
Google Vertex AI agents import but no activity appears	Observability isn't configured	Complete the roles, API, telemetry, and trace-export steps (2.5.3).
Google Vertex AI: no spans, but metrics appear	BigQuery roles or trace export are missing	Grant both BigQuery roles and enable trace export.
Google Vertex AI: queries fail although tables are readable	`roles/bigquery.jobUser` is missing	Grant BigQuery Job User to the service account.
Google Vertex AI: no request-count or latency metrics	Monitoring role or API is missing	Grant Monitoring Viewer and enable Cloud Monitoring API.
Google Vertex AI: `_AllSpans` doesn't exist	Trace Analytics BigQuery export isn't enabled	Enable the export and verify the dataset and view.
Google Vertex AI agents import but telemetry is empty	Connection scope doesn't match deployment scope	Use the deployed agent's project and region.

✅ What you proved

- [ ] A connection to at least one supported third-party platform validates, **Sync agents** runs, and the connection shows the expected number of synchronized agents with no synchronization errors.
- [ ] Every expected third-party test agent appears in **Agents** > **All agents** > **Registry** with its metadata.
- [ ] (Optional, Frontier preview) The **Activity** tab of a synchronized third-party agent shows sessions after the agent runs on the source platform.

References

- Connected platforms in Microsoft Agent 365
- Third-party agent observability (Frontier)
- Choose an Agent 365 integration option
- Connect Amazon Bedrock
- Connect Google Vertex AI
- Connect Salesforce Agentforce
- Connect Databricks Genie
- Connect Anthropic Claude Managed Agents
- Connect Oracle Generative AI Agents
- Connect Snowflake Cortex
- Troubleshoot connected platforms

Runbook index · Previous: Chapter 1 – Agent discovery and inventory · Agent 365 SDK guide · Next: Chapter 3 – Agent identity and ownership

Chapter 2 - Agent 365 SDK

Pillar: Observe **What it proves:** Custom agents are registered through the Agent 365 CLI with a Microsoft Entra agent identity blueprint and agent identity, then deployed and published for governance.

For no-code discovery from supported third-party platforms, use Connected platforms. A platform connection isn't required for this SDK path.

Success criteria: Agree the selected outcomes in What you proved before testing.

Topics and time guide

Approximate hands-on estimates for the selected scope, with prerequisites prepared. Service waiting time is separate.

Topic	Estimated hands-on time
2.1 Required permissions	~5 min
2.7 Prepare the custom-agent toolchain	~15-20 min
2.8 Create the agent blueprint with a365 setup	~15-20 min
2.9 Deploy the agent runtime to Azure (optional)	~15-25 min
2.10 Publish the agent and verify it in the registry	~10-15 min
2.11 Add observability and data protection (optional)	~20-40 min
2.12 Test and validation	~15-20 min
2.13 Evidence	~5-10 min
2.14 Troubleshooting	As needed

Outcome checklist · References

2.1 Required permissions

Use the shared account and role-assignment model with the task-specific permissions below.

Task	Least-privilege role	Section
Configure API permissions and consent on a tenant-owned **Agent 365 CLI** client app (only when the Microsoft-managed app isn't available)	Application Administrator (or Cloud Application Administrator)	2.7.3
Run `a365 setup all` (blueprint, inheritable permissions, agent identity, agent registration) and `a365 setup blueprint` (messaging endpoint)	Agent ID Developer plus Contributor on the Azure subscription	2.8, 2.9.4
Grant OAuth2 permissions (admin consent) for the blueprint; run `a365 setup permissions` subcommands	Global Administrator	2.8.4, 2.8.6

Task	Least-privilege role	Section
Grant service-to-service (S2S) agent identity grants (`--authmode s2s` or `both`)	Application Administrator or Global Administrator	2.8.2
Deploy the agent code to Azure	Contributor on the Azure subscription	2.9
Upload the published `manifest.zip` in the Microsoft 365 admin center	Global Administrator	2.10.2
Validation / read-only review	AI Reader (registry); any Microsoft Entra user (agent identities and blueprints lists); Directory Readers (`a365 query-entra`); **Security operations** > **Security data** > **Security data basics (read)** in Defender unified RBAC (advanced hunting)	2.8.5, 2.12

- Agent ID Developer and Agent ID Administrator can complete every `a365 setup all` step except the OAuth2 permission grants (admin consent), which require a Global Administrator. When setup completes, the CLI prints the next steps for the Global Administrator (2.8.4).

Before you start:

- [ ] Confirm the shared tenant checks and the Agent 365 SDK preparation.

2.7 Prepare the custom-agent toolchain

Documentation: Install and use the Agent 365 CLI · Agent 365 Skills for guided agent setup · Quickstart: Connect an existing agent to Agent 365 · Custom client app registration for Agent 365 CLI · Onboard a single agent with the Agent 365 CLI

2.7.1 Understand the custom-agent journey

A custom agent goes through three stages:

1. **Blueprint (2.8):** The agent blueprint defines the agent's identity, permissions, and infrastructure requirements. Every agent instance is created from it. `a365 setup all` creates the blueprint in Microsoft Entra, configures permissions (including inheritable permissions for agent instances), creates an agent identity, and registers the agent.
2. **Deploy (2.9, optional):** You deploy the agent code to the cloud. You can skip this step if the agent is already deployed to a cloud; it doesn't need to be Azure.
3. **Publish (2.10):** `a365 publish` packages the agent, and you upload the package to the Microsoft 365 admin center. After publishing, you can create agent instances and govern the agent.

The Agent 365 SDK isn't an alternative to your agent framework. Build the agent with your framework first (for example, the Microsoft 365 Agents SDK, as in `samples/sdk/`), then bring it into Agent 365.

2.7.2 Install the Agent 365 CLI and sign in to Azure

Performed by the **developer**.

1. Install .NET 8.0 or later and the Azure CLI.

2. Install the Agent 365 CLI as a global .NET tool, or update it if it's already installed:

```
dotnet tool install --global Microsoft.Agents.A365.DevTools.Cli
# If the CLI is already installed:
dotnet tool update --global Microsoft.Agents.A365.DevTools.Cli
```

3. Verify the installation:

```
a365 --version
```

4. Sign in to Azure with the account that holds the roles in 2.1, and select the subscription in the tenant where you onboard the agent. The CLI detects the tenant from the active Azure CLI account, and setup fails with authentication errors if you aren't signed in:

```
az login
az account list --output table
az account set --subscription "<subscription-id-or-name>"
az account show --query "{tenantId:tenantId, subscriptionId:id, subscriptionName:name}" --output table
```

Check result

- `a365 --version` returns a version, and `az account show` returns the PoC tenant and subscription.

2.7.3 Confirm the CLI client application

By default, Agent 365 setup uses the Microsoft-managed **Agent 365 CLI** enterprise application (application ID `f54280f4-395e-4ea8-9e48-bf2d4952aa14`) when it's available in your tenant. On native Windows it uses Web Account Manager (WAM); on Windows Subsystem for Linux, macOS, and Linux it uses device code. Security Defaults or a Conditional Access policy can block device code authentication and return `AADSTS530035`.

1. Open the Microsoft Entra admin center at https://entra.microsoft.com.
2. Select **Enterprise applications** > **All applications** and search for the application ID `f54280f4-395e-4ea8-9e48-bf2d4952aa14`.
3. If the application is present, continue with 2.8. Don't modify the Microsoft-managed application.
4. If it isn't present, config-free setup looks for a tenant-owned app named **Agent 365 CLI**. Create it by following Custom client app registration for Agent 365 CLI. Configuring its API permissions requires **Application Administrator** (recommended), **Cloud Application Administrator**, or **Global Administrator**. A Global Administrator can also create it from the `a365 setup requirements` prompt (2.8.1).

Check result

- Either the Microsoft-managed **Agent 365 CLI** enterprise application or a tenant-owned app named **Agent 365 CLI** exists in the tenant.

2.7.4 (Optional) Use the Agent 365 Skills in a coding assistant

Agent 365 Skills run in Claude Code, GitHub Copilot CLI, and VS Code agent mode. You state the outcome you want (for example, *set up this project for Agent 365* or *register this agent with Agent 365*), and the skill identifies the project, gathers the required values, makes the integration changes, and checks the result. `a365-setup` is the entry point and routes to `make-a365-agent` (standard agent) or `make-ai-teammate` (AI teammate, Frontier preview program only).

Installation options and the skill catalog are in `samples/sdk/README.md`. The rest of this chapter shows the equivalent CLI commands.

2.8 Create the agent blueprint with a365 setup

Documentation: Set up agent blueprint · Agent 365 CLI setup command reference · Onboard a single agent with the Agent 365 CLI · Agent 365 CLI query-entra command reference · View and filter agent identities · View and manage agent identity blueprints

2.8.1 Validate the prerequisites

Performed by **Agent ID Developer** (with Contributor on the Azure subscription).

1. Run the requirements check from any directory:

   ```
   a365 setup requirements
   ```

2. Resolve every failed required check. The command checks authentication, Azure access, tenant enrollment, roles, and the client application, and continues after a failed check so you can review all detected issues together.

3. If neither the Microsoft-managed application nor a tenant-owned app named **Agent 365 CLI** is available, the command prompts for a custom client app ID. A Global Administrator can enter `c` at the prompt to create and configure a tenant-owned app.

Check result

- All required checks pass.

2.8.2 Choose the setup variant

Scenario	Command	Notes
Agent, with `a365.config.json`	`a365 setup all`	Creates Azure infrastructure if it doesn't already exist (resource group, App Service plan, Azure Web App with managed identity enabled), registers the agent blueprint, configures API permissions, and saves generated IDs to `a365.generated.config.json`.
Agent, config-free	`a365 setup all --agent-name "<name>"`	No config file needed. The tenant ID is detected from `az account show` (override with `--tenant-id`). Display names are derived as `<name> Agent` (identity) and `<name> Blueprint` (blueprint). Infrastructure is always skipped (external hosting assumed).
Microsoft 365 agent (Teams, Copilot)	`a365 setup all --m365` (optionally `--messaging-endpoint <url>`)	Registers the messaging endpoint. If the endpoint is deferred, register it after deployment (2.9.4).
AI teammate (Frontier preview program only)	`a365 setup all --aiteammate`	Requires a manually created `a365.config.json`. Not used in this PoC.

Scenario	Command	Notes
Agent identity grants	`--authmode obo` (default), `s2s`, or `both`	`obo` creates principal-scoped delegated grants and needs no admin role. `s2s` creates app role assignments and requires Application Administrator or Global Administrator; if the role is absent, the CLI prints a PowerShell fallback. Not supported with `--aiteammate`.
Preview	add `--dry-run`	Shows what the command would do without executing it.

For this PoC, use a config-free standard agent with the default `obo` mode and a test base name, for example `PoC Custom`.

2.8.3 Run the setup

Performed by **Agent ID Developer** (with Contributor on the Azure subscription), or by **Global Administrator** to complete all steps in one run.

1. Create an empty working directory and change to it:

```
New-Item -ItemType Directory -Path .\poc-custom-agent
Set-Location .\poc-custom-agent
```

2. Preview the changes without creating tenant objects:

```
a365 setup all `
    --agent-name "PoC Custom" `
    --aiteammate false `
    --dry-run
```

3. Review the tenant ID, derived object names, selected client application, permissions, and authentication mode.

4. Run the same command without `--dry-run`:

```
a365 setup all `
    --agent-name "PoC Custom" `
    --aiteammate false
```

5. If you sign in as Global Administrator, complete any consent prompt that opens during setup. If you use the Agent ID Developer role, no browser window appears; the CLI completes the steps allowed by your role and prints the consent actions for a Global Administrator (2.8.4).

The blueprint agent flow runs these steps in sequence: requirements validation, blueprint creation, batch permissions (Microsoft Graph, Agent 365 Tools, Messaging Bot API, Power Platform, and any custom resources), agent identity creation, agent registration, and config sync. The CLI sets `managerApplications` on the blueprint, which is required for platform manageability. Setup typically takes 3 to 5 minutes.

Check result

- The setup summary shows each required step as completed, or lists the consent actions for a Global Administrator.

2.8.4 Complete the OAuth2 grants (Agent ID Developer path)

When you run `a365 setup all` without Global Administrator, the CLI completes all steps it can (blueprint creation and inheritable permissions), generates per-resource admin consent URLs and saves them to `a365.generated.config.json`, and displays the next steps for a Global Administrator.

Step	Who	Action
1	**Agent ID Developer**	Run `a365 setup all`. The CLI prints the next steps, including a consent URL for a Global Administrator to open.
2	**Agent ID Developer**	Share the consent URL from the CLI output with the Global Administrator.
3	**Global Administrator**	Open the consent URL in a browser signed in as Global Administrator and grant the requested permissions.

Check result

- The Global Administrator completed every consent URL printed by the CLI.

2.8.5 Verify the blueprint and agent identity

Performed by the **developer** (configuration file), **Directory Readers** (`a365 query-entra`), and any Microsoft Entra user (agent identities and blueprints lists).

1. Inspect the generated configuration:

   ```
   Get-Content .\a365.generated.config.json | ConvertFrom-Json
   ```

   Confirm that `agentBlueprintId` and `agentBlueprintObjectId` are populated, `completed` is `true`, and `resourceConsents` contains resources such as Microsoft Graph, Agent 365 Tools, and Messaging Bot API. If you ran setup as Agent ID Administrator or Agent ID Developer, `resourceConsents` might be empty and `completed` might be `false` until a Global Administrator completes the OAuth2 permission grants.

2. Check the blueprint's permissions and inheritance:

   ```
   a365 query-entra blueprint-scopes --agent-name "PoC Custom"
   a365 query-entra inheritance --agent-name "PoC Custom"
   ```

   `blueprint-scopes` shows the delegated scopes and app role assignments granted on the blueprint service principal (the same as the **API permissions** blade). `inheritance` reports `Effective inheritance: OK` when agent identities created from the blueprint will inherit permissions for a resource. `NONE` means no grants exist on the blueprint service principal (run `a365 setup permissions` as Global Administrator); `BROKEN` means the entry doesn't use `allAllowed` (run `a365 setup permissions` to reconcile).

   `a365 setup` configures every inheritable permission entry with `allAllowed` for both scopes and roles. What agent identities actually inherit is limited to the permissions granted on the blueprint service principal, so review `blueprint-scopes` before you grant more. Inheritable permissions on blueprints are described in 3.6 Configure inheritable permissions on a blueprint.

3. In the Microsoft Entra admin center (https://entra.microsoft.com), search for your `agentBlueprintId` or the agent name. Verify that the App Registration and Enterprise Application appear, that the **API permissions** tab of the blueprint app registration shows all permissions, and that the status shows **Granted for <your tenant>**.
4. Browse to **Entra ID** > **Agents** > **Agent blueprints**, add the **Blueprint App ID** filter with the `agentBlueprintId`, open the blueprint, and select **Linked agent identities**.
5. Browse to **Entra ID** > **Agents** > **Agent identities**, add the **Blueprint App ID** filter with the `agentBlueprintId`, and confirm that the agent identity is listed. Assign owners and sponsors as described in Chapter 3 – Agent identity and ownership.

Blueprints must have `managerApplications` set to be accepted by the platform. The CLI sets this automatically. If you have an existing blueprint created before this requirement was introduced, delete it and run `a365 setup all` again, or patch it via the Graph API.

Save both `a365.config.json` (if used) and `a365.generated.config.json`; you need these values for deployment and troubleshooting.

Check result

- The blueprint and the agent identity exist in Microsoft Entra with the same Blueprint App ID, and all permissions show **Granted for <your tenant>**.

2.8.6 (Optional) Apply custom permissions to the blueprint

Performed by **Global Administrator**.

1. Apply the additional scopes the agent needs inline, for example Microsoft Graph delegated scopes:

```
a365 setup permissions custom `
  --agent-name "PoC Custom" `
  --resource-app-id 00000003-0000-0000-c000-000000000000 `
  --scopes Mail.Read,Mail.Send,Chat.Read,Chat.ReadWrite,Chat.Create,User.Read
```

2. Run `a365 query-entra blueprint-scopes --agent-name "PoC Custom"` and confirm the new scopes.

The command configures OAuth2 delegated permission grants with admin consent, sets inheritable permissions, and reconciles Microsoft Entra with the configuration (it adds new permissions and removes any permissions that you deleted from the config).

Check result

- The new scopes appear in the `blueprint-scopes` output.

2.9 Deploy the agent runtime to Azure (optional)

Documentation: Deploy agent to Azure · Use agent code deployed in Amazon Web Services · Use agent code deployed in Google Cloud Platform

Skip this section if the agent is already deployed to a cloud. If it's deployed to Amazon Web Services or Google Cloud Platform, follow the corresponding article to update the messaging endpoint.

2.9.1 Prepare the deployment

Performed by the **developer**.

Confirm that you have:

- An Azure subscription with contributor access.
- An Azure Web App to deploy to. `a365 setup all` with an `a365.config.json` file creates the resource group, App Service plan, and Web App if they don't already exist (2.8.2). Config-free setup (`--agent-name`) doesn't create Azure hosting resources.
- Working agent code with a valid and reachable messaging endpoint, tested locally (and optionally tested with Microsoft 365 using Dev Tunnels).
- A valid agent blueprint from 2.8.
- Up-to-date configuration files `a365.config.json` (if used), `a365.generated.config.json`, and the config file in the code (for example, `.env`).
- The Azure CLI installed and authenticated.

2.9.2 Deploy the code and store secrets safely

Performed by the **developer** (Contributor on the Azure subscription).

1. Build the project. Example for .NET:

   ```
   dotnet publish -c Release -o ./publish
   ```

2. Deploy to the Azure Web App:

   ```
   az webapp deploy --name <your-web-app> --resource-group <your-resource-group> --src-path ./publish
   ```

   You can also use the Azure portal or the Azure Web Apps Deploy action for GitHub Actions.

3. Store environment variables, including API keys and secrets, as Azure App Settings rather than in code or configuration files. For production environments, use Azure Key Vault for sensitive secrets. Never commit `.env` files with sensitive information to source control.

2.9.3 Verify the deployment

Performed by the **developer**.

Check	How to verify
Deployment command completed without errors	Azure portal > your web app > **Deployment** > **Deployment Center** > logs for your latest deployment
Web app is running	`az webapp show --name <your-web-app> --resource-group <your-resource-group> --query state` returns `Running`
Application logs show successful startup	Azure portal > your web app > **Overview** > **Logs** > **Log Stream**, or `az webapp log tail --name <your-web-app> --resource-group <your-resource-group>`
Environment variables are configured	Azure portal > your web app > **Settings** > **Environment Variables**
Messaging endpoint responds	Test the endpoint shown on the web app **Overview** page

Check result

- All five checks pass.

2.9.4 Register or update the messaging endpoint (Microsoft 365 agents)

Performed by **Agent ID Developer**. Needed only when the messaging endpoint was deferred during `a365 setup all --m365`, or when it changes.

1. Register the messaging endpoint for the existing blueprint:

   ```
   a365 setup blueprint --agent-name "PoC Custom" --endpoint-only `
     --messaging-endpoint https://your-app.azurewebsites.net/api/messages
   ```

2. To delete the existing messaging endpoint and register a new one instead, run `a365 setup blueprint --agent-name "PoC Custom" --update-endpoint <url>`.

2.10 Publish the agent and verify it in the registry

Documentation: Publish agent to Microsoft admin center · Agent 365 CLI publish command reference · Upload Microsoft Copilot custom agents

2.10.1 Package the agent with a365 publish

Performed by the **developer**.

1. Confirm that the agent blueprint exists (2.8), the agent was tested locally, and `a365.config.json` (if used) and `a365.generated.config.json` are up to date. For agents provisioned with `--agent-name`, make sure `a365.generated.config.json` exists in your working directory, because the command reads the blueprint ID from this file.

2. Run the publish command (`a365 publish -h` shows all options):

   ```
   a365 publish
   ```

   The command updates `manifest.json` with your agent blueprint ID, packages `manifest.json` and icons into `manifest.zip`, and displays step-by-step instructions for uploading via the Microsoft 365 admin center.

3. Confirm that the manifest folder and files exist:

   ```
   Test-Path <deploymentProjectPath>/manifest/
   Test-Path <deploymentProjectPath>/manifest/manifest.json
   Test-Path <deploymentProjectPath>/manifest/manifest.zip
   # All should return: True
   ```

Check result

- The CLI shows `Manifest updated successfully` and `manifest.zip created successfully`, and prints the upload instructions.

2.10.2 Upload the package in the Microsoft 365 admin center

Performed by **Global Administrator**.

1. Open the Microsoft 365 admin center at https://admin.cloud.microsoft.
2. Navigate to **Agents** > **All agents**.
3. Select **Upload custom agent**.
4. Select **Choose File** and select the `manifest.zip` file from the `manifest` folder in your project. The ZIP file is validated.
5. Verify the agent's name, icon, and host products. Then select **Next**.
6. Select the assigned users. For testing, you can select a small audience, for example **Just me** or a single test group. Then select **Next**.
7. Review the agent's permissions and capabilities. Then select **Next**.
8. Select **Finish deployment**.

Check result

- The deployment finishes without validation errors.

2.10.3 Verify the custom agent in the registry

Performed by **AI Reader**.

1. After uploading, allow 5 to 10 minutes for the agent to appear in the Microsoft 365 admin center and Teams.
2. Go to **Agents** > **All agents** and confirm that your agent appears in the list.
3. Check **Name** (from `manifest.json`), **Version**, **Publisher** (your organization name), and **Availability**.
4. In **Agents** > **All agents**, find the agent identity that `a365 setup all` registered (**PoC Custom Agent**), open it, and confirm that its identity and registration details match `a365.generated.config.json`.

Check result

- The custom agent is listed with the expected name, version, publisher, and availability, and its identity details match the generated configuration.

2.11 Add observability and data protection (optional)

Documentation: Microsoft OpenTelemetry Distro · Agent 365 observability data model and concepts · Quickstart: Connect an existing agent to Agent 365

2.11.1 Instrument the agent for observability

Performed by the **developer**.

1. Add observability with one of these options:
   - In a coding assistant with the Agent 365 Skills installed, ask *add observability to this agent* (`instrument-observability`). When prompted, choose the authentication mode your agent uses: OBO, Agentic-User (Frontier preview program), or S2S.
   - Or install the Microsoft OpenTelemetry Distro and enable the Agent 365 exporter as described in the article: `pip install microsoft-opentelemetry` (Python), `npm install @microsoft/opentelemetry` (Node.js), or `dotnet add package Microsoft.OpenTelemetry` (.NET).

2. For blueprint agents, use S2S with an app-only token resolver. By default, the distro exports on the delegated route, which needs the delegated `Agent365.Observability.OtelWrite` permission and admin consent; `a365 setup all` doesn't configure that permission for blueprint agents. A registered agent instance can export on the S2S route without an Observability permission or admin consent.
3. Make sure each run has a valid `invoke_agent` span at its root. The Microsoft 365 admin center, Microsoft Defender agent-activity views, and Microsoft Purview depend on it. A run with only `chat`, `execute_tool`, or `output_messages` spans is queryable only in Defender advanced hunting (`CloudAppEvents`).
4. Redeploy the agent (2.9) and send test requests.

Check result

- Test runs appear in Defender advanced hunting (tested in 2.12.6).

2.11.2 Add the Purview DLP guard

Performed by the **developer**.

1. In a coding assistant with the Agent 365 Skills installed, ask *add Purview DLP to this agent* (`purview-dlp-integration`). The skill adds Microsoft Graph `processContent` checks that block prompts before they reach the LLM when an input-blocking policy matches, and guides permission and DLP policy setup. Optional response checks audit responses but don't filter sensitive output. By default, errors and timeouts stop the turn.
2. The skill supports agentic delegated authentication and Node.js S2S with client-secret FMI authentication. For .NET, verify the best-effort guard against your SDK version.
3. Configure the DLP policy and permissions as described in the skill's documentation: `purview-dlp-integration` and processContent API.

Check result

- The guard is added to the agent project and the DLP policy is configured.

2.12 Test and validation

2.12.5 Test: the custom agent appears with its blueprint and agent identity

Performed by the **developer** (configuration file), **AI Reader**, any Microsoft Entra user, and **Directory Readers**.

1. Open `a365.generated.config.json` and confirm that `completed` is `true` and note the `agentBlueprintId`.
2. In the Microsoft 365 admin center, open **Agents** > **All agents** and find **PoC Custom Agent** and the published agent (2.10.3).
3. In the Microsoft Entra admin center, browse to **Entra ID** > **Agents** > **Agent identities**, add the **Blueprint App ID** filter with the `agentBlueprintId`, and confirm that the agent identity is listed.
4. Browse to **Entra ID** > **Agents** > **Agent blueprints**, open the blueprint, and confirm that **Linked agent identities** lists the agent identity.
5. Run `a365 query-entra inheritance --agent-name "PoC Custom"` and confirm `Effective inheritance: OK` for each resource the agent uses. A resource shows `NONE` until permissions for it are granted on the blueprint service principal (2.8.4, 2.8.6).

Expected result

- `completed` is `true`, the custom agent and the published agent are listed in the Microsoft 365 admin center, and the agent identity is linked to the PoC blueprint with the same Blueprint App ID.

2.12.6 Test: custom agent activity is observable

Performed by the **developer** (send requests) and a user with **Security data basics (read)** (review). Requires 2.11.1 and the Microsoft 365 connector in Microsoft Defender (Connect the Microsoft 365 connector). Without the connector, `CloudAppEvents` returns no rows.

1. Send a few requests to the custom agent.
2. In the Microsoft Defender portal (https://security.microsoft.com), open advanced hunting and run:

```
CloudAppEvents
| where Timestamp > ago(1d)
| where ActionType in ("InvokeAgent", "InferenceCall", "ExecuteToolBySDK", "ExecuteToolByGateway",
"ExecuteToolByMCPServer")
| project Timestamp, ActionType, AccountDisplayName, RawEventData
| order by Timestamp desc
```

3. Confirm that `InvokeAgent` rows exist for the test runs. The per-span fields are inside `RawEventData`.

Expected result

- Each test run produces an `InvokeAgent` event.

2.13 Evidence

- `a365 setup all` summary output and a copy of `a365.generated.config.json` with the client secret and consent URLs removed.
- Output of `a365 query-entra blueprint-scopes` and `a365 query-entra inheritance`.
- Screenshots of the blueprint and the agent identity in the Microsoft Entra admin center with the matching Blueprint App ID.
- (If deployed) Output of `az webapp show ... --query state`.
- `a365 publish` output and a screenshot of the custom agent in **Agents** > **All agents** (name, version, publisher, availability).
- (Optional) Advanced hunting results for the custom agent's `InvokeAgent` events.

2.14 Troubleshooting

Symptom	Likely cause	Fix
`a365 setup` reports insufficient permissions	Missing Agent ID Developer (or Global Administrator) role, or missing Azure subscription contributor or owner access	Assign the roles, sign in again, and rerun the requirements check.
Microsoft-managed **Agent 365 CLI** application isn't available	Staged rollout hasn't reached the tenant	Update the CLI and run `a365 setup requirements` again, or use a tenant-owned app named **Agent 365 CLI**.
Device code authentication is blocked (`AADSTS530035`)	Security Defaults or Conditional Access block device code on WSL, macOS, or Linux	Don't disable the security policy. Run the CLI on native Windows to use WAM, or work with your identity administrator to use a tenant-owned custom application.

Symptom	Likely cause	Fix
Authentication fails with `AADSTS70007`	Older CLI version	Run `dotnet tool update --global Microsoft.Agents.A365.DevTools.Cli` and retry.
Setup completes but lists outstanding consent actions	OAuth2 grants require a Global Administrator	Have a Global Administrator open every consent URL printed by the CLI and grant the requested permissions.
`a365 query-entra inheritance` reports `Effective inheritance: NONE`	No grants on the blueprint service principal; most commonly a missing `wids` optional claim on the client app	Run `a365 setup requirements` to detect and repair the claim, then run `a365 setup permissions` as Global Administrator.
The blueprint or identity exists, but the agent isn't in **All agents**	Agent registration failure	Review the setup summary, resolve the error, then run `a365 setup all --agent-name "PoC Custom" --agent-registration-only`.
An existing blueprint isn't accepted by the platform	Blueprint created before `managerApplications` was required	Delete it and run `a365 setup all` again, or patch it via the Graph API.
`a365 publish` reports `Agent blueprint ID not found`	Blueprint setup isn't complete	Run `a365 setup` to complete blueprint setup.
`a365 publish` reports `Permissions missing`	Blueprint permissions not configured	Rerun setup with the `a365 setup permissions` command.
Upload fails in the admin center	Insufficient permissions, or an invalid or outdated `manifest.zip`	Use the Global Administrator role, run `a365 publish` again to generate a fresh `manifest.zip`, and upload again.
Web app is running but `/api/messages` returns 404	Route or endpoint handler not registered, or wrong entry point	Verify the route configuration and endpoint handler in the agent code and the entry point in the deployment.
App crashes on startup	Missing dependencies, missing environment variables, runtime version mismatch, or code errors	Check `az webapp log tail`, list and set app settings with `az webapp config appsettings list` and `az webapp config appsettings set`, then redeploy.
No custom-agent telemetry appears	No valid `invoke_agent` span at the root of the run, or an authentication mode that doesn't match the environment	Confirm that the run emits a root `invoke_agent` span and that the selected authentication mode matches your environment. Ask your coding assistant to *validate this Agent 365 integration*.
`CloudAppEvents` returns no rows at all	The Microsoft 365 connector isn't connected in Microsoft Defender	Connect it as described in Connect the Microsoft 365 connector, then run the agent again.

✅ What you proved

- [ ] For the custom test agent, `a365.generated.config.json` shows `completed: true`, and the agent identity blueprint and agent identity exist in Microsoft Entra with the same blueprint app ID.
- [ ] The custom test agent appears in **Agents** > **All agents** after registration, and the published package appears with the expected name, version, and publisher.
- [ ] (Optional) Test runs of the instrumented custom agent appear in Microsoft Defender advanced hunting (`CloudAppEvents`).

References

- Install and use the Agent 365 CLI
- Agent 365 Skills for guided agent setup
- Quickstart: Connect an existing agent to Agent 365
- Custom client app registration for Agent 365 CLI
- Onboard a single agent with the Agent 365 CLI
- Set up agent blueprint
- Agent 365 CLI setup command reference
- Agent 365 CLI query-entra command reference
- View and filter agent identities
- View and manage agent identity blueprints
- Deploy agent to Azure
- Use agent code deployed in Amazon Web Services
- Use agent code deployed in Google Cloud Platform
- Publish agent to Microsoft admin center
- Agent 365 CLI publish command reference
- Upload Microsoft Copilot custom agents
- Microsoft OpenTelemetry Distro
- Agent 365 observability data model and concepts
- processContent API
- advanced hunting

Additional resources

- `purview-dlp-integration`

Runbook index · Connected platforms · Next: Chapter 3 – Agent identity and ownership

Custom agent samples (Agent 365 SDK path)

> **Before you start:** Verify the chapter prerequisites and the sample-specific requirements below.

Minimal custom engine agents used in Chapter 2 – Agent 365 SDK, sections 2.7 to 2.11. Each file is the agent code only, built on the Microsoft 365 Agents SDK hosting layer. Registering the agent with Agent 365 (agent identity blueprint, agent identity, permissions, registration) is a separate step that you run with the Agent 365 CLI or the Agent 365 Skills.

Table of contents

- Files
- Path from code to the Agent 365 registry
- Use the Agent 365 Skills
- References

Files

File	Language	Package to install	Based on
`nodejs/agent.ts`	Node.js / TypeScript	`@microsoft/agents-hosting`, `@microsoft/agents-hosting-express`	Quickstart (Node.js)
`python/agent.py`	Python	`microsoft-agents-hosting-aiohttp`, `microsoft-agents-authentication-msal`, `python-dotenv` (imports `microsoft_agents.*`)	Quickstart (Python) and the Python quickstart sample
`dotnet/Agent.cs`	.NET (`Program.cs` of a `dotnet new web` project)	`Microsoft.Agents.Hosting.AspNetCore`	Quickstart (.NET)

The samples use the messaging endpoint `/api/messages`, so you can test them locally with Microsoft 365 Agents Playground before you register or deploy them, as in the quickstarts. The Node.js sample runs locally in anonymous mode, as in the Node.js quickstart. For the .NET sample, set the application URL in `launchSettings.json` as described in the .NET quickstart. The Python sample reads its service connection settings (`CONNECTIONS__SERVICE_CONNECTION__SETTINGS__CLIENTID`, `__CLIENTSECRET`, `__TENANTID`) from environment variables or a `.env` file, as in the Python quickstart sample. Never commit `.env` files with sensitive information to source control. These samples aren't production starter kits.

Path from code to the Agent 365 registry

Step	Agent 365 Skill (coding assistant)	Equivalent manual step	Result
1. Validate prerequisites and detect the stack	`a365-setup`	Install the Agent 365 CLI, run `a365 setup requirements`	CLI installed, Azure sign-in and roles checked. `a365-setup` routes you to step 2a or 2b.
2a. Register a standard agent	`make-a365-agent`	`a365 setup all` (or `a365 setup all --agent-name <name>`)	Agent identity blueprint and agent identity in Microsoft Entra, permissions configured, agent registered. Generated IDs saved in `a365.generated.config.json`.
2b. Make it an AI teammate (Frontier preview program only)	`make-ai-teammate`	`a365 setup all --aiteammate` (requires a manually created `a365.config.json`)	The CLI provisions the blueprint and permissions; the skill also adds the hosting layer, message routing, and notifications. The agent gets its own user account.
3. Deploy the runtime (optional)	–	`az webapp deploy` (see Deploy agent to Azure)	Agent code running behind a reachable messaging endpoint.
4. Publish	–	`a365 publish`, then upload `manifest.zip` in the Microsoft 365 admin center	Agent listed in **Agents** > **All agents**.
5. Add observability (optional)	`instrument-observability`	Microsoft OpenTelemetry Distro (`@microsoft/opentelemetry`, `microsoft-opentelemetry`, `Microsoft.OpenTelemetry`)	Spans with a root `invoke_agent` span reach Microsoft Defender, Microsoft Purview, and the Microsoft 365 admin center.
6. Add Work IQ tools (optional)	`add-workiq-tools`	`a365 develop add-mcp-servers`, then `a365 setup permissions mcp` by a Global Administrator	Governed Microsoft 365 MCP tools. Requires a delegated (OBO) permission model.
7. Add Purview DLP (optional)	`purview-dlp-integration`	–	Guard that calls Microsoft Graph `processContent` before the model call. See 2.11.2 Add the Purview DLP guard.
8. Validate the integration (optional)	`a365-code-validator`	`a365 query-entra blueprint-scopes`, `a365 query-entra inheritance`	Report of gaps in instrumentation, Entra artifacts, and permissions.

Use the Agent 365 Skills

For the guided workflow, see 2.7.4 Use the Agent 365 Skills. Use a coding assistant with terminal access; in VS Code, use agent mode.

Install the skills with one of the methods documented in the repository README and on Microsoft Learn:

- **GitHub CLI `gh skill` (GitHub Copilot CLI, VS Code agent mode, Claude Code).** The `gh skill` command is in preview in the GitHub CLI; `gh skill add` is an alias of `gh skill install`. The default installation scope is `project` (inside the current Git repository), and at project scope GitHub Copilot uses the `.agents/skills` directory. Use `--scope user` to install in your home directory, and `--agent` to select the coding assistant. Restart your assistant afterward so it picks up the new skills.

```
gh skill install microsoft/agent365-skills --all
# For Claude Code instead of GitHub Copilot:
gh skill install microsoft/agent365-skills --all --agent claude-code
```

- **Claude Code plugin marketplace.** Inside an active Claude Code session:

```
/plugin marketplace add https://github.com/microsoft/agent365-skills
/plugin install agent365@agent365-skills
```

- **Project-scoped copy (`.agents/skills/` open standard).** Clone the repository and run its installer from your agent project folder. VS Code agent mode, the Copilot cloud agent, and Copilot CLI load skills from `.agents/skills/`.

```
git clone https://github.com/microsoft/agent365-skills.git
cd <your-agent-project>
node <path-to>\agent365-skills\scripts\install.js
```

In GitHub Copilot CLI or VS Code, list the installed skills with `/skills list`. Skills are additive and idempotent: they don't delete or restructure existing code, and rerunning a skill doesn't duplicate completed work. Review the code, manifest, and configuration changes each skill makes.

References

- Quickstart (Node.js)
- Quickstart (Python)
- Quickstart (.NET)
- Deploy agent to Azure
- Agent 365 Skills for guided agent setup
- Quickstart: Connect an existing agent to Agent 365
- Install and use the Agent 365 CLI
- Agent 365 CLI reference
- Set up the agent blueprint
- Publish agent to the Microsoft 365 admin center
- Microsoft OpenTelemetry Distro
- Custom client app registration for Agent 365 CLI
- Microsoft 365 Agents SDK quickstart

Additional resources

- `microsoft/agent365-skills` README
- GitHub CLI `gh skill install`
- Python quickstart sample

Return to the chapter for registration, validation, and evidence.

Prerequisites - Chapter 3: Agent identity and ownership

> **Goal:** Prepare the accounts, access, and test assets needed for this chapter and its selected optional paths.

Apply the shared PoC preparation guidance and agree the chapter outcomes. Finish dependent build steps before their tests.

Table of contents

- Tenant readiness
- Prepare accounts and role assignments
- Test assets and dependencies
- References

Tenant readiness

Documentation: Get started with Microsoft Agent 365 · Agent management in the Microsoft 365 admin center · Frontier program

Use the task-specific roles in 3.1 Required permissions. Confirm availability rather than assuming the tenant is ready.

Follow the shared tenant readiness check using the chapter permissions and feature prerequisites linked above.

Prepare accounts and role assignments

Alongside the shared setup and reviewer accounts, prepare the following scenario accounts:

Account	Purpose
Standard test user	Runs the identity tests and acts as a sponsor where required. No admin role.
Developer account	Prepares the blueprint and agent identity if a custom or SDK-built agent is in scope.
PoC Sponsor and PoC Sponsor Manager	Sponsor continuity test with Lifecycle Workflows. The **Manager** attribute of PoC Sponsor is set to PoC Sponsor Manager.

Assign the roles in 3.1 Required permissions using the shared model. Verify each account can access the pages needed for its tasks.

Test assets and dependencies

Documentation: Fundamental concepts in Microsoft Entra Agent ID · Owners, sponsors, and managers · Agent's user account · View and filter agent identities in your tenant · Disable agent identities in your tenant · Add and manage owners and sponsors for agent identities and blueprints · Manage agents in end user experience · Agent identity sponsor tasks in Lifecycle Workflows · Lifecycle Workflows templates · Lifecycle Workflows tasks · Run a workflow on-demand · Check status of a workflow

- Verify the required inventory using Chapter 1 – Agent discovery and inventory so you know which agents are in scope.
- If you use custom or SDK-built agents, complete 2.8 Create the agent blueprint with a365 setup so their blueprint and agent identities exist in your tenant.
- Confirm that the **PoC Sponsor** and **PoC Sponsor Manager** accounts from Prepare accounts and role assignments exist, and that the **Manager** attribute of PoC Sponsor is set to PoC Sponsor Manager. The sponsorship transfer task in 3.10 requires a populated manager attribute, and the email tasks notify the manager and co-sponsors.
- Pick one non-production test agent that you can disable, mark as risky, and re-enable during the tests.
- Access packages for agents are covered in 6.6 Grant time-bound access with access packages. This chapter only links to it.

Build guidance: 1.2 Discover Microsoft-native agents

Check result

- Each asset needed for the selected paths is prepared or has a named build task that will finish before its dependent tests.

References

- Get started with Microsoft Agent 365
- Agent management in the Microsoft 365 admin center
- Frontier program
- Fundamental concepts in Microsoft Entra Agent ID
- Owners, sponsors, and managers
- Agent's user account
- View and filter agent identities in your tenant
- Disable agent identities in your tenant
- Add and manage owners and sponsors for agent identities and blueprints
- Manage agents in end user experience
- Agent identity sponsor tasks in Lifecycle Workflows
- Lifecycle Workflows templates
- Lifecycle Workflows tasks
- Run a workflow on-demand
- Check status of a workflow

> **Ready?** Continue to the chapter. →

Chapter 3 - Agent identity and ownership

Pillar: Govern **What it proves:** Agents become governable directory identities. Every in-scope agent has a Microsoft Entra Agent ID linked to a blueprint, named owners and sponsors who are accountable for it, observable sign-in and audit activity, a risk-response path, and a sponsorship that survives when people move or leave.

Success criteria: Agree the selected outcomes in What you proved before testing.

Topics and time guide

Approximate hands-on estimates for the selected scope, with prerequisites prepared. Service waiting time is separate.

Topic	Estimated hands-on time
3.1 Required permissions	~5 min
3.2 Understand the Agent ID object model	~5-10 min
3.3 Bring Copilot Studio agents under Microsoft Entra Agent ID	~10-15 min
3.4 View and filter agent identities and blueprints	~5-10 min
3.5 Assign owners and sponsors	~10-15 min
3.6 Configure inheritable permissions on a blueprint (optional)	~10-15 min
3.7 Tag agents with custom security attributes (optional)	~10-15 min
3.8 Monitor agent sign-ins and audit events	~10-15 min
3.9 Detect and respond with ID Protection	~10-15 min
3.10 Keep sponsorship continuous with Lifecycle Workflows	~20-30 min
3.11 Disable or restrict agents at the right scope	~5-10 min
3.12 Test and validation	~15-20 min
3.13 Evidence	~5-10 min
3.14 Troubleshooting	As needed

Outcome checklist · References

3.1 Required permissions

Use the shared account and role-assignment model with the task-specific permissions below.

Task	Least-privilege role	Section
View agent identities and blueprints in the Microsoft Entra admin center	Any Microsoft Entra user (no admin role)	3.3.1, 3.4.1, 3.4.2, 3.12.1
List agent identities with Microsoft Graph	Agent ID Administrator (owners can read their own agent identities)	3.4.3, 3.7.3

Task	Least-privilege role	Section
Migrate legacy Copilot Studio agents to Agent ID (preview)	Power Platform Administrator	3.3.2
Add or remove owners and sponsors	Agent ID Administrator (owners of the object can do this without a role)	3.5
Disable or re-enable agent identities and blueprints	Agent ID Administrator (owners can manage their own agents without a role; owners and sponsors can disable their agents in My Account)	3.11, 3.12.2
Configure inheritable permissions on a blueprint	Agent ID Developer (blueprints you own) or Agent ID Administrator	3.6
Create attribute sets and attribute definitions	Attribute Definition Administrator	3.7.1
Assign or remove custom security attribute values on agent identities	Attribute Assignment Administrator	3.7.2
Read custom security attribute assignments	Attribute Assignment Reader	3.7.3
Read sign-in and audit logs	Reports Reader	3.8
View the Risky Agents report	Security Reader	3.9.1, 3.12.3
Confirm compromised, confirm safe, dismiss risk (Microsoft Graph)	Security Administrator	3.9.2, 3.12.3
Create and run Lifecycle Workflows	Lifecycle Workflows Administrator	3.10, 3.12.4
Validation / read-only review	Any Microsoft Entra user; Reports Reader; Security Reader; Attribute Assignment Reader	3.12

Notes:

- By default, Global Administrator and other administrator roles can't read, define, or assign custom security attributes. Assign the attribute roles explicitly.
- Users with Agent ID admin roles aren't made sponsor automatically when they create agents.

Before you start:

- [ ] Confirm the chapter prerequisites for the selected topics.

3.2 Understand the Agent ID object model

Documentation: Fundamental concepts in Microsoft Entra Agent ID · Owners, sponsors, and managers · Agent's user account · View and filter agent identities in your tenant · Disable agent identities in your tenant

3.2.1 Identity objects

Object	What it is	PoC relevance
Agent identity blueprint	Template and authentication foundation for one or more agent identities. Holds the credentials and acquires tokens on behalf of all agent identities created from it. Policies applied to a blueprint, such as Conditional Access, take effect for all its agent identities.	For example the **Microsoft Copilot Studio agent identity blueprint**, or a blueprint you created in 2.8.
Agent identity blueprint principal	The object that records a blueprint's presence in a tenant, enables it to acquire tokens, and appears in audit logs. Listed under **Entra ID** > **Agents** > **Agent blueprints**.	Where you view linked agent identities, granted permissions, owners and sponsors, logs, and the **Disable** action for the blueprint.
Agent identity	The primary identity an agent uses to authenticate and access resources. It has no credentials of its own; it authenticates using tokens issued by its blueprint.	This is what you govern in this chapter.
Agent's user account (agent user)	Optional account paired 1:1 with an agent identity, used only when the agent must access systems that require a user object (mailbox, calendar, Teams, documents). It doesn't replace the agent identity; both must exist.	Shown in the sign-in logs **Agent type** filter as **Agent ID user**.

The **Agent identities** list also includes agents that use a service principal instead of an agent identity object. For those, **Uses agent identity** = No and **Blueprint App ID** is blank. Agents created in earlier versions of Copilot Studio and Azure AI Foundry might have been created this way; they're subject to the same policies, governance, and processes as all other applications and service principals in your tenant.

3.2.2 Owners, sponsors, and managers

	Owners	Sponsors	Managers
Purpose	Technical administration: setup, configuration, credential management	Business accountability: purpose, lifecycle decisions, access reviews, retention	User responsible for the agent in the organizational hierarchy
Required	Optional	At least one required on create for agent identities and agent identity blueprints (blueprint principals are exempt at creation)	Optional
Allowed types	Users (including guests) and service principals. Groups aren't supported.	Users (including guests); dynamic membership groups (security or Microsoft 365) and assigned membership Microsoft 365 groups. Role-assignable groups and assigned membership security groups aren't allowed.	Individual users
Limits	—	Up to 100 sponsors, no more than 5 groups	—
Can disable and delete agent identities	Yes	Yes (disable, soft-delete)	No

	Owners	Sponsors	Managers
Can re-enable, restore, or hard-delete	Yes	No — contact an owner or admin	No
Can edit settings and credentials, add owners and sponsors	Yes	Can modify the identity's sponsors only	No
Can request access packages	Yes (for owned agents)	Yes (for sponsored agents)	Yes (for their agents' user accounts)

When a group is a sponsor, all members of the group have sponsor rights. When you use a dynamic membership group as a sponsor, it can take up to 24 hours after a membership rule change or a user property change before the authorization check on sponsorship succeeds.

3.3 Bring Copilot Studio agents under Microsoft Entra Agent ID

Documentation: Manage Entra Agent IDs (Copilot Studio) · Migrate Copilot Studio agents to Microsoft Entra Agent ID (preview)

Copilot Studio automatically creates a Microsoft Entra Agent ID for each new agent, and you can no longer opt out of automatic agent identity creation. When the first agent identity is created, Copilot Studio adds the **Microsoft Copilot Studio agent identity blueprint** (Blueprint ID `25664c89-cea5-4ab6-b924-a54fd8a19ae0`) and a corresponding blueprint principal to your tenant. All Copilot Studio agent identities are children of this blueprint. Agents created before the Entra Agent ID rollout continue to use app registrations until they're migrated.

3.3.1 Confirm a Copilot Studio agent has an agent identity

Performed in Copilot Studio, then by **any Microsoft Entra user** in the Microsoft Entra admin center.

1. Open https://copilotstudio.microsoft.com and go to the **Settings** page for the PoC agent.
2. Select **Advanced** and expand the **Metadata** section.
3. Copy the GUID shown under **Entra Agent ID**.
4. Open https://entra.microsoft.com > **Entra ID** > **Agents** > **Agent identities** and use the GUID to find the agent identity.

Check result

- The agent identity is listed with **Uses agent identity** = Yes, and its parent blueprint is the Microsoft Copilot Studio agent identity blueprint.
- Agents created before the Entra Agent ID rollout continue to use app registrations; see 3.3.2.

3.3.2 Migrate legacy Copilot Studio agents (optional, preview)

Performed by **Power Platform Administrator**. Power Platform inventory must be enabled for the tenant.

Existing agents that use an app-registration identity are being migrated automatically by Microsoft. Use this task only to migrate a PoC agent ahead of that. Migration converts the identity in place, the agent keeps its application (client) ID, and an agent can be reverted if it doesn't pass validation.

1. Open https://admin.powerplatform.microsoft.com.
2. In the left navigation pane, select **Actions** > **Recommendations**, then the **Active** tab.
3. Search for and select **Migrate Copilot Studio agents to Microsoft Entra Agent ID for enhanced agent governance**.

4. Expand **Why is this important?** and review the guidance.
5. Use **Suggested migration order** and **Migration notes** to select only the non-critical PoC agent(s).
6. Select **Migrate**, review the confirmation, and confirm.
7. Review the **Action**, **Action state**, and **Action date** columns (or the **Action history** tab).
8. With the agent's maker, confirm the agent responds in every channel where it's published and that its actions, connectors, and flows run, then repeat 3.3.1.

Check result

- The agent has gained a Microsoft Entra Agent ID that appears under **Entra ID** > **Agents** > **Agent identities**.
- If the agent doesn't pass validation, revert it with the Power Platform API rollback endpoint described in the migration article before you continue.

3.4 View and filter agent identities and blueprints

Documentation: View and filter agent identities in your tenant · Manage agent identity blueprints in the Microsoft Entra admin center · List agentIdentity objects (Microsoft Graph)

3.4.1 Review the agent identities list

Performed by **any Microsoft Entra user**.

1. Open https://entra.microsoft.com > **Entra ID** > **Agents** > **Agent identities**.
2. Search by **name** or **object ID**, or add the **Blueprint App ID** filter.
3. Select **Choose columns** and add: **Name**, **Created On**, **Status**, **Object ID**, **View Access**, **Blueprint App ID**, **Owners and Sponsors**, **Uses agent identity**.
4. Select a PoC agent identity and review its overview (name, description, status, link to the parent agent identity blueprint), its owners and sponsors, its access (granted permissions and Microsoft Entra roles), and its audit and sign-in logs.

Column	Meaning
Status	Current operational state: **Active** or **Disabled**
Blueprint App ID	Identifier of the agent identity blueprint of this agent identity; blank for agents using service principals
Owners and Sponsors	Direct link to the owners and sponsors of the agent identity
Uses agent identity	**Yes** = agent identity object; **No** = service principal
View Access	Opens the agent's access pane on the **Permissions** tab

Check result

- Every in-scope PoC agent is listed with **Status** = Active and **Uses agent identity** = Yes.

3.4.2 Review agent blueprints and linked agent identities

Performed by **any Microsoft Entra user**.

1. Go to **Entra ID** > **Agents** > **Agent blueprints**.
2. Search by name or object ID, or select **Add filters** > **Blueprint App ID**.

3. Note the **Agent identities** count for each PoC blueprint, then select the blueprint principal to open its management page.
4. Select **Linked agent identities** to list each child agent with **Name**, **Status**, **View Access**, and **Owners and Sponsors**.
5. Under **Access**, select **Granted permissions** and review the **Admin consent** and **User consent** tabs (**API name**, **Claim value**, **Permission**, **Type**, **Granted through**, **Granted by**).
6. Under **Access**, select **Owners and sponsors** and review the **Agent blueprint** and **Agent blueprint principal** tabs.

Check result

- Each PoC blueprint lists the expected child agent identities and has at least one sponsor.

3.4.3 List agent identities with Microsoft Graph (optional)

Performed by **Agent ID Administrator** in Graph Explorer, with delegated permission `AgentIdentity.Read.All`.

```
GET https://graph.microsoft.com/v1.0/servicePrincipals/microsoft.graph.agentIdentity
```

Check result

- The response contains every PoC agent identity. For each, `agentIdentityBlueprintId` is the **appId** of its blueprint and `accountEnabled` is `true`.

3.5 Assign owners and sponsors

Documentation: Add and manage owners and sponsors for agent identities and blueprints · Manage agents in end user experience · Owners, sponsors, and managers

PoC pattern: the technical builder is the **owner**; the business accountable person (or a supported group of accountable people) is the **sponsor**. For the sponsor continuity test, make **PoC Sponsor** a sponsor of the test agent.

3.5.1 Assign owners and sponsors to an agent identity

Performed by **Agent ID Administrator** or an existing **owner** of the agent identity.

1. Open https://entra.microsoft.com > **Entra ID** > **Agents** > **Agent identities**.
2. Select the agent identity.
3. Under **Access**, select **Owners and sponsors**.
4. Select **Add** > **Add owner** or **Add sponsor**.
5. Search for and select the users, and groups (for sponsors only), that you want to add.
6. Select **Add**.

To remove an entry, select the checkbox next to it and select **Remove**.

Check result

- The agent identity shows the expected owner and at least one sponsor.

3.5.2 Assign owners and sponsors to a blueprint

Performed by **Agent ID Administrator** or an existing **owner** of the agent identity blueprint.

1. Go to **Entra ID** > **Agents** > **Agent blueprints** and select the blueprint.
2. Under **Access**, select **Owners and sponsors**.
3. Select the **Agent blueprint** tab or the **Agent blueprint principal** tab.
4. Select **Add** > **Add owner** or **Add sponsor**, search for and select the users and groups (for sponsors only), and select **Add**.

Check result

- The selected tab lists the intended owners and sponsors.

3.5.3 Review owned and sponsored agents in My Account (self-service)

Performed by **PoC Sponsor** (any owner or sponsor of at least one agent identity).

1. Sign in to https://myaccount.microsoft.com.
2. If you haven't opted in to the new homepage yet, select **Use new version** in the banner.
3. In the left menu, select **Manage agents**. This menu item appears only for users who own or sponsor at least one agent identity.
4. Choose the **Agents you sponsor** or **Agents you own** tab and select an agent to view its details.

Check result

- PoC Sponsor sees the test agent under **Agents you sponsor**. The disable test is in 3.12.2.

3.6 Configure inheritable permissions on a blueprint (optional)

Documentation: Inheritable permissions and required resource access · Configure inheritable permissions for agent identity blueprints · Create inheritablePermission (Microsoft Graph)

Inheritable permissions let agent identities created from a blueprint automatically inherit permissions from their parent blueprint, without interactive consent prompts. A permission is inherited only when **both** conditions are met:

- The resource scopes, roles, or both are listed in the inheritable permissions configuration on the agent identity blueprint.
- The permission is granted to the blueprint (static consent through required resource access, or dynamic consent with the permissions explicitly requested).

Inheritance patterns for delegated scopes (`inheritableScopes`):

Pattern	`@odata.type`	Description
Enumerated scopes	`microsoft.graph.enumeratedScopes`	Inherit only the listed scopes. Provides fine-grained control and supports gradual permission expansion.
All allowed scopes	`microsoft.graph.allAllowedScopes`	Inherit all available delegated scopes for the resource app. Newly granted scopes on the blueprint principal are automatically included.
None	`microsoft.graph.noScopes`	Inherit no scopes for the resource app.

Rules and limits:

- Maximum of 50 resource apps per agent identity blueprint.
- Enumerated `scopes` must be a non-empty list of unique scope identifiers and must not include globally blocked scopes.
- Inherited permissions aren't visible as permissions on agent identities in the Microsoft Entra admin center or through Microsoft Graph. They're only observable in the token: inherited scopes in the `scp` claim, inherited roles in the `roles` claim.
- Regularly review inherited scopes and roles, and remove unused permissions from both the blueprint principal and the inheritable permissions list.

Inheritable permissions are configured with Microsoft Graph through the `inheritablePermissions` navigation property on the `agentIdentityBlueprint` application resource. In the requests below, `{id}` is the `id` of the agent identity blueprint application.

3.6.1 Review current inheritable permissions

Performed by **Agent ID Developer** (blueprint owner) or **Agent ID Administrator**, with delegated permission `AgentIdentityBlueprint.Read.All`.

```
GET
https://graph.microsoft.com/v1.0/applications/{id}/microsoft.graph.agentIdentityBlueprint/inheritablePermissions
```

Check result

- You have a baseline list of `resourceAppId` entries and patterns to record as evidence before changing anything.

3.6.2 Add an enumerated-scopes entry for Microsoft Graph

Performed by **Agent ID Developer** (blueprint owner) or **Agent ID Administrator**, with delegated permission `AgentIdentityBlueprint.Create`.

```
POST
https://graph.microsoft.com/v1.0/applications/{id}/microsoft.graph.agentIdentityBlueprint/inheritablePermissions
Content-Type: application/json

{
  "resourceAppId": "00000003-0000-0000-c000-000000000000",
  "inheritableScopes": {
    "@odata.type": "microsoft.graph.enumeratedScopes",
    "scopes": [
      "User.Read",
      "Mail.Read"
    ]
  }
}
```

`00000003-0000-0000-c000-000000000000` is Microsoft Graph. Replace the scopes with the minimum your PoC agent needs. The inheritable permissions configuration doesn't grant access by itself; the permission must also be granted to the blueprint principal.

Check result

- The request returns `201 Created` with the new **inheritablePermission** object.

3.6.3 Update or remove an entry

Performed by **Agent ID Developer** (blueprint owner) or **Agent ID Administrator**, with delegated permission `AgentIdentityBlueprint.ReadWrite.All`.

- If an entry already exists for a `resourceAppId`, update it with `PATCH` on `.../inheritablePermissions/{resourceAppId}` instead of creating a duplicate, which results in `409 Conflict`.
- To remove inheritance for a resource app:

```
DELETE
https://graph.microsoft.com/v1.0/applications/{id}/microsoft.graph.agentIdentityBlueprint/inheritablePermissions
/00000003-0000-0000-c000-000000000000
```

Check result

- `DELETE` returns `204 No Content`, and the entry no longer appears in 3.6.1.

3.7 Tag agents with custom security attributes (optional)

Documentation: Custom security attributes overview · Add or deactivate custom security attribute definitions · Update agentIdentity (Microsoft Graph) · List agentIdentity objects (Microsoft Graph) · Manage custom security attributes for an application

Custom security attributes let you categorize agent identities with business-specific labels, and Conditional Access policies can target those attributes (see 6.2.3 (Optional) Exclude approved agents with custom security attributes).

Plan before you create: attribute set names and attribute names can't be renamed, attribute sets can't be deleted, and attribute definitions can only be deactivated, not deleted.

3.7.1 Create the AgentGovernance attribute set and attributes

Performed by **Attribute Definition Administrator**.

1. Open https://entra.microsoft.com > **Entra ID** > **Custom security attributes**.
2. Select **Add attribute set**, enter the name `AgentGovernance`, a description, and the maximum number of attributes, then select **Add**.
3. Open `AgentGovernance` and select **Add attribute** for each attribute below. Set **Data type** = String, **Allow multiple values to be assigned** = No, **Only allow predefined values to be assigned** = Yes, select **Add value** for each predefined value, and select **Save**.

Attribute	Predefined values	Purpose
`Project`	`Agent365PoC`	One filter returns every PoC agent.
`Environment`	`Pilot`, `Prod`	Distinguishes PoC agents from production agents.

Check result

- `AgentGovernance` lists `Project` and `Environment` with their predefined values.

3.7.2 Assign attribute values to PoC agent identities

Performed by **Attribute Assignment Administrator**, with delegated permissions `CustomSecAttributeAssignment.ReadWrite.All` and `AgentIdentity.ReadWrite.All`.

Repeat for each PoC agent identity (`{id}` = agent identity object ID):

```
PATCH https://graph.microsoft.com/v1.0/servicePrincipals/{id}/microsoft.graph.agentIdentity
Content-Type: application/json

{
  "customSecurityAttributes": {
    "AgentGovernance": {
      "@odata.type": "#Microsoft.DirectoryServices.CustomSecurityAttributeValue",
      "Project": "Agent365PoC",
      "Environment": "Pilot"
    }
  }
}
```

Check result

- Each request returns `204 No Content`.

3.7.3 Filter agents by attribute

Performed by **Agent ID Administrator** together with **Attribute Assignment Reader**, with delegated permissions `AgentIdentity.Read.All` and `CustomSecAttributeAssignment.Read.All`.

```
GET https://graph.microsoft.com/v1.0/servicePrincipals/microsoft.graph.agentIdentity?
$count=true&$select=id,displayName,customSecurityAttributes&$filter=customSecurityAttributes/AgentGovernance/Pro
ject eq 'Agent365PoC'
ConsistencyLevel: eventual
```

The filter value is case sensitive. Include `ConsistencyLevel: eventual` and `$count=true`.

Check result

- The result contains every PoC agent identity.

3.8 Monitor agent sign-ins and audit events

Documentation: Microsoft Entra Agent ID logs · List directoryAudits (Microsoft Graph)

3.8.1 Review agent sign-in logs

Performed by **Reports Reader**.

1. Have the standard test user exercise the PoC agent with a few prompts.
2. Open https://entra.microsoft.com > **Entra ID** > **Monitoring & health** > **Sign-in logs**.
3. Filter by **Agent type** (**Agent ID user**, **Agent Identity**, **Agent Identity Blueprint**, or **Not Agentic**) and **Is Agent** (**No** or **Yes**).
4. Because agents can sign in with either user-delegated or app-only permissions, check each of the sign-in log types.

Check result

- With **Is Agent** = Yes, sign-ins for the PoC agents are listed.

3.8.2 Review audit logs for agent changes

Performed by **Reports Reader**.

Agent activity is logged under the base identity type from which it originates:

Agent action	Audit activity	`agentType` value
Create an agent identity blueprint	Add application	`agenticApp`
Update an agent identity blueprint	Update application	`agenticApp`
Delete an agent identity blueprint	Delete application	`agenticApp`
Create an agent identity	Add service principal	`agenticAppInstance`
Update an agent identity	Update service principal	`agenticAppInstance`
Delete an agent identity	Delete service principal	`agenticAppInstance`
Create an agent's user account	Add user	`agentIDuser`

The blueprint principal uses `agentIdentityBlueprintPrincipal`, and `notAgentic` means the identity isn't an agent. The `agentType` property appears on the `initiatedBy`, `performedBy`, and `targetResources` fields; `blueprintId` correlates an agent identity back to its blueprint.

1. Open a PoC agent identity (3.4.1) and review its audit logs, or go to **Entra ID** > **Monitoring & health** > **Audit logs**.
2. Review the events for the PoC agent identities.

Check result

- Audit events for the PoC agent identities are listed.

3.8.3 Query agent logs with Microsoft Graph (optional)

Performed by **Reports Reader**, with delegated permission `AuditLog.Read.All`, on the beta endpoint.

Agent identity sign-ins:

```
GET https://graph.microsoft.com/beta/auditLogs/signIns?$filter=signInEventTypes/any(t: t eq 'servicePrincipal')
and agent/agentType eq 'agenticAppInstance'
```

Audit events (inspect `agentType` and `blueprintId` on `initiatedBy` and `targetResources`):

```
GET https://graph.microsoft.com/beta/auditLogs/directoryAudits
Prefer: include-unknown-enum-members
```

The `Prefer: include-unknown-enum-members` header is required to receive the `agentIdentityBlueprintPrincipal` and `agentIDuser` values.

Check result

- The sign-in query returns service principal sign-ins made by agent identities, and the audit query returns events with an `agentType` other than `notAgentic`.

3.9 Detect and respond with ID Protection

Documentation: ID Protection for agents · riskyAgent resource type (Microsoft Graph beta) · Manage agent identities in your organization

ID Protection detects identity-based risks on agents that have agent identities. All risk detections for risky agents are offline. In on-behalf-of flows, risky activity is attributed to the **user** rather than the agent. **Learning Mode** suppresses behavioral alerts for agents that lack sufficient activity history; a separate detection runs in parallel to catch malicious early-life behavior.

Agent risk detection	`riskEventType`
Confirmed compromised	`adminConfirmedAgentCompromised`
Early life malicious activity	`earlyLifeMaliciousActivity`
Entra Directory Reconnaissance	`entraDirectoryReconnaissance`
Failed access attempt	`failedAccessAttempt`
Microsoft Entra threat intelligence	`threatIntelligenceAccount`
Sign-in spike	`signInSpike`
Suspicious credential usage	`suspiciousCredentialUsage`
Unfamiliar resource access	`unfamiliarResourceAccess`

Blocking risky agents with a Conditional Access policy on agent risk is configured in 6.3 Block high-risk agent identities.

3.9.1 Review the Risky agents report

Performed by **Security Reader**.

1. Open https://entra.microsoft.com > **ID Protection** > **Dashboard** and select **View risky agents** (or open the **Risky Agents** report from the ID Protection navigation menu).
2. Filter and sort by agent, risk state, or risk level. Select an entry to view the agent display name and ID, risk state and risk level, agent type and sponsors, and the corresponding risk detections.
3. Open the **Risk Detections** report and select the **Agent detections** tab to view detection events from up to the past 90 days.

Check result

- The reviewer can open the **Risky Agents** report and the **Agent detections** tab.

3.9.2 Respond to a risky agent

Performed by **Security Administrator** (Microsoft Graph, permission `IdentityRiskyAgent.ReadWrite.All`). The same actions are available directly from the **Risky Agents** report and the risky agent details view.

Action	Effect	Graph (beta)
Confirm compromise	Sets the risk level to **High** and creates an event in the agent's risk detections. Triggers risk-based Conditional Access policies configured to block access on High agent risk.	`POST /identityProtection/riskyAgents/confirmCompromised`
Confirm safe	Clears active risk by setting the risk level to **None**; use for a false positive so the system avoids flagging similar activity.	`POST /identityProtection/riskyAgents/confirmSafe`
Dismiss risk	Marks the detected risk as no longer relevant, or a benign true positive where the system should continue to flag similar activity.	`POST /identityProtection/riskyAgents/dismiss`
Disable	Prevents all sign-ins for that agent across Microsoft Entra ID and connected apps.	—

The `confirmCompromised` and `confirmSafe` requests take a body of `{"agentIds": ["<agent object ID>"]}`.

Incident sequence:

1. **Detect** – review the Risky Agents report (3.9.1).
2. **Respond** – **Confirm compromise** and/or **Disable** the agent.
3. **Investigate** – review the risk detection details together with the agent's sign-in and audit logs (3.8). The agent's sponsor can determine whether the agent behavior is expected.
4. **Recover** – false positive: dismiss the risk and re-enable the agent. True compromise: rotate credentials before re-enabling, or retire the agent (see 5.8 Delete, restore, and permanently delete agents).

Risk data can also be exported through diagnostic settings in Microsoft Entra ID to a Log Analytics workspace, a storage account, an event hub, or a SIEM solution.

3.10 Keep sponsorship continuous with Lifecycle Workflows

Documentation: Agent identity sponsor tasks in Lifecycle Workflows · Lifecycle Workflows templates · Lifecycle Workflows tasks · Run a workflow on-demand · Check status of a workflow

If a sponsor leaves the organization, Microsoft Entra ID automatically reassigns sponsorship to the sponsor's manager. Lifecycle Workflows adds sponsor tasks that keep sponsorship continuous when a sponsor changes roles or leaves.

Built-in templates:

Template	Category	Default tasks
Agent sponsor job profile change	Mover	Send email to manager about sponsorship changes · Transfer agent sponsorships to manager · Remove all access package assignments for user
Offboard agent sponsors	Leaver	Send email to manager about sponsorship changes · Send email to co-sponsors about sponsor changes · Transfer agent sponsorships to manager

Sponsor tasks (mover and leaver tasks; available only under mover or leaver workflow templates):

Task	What it does
Send email to manager about sponsorship changes	Notifies the manager of a user who moved or left that the user sponsored one or more agent IDs, so the manager can decide whether another employee should become sponsor.
Send email to co-sponsors about sponsor changes	Notifies co-sponsors of the agent ID about the sponsorship change.
Transfer agent sponsorships to manager	Retrieves the user's manager, adds the manager as sponsor of each agent identity the user sponsors, and removes the user as sponsor. Requires a populated manager attribute.

3.10.1 Create the sponsor transition workflow

Performed by **Lifecycle Workflows Administrator**.

1. Open https://entra.microsoft.com > **ID Governance** > **Lifecycle workflows** > **Workflows**.
2. Create a new workflow based on the template **Agent sponsor job profile change**.
3. On the **Basics** tab, enter a unique display name (for example `PoC – Agent sponsor mover`) and description, select your trigger, and select **Next**.
4. On the **Configure scope** screen, select a scope that includes only the PoC test users, and select **Next**.
5. On the **Tasks** page, keep **Send email to manager about sponsorship changes** and **Transfer agent sponsorships to manager**, and optionally add **Send email to co-sponsors about sponsor changes**. Disable **Remove all access package assignments for user** so that the on-demand run in 3.10.2 doesn't remove PoC Sponsor's access package assignments. Select **Next**.
6. Review the workflow and select **Create**.

To be run on demand, the workflow must be enabled.

Check result

- The workflow appears in the workflows list with the sponsor tasks enabled and **Remove all access package assignments for user** disabled.

3.10.2 Run the workflow on demand

Performed by **Lifecycle Workflows Administrator**. Prerequisite: **PoC Sponsor** is a sponsor of the test agent (3.5.1) and has **PoC Sponsor Manager** as manager.

1. Select the workflow and select **Run on demand**.
2. On the **select users** tab, select **add users**, select **PoC Sponsor**, and select **Add**.
3. Confirm your choices and select **Run workflow**. An on-demand run doesn't take into account whether the user meets the workflow's execution conditions.
4. On the workflow overview screen, select **Workflow history** and review the user and task results.

Check result

- The tasks completed successfully for PoC Sponsor (see 3.12.4 for the full test).

3.11 Disable or restrict agents at the right scope

Documentation: Disable agent identities in your tenant · Manage agent identities in your organization · Manage agents in end user experience

Scope	Effect	Where
Individual agent identity	Blocks its access and token issuance; the agent identity and its metadata stay in the tenant.	Microsoft Entra admin center (administrators; the **All agent identities** page supports multi-select disable) or My Account portal (owners and sponsors)
Blueprint	Prevents new agent identities from being created from that blueprint and blocks existing ones.	Blueprint management page > **Disable**
Tenant-wide	Conditional Access policies block authentication of all agent identities, agents' user accounts, or users signing into agents; creation of agent identities can optionally be blocked through product-specific controls.	Conditional Access (see Chapter 6 – Conditional Access and least privilege)

Re-enabling a disabled agent identity at any scope restores access and token issuance.

Cautions:

- All Copilot Studio agent identities are children of the Microsoft Copilot Studio agent identity blueprint, so disabling that blueprint affects all of them.
- Globally disabling agent identities can cause existing agents to fail, degrade Microsoft product experiences, and push teams to use less transparent application or service principal identities. Evaluate the impact before enforcing. For a partial approach, use Conditional Access policies to block specific agents, and run policies in report-only mode first.

3.11.1 Disable and re-enable an individual agent identity

Performed by **Agent ID Administrator** (or the agent's owner).

1. Go to **Entra ID** > **Agents** > **Agent identities**.
2. Select the test agent identity, then select **Disable**.
3. After the test, enable the agent identity again from its overview.

Check result

- **Status** shows **Disabled**, then **Active** after re-enabling.

3.11.2 Disable a blueprint (only for a blueprint dedicated to the PoC)

Performed by **Agent ID Administrator**.

1. Go to **Entra ID** > **Agents** > **Agent blueprints** and select a blueprint that only PoC test agents use and that no other test in progress depends on (for example a blueprint created in 2.8).
2. Select **Disable** in the command bar of the blueprint's overview page. The confirmation dialog warns that existing agent identities created from this blueprint will no longer be able to authenticate. Confirm the action.
3. Re-enable the blueprint after the test.

Check result

- The blueprint **Status** is **Disabled**, and returns to **Active** after re-enabling.

3.12 Test and validation

3.12.1 Test identity and ownership coverage

Performed by **reviewer** (any Microsoft Entra user).

1. Open **Entra ID** > **Agents** > **Agent identities** with the columns from 3.4.1.
2. For each in-scope PoC agent, open **Owners and Sponsors**.
3. For each PoC blueprint, open **Owners and sponsors** as in 3.4.2.
4. (Optional) Run the custom security attribute filter in 3.7.3 (requires the roles listed there).

Expected result

- Every in-scope agent has an agent identity linked to the expected blueprint, **Status** = Active, at least one sponsor, and the intended owner.
- Every PoC blueprint has at least one sponsor.
- (Optional) The attribute filter returns every PoC agent identity.

3.12.2 Test sponsor self-service disable

Performed by **PoC Sponsor**, then **the agent owner** or **Agent ID Administrator**.

1. As PoC Sponsor, open https://myaccount.microsoft.com > **Manage agents** > **Agents you sponsor** and select the test agent.
2. Select **Disable agent**.
3. Confirm that PoC Sponsor can't re-enable the agent.
4. As the owner, select the agent in My Account and choose **Enable agent** (or, as Agent ID Administrator, re-enable it as in 3.11.1).

Expected result

- After step 2, the agent is disabled with the same effect as disabling it from the Microsoft Entra admin center: users can't access it and it isn't issued tokens. **Status** shows **Disabled** in the Microsoft Entra admin center.
- The sponsor can't re-enable the agent; the owner or admin can, and **Status** returns to **Active**.

3.12.3 Test risk response

Performed by **Security Administrator** (Graph Explorer, beta, `IdentityRiskyAgent.ReadWrite.All`), then **Security Reader**.

1. Confirm the test agent as compromised:

```
POST https://graph.microsoft.com/beta/identityProtection/riskyAgents/confirmCompromised
Content-Type: application/json

{
  "agentIds": ["<test agent identity object ID>"]
}
```

2. As Security Reader, open the **Risky Agents** report and the **Agent detections** tab (3.9.1).
3. As Security Administrator, confirm the agent as safe:

```
POST https://graph.microsoft.com/beta/identityProtection/riskyAgents/confirmSafe
Content-Type: application/json
```

```
{
  "agentIds": ["<test agent identity object ID>"]
}
```

Expected result

- After step 1, the test agent is listed in the **Risky Agents** report with risk level **High**, and a **Confirmed compromised** detection exists for the agent.
- After step 3, the risk level is **None**.

The end-to-end test that a Conditional Access policy blocks a confirmed-compromised agent is in 6.9 Test the risk-based block with the agent token flow (optional).

3.12.4 Test sponsor continuity

Performed by **Lifecycle Workflows Administrator**, then **reviewer**.

1. Confirm PoC Sponsor is a sponsor of the test agent and PoC Sponsor Manager is not.
2. Run the workflow on demand for PoC Sponsor (3.10.2).
3. Open **Workflow history** and select the total tasks for PoC Sponsor.
4. Open the test agent > **Owners and sponsors**.
5. Check the PoC Sponsor Manager mailbox (and any co-sponsor mailboxes).

Expected result

- The workflow history shows the sponsor tasks completed for PoC Sponsor.
- PoC Sponsor Manager is now a sponsor of the test agent, and PoC Sponsor is no longer a sponsor.
- PoC Sponsor Manager received the sponsorship change email. If you added **Send email to co-sponsors about sponsor changes** in 3.10.1, co-sponsors (if any) received the co-sponsor email.

3.13 Evidence

- Export or screenshot of **Agent identities** with columns **Name**, **Status**, **Object ID**, **Blueprint App ID**, **Owners and Sponsors**, **Uses agent identity** for all PoC agents.
- Screenshot of one PoC blueprint showing **Linked agent identities**, **Granted permissions**, and **Owners and sponsors**.
- Copilot Studio **Settings** > **Advanced** > **Metadata** showing the **Entra Agent ID** of a PoC agent (and the Power Platform admin center **Action history** if you migrated an agent).
- Screenshot of PoC Sponsor's My Account **Manage agents** page and the test agent in **Disabled** status after step 2 of 3.12.2.
- Screenshot of the test agent identity with **Status** = **Disabled** and then **Active** (3.11.1).
- (Optional) Graph responses for inheritable permissions before and after 3.6.2.
- (Optional) Graph response of the attribute filter in 3.7.3.
- Sign-in log export filtered by **Is Agent** = Yes, and audit log entries for the PoC agent identities.
- Screenshot of the **Risky Agents** report showing the test agent at **High**, then **None** after confirm safe.
- Lifecycle workflow **Workflow history** for PoC Sponsor, the agent's sponsors list after the run, and the manager notification email.

3.14 Troubleshooting

Symptom	Likely cause	Fix
A Copilot Studio agent isn't listed in **Agent identities** as an agent identity, or shows **Uses agent identity** = No	Agent was created before the Entra Agent ID rollout and still uses an app registration	Migrate it (3.3.2) or wait for the automatic migration by Microsoft.
Blueprint App ID is blank	The agent uses a service principal, not an agent identity	Expected for agents without agent identities; see 3.3 or Chapter 2.
Can't add a group as owner	Groups aren't supported as owners	Add users or service principals as owners; use a supported group as sponsor.
Can't add a security group as sponsor	Role-assignable groups and assigned membership security groups aren't allowed	Use a dynamic membership group or an assigned membership Microsoft 365 group.
Members of a dynamic group sponsor can't act as sponsors	Up to 24 hours after a rule or user-property change before sponsorship authorization succeeds	Wait and retry.
Manage agents missing in My Account	User doesn't own or sponsor any agent identity, or hasn't opted in to the new homepage	Add the user as owner or sponsor (3.5.1); select **Use new version**.
Sponsor can't re-enable an agent	Sponsors can't re-enable agents	Ask an owner or admin to re-enable it.
Inheritable permission `POST` returns `409 Conflict`	An entry for that `resourceAppId` already exists	Use `PATCH` on `.../inheritablePermissions/{resourceAppId}`.
Inheritable permission request returns `400 Bad Request`	`resourceAppId` isn't a valid GUID	Correct the GUID.
Inherited scope isn't visible on the agent identity	Inherited permissions are only observable in the token	Check the token `scp`/`roles` claims; confirm the resource app is listed as inheritable and the permission is granted to the blueprint.
Add attribute set is disabled	Attribute Definition Administrator isn't assigned (Global Administrator doesn't include it)	Assign Attribute Definition Administrator.
Attribute filter returns no results	Missing `ConsistencyLevel: eventual` or `$count=true`, or the value case doesn't match	Fix the request.
No agent entries in sign-in logs	Agent sign-ins can appear across each of the four sign-in log types	Use the **Agent type** / **Is Agent** filters and check every sign-in log type.
Risky Agents is empty	No offline detections yet, Learning Mode, or risk attributed to the user in on-behalf-of flows	Use 3.12.3 for a controlled test.
`riskyAgents` actions fail in Graph Explorer	Signed-in user isn't Security Administrator, or `IdentityRiskyAgent.ReadWrite.All` isn't consented	Assign the role and consent the permission.
Sponsor tasks aren't selectable when building a workflow	Sponsor tasks are available only under mover or leaver templates	Use a mover or leaver template (3.10).
Transfer agent sponsorships to manager doesn't transfer sponsorship	User's manager attribute isn't populated	Set the manager and run the workflow on demand again.
Workflow can't be run on demand	Workflow isn't enabled	Enable the workflow.

✅ What you proved

- [ ] Every in-scope PoC agent appears in **Entra ID** > **Agents** > **Agent identities** with **Uses agent identity** = Yes and the expected **Blueprint App ID**.
- [ ] Every in-scope agent identity and blueprint has at least one named sponsor, and the technical owner is recorded.
- [ ] A sponsor can see their agents in the My Account portal and can disable (but not re-enable) one of them.
- [ ] (Optional) Filtering on `AgentGovernance/Project = Agent365PoC` returns every PoC agent identity.
- [ ] Agent sign-ins are visible in the sign-in logs with the **Agent type** and **Is Agent** filters, and agent identity events are visible in the audit logs.
- [ ] A test agent can be confirmed compromised (risk level **High**) and then confirmed safe (risk level **None**).
- [ ] A Lifecycle Workflows run transfers a test sponsor's agent identity sponsorships to that sponsor's manager.
- [ ] An administrator can disable and re-enable a test agent identity, and its **Status** changes to **Disabled** and back to **Active**.

References

- Fundamental concepts in Microsoft Entra Agent ID
- Owners, sponsors, and managers
- Agent's user account
- View and filter agent identities in your tenant
- Disable agent identities in your tenant
- Manage Entra Agent IDs (Copilot Studio)
- Migrate Copilot Studio agents to Microsoft Entra Agent ID (preview)
- Manage agent identity blueprints in the Microsoft Entra admin center
- List agentIdentity objects (Microsoft Graph)
- Add and manage owners and sponsors for agent identities and blueprints
- Manage agents in end user experience
- Inheritable permissions and required resource access
- Configure inheritable permissions for agent identity blueprints
- Create inheritablePermission (Microsoft Graph)
- Custom security attributes overview
- Add or deactivate custom security attribute definitions
- Update agentIdentity (Microsoft Graph)
- Manage custom security attributes for an application
- Microsoft Entra Agent ID logs
- List directoryAudits (Microsoft Graph)
- ID Protection for agents
- riskyAgent resource type (Microsoft Graph beta)
- Manage agent identities in your organization
- Agent identity sponsor tasks in Lifecycle Workflows
- Lifecycle Workflows templates
- Lifecycle Workflows tasks
- Run a workflow on-demand
- Check status of a workflow

Runbook index · Previous: Chapter 2 – Third-party and custom agents · Next: Chapter 4 – Tools and MCP server governance

Prerequisites - Chapter 4: Tools and MCP server governance

> **Goal:** Prepare the accounts, access, and test assets needed for this chapter and its selected optional paths.

Apply the shared PoC preparation guidance and agree the chapter outcomes. Finish dependent build steps before their tests.

Table of contents

- Tenant readiness
- Prepare accounts and role assignments
- Prepare groups
- Test assets and dependencies
- References

Tenant readiness

Documentation: Get started with Microsoft Agent 365 · Agent management in the Microsoft 365 admin center · Frontier program

Use the task-specific roles in 4.1 Required permissions. Confirm availability rather than assuming the tenant is ready.

Follow the shared tenant readiness check using the chapter permissions and feature prerequisites linked above.

Prepare accounts and role assignments

Alongside the shared setup and reviewer accounts, prepare the following scenario accounts:

Account	Purpose
Standard test user	Tests plugin availability as a pilot-group member and invokes the test MCP tools. No admin role.
Second test user	User outside the plugin pilot group. No admin role.
Developer account	Builds and registers custom agents and MCP servers with the Agent 365 CLI.

Assign the roles in 4.1 Required permissions using the shared model. Verify each account can access the pages needed for its tasks.

Prepare groups

Documentation: Assign Microsoft Entra roles · Microsoft Entra built-in roles · Roles and role groups in Microsoft Purview

Performed by **Groups Administrator**.

Create or verify these PoC groups before the tasks that use them:

Group	Members
`A365-PoC-Plugin-Pilot`	Standard test user only

Check result

- Group membership matches the table, and the groups are recorded as PoC assets.

Test assets and dependencies

Documentation: Manage plugins, skills, and MCP servers in Microsoft 365 admin center · Build plugins for Copilot Cowork · Bring your own (BYO) MCP server in Microsoft 365 admin center · Agent 365 CLI develop-mcp command reference · Install and use the Agent 365 CLI · Add and manage tools – Set up service principal · Advanced hunting overview · CloudAppEvents table · BYO MCP server article · Connect MCP servers to developer clients

- Identify one plugin to restrict in 4.3.2. To also test **Delete**, use a plugin that your organization uploaded (packaged as described in Build plugins for Copilot Cowork); if you upload it yourself, complete 4.3.4 before 4.3.2. Delete isn't available for plugins from the Microsoft 365 Store.
- Prepare a test remote MCP server that you own, with a publicly accessible HTTPS endpoint (for example, `https://<your-host>/mcp`) and one of the supported authentication types (`NoAuth`, `APIKey`, `ExternalOAuth`, or `EntraOAuth`). Don't use a production server. During preview, you can't republish new versions of a registered server.
- Prepare a Copilot Studio environment and a test custom agent that you can add tools to.
- (Optional, for 4.5) An Azure API Management or Azure AI Gateway instance with at least one MCP server registered in it, or a LiteLLM gateway with its URL and API key.
- Confirm that Microsoft Defender advanced hunting is available to the security reviewer. See Chapter 8 – Threat detection and runtime protection (Defender).
- Tools Gateway, BYO MCP server, Power Platform connector usage, and the **Requests (preview)** tab are preview features and aren't meant for production use. Tool-level granular control is rolling out to tenants.

Build guidance: 4.6 Bring your own MCP server

Check result

- Each asset needed for the selected paths is prepared or has a named build task that will finish before its dependent tests.

References

- Get started with Microsoft Agent 365
- Agent management in the Microsoft 365 admin center
- Frontier program

- Assign Microsoft Entra roles
- Microsoft Entra built-in roles
- Roles and role groups in Microsoft Purview
- Manage plugins, skills, and MCP servers in Microsoft 365 admin center
- Build plugins for Copilot Cowork
- Bring your own (BYO) MCP server in Microsoft 365 admin center
- Agent 365 CLI develop-mcp command reference
- Install and use the Agent 365 CLI
- Add and manage tools – Set up service principal
- Advanced hunting overview
- CloudAppEvents table
- BYO MCP server article
- Connect MCP servers to developer clients

> **Ready?** Continue to the chapter. →

Chapter 4 - Tools and MCP server governance

Pillar: Govern **What it proves:** Every tool an agent can call (plugins, skills, MCP servers, and connectors) is inventoried in one registry, is approved before agents can use it, and can be blocked centrally so that every dependent agent loses access at runtime.

Success criteria: Agree the selected outcomes in What you proved before testing.

Topics and time guide

Approximate hands-on estimates for the selected scope, with prerequisites prepared. Service waiting time is separate.

Topic	Estimated hands-on time
4.1 Required permissions	~5 min
4.2 Review the Tools page and registry	~5-10 min
4.3 Govern plugins and skills	~15-20 min
4.4 Govern MCP servers	~10-15 min
4.5 Connect a Tools Gateway (preview) (optional)	~15-20 min
4.6 Bring your own MCP server (preview)	~15-20 min
4.7 Test and validation	~15-20 min
4.8 Evidence	~5-10 min
4.9 Troubleshooting	As needed

Outcome checklist · References

4.1 Required permissions

Use the shared account and role-assignment model with the task-specific permissions below.

Task	Least-privilege role	Section
Prepare the PoC plugin pilot group	Groups Administrator	Prerequisites
View the Tools registry, plugins, connectors, and settings (read-only)	AI Reader or Global Reader	4.2
Block or unblock tools; set plugin availability; upload, install, uninstall, delete, or block plugins and skills; review plugin access requests	AI Administrator	4.2.2, 4.3, 4.7.3, 4.7.4
Review and approve MCP server requests and grant tenant-wide consent; block, unblock, or delete MCP servers; manage individual tools	AI Administrator (Global Administrator also meets both requirements)	4.4, 4.6.4, 4.7.1, 4.7.2, 4.7.5

Task	Least-privilege role	Section
Connect a Tools Gateway, grant the one-time tenant-wide consent, and govern discovered servers	Global Administrator	4.5, 4.7.5
Provision the Agent 365 Tools service principal (one-time per tenant)	Global Administrator	4.6.1
Register a BYO MCP server with the Agent 365 CLI, and use the approved server in a supported client	Developer	4.6.1, 4.6.2, 4.6.5, 4.7.1, 4.7.2
Evaluate MCP tool-definition quality with the Agent 365 CLI	Developer (no admin role required)	4.6.3
Grant a BYO MCP server's permission to agent identities that are missing it (`a365 develop-mcp grant-agents-access`; needs a CLI version later than 1.1.221)	Global Administrator	4.9
Discover plugins and request access in the Copilot channel	Standard test user and second test user (no admin role)	4.7.3, 4.7.4
Validation / read-only review of MCP activity in advanced hunting	Security Reader	4.6.6, 4.7.6

Before you start:

- [ ] Confirm the chapter prerequisites for the selected topics.

4.2 Review the Tools page and registry

Documentation: Overview of the Tools page in Microsoft 365 admin center · Agent management roles and permissions · How do enterprises control what agents can do?

4.2.1 Open the Tools registry and identify the four tool types

Performed by **AI Reader** (or Global Reader).

1. Sign in to the Microsoft 365 admin center at `https://admin.cloud.microsoft`.
2. Select **Agents** > **Tools** > **Registry**.
3. Review the tool types that the page brings together:

Tool type	What it is
Plugins	Installable packages that bundle capabilities an agent can use, such as skills, MCP servers, and connectors.
Skills	Reusable instructions or workflows that teach an agent how to do a task consistently.
MCP servers	Standardized servers that expose tools and resources that an agent can dynamically discover and use.
Connectors	Prebuilt links to external systems and data sources that an agent can read from and write to.

4. Note the two tabs: **Registry** (discover and manage tools in the tenant) and **Requests** (review and approve requests for tools).

Check result

- The registry loads and shows tools of more than one type.
- As AI Reader or Global Reader, you can view tools and settings but can't change them.

4.2.2 Filter the registry and act on a tool

Performed by **AI Reader** (view) and **AI Administrator** (actions).

1. On **Agents** > **Tools** > **Registry**, filter by **Status** (**Available** or **Blocked**).
2. Filter by **Publisher** to separate Microsoft tools from tools published by other providers.
3. Review the columns:

Column	Description
Name	Display name of the tool, for example **Microsoft Teams MCP Server**.
Status	**Available** or **Blocked**.
Type	Tool category, for example **MCP Server**.
Publisher	Who published the tool, for example Microsoft for first-party tools.

4. Select a tool to open its overview page. As AI Administrator, you can select **Block** to prevent agents or workflows from using the tool, or **Unblock** to restore access. Don't block tools in this step; blocking is tested in 4.7.2.

Check result

- Filters narrow the list to the selected status or publisher.
- AI Reader and Global Reader have read-only access to tools and settings; AI Administrator can take the **Block** and **Unblock** actions.

4.2.3 Review Power Platform connector usage (preview)

Performed by **AI Reader** (or Global Reader).

This view provides visibility into which Power Platform connectors agents use. It doesn't include management controls.

1. Select **Agents** > **Tools** > **Registry**.
2. Select **Connectors**.
3. Review the list of Power Platform connectors used by agents in your organization.
4. Select a connector to view the agents that use it. The connector's overview page shows the agents associated with the selected connector.
5. Note the **View in Power Platform** option on the connector's overview page. It opens the Power Platform admin center (`https://admin.powerplatform.microsoft.com`), where the connector's use is blocked, allowed, and governed (see Advanced connector policies). This chapter doesn't change connector policies.

Check result

- At least one connector shows the agents that use it.

4.3 Govern plugins and skills

Documentation: Manage plugins, skills, and MCP servers in Microsoft 365 admin center · Build plugins for Copilot Cowork

Plugins are primarily available to end users in the Copilot channel. Managing a skill uses the same actions and steps as managing a plugin, so every task in this section applies to both.

4.3.1 Set organization-level plugin availability by publisher category

Performed by **AI Administrator**.

1. In the Microsoft 365 admin center, select **Agents** > **Settings** > **Agent and plugin access**.
2. Record the publisher categories that are currently selected; the availability test in 4.7.4 uses this baseline.
3. Under the installation settings, choose which publisher categories users can access: **Microsoft**, **your organization**, or **certified external publishers**.
4. Select a category to allow users to discover and install plugins from that publisher type. To block all third-party plugins, clear **certified external publishers**.

Check result

- Plugins from a category that isn't selected remain discoverable but display "This plugin is blocked by your organization's policy." Eligible users can still select **Request access**. This is tested in 4.7.4.

4.3.2 Restrict a plugin to specific users and groups

Performed by **AI Administrator**.

1. Select **Agents** > **Tools** > **Plugins**.
2. Select the plugin you identified in **Before you start**.
3. Open **Users** and choose one of the availability options:
   - **All users** – available to everyone in the organization.
   - **No users** – unavailable to all users.
   - **Specific users and groups** – available only to the selected users or groups.
4. For the PoC, select **Specific users and groups** and add `A365-PoC-Plugin-Pilot`.
5. Review the selection, and then save the changes.

Check result

- The plugin's **Users** setting shows only the pilot group. The end-user result is validated in 4.7.3.

4.3.3 Review plugin access requests

Performed by **AI Administrator**.

A plugin access request is created when a user selects **Request access** on a restricted plugin. The end-to-end flow is tested in 4.7.3.

1. Select **Agents** > **Tools** > **Requests**.
2. Select the plugin access request that you want to review.
3. Review the request details, including the user and the requested plugin.
4. Select **Approve** to grant access, or **Reject** to deny the request.

Check result

- **Approve** grants the user access to the plugin; **Reject** denies the request.

4.3.4 Upload a plugin or skill

Performed by **AI Administrator**.

A developer must first package the plugin or skill into a manifest file.

1. Select **Agents** > **Tools** > **Registry**.
2. Select **Upload**.
3. Upload the manifest file for the plugin or skill.
4. Review the components included in the package, such as MCP servers and skills, and then select **Next**.
5. In the **Scope users** pane, select **All users** or **Specific users or groups**. For the PoC, scope it to `A365-PoC-Plugin-Pilot`.
6. Review the configuration, and then select **Install**.

Check result

- The upload wizard listed the components included in the package, such as MCP servers and skills, and the installation completed for the selected users.

4.3.5 Install or uninstall a plugin or skill

Performed by **AI Administrator**.

Install a plugin or skill so that users don't have to install it manually:

1. Select **Agents** > **Tools** > **Plugins**.
2. Select a plugin or skill. The **Overview** pane opens.
3. Select **Install**.
4. In the **Select users** pane, confirm the details and choose **All users** or **specific users or groups**.
5. Select **Next**.
6. In the **Review and install** pane, confirm the details and select **Install**.

Uninstall a plugin or skill so that agents can't use it unless it's installed again:

1. Select **Agents** > **Tools** > **Plugins**.
2. Select the plugin or skill. The **Overview** pane opens.
3. Select **Uninstall**, and then confirm by selecting **Uninstall**.

Check result

- After install, the plugin or skill is ready to use for the selected users without manual installation by end users.
- After uninstall, it isn't available to agents unless you install it again.

4.3.6 Delete or block a plugin or skill

Performed by **AI Administrator**.

Perform these actions only on test plugins or skills.

Action	Effect	Applies to
Uninstall	Removes the plugin or skill from the environment. It isn't available to agents unless it's installed again.	Any installed plugin or skill

Action	Effect	Applies to
Delete	Removes the package across the organization. Agents can no longer use it, and it's removed from the registry.	Only plugins and skills uploaded to your tenant (not Microsoft 365 Store items)
Block	Prevents users and agents across the organization from accessing it, without deleting the package. Can be reversed with **Unblock**.	Any plugin or skill

To delete:

1. Select **Agents** > **Tools** > **Plugins**.
2. Select the plugin or skill. The **Overview** pane opens.
3. Select **Delete**, and then confirm by selecting **Delete**.

To block:

1. Select **Agents** > **Tools** > **Plugins**.
2. Select the plugin or skill. The **Overview** pane opens.
3. Select **Block**, and then confirm by selecting **Block**.

> Caution: Blocking cascades. Blocking a plugin prevents users from accessing the plugin and any agents that depend on it; its associated MCP servers and connectors remain available to other plugins. Blocking an MCP server also blocks dependent plugins and linked connectors. Blocking a plugin package blocks the agents included in it, because they share the same governance controls; unblocking the package restores access to those agents. To see which plugins use an MCP server, select the MCP server in the registry before you block it.

Check result

- A blocked plugin or skill can no longer be used by agents. A deleted plugin or skill is removed from the registry.

4.4 Govern MCP servers

Documentation: Manage plugins, skills, and MCP servers – Review and approve MCP requests · Grant tenant-wide admin consent to an application · Agent 365 tooling servers overview

To approve an MCP server, the reviewer must have access to the **Tools** page and must be able to grant tenant-wide consent. AI Administrator and Global Administrator meet both requirements; use AI Administrator for least privilege. Each MCP server corresponds to a permission on the Agent 365 application, and agents gain access to the server only after consent. A blocked MCP server is blocked for every user and every agent. The ability to allow or disallow tooling and MCP servers in the Microsoft 365 admin center might not be available in your region yet (Agent 365 tooling servers overview).

4.4.1 Review and approve MCP server requests

Performed by **AI Administrator**.

1. Sign in to the Microsoft 365 admin center at `https://admin.cloud.microsoft`.

2. Select **Agents** > **Tools**, and then select the **Requests (preview)** tab.
3. Review the server name, publisher, requester, and request date.
4. Review the server information and declared tools for accuracy and compliance.
5. Select **Approve** to make the server available in the organizational registry, or **Reject** to deny the request.
6. After approval, consent to the Microsoft Entra permissions that the MCP server requires. The server becomes available to agent-building surfaces only after consent is granted.

Check result

- The server appears in **Agents** > **Tools** > **Registry** with status **Available**.
- After approval and consent, allow up to 30 minutes for the server to appear in all Copilot Studio environments in the tenant.

4.4.2 Block or unblock an MCP server

Performed by **AI Administrator**.

1. Select **Agents** > **Tools**, and then select the **Registry** tab.
2. Select an MCP server to open its overview pane. Selecting the server shows the plugins that use it.
3. Select **Block** to restrict the server across the organization, or **Unblock** to restore access to a previously blocked server.
4. If one or more plugins use the server, you can also block all linked plugins when you block the server.

Check result

- The server status is **Blocked** (or **Available** after unblocking). Blocking disables all the tools the server exposes.

4.4.3 Enable or disable individual tools (where supported)

Performed by **AI Administrator**.

Tool-level granular control is rolling out to tenants and supports only specific types of MCP servers registered on Agent 365, and MCP servers registered in an AI Gateway tier of Azure API Management. Support for BYO MCP servers is planned for a future release. Use it to keep high-risk tools (for example, tools that write, delete, or process payments) disabled while lower-risk tools on the same server stay enabled.

1. Select **Agents** > **Tools**, and then select the **Registry** tab.
2. Select an MCP server to open its overview pane.
3. Select the **Tools** tab. The list shows every tool the server exposes, its description, and an **Enabled** or **Disabled** toggle.
4. Turn individual tools on or off as needed.
5. Select **Save** to apply your changes, or **Discard changes** to cancel.

Check result

- The tool's toggle is **Disabled** after you save. The policy applies wherever the tool is used and is enforced at runtime by the Agent 365 Tooling Gateway.
- If the **Tools** tab shows that the server doesn't support tool discovery, use 4.4.2 to block or allow the whole server instead.

4.5 Connect a Tools Gateway (preview)

Documentation: Manage Tools Gateway in Microsoft 365 admin center · About MCP servers in Azure API Management · AI Gateway tier overview

Tools Gateway connects an existing Azure API Management (APIM) instance, Azure AI Gateway, or LiteLLM gateway to the Microsoft 365 admin center. After a Global Administrator grants a single tenant-wide consent, the MCP servers registered in that gateway appear on the Tools page and are governed like any other MCP server. Servers added to the gateway later are discovered automatically.

4.5.1 Confirm gateway prerequisites

Performed by **Global Administrator**.

1. Confirm that an AI Gateway or APIM instance is configured.
2. Confirm that at least one MCP server is registered in the gateway (in APIM, for example by exposing a REST API as an MCP server or exposing an existing MCP server).
3. Confirm that a Global Administrator is available to grant consent and has access to the Microsoft 365 admin center.
4. For a LiteLLM gateway, collect the gateway name, gateway URL, and API key.

Check result

- You can name at least one MCP server that is registered in the gateway, to look for in 4.5.3.

4.5.2 Connect an Azure or LiteLLM gateway

Performed by **Global Administrator**.

1. Sign in to the Microsoft 365 admin center as a Global Administrator.
2. Go to **Agents** > **Settings**.
3. Select **Gateways**.
4. Under **Gateways**, connect one of the following:
    - **Microsoft Azure**: turn on the toggle to connect an Azure API Management service or Azure AI Gateway, and then review and accept the requested permissions.
    - **LiteLLM gateway**: select **Connect a gateway**, and then enter the gateway name, gateway URL, and API key.

Check result

- Consent is granted once per tenant and applies to all MCP servers in the connected gateway; individual servers don't require separate consent.
- Discovery starts automatically after consent is granted.

4.5.3 Govern gateway-discovered MCP servers

Performed by **Global Administrator**.

Discovery starts automatically after consent and typically completes within a few minutes.

1. Go to **Agents** > **Tools**.
2. Filter the list by source, and then select **AI gateway**.
3. Review the discovered MCP servers.

4. Block or allow each server as in 4.4.2. For servers in an AI Gateway tier of APIM, you can also manage individual tools as in 4.4.3.

Check result

- The MCP server you noted in 4.5.1 appears under the **AI gateway** source.
- Only MCP servers registered in the connected gateway are discovered. New servers added to the gateway appear on subsequent refreshes.

4.6 Bring your own MCP server (preview)

Documentation: Bring your own (BYO) MCP server in Microsoft 365 admin center · Agent 365 CLI develop-mcp command reference · Install and use the Agent 365 CLI · Add and manage tools – Set up service principal · Advanced hunting overview · CloudAppEvents table

BYO MCP server routes an organization's internally built remote MCP servers through the Agent 365 Tooling Gateway, so administrators control them in the Microsoft 365 admin center and security teams get observability. The flow is:

1. A developer registers the server with the Agent 365 CLI (server URL, authentication type, and tools).
2. An administrator reviews and approves or rejects the request, and then grants the required Microsoft Entra permissions.
3. Supported clients use the approved server. During preview, the supported clients are Copilot Studio, Visual Studio Code, Claude Code, and GitHub Copilot CLI. Azure AI Foundry and Microsoft 365 declarative agents aren't supported.
4. The security team monitors activity and tool invocations in Microsoft Defender advanced hunting.

4.6.1 Confirm developer prerequisites

Performed by **developer**, with **Global Administrator** for the service principal.

1. Install the Agent 365 CLI (requires .NET; .NET 8.0 is recommended):

   ```
   dotnet tool install --global Microsoft.Agents.A365.DevTools.Cli
   ```

   If it's already installed, update it:

   ```
   dotnet tool update --global Microsoft.Agents.A365.DevTools.Cli
   ```

2. Verify the installation by running the `--help` command. BYO MCP server registration requires Agent 365 CLI version **1.1.165-preview or later**. For available versions, see the Microsoft.Agents.A365.DevTools.Cli NuGet page.

   ```
   a365 -h
   ```

3. Confirm that the Agent 365 service principal associated with app ID `ea9ffc3e-8a23-4a7d-836d-234d7c7565c1` is provisioned in your tenant.

4. If you can't find it, a Global Administrator provisions it once per tenant: download New-Agent365ToolsServicePrincipalProdPublic.ps1, open PowerShell as Administrator, go to the script directory, run the script, and sign in by using your Azure credentials when prompted:

```
.\New-Agent365ToolsServicePrincipalProdPublic.ps1
```

5. Confirm that the test MCP server has a publicly accessible endpoint and uses one of the supported authentication types: `NoAuth`, `APIKey` (header or query parameter), `ExternalOAuth`, or `EntraOAuth`.

6. Plan the target tenant for registration: the registration command's `--tenant-id` option defaults to the current `az login` tenant. Pass `--tenant-id` with the PoC tenant ID to target the PoC tenant explicitly.

Check result

- `a365 -h` displays the help information, and the service principal is provisioned.

4.6.2 Register the MCP server with the Agent 365 CLI

Performed by **developer**.

Rules from the CLI reference:

- The server name must start with `ext_` and be at most 20 characters, for example `ext_PocTest`.
- Tool names must exactly match the names exposed by the remote MCP server. Mismatched names cause tool invocations to fail at runtime.
- Command-line options override values in a JSON input file. Omitted required values are prompted for interactively.
- If registration fails after the Microsoft Entra app registrations are created, they aren't rolled back automatically. Delete them manually in the Azure portal before you retry.

1. Register the server by using one of the following methods.

   Option A – command-line options. This example is for a server that requires no authentication (Bash syntax):

   ```
   a365 develop-mcp register-external-mcp-server \
   --server-name "ext_PocTest" \
   --server-url "https://<your-host>/mcp" \
   --publisher "Contoso" \
   --description "PoC test MCP server" \
   --auth-type "NoAuth" \
   --tools "tool1,tool2"
   ```

   For a server that takes an API key in a header, use these authentication options instead:

   ```
   a365 develop-mcp register-external-mcp-server \
   --server-name "ext_PocTest" \
   --server-url "https://<your-host>/mcp" \
   --publisher "Contoso" \
   --description "PoC test MCP server" \
   --auth-type APIKey \
   --api-key-location Header \
   --api-key-name token \
   --tools "tool1,tool2"
   ```

   For `EntraOAuth`, use `--auth-type EntraOAuth --remote-scopes "<scope>"`. For `ExternalOAuth`, also pass `--idp-authorization-url`, `--idp-token-url`, `--idp-scopes`, `--idp-client-id`, `--idp-client-secret`, and `--remote-scopes`, and after registration add the redirect URI that the CLI displays to your external identity provider application. For complete examples of each type, see the BYO MCP server article.

   Option B – JSON file. Create `ext-poctest.json` by using the structure of the Learn `NoAuth` example:

```
{
  "serverName": "ext_PocTest",
  "serverUrl": "https://<your-host>/mcp",
  "authType": "NoAuth",
  "description": "PoC test MCP server",
  "publisherName": "Contoso",
  "tools": [
    {
      "name": "tool1",
      "description": "<what tool1 does>"
    }
  ],
  "remoteScopes": null,
  "externalOAuth": null,
  "apiKey": null
}
```

Preview the actions with `--dry-run`, and then register:

```
a365 develop-mcp register-external-mcp-server -f ./ext-poctest.json --dry-run
a365 develop-mcp register-external-mcp-server -f ./ext-poctest.json
```

2. After successful registration, submit the MCP server for admin review in the Microsoft 365 admin center, and ask the AI Administrator to review it. Before the request is approved, run steps 1–2 of 4.7.1.

Check result

- The registered server appears in the Microsoft 365 admin center on **Agents** > **Tools** > **Requests (preview)** for review and approval.

4.6.3 Evaluate tool-definition quality

Performed by **developer** (no admin role required).

`a365 develop-mcp evaluate` reads the server's tool schemas through a standard MCP `tools/list` call and produces a report with a score and prioritized improvements. Semantic checks are scored by a coding agent CLI (GitHub Copilot CLI or Claude Code) that runs locally under the developer's own account. The command doesn't send tool-schema data to Microsoft and doesn't call Azure or Microsoft Graph. Run it only against MCP servers you trust.

1. (Optional, for semantic scoring) Install one coding agent CLI locally, for example GitHub Copilot CLI:

```
npm install -g @github/copilot
```

2. Run the evaluation against the test server:

```
a365 develop-mcp evaluate --server-url "https://<your-host>/mcp" --output-dir "./eval"
```

If the server requires authentication, supply the bearer token through the `A365_MCP_AUTH_TOKEN` environment variable instead of the command line:

```
$env:A365_MCP_AUTH_TOKEN = "<bearer-token>"
a365 develop-mcp evaluate --server-url "https://<your-host>/mcp" --output-dir "./eval"
```

To skip AI scoring and generate the checklist only, so that you can score it with your own LLM:

```
a365 develop-mcp evaluate --server-url "https://<your-host>/mcp" --eval-engine none
```

3. Open `<server-name>_eval_report.html` from the output directory (`./eval` in these examples; the current directory if you omit `--output-dir`). Review the overall score (0-100), the maturity level, the per-tool scores (tool name, tool description, parameter name, parameter description, schema structure), and the prioritized action items.

Check result

- The output directory contains `<server-name>_checklist.json`, `<server-name>_eval_report.html`, and `<server-name>_eval_report.json`.
- Keep the HTML report as PoC evidence that the tool definitions were evaluated.

4.6.4 Review, approve, and consent

Performed by **AI Administrator**.

Complete steps 1–2 of 4.7.1 before you approve the request.

1. Follow 4.4.1 for the `ext_PocTest` request: review the server name, publisher, requester, request date, server information, and declared tools.
2. Compare the declared tools with the evaluation report from 4.6.3.
3. Select **Approve**, and then consent to the Microsoft Entra permissions that the server requires.

The governance controls that apply to a BYO MCP server are:

Control	Description
Approval or rejection	An administrator approves or rejects each BYO MCP server before it can be used.
Server-level block	An administrator can block an approved server at any time.
Runtime enforcement	Blocked MCP servers can't be invoked at runtime from any client surface.
Tool-level block	Rolling out for supported MCP servers registered on Agent 365; support for BYO MCP servers is planned for a future release.
Tools snapshot	An administrator can view the tools declared by each MCP server.
Delete	An administrator can delete a registered BYO MCP server that's no longer needed.

Check result

- `ext_PocTest` has status **Available** in the organizational registry, and it becomes available to agent-building surfaces after consent is granted.

4.6.5 Use the approved server in Copilot Studio

Performed by **developer**.

After approval and consent, the server can take up to 30 minutes to appear in all Copilot Studio environments in the tenant.

1. Go to Copilot Studio (`https://copilotstudio.microsoft.com`) in the test environment.

2. Create a new custom agent, or open the test agent.
3. Go to the **Tools** section and select **MCP Server**.
4. Select `ext_PocTest` from the registry.
5. Test the agent by entering a prompt that invokes one of the server's tools.
6. If you're prompted to complete a one-time connection setup (for example, entering the API key for an `APIKey` server), follow the provided URL to create the connection, return to the agent, and retry the prompt.

Check result

- After a successful invocation, the MCP server returns the tool output.

To configure a developer client's MCP connection, see Connect MCP servers to developer clients. Verify that the server and authentication model are supported by the selected client.

4.6.6 Monitor MCP activity in Microsoft Defender

Performed by **Security Reader**.

1. Sign in to the Microsoft Defender portal (`https://security.microsoft.com`).
2. Go to **Advanced hunting**.
3. Run the sample query from Learn, replacing `<tool name>` with a tool exposed by `ext_PocTest`:

```
CloudAppEvents
| where ActionType in ("ExecuteToolByGateway")
| where RawEventData contains "<tool name>"
```

4. To review all gateway tool invocations for the PoC window:

```
CloudAppEvents
| where Timestamp > ago(7d)
| where ActionType == "ExecuteToolByGateway"
| project Timestamp, ActionType, AccountDisplayName, RawEventData
| order by Timestamp desc
```

Check result

- The query returns details including agent name, MCP server name, and invocation metadata for the test prompt from 4.6.5.

4.7 Test and validation

Documentation: Manage plugins, skills, and MCP servers · Bring your own (BYO) MCP server · Manage Tools Gateway

4.7.1 Approved BYO MCP server appears in the registry only after approval

Performed by **developer** and **AI Administrator**.

1. After registration (4.6.2) and before approval (4.6.4), as AI Administrator, open **Agents** > **Tools** > **Requests (preview)** and confirm that the `ext_PocTest` request is listed.
2. As the developer, in Copilot Studio, confirm that `ext_PocTest` isn't yet available to select as an MCP server.

3. As AI Administrator, approve the request and grant consent (4.6.4).
4. Open **Agents** > **Tools** > **Registry** and locate `ext_PocTest`.
5. As the developer, add the server to the test agent and invoke it (4.6.5).

Expected result

- Before approval and consent, the server is a request on the **Requests (preview)** tab and isn't available to agent-building surfaces.
- After approval and consent, the server is **Available** in the organizational registry, and the test agent's invocation returns the tool output.

4.7.2 Blocking the MCP server stops the dependent agent

Performed by **AI Administrator** and **developer**.

1. In Copilot Studio, confirm that the test agent successfully invokes `ext_PocTest`.
2. In the Microsoft 365 admin center, block `ext_PocTest` (4.4.2). If any plugins use the server, you can also block all linked plugins.
3. In Copilot Studio, enter the same prompt in the test agent.
4. Unblock `ext_PocTest`, and then repeat the prompt.

Expected result

- While blocked, the server's status is **Blocked**, and runtime enforcement prevents agents from invoking it.
- After unblocking, access to the server is restored and the MCP server returns the tool output again.

4.7.3 Plugin restricted to a group triggers Request access

Performed by **AI Administrator**, **standard test user** (member of `A365-PoC-Plugin-Pilot`), and **second test user** (not a member).

1. As AI Administrator, confirm that the plugin from 4.3.2 is set to **Specific users and groups** with `A365-PoC-Plugin-Pilot`.
2. As the standard test user, find the plugin in the Copilot channel.
3. As the second test user, find the same plugin in the Copilot channel, and select **Request access**.
4. As AI Administrator, open **Agents** > **Tools** > **Requests**, select the request, review the user and the requested plugin, and select **Approve** (or **Reject** to test denial).

Expected result

- The plugin is available to the standard test user, because it's available to the selected group.
- The second test user can still discover the plugin, which appears as blocked by organizational policy, and can select **Request access**.
- The request is sent to the Microsoft 365 admin center and is listed on the **Requests** tab with the user and the requested plugin.
- **Approve** grants the user access; **Reject** denies the request.

4.7.4 (Optional) Organization-level publisher category block

Performed by **AI Administrator** and **second test user**.

1. As AI Administrator, in **Agents** > **Settings** > **Agent and plugin access**, clear **certified external publishers** (4.3.1).
2. As the second test user, find a plugin from a certified external publisher in the Copilot channel.
3. As AI Administrator, restore the publisher categories that you recorded in 4.3.1.

Expected result

- The plugin is still discoverable but displays "This plugin is blocked by your organization's policy."

4.7.5 (Optional) Tool-level control and gateway discovery

Performed by **Global Administrator** (gateway) and **AI Administrator** (tool-level control).

1. If you connected a gateway (4.5), as Global Administrator, go to **Agents** > **Tools**, filter the list by source, select **AI gateway**, and confirm that the server you noted in 4.5.1 is listed.
2. Register another MCP server in the gateway, and then refresh the Tools page.
3. As AI Administrator, for a server whose **Tools** tab lists its tools, disable one tool and select **Save** (4.4.3).

Expected result

- Gateway servers are listed under **AI gateway** without separate per-server consent, and the newly registered server is discovered during a subsequent refresh.
- The disabled tool's toggle is **Disabled**. The policy applies wherever the tool is used and is enforced at runtime by the Agent 365 Tooling Gateway, while the other enabled tools on the same server remain available.

4.7.6 MCP invocations are visible in Defender

Performed by **Security Reader**.

1. Run the queries from 4.6.6 for the PoC window.

Expected result

- The query returns details including agent name, MCP server name, and invocation metadata for the invocations made in 4.7.1 and 4.7.2.

4.8 Evidence

- Screenshot of **Agents** > **Tools** > **Registry** showing the tool categories, with the **Status** and **Publisher** filters applied (4.2.1, 4.2.2).
- Screenshot of a connector's overview page showing the agents that use it (4.2.3).
- Screenshot of **Agents** > **Settings** > **Agent and plugin access** showing the selected publisher categories (4.3.1).
- Screenshot of the restricted plugin's **Users** setting with only `A365-PoC-Plugin-Pilot` selected (4.3.2).
- CLI output of the `register-external-mcp-server` command, with secrets removed (4.6.2).
- The `<server-name>_eval_report.html` evaluation report (4.6.3).
- Screenshot of the `ext_PocTest` request on the **Requests (preview)** tab before approval (4.7.1).
- Screenshot of `ext_PocTest` with status **Available** in the registry, and of the Microsoft Entra consent prompt (4.7.1).
- Screenshots of the Copilot Studio test agent: successful invocation, the invocation attempt while the server is blocked with the server's **Blocked** status, and successful invocation after unblocking, with timestamps (4.7.1, 4.7.2).
- Screenshots of the plugin available to the standard test user, the plugin blocked by organizational policy with **Request access** for the second test user, and the request on the **Requests** tab with its decision (4.7.3).
- (Optional) Screenshot of the "This plugin is blocked by your organization's policy." message (4.7.4).
- (Optional) Screenshot of **Agents** > **Settings** > **Gateways** after connecting the gateway, the registry filtered by source **AI gateway**, and an MCP server's **Tools** tab with a disabled tool (4.7.5).
- Advanced hunting query results exported to CSV (4.7.6).

4.9 Troubleshooting

Symptom	Likely cause	Fix
Reviewer can't approve an MCP server request or the consent step fails	The account can't grant tenant-wide consent	Use AI Administrator or Global Administrator.
Registration fails because the server name is rejected	Name doesn't start with `ext_` or is longer than 20 characters	Use a name such as `ext_PocTest`.
Registration fails partway and a retry fails	Microsoft Entra app registrations created by the failed run aren't automatically rolled back	Delete them manually in the Azure portal, and then retry.
The Agent 365 service principal can't be found in the tenant	The service principal associated with app ID `ea9ffc3e-8a23-4a7d-836d-234d7c7565c1` isn't provisioned	Have a Global Administrator run `New-Agent365ToolsServicePrincipalProdPublic.ps1` (4.6.1).
Registration targets the wrong tenant	`--tenant-id` defaults to the current `az login` tenant	Pass `--tenant-id` with the PoC tenant ID.
Approved server isn't listed in Copilot Studio	Consent not granted, or the server hasn't yet appeared in all environments	Confirm consent was granted; after approval and consent, the server can take up to 30 minutes to appear in all Copilot Studio environments.
Tool invocations fail at runtime after approval	Tool names in the registration don't exactly match the names exposed by the remote MCP server	Register tool names that exactly match the names exposed by the server. You can't republish new versions of a remote MCP server during preview.
First invocation of an `APIKey` server prompts for setup	One-time connection setup is required	Follow the provided URL to create the connection, enter the API key, return to the agent, and retry the prompt.
An agent identity can't call the BYO server	Agent identities are missing the `Tools.ListInvoke.All` permission for the server	A Global Administrator runs `a365 develop-mcp grant-agents-access --mcp-server-name <NAME> --agent-blueprint-id <GUID>` (use `--dry-run` to report without granting). The command isn't available in Agent 365 CLI 1.1.221; update the CLI (`dotnet tool update --global Microsoft.Agents.A365.DevTools.Cli`) and confirm it appears in `a365 develop-mcp -h`.
`evaluate` returns `Unauthorized` from `tools/list`	Wrong or expired bearer token	Reacquire the token and pass it through `A365_MCP_AUTH_TOKEN`.
`evaluate` stops after step `[2/5]`	No coding agent on `PATH`	Install GitHub Copilot CLI or Claude Code, or score the checklist yourself and rerun.
`evaluate` reports `Unknown eval engine`	Invalid `--eval-engine` value	Use `auto`, `github-copilot`, `claude-code`, or `none`.
`evaluate` reports `Failed to read existing checklist`	The checklist file is locked or malformed	Delete the checklist file to force a fresh discovery on the next run.
No servers appear under source **AI gateway**	Discovery not yet complete, or the servers aren't registered in the connected gateway	Refresh the Tools page and allow more time. Only MCP servers registered in the connected gateway are discovered.
Tools tab shows that the server doesn't support tool discovery	Individual tools aren't available to manage for this server	Block or allow the whole server (4.4.2).

Symptom	Likely cause	Fix
Blocking a server affected plugins and connectors	Blocking an MCP server also blocks dependent plugins and linked connectors	Select the server to view the plugins that use it before blocking; unblock to restore access.
Allow or block options for tooling and MCP servers aren't available	The capability might not be available in your region yet	See Agent 365 tooling servers overview.

✅ What you proved

- [ ] The Tools registry in the Microsoft 365 admin center lists plugins, skills, MCP servers, and connectors with their status, type, and publisher.
- [ ] A test bring-your-own (BYO) MCP server registered with the Agent 365 CLI appears as a request on the **Requests (preview)** tab, and is listed as **Available** in the registry only after an administrator approves it and grants consent.
- [ ] After the test MCP server is blocked, a test agent in Copilot Studio can no longer invoke it; after it is unblocked, the agent can invoke it again.
- [ ] A plugin scoped to a pilot group is available to the group member, appears to a non-member as blocked by organizational policy with **Request access**, and the request appears on the **Requests** tab for approval or rejection.
- [ ] Microsoft Defender advanced hunting returns tool-invocation records for the test MCP server.
- [ ] (Optional) MCP servers registered in a connected Azure API Management, Azure AI Gateway, or LiteLLM gateway appear in the registry under the **AI gateway** source without per-server registration.

References

- Overview of the Tools page in Microsoft 365 admin center
- Agent management roles and permissions
- How do enterprises control what agents can do?
- Advanced connector policies
- Manage plugins, skills, and MCP servers in Microsoft 365 admin center
- Build plugins for Copilot Cowork
- Manage plugins, skills, and MCP servers – Review and approve MCP requests
- Grant tenant-wide admin consent to an application
- Agent 365 tooling servers overview
- Manage Tools Gateway in Microsoft 365 admin center
- About MCP servers in Azure API Management
- AI Gateway tier overview
- exposing a REST API as an MCP server
- exposing an existing MCP server
- Bring your own (BYO) MCP server in Microsoft 365 admin center
- Agent 365 CLI develop-mcp command reference
- Install and use the Agent 365 CLI
- Add and manage tools – Set up service principal
- Advanced hunting overview
- CloudAppEvents table
- BYO MCP server article
- Connect MCP servers to developer clients

Additional resources

- Microsoft.Agents.A365.DevTools.Cli NuGet page
- New-Agent365ToolsServicePrincipalProdPublic.ps1

Runbook index · Previous: Chapter 3 – Agent identity and ownership · Next: Chapter 5 – Agent lifecycle and audit

Prerequisites - Chapter 5: Agent lifecycle and audit

> **Goal:** Prepare the accounts, access, and test assets needed for this chapter and its selected optional paths.

Apply the shared PoC preparation guidance and agree the chapter outcomes. Finish dependent build steps before their tests.

Table of contents

- Tenant readiness
- Prepare accounts and role assignments
- Prepare groups
- Audit recording
- Test assets and dependencies
- References

Tenant readiness

Documentation: Get started with Microsoft Agent 365 · Agent management in the Microsoft 365 admin center · Frontier program

Use the task-specific roles in 5.1 Required permissions. Confirm availability rather than assuming the tenant is ready.

Follow the shared tenant readiness check using the chapter permissions and feature prerequisites linked above.

Prepare accounts and role assignments

Alongside the shared setup and reviewer accounts, prepare the following scenario accounts:

Account	Purpose
Standard test user	Tests installation, availability, and the ownership transfer. No admin role.
Maker account	Creates the disposable lifecycle agent and any additional Agent Builder or Foundry assets.

Assign the roles in 5.1 Required permissions using the shared model. Verify each account can access the pages needed for its tasks.

Prepare groups

Documentation: Assign Microsoft Entra roles · Microsoft Entra built-in roles · Roles and role groups in Microsoft Purview

Performed by **Groups Administrator**.

Create or verify these PoC groups before the tasks that use them:

Group	Members
`A365-PoC-Users`	Standard test user

Check result

- Group membership matches the table, and the groups are recorded as PoC assets.

Audit recording

Documentation: Turn auditing on or off · Audit logs for Copilot and AI applications

Confirm auditing before test activity that must appear in Microsoft Purview or the Microsoft 365 connector.

Confirm that auditing is on

Performed by a member of the **Audit Manager** role group.

1. Open the Microsoft Purview portal (https://purview.microsoft.com) and go to **Audit**.
2. If a banner prompts you to start recording user and admin activity, select it.
3. Record the date and time.

Check result

- **Audit** search is available and no banner asks you to start recording.

Test assets and dependencies

Documentation: Governance and lifecycle actions for agents – Install agents and Activate agents · Agent requests in the Microsoft 365 admin center · Policy templates · Search the audit log · Audit logs for Copilot and AI applications · Audit log activities – Agent 365 activities · Agent sign-in and audit logs in Microsoft Entra · Access activity logs in Microsoft Entra · Discover AI agents in Microsoft Defender · AgentsInfo table

- Verify the required inventory using Chapter 1 – Agent discovery and inventory so the test agents are visible in **Agents** > **All agents** > **Registry**.
- Review Chapter 3 – Agent identity and ownership for owners and sponsors in Microsoft Entra. This chapter covers ownership as managed in the Microsoft 365 admin center.
- Complete Enable security for AI and Connect the Microsoft 365 connector before you run the Advanced Hunting tasks in 5.9.4.
- As the maker, create the lifecycle test agent `PoC-Lifecycle-Agent`: a shared Agent Builder or Copilot Studio agent that you're allowed to delete permanently. **Assign new owner** is only available for shared Agent Builder and Copilot Studio agents. The maker is the agent's first owner; the standard test user becomes the new owner

in 5.10.6.
- For 5.4.2 and 5.4.3, use an Agent Builder agent, because Agent Builder agents support multiple owners. If `PoC-Lifecycle-Agent` isn't an Agent Builder agent, create a separate one with Agent Builder.
- For the optional 5.3.2, use the Microsoft Foundry test agent from 1.2.2 Create and publish a Foundry test agent.
- Note the PoC start time in UTC. The audit searches in 5.9 and 5.10 start from this time.

Build guidance: 1.2 Discover Microsoft-native agents

Check result

- Each asset needed for the selected paths is prepared or has a named build task that will finish before its dependent tests.

References

- Get started with Microsoft Agent 365
- Agent management in the Microsoft 365 admin center
- Frontier program
- Assign Microsoft Entra roles
- Microsoft Entra built-in roles
- Roles and role groups in Microsoft Purview
- Turn auditing on or off
- Audit logs for Copilot and AI applications
- Governance and lifecycle actions for agents – Install agents and Activate agents
- Agent requests in the Microsoft 365 admin center
- Policy templates
- Search the audit log
- Audit log activities – Agent 365 activities
- Agent sign-in and audit logs in Microsoft Entra
- Access activity logs in Microsoft Entra
- Discover AI agents in Microsoft Defender
- AgentsInfo table
- Agent Builder

> **Ready?** Continue to the chapter. →

Chapter 5 - Agent lifecycle and audit

Pillar: Govern **What it proves:** IT can govern agents through their lifecycle from the Agent Registry in the Microsoft 365 admin center (install, activate, block, manage ownership, connect, delete, and restore) and can review the resulting records in Microsoft Purview Audit, Microsoft Entra logs, and Microsoft Defender Advanced Hunting.

Success criteria: Agree the selected outcomes in What you proved before testing.

Topics and time guide

Approximate hands-on estimates for the selected scope, with prerequisites prepared. Service waiting time is separate.

Topic	Estimated hands-on time
5.1 Required permissions	~5 min
5.2 Install and activate agents	~10-15 min
5.3 Control agent availability	~10-15 min
5.4 Manage agent ownership	~10-15 min
5.5 Govern ownerless agents	~10-15 min
5.6 Manage connected agents	~10-15 min
5.7 Manage agent instances	~10-15 min
5.8 Delete, restore, and permanently delete agents	~10-15 min
5.9 Audit the agent lifecycle	~10-15 min
5.10 Test and validation	~15-20 min
5.11 Evidence	~5-10 min
5.12 Troubleshooting	As needed

Outcome checklist · References

5.1 Required permissions

Use the shared account and role-assignment model with the task-specific permissions below.

Task	Least-privilege role	Section
Prepare the PoC installation audience group	**Groups Administrator**	Prerequisites
Install, uninstall, and activate agents; approve activation requests	**AI Administrator**	5.2, 5.10
Block and unblock agents	**AI Administrator**	5.3, 5.10
Start or stop a Microsoft Foundry agent	**AI Administrator** plus the **Azure AI Owner** role	5.3

Task	Least-privilege role	Section
Disable and enable an agent identity in Microsoft Entra (optional)	**Agent ID Administrator**	5.3
Assign a new owner, add or remove owners	**AI Administrator**	5.4, 5.10
Act on ownerless agents; create and run Agent management rules	**AI Administrator**	5.5
Connect or remove connected agents	**AI Administrator**	5.6
Block, unblock, and delete agent instances	**AI Administrator**	5.7
Delete, restore, and permanently delete agents	**AI Administrator**	5.8, 5.10
Search and export the Microsoft Purview audit log	**Audit Reader** role group in Microsoft Purview	5.9, 5.10
Review Microsoft Entra audit and sign-in logs	**Reports Reader**	5.9, 5.10
Run Advanced Hunting queries on `AgentsInfo` and `CloudAppEvents`	**Security Reader**	5.9, 5.10
Validation / read-only review of the Agent Registry, ownerless agents, connected agents, and instances	**AI Reader** or **Global Reader**	5.5, 5.6, 5.7, 5.10

In the Microsoft 365 admin center, only **AI Administrator** and **Global Administrator** can install, modify, approve, and manage agent configurations. AI Reader and Global Reader have read-only access to tools and settings. Global Administrator is a highly privileged role; use AI Administrator for the changes in this chapter.

Before you start:

- [ ] Confirm the chapter prerequisites for the selected topics.

5.2 Install and activate agents

Documentation: Governance and lifecycle actions for agents – Install agents and Activate agents · Agent requests in the Microsoft 365 admin center · Policy templates

Installing an agent makes it ready to use for the selected users without manual installation by end users. It affects the agent's availability and functionality in Copilot and in other host products, such as Outlook, Teams, or Microsoft 365. Activation is a governance step for new agents: when a user requests activation so they can create instances of an agent, the request requires AI admin approval before the requester can create instances.

5.2.1 Install an agent for users or groups

Performed by **AI Administrator**.

1. Sign in to the Microsoft 365 admin center at `https://admin.cloud.microsoft`.
2. Go to **Agents** > **All agents**, and make sure **Registry** is selected.
3. Select the **Status** filter, and then select **Available**.
4. Select an agent that isn't already installed, for example `PoC-Lifecycle-Agent`.
5. In the agent details pane, immediately under the agent's name, select **Install**.

6. In the **Deploy agent to selected users** pane, choose to install the agent for all users or for specific users or groups. For the PoC, select `A365-PoC-Users`. Select **Next**.
7. In the **Review permissions** pane, review the permissions requested for the agent. If they're acceptable, select **Grant admin consent**.
8. In the **Permissions requested** window, select **Accept**, and then select **Next**.
9. In the **Review & finish** pane, select **Finish deployment**.

Check result

- The agent is ready to use for the members of `A365-PoC-Users` without manual installation.

5.2.2 Uninstall an agent

Performed by **AI Administrator**.

Run this task only on an agent that you installed for the PoC.

1. Go to **Agents** > **All agents** > **Registry**, select the **Status** filter, and then select **Available**.
2. Select the installed agent.
3. In the agent details pane, immediately under the agent's name, select **Uninstall**. If you don't see **Uninstall**, the selected agent might not be installed.
4. In the **Remove agent** pane, select **Remove agent**, and then select **Uninstall Agent**.

Check result

- The agent is no longer installed for the users it was removed from.

5.2.3 Review and approve an activation request

Performed by **AI Administrator**.

Run this task only if an activation request is pending in **Requests**.

1. Go to **Agents** > **All agents** > **Requests**.
2. Filter **State** to **Pending activate**. New submissions appear with the status **Allow activation**, with the agent name, publisher name, status, and date requested.
3. Select the pending agent and review its description, capabilities, connected data sources, and tools.
4. Select the **Request** tab to open the request wizard.
5. Select the users and security groups that you approve for activating the agent. You can include the original requesters.
6. Apply a template:
    - **Default template** – Microsoft's template with essential security and compliance controls from Microsoft Entra, Microsoft Purview, and SharePoint.
    - **Custom template** – a template you create to apply extra policies, such as restricting external content sharing. See Policy templates.
7. Review the permissions requested by the agent, and grant admin consent if appropriate.
8. Review all configurations, and then select **Finish**.

AI Administrators aren't authorized to configure Conditional Access policies or Microsoft Entra access package policies. These actions require a highly privileged administrator role with the appropriate Microsoft Graph permissions.

Check result

- The agent becomes available for creating instances in the Teams app store, Microsoft 365 Copilot, and Copilot Studio, and the requester is notified.

- For future user additions, you only approve or reject requests by using the existing template, without repeating the full activation steps.

5.2.4 Activate an agent without a request (admin-initiated)

Performed by **AI Administrator**.

Administrators can proactively activate an agent through the Agent Registry without waiting for an activation request. Run this task only if your registry contains an agent that users create instances from.

1. Go to **Agents** > **All agents** > **Registry** and select the agent.
2. Activate the agent, and make it available to all users, specific users, or specific security groups.

Check result

- The agent is activated for the selected audience without a pending request.

5.3 Control agent availability

Documentation: Governance and lifecycle actions for agents – Block or unblock agents and Start or stop a Foundry agent · Disable agent identities in your tenant

The agent actions in the Microsoft 365 admin center are **Install and uninstall**, **Activate**, **Block and unblock**, **Delete, restore, and permanently delete**, **Start and stop** (Microsoft Foundry agents only), **Assign a new owner**, **Add an owner**, **Remove an owner**, **Publish to store**, and **Reject submission**. To stop an agent identity from receiving tokens, use **Disable** in the Microsoft Entra admin center (5.3.3).

5.3.1 Block or unblock an agent

Performed by **AI Administrator**.

1. Go to **Agents** > **All agents** and select the agent.
2. In the agent details pane, immediately under the agent's name, select **Block** or **Unblock**.
3. In the **Block agent** or **Unblock agent** pane, select **Block agent** or **Unblock agent**, and then select **Save**.

The impact depends on how the agent was created:

Agent platform	Effect of Block or Unblock
Agent Builder, Copilot Studio	Affects availability and functionality in Microsoft Copilot and in other host products, such as Outlook, Teams, or other Microsoft 365 applications.
SharePoint, Microsoft Foundry	Blocking only affects availability in Microsoft Copilot Chat.
Researcher, Analyst	The **Edit users** panel is disabled. Use **Block** to manage availability for the entire tenant.

Check result

- No user in the organization can use the blocked agent, within the scope shown in the table.

5.3.2 Start or stop a Microsoft Foundry agent (optional)

Performed by **AI Administrator** with the **Azure AI Owner** role.

Stop and **Start** operate on individual deployments by deallocating or provisioning Azure compute resources. These actions affect your underlying Azure infrastructure, not just how the agent is used in your organization.

1. Go to **Agents** > **All agents**, and find and select the Microsoft Foundry test agent.
2. If required, in the agent details pane select **Add role** to add the **Azure AI Owner** role.
3. Select **Stop**.
4. After the check, select **Start**.

Check result

- **Stop** deallocates and **Start** provisions the Azure compute resources for the deployment.

5.3.3 Disable the agent identity in Microsoft Entra (optional)

Performed by **Agent ID Administrator**.

Disabling prevents an agent identity from receiving tokens and authenticating, while the agent identity and its metadata stay in your tenant. Run this task only on a test agent identity, as in 3.11.1 Disable and re-enable an individual agent identity.

1. Sign in to the Microsoft Entra admin center at `https://entra.microsoft.com`.
2. Go to **Entra ID** > **Agents** > **Agent identities**.
3. Select the test agent identity, and then select **Disable**.
4. After the check, enable the agent identity again from its overview.

Check result

- While disabled, the agent identity can't receive tokens or authenticate.
- After step 4, the agent identity is enabled again.

5.4 Manage agent ownership

Documentation: Governance and lifecycle actions for agents – Assign new owner, Add an owner, Remove an owner

Action	Applies to	Result
Assign new owner	Shared Agent Builder and Copilot Studio agents that are ownerless or active	The new owner gets full edit and delete permissions, plus access to any files the previous owner uploaded. The previous owner loses all access, including read rights.
Add owner	Agent Builder agents that already have an owner	All owners have the same rights to edit, share, manage, and maintain the agent; there are no primary or secondary owners. Only individual users, not groups. **Created by** doesn't change.
Remove owner	Agent Builder agents with multiple owners	The last remaining owner can't be removed.

5.4.1 Assign a new owner

Performed by **AI Administrator**.

1. Go to **Agents** > **All agents**.
2. Use the **Platform** filter to show Agent Builder or Copilot Studio agents.
3. Select the agent that you want to reassign.
4. In the agent details pane, immediately under the agent's name, select **Assign new owner**.
5. In the **Assign a new owner** pane, enter a new owner from your organization, and then select **Assign**.

Check result

- The new owner has full edit and delete permissions and access to the files the previous owner uploaded.
- The previous owner has lost all access, including read rights.

5.4.2 Add an owner to an Agent Builder agent

Performed by **AI Administrator**.

1. Go to **Agents** > **All agents**, use the **Platform** filter to show Agent Builder agents, and select the Agent Builder test agent.
2. On the **Details** tab of the agent details pane, select the user link next to **Owner**.
3. In the **Owners** pane, select **Add owner**.
4. Search for and select the standard test user, and then select **Add**.

Check result

- The owner list change is reflected for the agent in Microsoft Agent 365 and in Agent Builder.
- The **Created by** value is unchanged.

5.4.3 Remove an owner from an Agent Builder agent

Performed by **AI Administrator**.

1. On the **Details** tab of the agent details pane of the Agent Builder test agent, select the user link next to **Owner**.
2. In the **Owners** pane, select the standard test user that you added in 5.4.2.
3. Select **Remove owner**, review the change, and then select **Remove**.

Check result

- The standard test user no longer appears in the owner list.
- The last remaining owner can't be removed. Add another owner before you remove the current owner. For an ownerless agent, use **Assign new owner** (5.4.1).

5.5 Govern ownerless agents

Documentation: Agent registry – Agents without owners · Agent settings – Agent management rules · Governance and lifecycle actions for agents – Assign new owner · Reassign ownership of orphaned agents with the Power Platform API

Shared agents can become ownerless when you delete the user who created them from the organization. The Agent Registry shows the number of ownerless shared agents, provides a one-click filter to isolate them, and updates the count when you hard delete a user. 1.3.3 Find agents without owners shows the same view during

discovery.

5.5.1 Identify and triage ownerless agents

Performed by **AI Administrator** (or **AI Reader** for review only).

1. Go to **Agents** > **All agents** > **Registry** and clear any previous filter.
2. Select the **Agents without owners** card. The agent list is filtered by **Publisher type** and **Owner** to show shared agents missing an owner.
3. Select **Export** to keep a copy of the list for the evidence pack.
4. Select each agent to open the agent details pane and review its details.

Check result

- The filtered list shows the shared agents without a valid owner, and the export is saved.

5.5.2 Decide: reassign, block, or delete

Performed by **AI Administrator**.

For each ownerless agent from 5.5.1, agree the action with the customer, and then take it:

Action	Section
Assign new owner (shared Agent Builder and Copilot Studio agents)	5.4.1
Block	5.3.1
Delete	5.8.1

Check result

- Every ownerless agent from 5.5.1 has been reassigned, blocked, or deleted, and the action is recorded next to the agent in the export.

5.5.3 Automate with Agent management rules

Performed by **AI Administrator**.

Agent management rules apply governance and lifecycle controls across agents by using bulk administrative actions. You identify agents that meet defined conditions, review the impacted agents before running the rule, and apply the action across the affected agents. Supported rule-based actions:

- **Install Microsoft agents** – install Microsoft-published agents for all users in one bulk action.
- **Reassign ownerless agents created with Agent Builder to manager** – transfer ownership in bulk to the manager of the previous owner, based on the Microsoft Entra ID hierarchy. This rule only supports agents created with Microsoft Copilot Agent Builder.
- **Block ownerless agents without usage** – a custom rule that blocks multiple ownerless agents that have no usage.
- **Apply a template** – apply security policies to existing agents. The rule evaluates agents in the registry that have an agent identity; it doesn't apply to agent blueprints or AI teammates. It's a one-time bulk operation, not a scheduled rule.
- **Reject agent publish requests older than a specified number of days**.

All rules are created from **Agents** > **Settings** > **Agent management rules** > **Add rule**. To apply a template to existing PoC agents:

1. In the Microsoft 365 admin center, go to **Agents** > **Settings** > **Agent management rules**, and then select **Add rule**.
2. Select **Apply template** as the rule action.
3. Define the criteria that determine which agent instances the rule applies to.
4. Choose a preexisting template, or select policies from the **Custom** section. Review the selected policies, and then save the rule.
5. In the rule details pane, select only the PoC test agents from the list. Run the rule to apply the template.

Check result

- You reviewed the impacted agents before running the rule, and the template was applied only to the selected PoC test agents.

If an agent resides in an environment configured with Power Platform IP firewall in active enforcement mode, admin actions initiated from the Microsoft 365 admin center fail. Run the action directly against the Power Platform API: Reassign ownership of orphaned agents or Quarantine noncompliant agents.

5.6 Manage connected agents

Documentation: Manage connected agents in the Microsoft 365 admin center

Connected agents are other Copilot-enabled agents that a primary agent can invoke during a conversation. When a user's request falls within a connected agent's domain, the primary agent can delegate the request to that agent.

- You can add connected agents to **Researcher** and **Sales**. For other primary agents, you can only view the connections configured by the agent developer.
- Primary agents can connect to declarative agents and to agents that are available over the Agent2Agent (A2A) protocol. An agent can connect to up to 10 other agents.
- You can't remove connected agents configured by the agent developer.
- Both the primary agent and each connected agent must be acquired for the user.
- If a connected agent is external to your organization, the primary agent can include information from its context in the prompt it sends to the connected agent. Review the connected agent and your organization's data policies before making it available to users.

Tasks 5.6.2 to 5.6.4 are optional.

5.6.1 Review the connected agents of an agent

Performed by **AI Administrator** (or **AI Reader** for review only).

1. Go to **Agents** > **All agents** and select an agent.
2. Select the **Connected Agents** tab.

Check result

- The tab lists the agents that provide information and answers when users interact with the primary agent.

5.6.2 Connect agents to Researcher or Sales

Performed by **AI Administrator**.

1. Go to **Agents** > **All agents**.
2. Select **Researcher** or **Sales**, and then select the **Connected Agents** tab.

3. Select **+ Connect agents**.
4. In the **Select agent to connect** pane, use the search bar to find a PoC test agent, and select the check box next to it.
5. Select **Save**.

Check result

- The connected PoC test agent appears on the **Connected Agents** tab.

5.6.3 Make connected agents available to users

Performed by **AI Administrator**.

1. On the **Connected Agents** tab, select the connected agent. The agent's details page opens.
2. Install the agent for `A365-PoC-Users` (5.2.1).
3. Install the primary agent for the same users if it isn't installed.

Check result

- The primary agent and the connected agent are acquired for the members of `A365-PoC-Users`, so the primary agent can invoke the connected agent.

5.6.4 Remove or reset connected agents

Performed by **AI Administrator**.

1. To remove one connection, on the **Connected Agents** list select an agent that an admin added, and then select **Remove**.
2. To remove all admin-added connections, select **Reset to default**. This restores the connections configured by the agent developer and also removes admin-added connections that existed before the PoC.

Check result

- The connection that you added in 5.6.2 no longer appears on the **Connected Agents** tab.

5.7 Manage agent instances

Documentation: Manage agent instances in Microsoft 365 admin center

After an admin or AI admin activates an agent (5.2.3 or 5.2.4), the requesters can create instances of it. The Microsoft 365 admin center provides a centralized view for managing these instances. Run this section only if your registry contains agents with the **AI teammate** tag.

5.7.1 View instances

Performed by **AI Administrator** (or **AI Reader** for review only).

1. Go to **Agents** > **All agents** > **Registry**.
2. Find agents with the **AI teammate** tag and note the number of instances created for each.
3. Select the agent to open the agent details flyout, and then select **See details** to list all instances created under that agent.

Check result

- From the instance list you can manage individual instances and review the security and compliance status of each instance.

5.7.2 Block or unblock an instance

Performed by **AI Administrator**.

1. Go to **Agents** > **All agents** and select an agent from the list.
2. Select the **Instances** tab to see all instances created by that agent.
3. Select a test instance and choose **Block**.
4. To restore functionality, select **Unblock** for the instance.

Check result

- Blocking an instance stops it and any actions it's performing.

5.7.3 Delete an instance

Performed by **AI Administrator**.

1. Go to **Agents** > **All agents**, filter the list by setting **Agent template** to **Yes**, and select the agent.
2. Select the **Instances** tab.
3. Select the test instance you want to delete, select **Delete**, and confirm the deletion when prompted.
4. Notify the owner of the deletion.

Check result

- Once deleted, the instance no longer appears in the list.
- After 30 days, all instance accounts and data are permanently deleted. Audit logs are kept.

5.8 Delete, restore, and permanently delete agents

Documentation: Governance and lifecycle actions for agents – Delete agents

When you delete an agent in the Microsoft 365 admin center, the agent is soft deleted. A soft-deleted agent is unavailable to its maker and end users, but its properties and associated resources are retained for 30 days, including the agent ID, metadata, channels, ownership, and connections. During the 30-day retention period you can restore the agent or permanently delete it. If you do neither, Microsoft 365 permanently deletes it automatically when the retention period ends. Soft-delete, restore, and permanent-delete events are recorded in the audit log with information about the system that initiated the action.

5.8.1 Delete an agent (soft delete)

Performed by **AI Administrator**.

1. Go to **Agents** > **All agents** and select the agent.
2. In the agent details pane, select **Delete agent**.
3. In the confirmation pane, select **Delete**.

Check result

- The agent is unavailable to its maker and end users.
- The agent can be selected under **Agents** > **All agents** > **Deleted** during the 30-day retention period.

5.8.2 Restore a deleted agent

Performed by **AI Administrator**.

1. Go to **Agents** > **All agents**, and then select **Deleted**.
2. Select the agent.
3. In the agent details pane, select **Restore agent**, and then in the confirmation dialog select **Restore**.

Check result

- The agent returns to an active state and is available to its maker and end users with its previous configuration.
- Restore is only possible while the agent is soft deleted and within its 30-day retention period.

5.8.3 Permanently delete an agent

Performed by **AI Administrator**.

Permanently deleting an agent removes the agent and its associated resources and can't be undone. You can permanently delete only an agent that's already soft deleted.

1. Go to **Agents** > **All agents**, and then select **Deleted**.
2. Select the agent.
3. In the agent details pane, select **Permanently delete**, and then in the confirmation dialog select **Delete**.

Check result

- The agent and its associated resources are removed, and the action can't be undone.

5.9 Audit the agent lifecycle

Documentation: Search the audit log · Audit logs for Copilot and AI applications · Audit log activities – Agent 365 activities · Agent sign-in and audit logs in Microsoft Entra · Access activity logs in Microsoft Entra · Discover AI agents in Microsoft Defender · AgentsInfo table

Microsoft Purview Audit is turned on in Audit recording. If auditing was turned on within the last 180 days, the search date range can't start before the date auditing was turned on.

5.9.1 Search admin actions on agents in Purview Audit

Performed by **Audit Reader**.

1. Sign in to the Microsoft Purview portal at `https://purview.microsoft.com` and select the **Audit** solution card.
2. On the **Search** page, set **Date and time range (UTC)** to cover the lifecycle actions, starting at the PoC start time.
3. In **Users**, select the test admin account that performed the actions.
4. Optionally, enter `PoC-Lifecycle-Agent` in **Keyword Search**.
5. Enter a **Search name**, for example `PoC-Ch5-Admin-Actions`, and run the search.
6. When the search job completes, open it. Select an activity to see its details in the fly-out window.
7. Select **Export** on the command bar to export the search job items to a .csv file.

Check result

- The soft-delete, restore, and permanent-delete events for the test agent are in the results, with information about the system that initiated the action.

5.9.2 Search agent interactions in Purview Audit

Performed by **Audit Reader**.

Run two searches: one for the Copilot interaction record types and one for the Agent 365 activity operation names.

1. On the Microsoft Purview **Audit** > **Search** page, set the date range to cover the test interactions, and in **Users** select the standard test user.
2. Search 1: in **Record types**, select `CopilotInteraction`. For custom-built or third-party AI applications deployed and registered in your organization, also select `ConnectedAIAppInteraction`. Enter a search name, run the search, and export the results.
3. Search 2: clear **Record types**, and in **Activities - operations names** enter the Agent 365 activities that are logged when an agent is invoked or performs actions or tool calls: `AIExecuteTool,AIInvokeAgent,AIInferenceCall,AIGuardrail`. Enter a search name, run the search, and export the results.
4. In the exports, filter the `AgentId` or `AgentName` property for the test agent. To filter on `AppIdentity`, export the results first and filter offline.

Check result

- The exports contain the standard test user's interactions with the test agent, with properties such as `AgentId`, `AgentName`, `AppHost`, and `AccessedResources` (including `SensitivityLabelId`, `PolicyDetails`, and `Status` where applicable).

5.9.3 Review Entra audit and sign-in logs for the agent identity

Performed by **Reports Reader**.

Run this task only if the test agent has an agent identity.

1. Sign in to the Microsoft Entra admin center at `https://entra.microsoft.com`.
2. Go to **Entra ID** > **Agents** > **Agent identities** and find the test agent's identity.
3. Go to **Entra ID** > **Monitoring & health** > **Audit logs** and adjust the filter for the PoC date range. Agent identity activities appear as service principal events, such as **Add service principal**, **Update service principal**, and **Delete service principal**, with the `agentType` value `agenticAppInstance`.
4. Go to **Entra ID** > **Monitoring & health** > **Sign-in logs**, and use the filters **Is Agent** = **Yes** and **Agent type** = **Agent Identity**. Because agents can sign in with either user-delegated or app-only permissions, check each of the sign-in log types.

To retrieve agent identity sign-in events by using Microsoft Graph:

```
GET https://graph.microsoft.com/beta/auditLogs/signIns?$filter=signInEventTypes/any(t: t eq 'servicePrincipal')
and agent/agentType eq 'agenticAppInstance'
```

Check result

- Audit events for the agent identity show `agentType = agenticAppInstance` on the `initiatedBy`, `performedBy`, or `targetResources` fields.
- Sign-in events for the agent identity are returned by the sign-in log filters or the Graph request.

5.9.4 Track lifecycle state in Defender Advanced Hunting

Performed by **Security Reader**.

The `AgentsInfo` table stores multiple snapshots of each agent over time; use `arg_max(Timestamp, *)` to return the latest state of each agent. `LifecycleStatus` is the agent's current operational state in the tenant, with the possible values `Active`, `Blocked`, `Uninstalled`, and `Deleted`.

1. Sign in to the Microsoft Defender portal at `https://security.microsoft.com`.
2. Select **Investigation & response** > **Hunting** > **Advanced hunting**.
3. Run the queries below.

Latest record for each agent:

```
AgentsInfo
| summarize arg_max(Timestamp, *) by AgentId
| where LifecycleStatus != "Deleted"
```

All recorded snapshots of the test agent:

```
AgentsInfo
| where Name has "PoC-Lifecycle-Agent"
| project Timestamp, AgentId, Name, Platform, LifecycleStatus, PublishedStatus, Availability, Owners,
EntraAgentID
| sort by Timestamp asc
```

`LifecycleStatus` changes between snapshots:

```
AgentsInfo
| where Timestamp > ago(30d)
| project Timestamp, AgentId, Name, LifecycleStatus
| sort by AgentId asc, Timestamp asc
| extend PrevAgentId = prev(AgentId), PrevStatus = prev(LifecycleStatus)
| where AgentId == PrevAgentId and LifecycleStatus != PrevStatus
| project Timestamp, Name, PrevStatus, LifecycleStatus
| sort by Timestamp desc
```

`CloudAppEvents` contains Agent 365 observability data for AI agent activity, including agent actions, tool invocations, and data access events:

```
CloudAppEvents
| where Timestamp between (datetime(<START>) .. datetime(<END>))
| where RawEventData has "<test agent name or AgentId>"
| project Timestamp, Application, ActionType, AccountDisplayName, AccountObjectId, RawEventData
| sort by Timestamp desc
```

Check result

- The snapshots of the test agent show the `LifecycleStatus` values recorded during the test walk.

5.10 Test and validation

Sections 5.2 to 5.8 describe each action. This section runs them in order on `PoC-Lifecycle-Agent`. After each step, record the UTC time and a screenshot of the agent details pane. Collect the Microsoft Purview Audit records (5.9.1, 5.9.2) and the `AgentsInfo` snapshots (5.9.4) for all steps in 5.10.10.

Step	Action	Documented outcome
5.10.2	Install	Agent ready to use for `A365-PoC-Users`
5.10.3	Block	No user can use the agent
5.10.4	User tries to use the agent	Agent not available
5.10.5	Unblock	Agent available again
5.10.6	Assign new owner	New owner has full access; previous owner loses all access
5.10.7	Delete	Soft deleted, unavailable to maker and end users, audit record
5.10.8	Restore	Active state with previous configuration, audit record
5.10.9	Permanently delete	Agent and resources removed, can't be undone, audit record

5.10.1 Prepare the test agent and record a baseline

Performed by **AI Reader**, **Security Reader**, and the **standard test user**.

1. As AI Reader, in **Agents** > **All agents** > **Registry**, search for `PoC-Lifecycle-Agent` and capture the agent details pane, including the owner.
2. As the standard test user, check whether the agent is already available to you.
3. As Security Reader, run the "All recorded snapshots of the test agent" query from 5.9.4 and save the result.

Expected result

- You have a baseline of the agent's details, its owner (the maker), and its `AgentsInfo` snapshots before the walk.

5.10.2 Install the agent for the test group

Performed by **AI Administrator**, then the **standard test user**.

1. Install the agent for `A365-PoC-Users` (5.2.1).
2. As the standard test user, open Microsoft 365 Copilot and start a conversation with the agent.

Expected result

- The agent is ready to use for the standard test user without manual installation.
- Audit evidence: a `CopilotInteraction` record for the standard test user's conversation (5.9.2).

5.10.3 Block the agent

Performed by **AI Administrator**.

1. Block the agent (5.3.1).

Expected result

- The agent is blocked for the organization.
- Audit evidence: an `AgentsInfo` snapshot of the test agent with `LifecycleStatus = Blocked` (5.9.4).

5.10.4 Confirm that the user can't use the blocked agent

Performed by the **standard test user**.

1. Try to open or invoke the agent in Microsoft 365 Copilot.
2. Try to open the agent in Outlook and Teams.

Expected result

- The agent isn't available to the standard test user in Microsoft Copilot or in other host products such as Outlook and Teams (Agent Builder and Copilot Studio agents).

5.10.5 Unblock the agent

Performed by **AI Administrator**, then the **standard test user**.

1. Unblock the agent (5.3.1).
2. As the standard test user, start a new conversation with the agent.

Expected result

- The agent is unblocked, and the standard test user can use it again.
- Audit evidence: a new `CopilotInteraction` record for the standard test user's conversation (5.9.2).

5.10.6 Reassign the owner

Performed by **AI Administrator**, then the maker and the **standard test user**.

1. Assign the standard test user as the new owner of the agent (5.4.1).
2. As the maker (the previous owner), try to open the agent.
3. As the standard test user (the new owner), open the agent for editing.

Expected result

- The standard test user has full edit and delete permissions and access to any files the maker uploaded.
- The maker has lost all access, including read rights.

5.10.7 Delete the agent

Performed by **AI Administrator**, then the **standard test user**.

1. Delete the agent (5.8.1).
2. As the standard test user, try to use the agent.

Expected result

- The agent is unavailable to its owner and end users and can be selected under **Deleted**.
- Audit evidence: the soft-delete event in Microsoft Purview Audit (5.9.1), and an `AgentsInfo` snapshot with `LifecycleStatus = Deleted` (5.9.4).

5.10.8 Restore the agent

Performed by **AI Administrator**, then the **standard test user**.

1. Restore the agent (5.8.2).
2. Compare the agent details pane with the screenshot from 5.10.6.

3. As the standard test user, start a conversation with the agent.

Expected result

- The agent is back in an active state and available to its owner and end users with its previous configuration, including ownership, channels, and connections.
- Audit evidence: the restore event in Microsoft Purview Audit (5.9.1), and an `AgentsInfo` snapshot with `LifecycleStatus = Active` (5.9.4).

5.10.9 Permanently delete the agent

Performed by **AI Administrator**.

1. Delete the agent again (5.8.1).
2. Permanently delete it from the **Deleted** view (5.8.3).

Expected result

- The agent and its associated resources are removed, and the action can't be undone.
- Audit evidence: the second soft-delete event and the permanent-delete event in Microsoft Purview Audit (5.9.1).

5.10.10 Reconcile the audit trail

Performed by **Audit Reader**, **Security Reader**, and **Reports Reader**.

1. As Audit Reader, run 5.9.1 and 5.9.2 for the full walk window and export the results.
2. As Security Reader, run the "All recorded snapshots of the test agent" and "`LifecycleStatus` changes between snapshots" queries from 5.9.4 and export the results.
3. As Reports Reader, if the test agent has an agent identity, run 5.9.3 and download the Microsoft Entra audit and sign-in records.
4. Build one timeline: step, UTC time, admin action, Microsoft Purview record, `LifecycleStatus`.

Expected result

- The soft-delete, restore, and permanent-delete events of the test agent each have a Microsoft Purview audit record.
- The timeline shows `LifecycleStatus = Blocked` after 5.10.3 and `Deleted` after 5.10.7.

5.11 Evidence

- Export of **Agents** > **All agents** > **Registry** before and after the PoC, and the export of **Agents without owners** with the action taken for each agent (5.5.1, 5.5.2).
- Screenshots of the agent details pane after each step of the test walk (5.10.1 to 5.10.9), with UTC time.
- Screenshot of the **Deleted** view showing the test agent (5.10.7).
- Screenshot of the activation wizard with the selected template, if you ran 5.2.3 or 5.2.4.
- Screenshots of the Foundry agent after **Stop** and **Start** (5.3.2) and of the disabled agent identity (5.3.3), if you ran these tasks.
- Screenshots of the **Owners** pane (5.4.2, 5.4.3) and of the **Assign a new owner** pane (5.4.1).
- Screenshot of the Agent management rule and its impacted agents, if you ran 5.5.3.
- Screenshots of the **Connected Agents** tab before and after changes, if you ran 5.6.
- Screenshot of the **Instances** tab, if you ran 5.7.
- Microsoft Purview Audit exports: admin actions (5.9.1) and agent interactions (5.9.2) for the full walk window.

- Microsoft Entra audit log and sign-in log downloads for the agent identity (5.9.3), if the test agent has one.
- Advanced Hunting result exports for the `AgentsInfo` queries (5.9.4).
- The reconciled timeline from 5.10.10.

5.12 Troubleshooting

Symptom	Likely cause	Fix
You can view agents but can't install, modify, approve, or manage them	Signed in with a read-only role (AI Reader, Global Reader, Security Reader, Reports Reader)	Use an account with **AI Administrator**.
An admin action fails and isn't applied to the agent	The agent's environment uses Power Platform IP firewall in active enforcement mode	In the Power Platform admin center, go to **Security** > **Identity and access** > **IP firewall**, select the environment, and check the **Advanced** tab. Run the action against the Power Platform API instead.
Uninstall isn't shown	The selected agent might not be installed	Install it first (5.2.1).
You can't reassign ownership	Ownership reassignment is only available for shared Agent Builder and Copilot Studio agents	Use a shared Agent Builder or Copilot Studio agent.
You can't remove an owner	The owner is the last remaining owner	Add another owner first (5.4.2).
Edit users is disabled for Researcher or Analyst	The **Edit users** panel is disabled for these agents	Use **Block** to manage availability for the entire tenant.
A blocked SharePoint or Microsoft Foundry agent is still available outside Copilot Chat	Blocking these agents only affects availability in Microsoft Copilot Chat	No fix needed; this is the documented scope of **Block** for these agents.
Stop or **Start** isn't possible for a Foundry agent	The **Azure AI Owner** role is missing	Select **Add role** in the agent details pane to add the **Azure AI Owner** role.
Restore agent isn't possible	The agent was permanently deleted, or the 30-day retention period has ended	The action can't be undone.
Microsoft Purview Audit search returns no results for older activity	The date range starts before auditing was turned on	Set the start date on or after the date auditing was turned on.
Microsoft Purview Audit search job takes a long time	Broad search scope in a large tenant	Narrow the date range and users. Broadly scoped search jobs in large tenants may take up to 48 hours.
No `CopilotInteraction` records for a custom or third-party AI app	Interactions with custom-built or third-party apps registered in the organization use a different record type	Also search `ConnectedAIAppInteraction`.
A query on `AIAgentsInfo` fails	`AIAgentsInfo` is transitioning to `AgentsInfo`	Use the `AgentsInfo` table.
Agents without owners count doesn't change after deleting a user	The count updates when a user is hard deleted	Check again after the user is hard deleted.

✅ What you proved

- [ ] The test agent is installed for `A365-PoC-Users` and is ready to use for the standard test user without a manual install.

- [ ] After **Block**, the standard test user can't use the test agent. After **Unblock**, the standard test user can use it again.
- [ ] **Assign new owner** transfers the test agent to a new owner, and the previous owner loses all access.
- [ ] Each agent under **Agents without owners** has been reassigned, blocked, or deleted.
- [ ] The test agent is deleted, restored from the **Deleted** view with its previous configuration, and then permanently deleted.
- [ ] Microsoft Purview Audit returns the soft-delete, restore, and permanent-delete records for the test agent.
- [ ] `AgentsInfo` in Defender Advanced Hunting returns `LifecycleStatus` values `Blocked` and `Deleted` for the test agent after the corresponding steps.

References

- Governance and lifecycle actions for agents – Install agents and Activate agents
- Agent requests in the Microsoft 365 admin center
- Policy templates
- Disable agent identities in your tenant
- Agent registry – Agents without owners
- Agent settings – Agent management rules
- Reassign ownership of orphaned agents with the Power Platform API
- Quarantine noncompliant agents
- Manage connected agents in the Microsoft 365 admin center
- Manage agent instances in Microsoft 365 admin center
- Search the audit log
- Audit logs for Copilot and AI applications
- Audit log activities – Agent 365 activities
- Agent sign-in and audit logs in Microsoft Entra
- Access activity logs in Microsoft Entra
- Discover AI agents in Microsoft Defender
- AgentsInfo table

Runbook index · Previous: Chapter 4 – Tools and MCP server governance · Next: Chapter 6 – Conditional Access and least privilege

Prerequisites - Chapter 6: Conditional Access and least privilege

> **Goal:** Prepare the accounts, access, and test assets needed for this chapter and its selected optional paths.

Apply the shared PoC preparation guidance and agree the chapter outcomes. Finish dependent build steps before their tests.

Table of contents

- Tenant readiness
- Prepare accounts and role assignments
- Prepare groups
- Test assets and dependencies
- References

Tenant readiness

Documentation: Get started with Microsoft Agent 365 · Agent management in the Microsoft 365 admin center · Frontier program

Use the task-specific roles in 6.1 Required permissions. Confirm availability rather than assuming the tenant is ready.

Follow the shared tenant readiness check using the chapter permissions and feature prerequisites linked above.

Prepare accounts and role assignments

Alongside the shared setup and reviewer accounts, prepare the following scenario accounts:

Account	Purpose
Standard test user	Sponsors the approved agent and runs the access-package tests. No admin role.
PoC approver	Approves access package requests. No admin role.

Assign the roles in 6.1 Required permissions using the shared model. Verify each account can access the pages needed for its tasks.

Prepare groups

Documentation: Assign Microsoft Entra roles · Microsoft Entra built-in roles · Roles and role groups in Microsoft Purview

Performed by **Groups Administrator**.

Create or verify these PoC groups before the tasks that use them:

Group	Members
`PoC-Agent-Resource-Access`	None at the start

Use assigned (not dynamic) membership for `PoC-Agent-Resource-Access`, with no members before the access-package test.

Check result

- Group membership matches the table, and the groups are recorded as PoC assets.

Test assets and dependencies

Documentation: Conditional Access for agents · Target agents in Conditional Access policies · Secure autonomous agents with Conditional Access · Conditional Access report-only mode · Access packages for agent identities · Create and manage a catalog · Change resource roles for an access package · Change lifecycle settings for an access package · Request access packages on behalf of other identities · View, add, and remove assignments · accessPackageAssignmentRequest

- Verify the required inventory using Chapter 1 – Agent discovery and inventory so you know which agents exist and which ones must keep working.
- Verify the required identity setup using Chapter 3 – Agent identity and ownership. Sponsors must be assigned (3.5), and you need the sign-in log filters (3.8) and the Risky agents report (3.9). Sponsor transitions with Lifecycle Workflows are covered in 3.10 Keep sponsorship continuous with Lifecycle Workflows.
- Prepare a lab agent identity blueprint with a client secret you control, for example with 2.8 Create the agent blueprint with a365 setup. Then, as an **Agent ID Administrator**, create the three agent identities below from that blueprint: open https://entra.microsoft.com > **Entra ID** > **Agents** > **Agent identities** > **New agent identity (Preview)**, select the lab blueprint under **Agent blueprint**, enter the name in **Agent identity name**, and complete the wizard with **Create**. Record each agent identity's ID. For agent identities, the object ID and app ID are always the same value.

Test object	Purpose	Approved (excluded from default deny)
`PoC-Agent-Approved`	Control agent for all tests; receives the access package	Yes
`PoC-Agent-Risk`	Marked as compromised in the optional risk test (6.9)	Yes
`PoC-Agent-Unapproved`	Not on the allow list	No
`PoC-Agent-Resource-Access` (security group prepared on this page)	Resource granted through the access package	–
PoC approver (test user prepared on this page)	Approves access package requests	–

- Set the standard test user as a sponsor of `PoC-Agent-Approved`, as in 3.5.1 Assign owners and sponsors to an agent identity. The standard test user is the sponsor in 6.6.3 and uses the published agent in 6.8.3.
- For 6.7, have a member of your organization submit a test agent for admin approval, so that it appears in **Agents** > **All agents** > **Requests** in the Microsoft 365 admin center with state **Pending review**.
- Security defaults must be off in the tenant. Conditional Access policies don't apply while security defaults are enabled.

Build guidance: 6.7 Apply Conditional Access and custom security attributes at publish time

Check result

- Each asset needed for the selected paths is prepared or has a named build task that will finish before its dependent tests.

References

- Get started with Microsoft Agent 365
- Agent management in the Microsoft 365 admin center
- Frontier program
- Assign Microsoft Entra roles
- Microsoft Entra built-in roles
- Roles and role groups in Microsoft Purview
- Conditional Access for agents
- Target agents in Conditional Access policies
- Secure autonomous agents with Conditional Access
- Conditional Access report-only mode
- Access packages for agent identities
- Create and manage a catalog
- Change resource roles for an access package
- Change lifecycle settings for an access package
- Request access packages on behalf of other identities
- View, add, and remove assignments
- accessPackageAssignmentRequest

> **Ready?** Continue to the chapter. →

Chapter 6 - Conditional Access and least privilege

Pillar: Secure **What it proves:** Agents get only the access you approve. Conditional Access refuses a token to agent identities you haven't approved or that ID Protection flags as high risk, and the denial is visible in the sign-in log. Access packages grant resource access to agents with approval, and the access is removed automatically when it expires.

Success criteria: Agree the selected outcomes in What you proved before testing.

Topics and time guide

Approximate hands-on estimates for the selected scope, with prerequisites prepared. Service waiting time is separate.

Topic	Estimated hands-on time
6.1 Required permissions	~5 min
6.2 Allow only approved agent identities (default deny)	~15-20 min
6.3 Block high-risk agent identities	~10-15 min
6.4 Protect agents that act as users (agent user accounts) (when agent user accounts are in scope)	~15-20 min
6.5 Use ID Protection signals for agents	~10-15 min
6.6 Grant time-bound access with access packages	~20-30 min
6.7 Apply Conditional Access and custom security attributes at publish time (policy templates)	~15-20 min
6.8 Test and validation	~15-20 min
6.9 Test the risk-based block with the agent token flow (optional)	~15-25 min
6.10 Evidence	~5-10 min
6.11 Troubleshooting	As needed

Outcome checklist · References

6.1 Required permissions

Use the shared account and role-assignment model with the task-specific permissions below.

Task	Least-privilege role	Section
Create the three lab agent identities from the lab blueprint	Agent ID Administrator	Before you start
Review agent sign-ins and Conditional Access results in the sign-in logs	Reports Reader	6.2.1, 6.2.4, 6.8.1, 6.8.3, 6.9.4

Task	Least-privilege role	Section
Create, edit, and enable Conditional Access policies for agent identities and agent users	Conditional Access Administrator	6.2, 6.3, 6.4
Select custom security attributes in a Conditional Access policy	Conditional Access Administrator and Attribute Definition Reader	6.2.3
Create attribute definitions / assign or remove attribute values on agent identities	Attribute Definition Administrator / Attribute Assignment Administrator	6.2.3
View the Risky agents report and agent risk detections	Security Reader	6.5.1
Confirm compromise, confirm safe, or dismiss risk (Microsoft Entra admin center or Microsoft Graph)	Security Administrator	6.5.2, 6.9.2, 6.9.5
Configure diagnostic settings to export risk and sign-in data (optional)	Security Administrator	6.5.3
Query the exported data in the Log Analytics workspace (optional)	Permission to access data in the Log Analytics workspace	6.5.3
Create catalogs, access packages, and policies, and assign or remove access	Identity Governance Administrator	6.6, 6.8.2
Add API permissions or directory roles as resource roles (optional)	Global Administrator	6.6.2
Request an access package for an agent in My Access	Owner or sponsor of the agent identity (no admin role)	6.6.3
Approve access package requests	Approver named in the policy (no admin role)	6.6.3
Create a Conditional Access policy that a template can select	Conditional Access Administrator	6.7.1
Create a custom policy template with Conditional Access and custom security attribute policies, and apply it when publishing an agent	Global Administrator with Attribute Assignment Administrator	6.7.2, 6.7.3
Verify custom security attribute values on the published agent	Agent ID Administrator together with Attribute Assignment Reader	6.8.3
Run the community reference script (optional)	Owner of the lab agent identity blueprint	6.9.1, 6.9.3
Validation / read-only review	Global Reader (policies, access packages, group members), Reports Reader (sign-in logs), Security Reader (risky agents)	6.6.4, 6.8, 6.9, 6.10

Notes:

- By default, Global Administrator and other administrator roles can't read, define, or assign custom security attributes. Assign the attribute roles explicitly.
- To keep the PoC least-privileged, the core path in 6.6 uses only a security group, because adding OAuth API permissions or directory roles to an access package marks the catalog as privileged and requires a Global Administrator.
- The role requirements for policy templates are described in 6.7.

Before you start:

- [ ] Confirm the chapter prerequisites for the selected topics.

6.2 Allow only approved agent identities (default deny)

Documentation: Conditional Access for agents · Target agents in Conditional Access policies · Secure autonomous agents with Conditional Access · Conditional Access report-only mode

Conditional Access evaluates the **token subject**. It applies when Microsoft Entra ID issues or refreshes an access token. Agents can use three access patterns, and each one needs its own policy:

If the agent	Access pattern	Token subject / policy target	Covered in
Accesses resources for a signed-in user	On-behalf-of (delegated)	The **user**. Target users and groups with your existing user policies.	Out of scope for agent policies
Accesses resources with its own identity and no signed-in user	App-only (autonomous)	**Agent identity** or **agent identity blueprint**	6.2, 6.3
Accesses resources through its own user account (mailbox, chat, team member)	Agent user	**Agent's user account**	6.4

A policy that targets agent identities doesn't apply to the agent's user account, and the reverse is also true. A policy that targets a blueprint covers every agent identity created from it, including agent identities created later. The blueprint target doesn't cover the agents' user accounts.

Conditions and grant controls depend on the token subject:

	Agent identities	Agent user accounts (Preview)
Include options	**All agent identities**, **Select agent identities** (individual agents or blueprints, or custom security attributes)	**All agent users (Preview)**, **Select agent users (Preview)** (individual accounts or custom security attributes). Group-based include and exclude isn't supported.
Conditions	**Agent risk (Preview)** only	**Agent risk (Preview)**, **Agent execution environments (Preview)**, **Device platforms**, **Filter for devices**, **Network**. Device and network signals are available only for agents that run on endpoints.
Grant controls	**Block access** only. Interactive remediation isn't possible.	**Block access**, or **Grant access** with **Require device to be marked as compliant** or **Require compliant network**

Conditional Access doesn't apply in these cases:

- A blueprint acquires a Microsoft Graph token to create an agent identity or an agent's user account.
- A blueprint or agent identity performs the intermediate token exchange at **AAD Token Exchange Endpoint: Public** (resource ID `fb60f99c-7a34-4190-8149-302f77469936`). In the autonomous agent flow, this is the blueprint's token request (T1, see 6.9). Conditional Access protects the token request that follows, made by the agent identity (T2).
- Security defaults are enabled.
- The agent authenticates to a resource without Microsoft Entra ID, for example with an API key.

Also keep in mind:

- Policies that target **All users** don't include agents' user accounts.
- Target resources must have an enterprise application (service principal) in your tenant. To protect a custom MCP server or tool, register it in Microsoft Entra ID and expose its permissions.

6.2.1 Map each PoC agent to its token subject

Performed by **Reports Reader**.

1. Have the owner of each PoC agent exercise it once. For the three lab agent identities, use the community reference script in 6.9.1.
2. Open https://entra.microsoft.com > **Entra ID** > **Monitoring & health** > **Sign-in logs**.
3. Add the filter **Is Agent** = **Yes**, and add the filter **Agent type**.
4. Check the **Service principal sign-ins** tab and the user sign-in tabs. Note for each agent where it appears:
   - **Agent type** = **Agent Identity** on **Service principal sign-ins**: app-only. Use 6.2 and 6.3.
   - **Agent type** = **Agent ID user**: agent user account. Use 6.4.

Check result

- Every PoC agent is mapped to one or more token subjects. The three lab agent identities appear on **Service principal sign-ins** with **Agent type** = **Agent Identity**.

6.2.2 Create the default-deny policy with the enhanced object picker

Performed by **Conditional Access Administrator**.

1. Open https://entra.microsoft.com > **Entra ID** > **Conditional Access** > **Policies** > **New policy**.
2. Name: `PoC - Agents - Default deny except approved`.
3. Under **Assignments**, select **Users, agents or workload identities**.
4. Under **What does this policy apply to?**, select **Agents**.
5. Under **Include**, select **All agent identities**.
6. Under **Exclude**, select **Select individual agent identities**. In the enhanced object picker, use the **All**, **Agent blueprint principals**, and **Agent identities** tabs to select `PoC-Agent-Approved`, `PoC-Agent-Risk`, and every other agent identity or blueprint that must keep working. Then select **Select**.
7. Under **Target resources** > **Include**, select **All resources (formerly 'All cloud apps')**.
8. Under **Access controls** > **Grant**, select **Block**, and then select **Select**.
9. Set **Enable policy** to **Report-only**, and then select **Create**.

> **All agent identities** applies the policy to every agent identity in your tenant. Report-only mode evaluates and logs the policy but doesn't block anything. Before you switch to **On** (6.2.4), make sure every agent from your Chapter 1 inventory that must keep working is excluded. To exclude a whole fleet, exclude its blueprint.

Check result

- The policy is listed with state **Report-only**. Its exclusions contain both approved test agents.

6.2.3 (Optional) Exclude approved agents with custom security attributes

Use this variant when you have many agents. Every agent that carries the approval attribute is excluded automatically, including agents added later.

Performed by **Attribute Definition Administrator** (steps 1–2), **Attribute Assignment Administrator** (step 3), and **Conditional Access Administrator** with **Attribute Definition Reader** (steps 4–8).

1. Open https://entra.microsoft.com > **Entra ID** > **Custom security attributes**, and add an attribute set named `AgentAttributes`.

2. In `AgentAttributes`, add the attribute `AgentApprovalStatus` with data type **String**. Select **Allow multiple values to be assigned** and **Only allow predefined values to be assigned**, and add the predefined values `New`, `In_Review`, `HR_Approved`, `Finance_Approved`, and `IT_Approved`. Conditional Access supports only attributes of type string.

3. Assign `IT_Approved` to `PoC-Agent-Approved` and `PoC-Agent-Risk`. Use the Graph method from 3.7.2 Assign attribute values to PoC agent identities (`PATCH https://graph.microsoft.com/v1.0/servicePrincipals/{id}/microsoft.graph.agentIdentity`, delegated permissions `CustomSecAttributeAssignment.ReadWrite.All` and `AgentIdentity.ReadWrite.All`), with this body for a multi-valued attribute:

   ```
   {
     "customSecurityAttributes": {
       "AgentAttributes": {
         "@odata.type": "#Microsoft.DirectoryServices.CustomSecurityAttributeValue",
         "AgentApprovalStatus@odata.type": "#Collection(String)",
         "AgentApprovalStatus": ["IT_Approved"]
       }
     }
   }
   ```

4. Create a new policy as in 6.2.2, steps 1–5. Name it `PoC – Agents – Default deny except approved (attributes)`.

5. Under **Exclude**, select **Select agent identities based on attributes**, and set **Configure** to **Yes**.

6. Select the attribute **AgentApprovalStatus**, set **Operator** to **Contains** and **Value** to `IT_Approved`, and then select **Done**.

7. Complete 6.2.2, steps 7–8.

8. Set **Enable policy** to **Report-only**, and select **Create**.

Use only one of the two default-deny variants for the tests in 6.8.

Check result

- Each `PATCH` request returns `204 No Content`. Both approved agents carry `AgentApprovalStatus = IT_Approved`, and the policy is in **Report-only**.

6.2.4 Review the report-only results, then switch to On

Performed by **Reports Reader** (review) and **Conditional Access Administrator** (switch).

1. Run the allow-list test in 6.8.1.
2. In the sign-in log, open recent agent sign-ins and select the **Report-only** tab. Confirm that only agents you intend to block show **Report-only: Failure**.
3. Add any agent that must keep working to the exclusions.
4. When the results match your intent, open the policy and move **Enable policy** from **Report-only** to **On**. In a shared or production tenant, you can keep the policy in report-only for the PoC and record the results as evidence.

> The **Policy impact** view shows the impact of policies on interactive sign-ins. Report-only results for each sign-in are logged on the **Conditional Access** and **Report-only** tabs of the sign-in log details.

Check result

- No agent that must keep working shows **Report-only: Failure**. The final state of the policy (**On** or **Report-only**) is recorded in the evidence.

6.3 Block high-risk agent identities

Documentation: Secure autonomous agents with Conditional Access – Block high-risk agent identities · ID Protection for agents

This policy blocks agent identities dynamically when ID Protection raises their risk. It works alongside the static allow list in 6.2. Microsoft recommends **High** as the starting level.

6.3.1 Create the risk-based policy

Performed by **Conditional Access Administrator**.

1. Open https://entra.microsoft.com > **Entra ID** > **Conditional Access** > **Policies** > **New policy**.
2. Name: `PoC – Agents – Block high-risk agent identities`.
3. Under **Assignments**, select **Users, agents or workload identities**, and under **What does this policy apply to?**, select **Agents**.
4. Under **Include**, select **All agent identities**.
5. Under **Target resources** > **Include**, select **All resources (formerly 'All cloud apps')**.
6. Under **Conditions** > **Agent risk (Preview)**, set **Configure** to **Yes**. Under **Configure agent risk levels needed for policy to be enforced**, select **High**.
7. Under **Access controls** > **Grant**, select **Block**, and then select **Select**.
8. Set **Enable policy** to **Report-only**, and then select **Create**.
9. After you confirm the settings with report-only mode (the **Report-only** tab of agent sign-ins), move **Enable policy** from **Report-only** to **On**.

The optional risk test in 6.9 requires this policy to be **On**. Report-only mode evaluates the policy but doesn't enforce it.

> Microsoft Learn also provides a ready-made Conditional Access template for this policy: https://aka.ms/CreateAgentRiskPolicy.

Check result

- The policy is **On**. It includes **All agent identities** with **Agent risk (Preview)** = **High** and the grant control **Block**.

6.4 Protect agents that act as users (agent user accounts)

Documentation: Secure agent users with Conditional Access · Target agents in Conditional Access policies · Agents' user accounts · What is Windows 365 for Agents? · What is Global Secure Access?

Complete this section only if a PoC agent uses its own agent user account. Agent user targeting is in preview. Agents that run directly in cloud infrastructure, such as Copilot Studio hosted agents, have no device. A device or network requirement without the **Agent execution environments (Preview)** condition blocks those agents with no path to remediation.

Prerequisites:

- An agent user account linked to an agent identity.
- For 6.4.2: an agent that runs on an Intune-managed Windows 365 Cloud PC for Agents. Device compliance requires Intune enrollment, which today is supported only on Windows 365 Cloud PCs for Agents.
- For 6.4.3: the same Cloud PC with the Global Secure Access client installed.

6.4.1 Block risky agent user accounts

Performed by **Conditional Access Administrator**.

1. Open https://entra.microsoft.com > **Entra ID** > **Conditional Access** > **Policies** > **New policy**.
2. Name: `PoC – Agent users – Block medium and high risk`.
3. Under **Assignments**, select **Users, agents or workload identities**. Under **What does this policy apply to?**, select **Agents**, and under **Include**, select **All agent users (Preview)**.
4. Under **Target resources** > **Include**, select **All resources (formerly 'All cloud apps')**.
5. Under **Conditions** > **Agent risk (Preview)**, set **Configure** to **Yes**, and select **Medium** and **High**.
6. Under **Access controls** > **Grant**, select **Block**, and then select **Select**.
7. Set **Enable policy** to **Report-only**, and then select **Create**. Move it to **On** after you review the report-only results.

Check result

- The policy targets **All agent users (Preview)** with **Agent risk (Preview)** = **Medium**, **High**, and **Block**.

6.4.2 Require a compliant device (endpoint agents only)

Performed by **Conditional Access Administrator**.

1. Create a new policy named `PoC – Agent users – Require compliant device (endpoints)`.
2. Under **Assignments**, select **Users, agents or workload identities** > **Agents**, and under **Include**, select **All agent users (Preview)**.
3. Under **Target resources** > **Include**, select **All resources (formerly 'All cloud apps')**.
4. Under **Conditions** > **Agent execution environments (Preview)**, set **Configure** to **Yes**. Under **Include**, select **Agent user sessions initiated from endpoints**.
5. Under **Access controls** > **Grant**, select **Grant access** and **Require device to be marked as compliant**, and then select **Select**.
6. Set **Enable policy** to **Report-only**, and then select **Create**.

Check result

- The policy includes the condition **Agent user sessions initiated from endpoints**. Agents that don't run on a device are excluded from evaluation.

6.4.3 Require a compliant network (endpoint agents only)

Performed by **Conditional Access Administrator**.

1. Create a new policy named `PoC – Agent users – Require compliant network (endpoints)`.
2. Configure the assignments, target resources, and **Agent execution environments (Preview)** as in 6.4.2, steps 2–4.
3. Under **Access controls** > **Grant**, select **Grant access** and **Require compliant network**, and then select **Select**.
4. Set **Enable policy** to **Report-only**, and then select **Create**.

Check result

- The policy includes the condition **Agent user sessions initiated from endpoints** and the grant control **Require compliant network**.

6.5 Use ID Protection signals for agents

Documentation: ID Protection for agents · riskyAgent resource type (Microsoft Graph beta) · riskyAgent: confirmCompromised · List agentRiskDetections · Export risk data

ID Protection supplies the **Agent risk (Preview)** signal used in 6.3 and 6.4.1. All agent risk detections are offline. In on-behalf-of flows, risk is attributed to the user, not the agent. The detection list and incident sequence are in 3.9 Detect and respond with ID Protection. This section focuses on how risk feeds Conditional Access.

6.5.1 Review risky agents and agent detections

Performed by **Security Reader**.

1. Open https://entra.microsoft.com > **ID Protection** > **Dashboard**, and select **View risky agents**. You can also open the **Risky agents** report from the ID Protection menu.
2. Filter by risk level and risk state. Open an agent to see its risk state, risk level, agent type, sponsors, and detections.
3. Open the **Risk detections** report, and select the **Agent detections** tab. Detections are kept for up to 90 days.
4. Optional, with Graph Explorer and the delegated permissions `IdentityRiskyAgent.Read.All` and `IdentityRiskEvent.Read.All` (beta):

```
GET https://graph.microsoft.com/beta/identityProtection/riskyAgents?$filter=riskLevel eq 'high'
GET https://graph.microsoft.com/beta/identityProtection/agentRiskDetections
```

Check result

- The reviewer can open the **Risky agents** report and the **Agent detections** tab.

6.5.2 Respond to a risky agent

Performed by **Security Administrator**.

Action (Risky agents report)	Effect	Graph (beta)
Confirm compromise	Sets risk level to **High** and adds a detection. Triggers policies that block on high agent risk (6.3).	`POST /identityProtection/riskyAgents/confirmCompromised`
Confirm safe	Sets risk level to **None**. Use it for a false positive. Similar activity isn't flagged again.	`POST /identityProtection/riskyAgents/confirmSafe`
Dismiss risk	Sets risk level to **None**. Similar activity is still flagged in future.	`POST /identityProtection/riskyAgents/dismiss`
Disable	Prevents all sign-ins for the agent across Microsoft Entra ID and connected apps.	–

All three Graph actions take the body `{"agentIds": ["<object ID>"]}` and return `204 No Content`. They require the permission `IdentityRiskyAgent.ReadWrite.All`, and **Security Administrator** is the least-privileged supported role. The IDs can be agent identities, blueprint principals, or agent users.

Check result

- The responder can see the actions on a risky agent. The optional test in 6.9 uses them.

6.5.3 Export agent risk and sign-in data

Performed by **Security Administrator** (steps 1–2) and a user with permission to access data in the Log Analytics workspace (step 3).

This task is optional. It requires an Azure subscription with a Log Analytics workspace.

1. Open https://entra.microsoft.com > **Entra ID** > **Monitoring & health** > **Diagnostic settings**, and select **+ Add diagnostic setting**.
2. Enter a **Diagnostic setting name**, and select the log categories **RiskyAgents**, **AgentRiskEvents**, and **ServicePrincipalSignInLogs**. Under **Destination Details**, select **Send to Log Analytics workspace**, select the **Subscription** and **Log Analytics workspace**, and then select **Save**. Data can take about 15 minutes to appear, and only events from after you enable the setting are exported.
3. In the workspace, run:

```
AADRiskyAgents
| where TimeGenerated > ago(7d)
| project TimeGenerated, AgentDisplayName, Id, IdentityType, RiskLevel, RiskState, RiskDetail
| order by TimeGenerated desc
```

```
AADServicePrincipalSignInLogs
| where TimeGenerated > ago(1d)
| where ServicePrincipalId in ("<PoC-Agent-Risk ID>", "<PoC-Agent-Approved ID>", "<PoC-Agent-Unapproved
ID>")
| project TimeGenerated, ServicePrincipalName, ServicePrincipalId, ResourceDisplayName, ResultType,
ConditionalAccessStatus, ConditionalAccessPolicies
| order by TimeGenerated desc
```

Check result

- After the tests in 6.8 and 6.9, `AADServicePrincipalSignInLogs` contains the agent identity sign-ins, and `AADRiskyAgents` contains `PoC-Agent-Risk` if you ran 6.9.

6.6 Grant time-bound access with access packages

Documentation: Access packages for agent identities · Create and manage a catalog · Change resource roles for an access package · Change lifecycle settings for an access package · Request access packages on behalf of other identities · View, add, and remove assignments

Conditional Access decides whether an agent gets a token. Access packages decide what the agent can reach. Access packages for agents support these resource roles:

- Security group membership.
- OAuth API permissions, application or delegated. Certain high-risk Microsoft Graph permissions are blocked for agents.
- Directory roles allowed for agents.

You can't add agent identities and service principals to application roles, SAP roles, or SharePoint Online site roles through access packages. For this reason, you can't reuse an existing access package that contains those roles. Create a new one.

6.6.1 Prepare the catalog and the resource

Performed by **Identity Governance Administrator**.

This task uses the security group `PoC-Agent-Resource-Access` from Prepare groups. It has assigned (not dynamic) membership and no members at the start.

1. Open https://entra.microsoft.com > **ID Governance** > **Catalogs** > **New catalog**. Enter the name `PoC – Agent access` and a description, set **Enabled** to **Yes**, and then select **Create**.
2. Open the catalog, select **Resources** > **Add resources** > **Groups and Teams**, select `PoC-Agent-Resource-Access`, and then select **Add**.

Check result

- The catalog lists the group as a resource.

6.6.2 Create the access package and the request policy for agents

Performed by **Identity Governance Administrator**.

1. Open https://entra.microsoft.com > **ID Governance** > **Entitlement management** > **Access packages** > **New access package**.
2. On **Basics**, enter the name `PoC – Agent resource access` and a description. Under **Catalog**, select `PoC – Agent access`.
3. Select **Next: Resource roles**. Select **Groups and Teams**, select `PoC-Agent-Resource-Access`, and set **Role** to **Member**. Don't add application roles, SAP roles, or SharePoint Online site roles.
   - Optional: to grant API permissions, select **API Permissions**. Choose the source application, select **delegated** or **application**, select the permissions, and then select **Update permissions**. This step requires a **Global Administrator** and marks the catalog as privileged. Users can't receive API permissions, so the policy must be scoped to agents or service principals. Adding directory roles has the same requirements.
4. Select **Next: Requests**. Under **Who can get access**, select **For users, service principals, and agent identities in your directory**. Under **Select specific scope**, select **All agents**.
   - If some agents use plain service principals instead of agent identities, create a second policy later with **All Service principals**.

5. Configure approval: set **How many stages** to **1**. For the first approver, select **Choose specific approvers** > **Add approvers**, and select the PoC approver. Set **Decision must be made in how many days?** to 2, and set **Require approver justification** to **Yes**.
6. Select **Next: Requestor Information**, and then select **Next: Lifecycle**.
7. Under **Expiration**, set **Access package assignments expire** to **Number of hours** and enter 2. Select **Show advanced expiration settings**. Set **Allow users to extend access** to **Yes**, and **Require approval to grant extension** to **Yes**.
8. Select **Next: Rules**, then **Next: Review + create**. Fix any validation errors, and then select **Create**.

Check result

- The access package shows one policy with the scope **All agents**, one approval stage, and a 2-hour expiration with extension allowed.

6.6.3 Assign the access package to an agent (three pathways)

Agents receive access packages in three ways. After a request is submitted, it goes to the approvers configured in the policy.

Pathway	Who	How
Agent self-request	The agent identity	Programmatically, by creating an accessPackageAssignmentRequest (developer scenario, optional in the PoC)
Sponsor or owner request	Sponsor or owner of the agent identity	My Access portal (**Sponsor request** below)
Admin direct assignment	Identity Governance Administrator	Microsoft Entra admin center (**Admin direct assignment** below). No approval is needed.

Sponsor request, performed by the **standard test user** (sponsor of `PoC-Agent-Approved`) and the **PoC approver**:

1. Sign in to https://myaccess.microsoft.com, and select **Access packages**.
2. Locate `PoC – Agent resource access`, and select **Request**.
3. Under **Request details**, select **Requesting for Sponsored agent** (or **Requesting for Owned agent**).
4. Select `PoC-Agent-Approved`, select **Continue**, enter a justification, and submit the request.
5. As the **PoC approver**, sign in to https://myaccess.microsoft.com, and select **Approvals** > **Pending**.
6. Open the request, enter a justification, and approve it.

Admin direct assignment (use it for another agent identity, or if the sponsor pathway isn't available), performed by **Identity Governance Administrator**:

1. Open https://entra.microsoft.com > **ID Governance** > **Entitlement management** > **Access packages**, and open `PoC – Agent resource access`.
2. Select **Assignments** > **New assignment**.
3. In **Select policy**, select the policy from 6.6.2, and add the agent identity.
4. Optionally set the start and end of the assignment. If you set no end date, the policy's lifecycle settings apply.
5. Enter a justification, select **Add**, and after a few moments select **Refresh**.

Check result

- The assignment for `PoC-Agent-Approved` shows the status **Delivered**, and the agent identity is a member of `PoC-Agent-Resource-Access`.

6.6.4 Track expiry and sponsor extension

Performed by **Global Reader** (review).

When an assignment belongs to an agent identity with a sponsor, the sponsor is notified as the expiry date approaches. The sponsor can request an extension, if the policy allows it, which starts a new approval cycle. If the sponsor takes no action, the assignment expires automatically on its end date, and the agent identity loses access to the target resources. For a group resource role, the identity is removed from the group when its assignment expires, unless another access package assignment includes the same group.

1. Open https://entra.microsoft.com > **ID Governance** > **Entitlement management** > **Access packages**, and open `PoC – Agent resource access`.
2. Select **Assignments**, and review the assignment of `PoC-Agent-Approved`.
3. Select **Policies**, open the policy from 6.6.2, and review its lifecycle (expiration and extension) settings.

Check result

- The assignment has an end date. The policy expires assignments after 2 hours and allows extension with approval.

6.7 Apply Conditional Access and custom security attributes at publish time (policy templates)

Documentation: Policy templates · Agent requests in the Microsoft 365 admin center · Agent actions – Activate agents · Agent settings – Apply a template · Conditional Access for agents – Attribute-driven Conditional Access

An Agent 365 policy template is a collection of predefined governance and security policies that you apply to agents to enforce organizational standards. You select a template when you publish a requested agent or activate an agent. Its policies are applied to that agent.

Template type	Content
Default templates	Predefined policies from Microsoft Entra, Microsoft Purview, SharePoint Online, and Microsoft Defender. There are two default templates: one for all agents except AI teammates, and one for AI teammates in Frontier. Microsoft's built-in default policies appear locked and can't be edited.
Custom templates	Extend the default policies with Microsoft Entra policies that you apply case by case. The supported custom policies are **Conditional access**, **Access packages**, and **Custom security attribute**.

When templates apply:

- Policy templates support only new agent activation. You can't apply a template to agents that are already approved.
- An edited template applies to all new activations. The changes don't affect agents that are already approved.
- To apply policies to existing agents, use an agent management rule with the **Apply template** action. It's a one-time bulk operation that evaluates agents in the registry that have an agent identity. It doesn't apply to agent blueprints or AI teammates.
- A Microsoft Entra policy requires the agent to authenticate with its Microsoft Entra identity when it accesses resources. If the agent doesn't use Entra-based authentication, you can assign the policy, but it might not be enforced at runtime. Work with the agent developer to verify that Entra-based authentication is enabled.

Prerequisites:

- Create every policy in Microsoft Entra first. If a policy doesn't exist yet, you can't select it when you create a template.
- **Conditional access:** only Conditional Access policies that include at least one individual agent identity are visible in the Microsoft 365 admin center when you create a template. A policy scoped to all agent identities is selected automatically in the Microsoft 365 admin center experience and can't be overridden. This applies to the policies from 6.2 and 6.3. Create the selectable policy in 6.7.1.
- **Custom security attribute:** the attribute set `AgentAttributes` and the attribute `AgentApprovalStatus` from 6.2.3 (steps 1–2) must exist. You can also use the `AgentGovernance` attributes from 3.7 Tag agents with custom security attributes.
- **Access packages (optional):** the access package from 6.6.
- A test agent that a member of your organization submitted for admin approval (for example from Copilot Studio, Microsoft Foundry, or Microsoft 365 Agents Toolkit), as prepared in the chapter prerequisites. It must be listed in **Agents** > **All agents** > **Requests** in the Microsoft 365 admin center with the state **Pending review**.
- Roles: the Global Administrator and the AI Administrator both need the **Attribute Assignment Administrator** role for custom security attribute policies. The AI Administrator can create and apply access packages, but doesn't have enough privileges for Conditional Access and custom security attributes. Because this section uses both, perform 6.7.2 and 6.7.3 as a **Global Administrator** with **Attribute Assignment Administrator**.

6.7.1 Create a Conditional Access policy that the template can select

Performed by **Conditional Access Administrator**.

1. Open https://entra.microsoft.com > **Entra ID** > **Conditional Access** > **Policies** > **New policy**.
2. Name: `PoC – Agents – Template – Block high-risk agent identities`.
3. Under **Assignments**, select **Users, agents or workload identities**, and under **What does this policy apply to?**, select **Agents**.
4. Under **Include**, select **Select agent identities**, and select at least one individual agent identity, for example `PoC-Agent-Approved`.
5. Under **Target resources** > **Include**, select **All resources (formerly 'All cloud apps')**.
6. Under **Conditions** > **Agent risk (Preview)**, set **Configure** to **Yes**, and select **High**.
7. Under **Access controls** > **Grant**, select **Block**, and then select **Select**.
8. Set **Enable policy** to **Report-only**, and then select **Create**.

Check result

- The policy includes at least one individual agent identity. It doesn't use **All agent identities**.

6.7.2 Create the custom policy template

Performed by **Global Administrator** with **Attribute Assignment Administrator**.

1. Open the Microsoft 365 admin center at https://admin.cloud.microsoft.
2. In the navigation pane, expand **Agents**.
3. Select **Settings** > **Templates** > **Add a New Template**.
4. Enter the template details:
    1. Template name: `PoC – Agent CA and attributes`.
    2. A description.
    3. Indicate whether this template applies to agents with their own access.
5. Select **Next**, and choose the custom policies to add:
    - **Conditional access**: the policy `PoC – Agents – Template – Block high-risk agent identities` from 6.7.1.

    - **Custom security attribute**: `AgentApprovalStatus` in the attribute set `AgentAttributes`, with the value `IT_Approved`.
    - **Access packages** (optional): `PoC – Agent resource access` from 6.6.
6. Review, and finish adding the template.
7. Select **Save template**.

Check result

- The template appears under **Agents** > **Settings** > **Templates** with the selected custom policies. The default policies appear locked.

6.7.3 Apply the template when you publish a requested agent

Performed by **Global Administrator** with **Attribute Assignment Administrator**.

1. Sign in to the Microsoft 365 admin center at https://admin.cloud.microsoft.
2. Select **Agents** > **All agents** > **Requests**.
3. Select the requested test agent and view its details. Confirm the capabilities, data sources, security and permissions, and custom actions that the agent can invoke.
4. Select **Publish to store** in the agent details pane to open the agent publishing wizard.
5. Select the users or groups that can install the agent. Publishing makes the agent available for installation to the selected audience.
6. (Optional) Select the users or groups who will have the agent preinstalled.
7. Select **Next** to view the template options.
8. Choose the policy template `PoC – Agent CA and attributes`.
9. Select **Next** to review permissions.
10. In the **Review permissions** step, view the permissions requested by the agent, and grant admin consent if appropriate.
11. Select **Next** to complete the process.
12. After you review the agent details, select **Publish**.

For an agent request with the state **Pending activate**, the activation wizard contains the same **Apply a template** step. Choose the custom template there, review the permissions, and select **Finish**.

Check result

- The publishing wizard completes with the template `PoC – Agent CA and attributes` selected. Verify the applied policies in 6.8.3.

6.8 Test and validation

Documentation: Microsoft Entra Agent ID logs · Conditional Access report-only mode · View, add, and remove assignments · Change resource roles for an access package · Policy templates · Manage custom security attributes for an application

6.8.1 Test the allow list (default deny)

Performed by **Reports Reader**. The agents are exercised by their owners or developers.

Precondition: the default-deny policy from 6.2.2 (or 6.2.3) is in **Report-only**.

1. Exercise `PoC-Agent-Unapproved` and `PoC-Agent-Approved` so that each one requests a token with its own agent identity. For lab agent identities, you can use the community reference script from 6.9.1.

2. Open https://entra.microsoft.com > **Entra ID** > **Monitoring & health** > **Sign-in logs**, and select the **Service principal sign-ins** tab.
3. Add the filters **Is Agent** = **Yes** and **Agent type** = **Agent Identity**, open the entry for each agent, and select the **Report-only** tab.
4. Optional, enforced run: do this only in a dedicated PoC tenant, or where every agent that must keep working is excluded (6.2.4). Switch the default-deny policy to **On**, exercise both agents again, and open their entries on the **Conditional Access** tab. Switch the policy back to **Report-only** if required.

Expected result

- Report-only run: the policy isn't enforced, so both sign-ins succeed. `PoC-Agent-Unapproved` shows **Report-only: Failure** for `PoC – Agents – Default deny except approved`, and `PoC-Agent-Approved` shows **Report-only: Not applied**.
- Enforced run: the sign-in for `PoC-Agent-Unapproved` fails with `AADSTS53003` (`BlockedByConditionalAccess`), and the **Conditional Access** tab shows the policy with result **Failure**. The sign-in for `PoC-Agent-Approved` succeeds.

6.8.2 Test access package expiry

Performed by **Identity Governance Administrator** or **Global Reader**.

1. Right after the assignment in 6.6.3, open the access package and select **Assignments**. Confirm that `PoC-Agent-Approved` is **Delivered**, and record the expiry time.
2. Open https://entra.microsoft.com > **Entra ID** > **Groups** > **All groups** > `PoC-Agent-Resource-Access` > **Members**. Confirm that the agent identity is a member.
3. After the expiry time, open **Assignments** again, set the status filter to **Expired**, and confirm the assignment. Select **Download** to export the list.
4. Check the group members again.

Expected result

- Before expiry, the assignment is **Delivered** and the agent identity is a member of the group.
- After expiry, the assignment appears in the **Expired** list, and the agent identity is no longer a member of the group. The agent identity loses access to the target resources of the access package.

6.8.3 Test the policy template applied at publish time

Performed by **Agent ID Administrator** together with **Attribute Assignment Reader** (steps 1–2), the **standard test user** (step 3), and **Reports Reader** (steps 4–5).

1. Find the agent identity of the agent published in 6.7.3 in **Entra ID** > **Agents** > **Agent identities**, as in 3.4.1 Review the agent identities list, and record its object ID.

2. With the delegated permissions `AgentIdentity.Read.All` and `CustomSecAttributeAssignment.Read.All`, run the attribute filter from 3.7.3 Filter agents by attribute with the attribute from the template:

   ```
   GET https://graph.microsoft.com/v1.0/servicePrincipals/microsoft.graph.agentIdentity?
   $count=true&$select=id,displayName,customSecurityAttributes&$filter=customSecurityAttributes/AgentAttribute
   s/AgentApprovalStatus eq 'IT_Approved'
   ConsistencyLevel: eventual
   ```

3. Use the published agent.

4. Open https://entra.microsoft.com > **Entra ID** > **Monitoring & health** > **Sign-in logs**, and add the filters **Is Agent** = **Yes** and **Agent type** = **Agent Identity**. Check the **Service principal sign-ins** tab and the user sign-in tabs.

5. Open a sign-in entry of the published agent's agent identity, and select the **Conditional Access** and **Report-only** tabs.

Expected result

- The query in step 2 returns the object ID recorded in step 1, with `AgentApprovalStatus = IT_Approved`.
- The sign-in details of the published agent's agent identity show the evaluation result for `PoC – Agents – Template – Block high-risk agent identities`.
- If you use the attribute-based default-deny policy (6.2.3), the published agent is excluded from it by its `IT_Approved` value, and its sign-in shows **Report-only: Not applied** for that policy.
- Microsoft Entra policies in a template require the agent to authenticate with its Microsoft Entra identity. If the agent doesn't use Entra-based authentication, the policies might not be enforced at runtime. Work with the agent developer to verify that Entra-based authentication is enabled (6.7).

6.9 Test the risk-based block with the agent token flow (optional)

Documentation: Agent autonomous app OAuth flow · Authenticate and acquire tokens for autonomous agents · riskyAgent: confirmCompromised · Microsoft Entra Agent ID logs · Microsoft Entra authentication and authorization error codes

Community reference (not Microsoft documentation): Conditional Access for Agents: Blocking Agent Identities with Risk and Custom Security Attributes · sample script AgentID-AuthenticationFlow.ps1

This optional test proves the policy from 6.3 end to end. It follows the community lab article above. You mark a lab agent identity as compromised, run the autonomous agent token flow with the community sample script, and confirm the block in the sign-in log.

The autonomous agent flow, as documented on Microsoft Learn, has two token requests:

1. **T1 – blueprint token request.** The agent identity blueprint presents its credential in a `client_credentials` request with `scope=api://AzureADTokenExchange/.default` and `fmi_path=<agent identity client ID>`. Conditional Access doesn't apply to this intermediate token exchange.
2. **T2 – agent identity token request.** The agent identity sends a `client_credentials` request with `client_id=<agent identity client ID>` and the resource scope, and presents T1 as `client_assertion` with `client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer`. Conditional Access protects this token acquisition by the agent identity.

Microsoft Learn states that client secrets shouldn't be used as client credentials for agent identity blueprints in production. The community script uses a blueprint client secret, so run it only against a lab blueprint.

Precondition: the policy from 6.3 is **On**. `PoC-Agent-Risk` and `PoC-Agent-Approved` are excluded from the default-deny policy (6.2).

6.9.1 Prepare the community reference script

Performed by the **owner of the lab agent identity blueprint**.

1. Open the community sample script AgentID-AuthenticationFlow.ps1 and review it before you use it.
2. In its configuration block, set `$tenantId`, `$blueprintAppId`, and `$blueprintClientSecret` for the lab blueprint. The script doesn't run while placeholders are still in place.
3. Note how the script works. It takes `-AgentIdentityClientId <agent identity ID>`, requests T1 with `fmi_path`, and then requests T2 for `https://graph.microsoft.com/.default`. It reports one of these results for T2:
   - `[BLOCKED]`: Conditional Access denied the token (`AADSTS53003`).

- `[FAIL]`: T2 failed for a reason other than Conditional Access.
- `[WARN]`: the agent obtained a token, so Conditional Access didn't block it.

Check result

- Running the script for `PoC-Agent-Approved` prints `[OK] Blueprint authenticated - T1 obtained` and a T2 result.

6.9.2 Mark the test agent as compromised

Performed by **Security Administrator**.

1. In Graph Explorer, sign in, consent to `IdentityRiskyAgent.ReadWrite.All`, and send:

   ```
   POST https://graph.microsoft.com/beta/identityProtection/riskyAgents/confirmCompromised
   Content-Type: application/json

   { "agentIds": [ "<PoC-Agent-Risk ID>" ] }
   ```

   The response is `204 No Content`. Alternatively, open **ID Protection** > **Risky agents**, select the agent, and select **Confirm compromise**.

2. Read the risk state:

   ```
   GET https://graph.microsoft.com/beta/identityProtection/riskyAgents/<PoC-Agent-Risk ID>
   ```

3. Record the time (UTC).

Check result

- `riskLevel` is `high` and `riskState` is `confirmedCompromised`. Confirm compromise triggers risk-based Conditional Access policies that are configured to block on high agent risk.

6.9.3 Run the token flow for the risky agent and the control agent

Performed by the **owner of the lab agent identity blueprint**.

1. Run the script for the risky agent:

   ```
   .\AgentID-AuthenticationFlow.ps1 -AgentIdentityClientId <PoC-Agent-Risk ID>
   ```

2. Run the script for the control agent:

   ```
   .\AgentID-AuthenticationFlow.ps1 -AgentIdentityClientId <PoC-Agent-Approved ID>
   ```

3. Save the console output of both runs.

Expected result

- `PoC-Agent-Risk`: T1 is obtained, and T2 returns `[BLOCKED]` with `AADSTS53003`.
- `PoC-Agent-Approved`: T1 is obtained, and the T2 result isn't `[BLOCKED]`.

6.9.4 Confirm the block in the sign-in log

Performed by **Reports Reader**.

1. Open https://entra.microsoft.com > **Entra ID** > **Monitoring & health** > **Sign-in logs**, and select the **Service principal sign-ins** tab.
2. Set the date range to include the test, and add the filters **Is Agent** = **Yes** and **Agent type** = **Agent Identity**.
3. Open the entry for `PoC-Agent-Risk` at the recorded time, and select the **Conditional Access** tab.
4. Open the entry for `PoC-Agent-Approved`, and select the **Conditional Access** tab.
5. Optional, with Graph Explorer and `AuditLog.Read.All` (beta):

```
GET https://graph.microsoft.com/beta/auditLogs/signIns?$filter=signInEventTypes/any(t: t eq
'servicePrincipal') and agent/agentType eq 'agenticAppInstance'
```

Expected result

- `PoC-Agent-Risk`: the sign-in failed, and `PoC – Agents – Block high-risk agent identities` shows the result **Failure**.
- `PoC-Agent-Approved`: the risk policy doesn't block the sign-in. This proves that the policy acts on risk, not on identity.
- If the risk policy is in **Report-only**, it isn't enforced. The entry for `PoC-Agent-Risk` then shows **Report-only: Failure** on the **Report-only** tab.

6.9.5 Reset the test agent's risk

Performed by **Security Administrator**.

1. Reset the risk with **Dismiss risk** or **Confirm safe**. You can use **ID Protection** > **Risky agents** > select the agent, or Graph:

```
POST https://graph.microsoft.com/beta/identityProtection/riskyAgents/dismiss
Content-Type: application/json

{ "agentIds": [ "<PoC-Agent-Risk ID>" ] }
```

   Confirm safe (`/confirmSafe`, same body) tells the system to avoid flagging similar activity. **Dismiss risk** keeps flagging similar activity. Both set the risk level to none.

2. Read `riskyAgents/<PoC-Agent-Risk ID>` again.

Check result

- `riskLevel` is `none`.

6.10 Evidence

- **Policies (6.2–6.4, 6.7.1):** screenshots of each Conditional Access policy, or its JSON from Microsoft Graph (`GET https://graph.microsoft.com/beta/identity/conditionalAccess/policies/{id}`, permission `Policy.Read.All`), showing the name, assignments, conditions, grant control, and state. Include the default-

deny policy with its final state (6.2.4), the high-risk policy (**On**), the template policy, and the agent-user policies if in scope.

- **Allow list (6.8.1):** sign-in log details (the **Conditional Access** and **Report-only** tabs) for `PoC-Agent-Unapproved` and `PoC-Agent-Approved`.
- **Access package (6.6, 6.8.2):** the policy summary (scope **All agents**, one approval stage, 2-hour expiration, extension with approval), the approval record from My Access, the assignment list exported as **Delivered** and then **Expired**, and the group members of `PoC-Agent-Resource-Access` before and after expiry.
- **Policy template (6.7, 6.8.3):** a screenshot of the template under **Agents** > **Settings** > **Templates** with its custom policies, a screenshot of the template step in the publishing wizard, the output of the attribute query, and the sign-in log details of the published agent showing the template's Conditional Access policy.
- **Optional risk test (6.9):**
  - the `confirmCompromised` request with its `204` response, and the `riskyAgents/<id>` response showing `riskLevel = high`;
  - the console output of the community script for `PoC-Agent-Risk` and `PoC-Agent-Approved`;
  - the sign-in log details showing the risk policy with result **Failure**;
  - the **Risky agents** report before and after the reset.
- **Optional export (6.5.3):** the results of the KQL queries.

6.11 Troubleshooting

Symptom	Likely cause	Fix
A policy has no blocking effect	Policy is in **Report-only**, which evaluates but doesn't enforce	Check the **Report-only** tab for the result. Switch to **On** to enforce.
Policy doesn't apply to an agent's user account	Policy targets agent identities (or a blueprint), which don't cover agent users	Create a separate agent-user policy (6.4).
Existing **All users** policies don't affect agent users	**All users** doesn't include agents' user accounts	Target **All agent users (Preview)** or select agent users. Group-based targeting isn't supported.
Delegated (on-behalf-of) agent isn't affected by agent policies	In delegated access, the token subject is the user	Use user-targeted policies for that access pattern.
Unapproved agent shows **Report-only: Not applied**	Agent is excluded directly, through its blueprint, or through a custom security attribute	Review the exclusions and the agent's attribute values.
No agent entries in the sign-in log	Wrong tab or missing filters	Use **Service principal sign-ins** with **Is Agent** = **Yes** and **Agent type** = **Agent Identity**.
Community script: T1 fails	Wrong blueprint app ID or secret, or `fmi_path` isn't a child agent identity of this blueprint (a blueprint can only impersonate its own child agent identities)	Check the IDs in **Entra ID** > **Agents** > **Agent identities** (3.4.1) and the blueprint credential.
Community script: `[WARN]` for `PoC-Agent-Risk` after confirm compromise	Risk policy in **Report-only**; agent excluded from the risk policy; or the policy targets agent users instead of agent identities	Check the policy state and assignments, and check that `riskyAgents/<id>` shows `riskLevel = high`.
A blueprint token-exchange sign-in succeeds while the agent identity is blocked	Conditional Access doesn't apply to the intermediate token exchange	Evaluate the agent identity's sign-in entry.
Attribute isn't selectable in the policy	Missing **Attribute Definition Reader** role, or the attribute isn't of type string	Assign the role, and use a string attribute.

Symptom	Likely cause	Fix
Device or network policy blocks cloud-hosted agents	**Agent execution environments (Preview)** isn't configured	Include only **Agent user sessions initiated from endpoints**.
Can't add API permissions or directory roles to the access package	You aren't a Global Administrator; the policy isn't scoped to agents or service principals; or the permission is blocked for agents	Use a Global Administrator, scope the policy to **All agents**, or choose another permission.
Existing access package can't be used for agents	It contains application, SAP, or SharePoint Online site roles	Create a new access package (6.6.2).
Sponsor doesn't see **Requesting for Sponsored agent**	The user isn't a sponsor or owner of the agent identity	Assign the sponsor (Chapter 3.5.1).
Agent identity can't be assigned in **New assignment**	The identity isn't eligible under the selected policy	Select the policy with **For users, service principals, and agent identities in your directory** > **All agents**.
Conditional Access policy isn't listed when you create the template	The policy doesn't include at least one individual agent identity, or it wasn't created in Microsoft Entra first	Create the policy with **Select agent identities** and at least one agent identity (6.7.1).
Policies from 6.2 and 6.3 are selected in the template and can't be removed	Policies scoped to all agent identities are selected automatically and can't be overridden	No action needed. Use **Select agent identities** for policies that you want to choose per template.
Conditional access or custom security attribute policies can't be added to the template	The admin is an AI Administrator, or lacks **Attribute Assignment Administrator**	Use a Global Administrator who also has **Attribute Assignment Administrator**.
An edited template doesn't change an agent that's already published	Templates apply only to new activations	Use an agent management rule with the **Apply template** action (agents with an agent identity).
Template's Entra policies have no effect on the published agent	The agent doesn't authenticate with its Entra identity	Work with the agent developer to enable Entra-based authentication.

✅ What you proved

- [ ] A default-deny policy for agent identities exists. In report-only mode, the sign-in log shows **Report-only: Failure** for an unapproved test agent and **Report-only: Not applied** for an approved test agent.
- [ ] The policy **Block high-risk agent identities** is **On**. It targets **All agent identities** with **Agent risk (Preview)** = **High** and the grant control **Block**.
- [ ] Optional (6.9): after a test agent is marked as compromised, its token request as the agent identity fails with `AADSTS53003`, and the sign-in log lists the risk policy with result **Failure**. A control agent that isn't risky still gets a token.
- [ ] If agent user accounts are in scope, the agent-user policies exist in report-only mode, and the device and network policies are limited to sessions from endpoints with the **Agent execution environments (Preview)** condition.
- [ ] Optional (6.9): the **Risky agents** report shows the test agent as high risk during the test, and as no longer at risk after the reset.
- [ ] An access package for agent identities exists, with a request policy for **All agents**, an approval stage, and an expiration. A test agent receives the access, and loses it automatically when the assignment expires.
- [ ] A custom policy template with a Conditional Access policy and a custom security attribute is applied when a requested agent is published. The published agent's identity carries the attribute value, and its sign-in details show the template's Conditional Access policy.

References

- Conditional Access for agents
- Target agents in Conditional Access policies
- Secure autonomous agents with Conditional Access
- Conditional Access report-only mode
- Secure autonomous agents with Conditional Access – Block high-risk agent identities
- ID Protection for agents
- Secure agent users with Conditional Access
- Agents' user accounts
- What is Windows 365 for Agents?
- What is Global Secure Access?
- riskyAgent resource type (Microsoft Graph beta)
- riskyAgent: confirmCompromised
- List agentRiskDetections
- Export risk data
- Access packages for agent identities
- Create and manage a catalog
- Change resource roles for an access package
- Change lifecycle settings for an access package
- Request access packages on behalf of other identities
- View, add, and remove assignments
- accessPackageAssignmentRequest
- Policy templates
- Agent requests in the Microsoft 365 admin center
- Agent actions – Activate agents
- Agent settings – Apply a template
- Conditional Access for agents – Attribute-driven Conditional Access
- Microsoft Entra Agent ID logs
- Manage custom security attributes for an application
- Agent autonomous app OAuth flow
- Authenticate and acquire tokens for autonomous agents
- Microsoft Entra authentication and authorization error codes

Additional resources

- Community article: Conditional Access for Agents: Blocking Agent Identities with Risk and Custom Security Attributes
- Community sample: AgentID-AuthenticationFlow.ps1

Runbook index · Previous: Chapter 5 – Agent lifecycle and audit · Next: Chapter 7 – Sensitive data protection (Purview)

Prerequisites - Chapter 7: Sensitive data protection (Purview)

> **Goal:** Prepare the accounts, access, and test assets needed for this chapter and its selected optional paths.

Apply the shared PoC preparation guidance and agree the chapter outcomes. Finish dependent build steps before their tests.

Table of contents

- Tenant readiness
- Prepare accounts and role assignments
- Audit recording
- Test assets and dependencies
- Plan propagation and test timing
- References

Tenant readiness

Documentation: Get started with Microsoft Agent 365 · Agent management in the Microsoft 365 admin center · Frontier program

Use the task-specific roles in 7.1 Required permissions. Confirm availability rather than assuming the tenant is ready.

Follow the shared tenant readiness check using the chapter permissions and feature prerequisites linked above.

Prepare accounts and role assignments

Alongside the shared setup and reviewer accounts, prepare the following scenario accounts:

Account	Purpose
Maker account	Creates the PoC SharePoint site and test agents; needs the permissions specified in the chapter.
Authorized and unauthorized test users	Chapter 7 acceptance tests: one user inside and one user outside the sharing scope of the PoC site and agents. No admin role.
Intended non-owner account	Discovers and installs the published Copilot Studio agent. Can be the authorized test user if it didn't create the agents. No admin role.
Controlled recipient mailbox	Receives the test email from the Copilot Studio email agent. Never use a real personal address.

Assign the roles in 7.1 Required permissions using the shared model. Verify each account can access the pages needed for its tasks.

Audit recording

Documentation: Turn auditing on or off · Audit logs for Copilot and AI applications

Performed by a member of **Audit Manager**. Follow the shared audit recording check before the activity to be reviewed.

Test assets and dependencies

Documentation: Manage site creation in SharePoint · Manage sensitivity labels in Office apps · Enable sensitivity labels for Office files in SharePoint and OneDrive · Build agents in Agent Builder · Add knowledge sources to an agent in Agent Builder · Share and manage agents built in Agent Builder

- Make sure a **Confidential** sensitivity label exists and is published to the PoC maker account and test users. If you plan to keep the Excel workbook in 7.4.4 as a comparison source, make sure a **General** label is published as well.
- Prepare the test accounts: an authorized test user, an intended non-owner account (it can be the authorized test user, as long as it didn't create the agents), an unauthorized test user who is outside the sharing scope you choose in 7.4.6 and 7.5.3, and one controlled email recipient.
- Review who has permission to enable blocking and organization-wide sharing before turning anything on.
- Build the sections in order: the two DLP policies (7.2, 7.3), the SharePoint site and files (7.4), the two agents (7.5, 7.6), then the supervision policies (7.7, 7.8), and finally the acceptance checklist (7.9).

Build guidance: 7.4 SharePoint communication site and PoC files · 7.5 Microsoft 365 Copilot knowledge agent · 7.6 Copilot Studio email agent and publication

Check result

- Each asset needed for the selected paths is prepared or has a named build task that will finish before its dependent tests.

Plan propagation and test timing

Documentation: Create and configure sensitivity labels and their policies · Learn about the Microsoft 365 Copilot and Copilot Chat DLP location

Configuration	Documented propagation window
Sensitivity label and label policy changes	Up to 24 hours
DLP policy changes for the Microsoft 365 Copilot and Copilot Chat location	Up to 4 hours

> ⚠️ **Watch out:** Allow for the documented propagation window and verify the actual configuration or signal before testing. Elapsed time alone is not proof that a change has applied.

References

- Get started with Microsoft Agent 365
- Agent management in the Microsoft 365 admin center
- Frontier program
- Turn auditing on or off
- Audit logs for Copilot and AI applications
- Manage site creation in SharePoint
- Manage sensitivity labels in Office apps
- Enable sensitivity labels for Office files in SharePoint and OneDrive
- Build agents in Agent Builder
- Add knowledge sources to an agent in Agent Builder
- Share and manage agents built in Agent Builder
- Create and configure sensitivity labels and their policies
- Learn about the Microsoft 365 Copilot and Copilot Chat DLP location

---

> **Ready?** Continue to the chapter. →

Chapter 7 - Sensitive data protection (Purview)

Pillar: Secure **What it proves:** Microsoft Purview and Microsoft 365 Copilot work together to protect sensitive data that an agent can access or generate: sensitivity labels and DLP stop Copilot from processing or emailing regulated content, two agents built on a labeled SharePoint knowledge source respect those controls, and Communication Compliance and Insider Risk Management supervise what agents do.

Success criteria: Agree the selected outcomes in What you proved before testing.

Topics and time guide

Approximate hands-on estimates for the selected scope, with prerequisites prepared. Service waiting time is separate.

Topic	Estimated hands-on time
7.1 Required permissions	~5 min
7.2 DLP policy for Copilot content protection	~10-15 min
7.3 DLP policy for sensitive email content	~10-15 min
7.4 SharePoint communication site and PoC files	~15-20 min
7.5 Microsoft 365 Copilot knowledge agent	~10-15 min
7.6 Copilot Studio email agent and publication	~20-30 min
7.7 Communication Compliance policy	~10-15 min
7.8 Insider Risk Management verification	~5-10 min
7.9 Acceptance checklist (tests)	~15-25 min
7.10 Evidence	~5-10 min
7.11 Troubleshooting	As needed

Outcome checklist · References

7.1 Required permissions

Use the shared account and role-assignment model with the task-specific permissions below.

Task	Least-privilege role	Section
Create the DLP policy for Microsoft 365 Copilot and Copilot Chat	**Data Security AI Admins** (Purview role group) or **AI Administrator** (Microsoft Entra)	7.2
Create the DLP policy for Exchange email	**Information Protection Admins** (Purview role group)	7.3
Create the communication site, add and label the PoC files, share the site	PoC maker account allowed to create SharePoint sites (becomes the site owner), with the PoC sensitivity labels published to it	7.4

Task	Least-privilege role	Section
Build and share the Microsoft 365 Copilot knowledge agent	PoC maker account	7.5
Build and publish the Copilot Studio email agent	PoC maker account with permissions to create and access a Copilot Studio environment	7.6.1–7.6.8
Review, approve, and publish the agent to the store	**AI Administrator** (Microsoft Entra)	7.6.9
Create the Communication Compliance policy; review its configuration	**Communication Compliance Administrators** (Purview role group)	7.7, 7.9.1
Review Communication Compliance policy matches	**Communication Compliance Investigators** or **Communication Compliance Analysts** (Purview role groups), and named as reviewer in the policy	7.7, 7.9.8
Verify the Insider Risk Management agent policy	**Insider Risk Management Admins** (Purview role group)	7.8, 7.9.1, 7.9.9
Run the agent, email, availability, and permission tests	Test accounts without admin roles: an authorized test user, an intended non-owner account, and an unauthorized test user	7.9.2–7.9.7
Validation / read-only review of the DLP policies	**Global Reader** (Microsoft Entra) + **Information Protection Analysts** (Purview role group, view-only access to DLP policies and sensitivity labels)	7.9.1

Before you start:

- [ ] Confirm the chapter prerequisites for the selected topics.

7.2 DLP policy for Copilot content protection

Documentation: Microsoft Purview DLP for Microsoft 365 Copilot and Cowork · Create and deploy a DLP policy

Agents and Copilot can surface confidential content in a chat response unless sensitive data is explicitly protected at the point Copilot processes it. This section builds a Purview DLP policy aimed at the Copilot location itself – not at a file share or a mailbox – that stops Microsoft 365 Copilot and Copilot Chat from processing content carrying the **Confidential** sensitivity label. It controls what an AI surface is allowed to read and act on, not only where a document is allowed to live.

7.2.1 Create the DLP policy

Performed by **Data Security AI Admins**.

1. In Microsoft Purview (`https://purview.microsoft.com`), open **Solutions** > **Data Loss Prevention** > **Policies** and select **Create policy**.
2. Create a separate policy for this PoC rather than editing an existing one, so the Copilot restriction stays isolated and easy to remove afterward.
3. On the category screen, select **Enterprise applications and devices**.
4. Select **Custom** under **Categories** and **Custom** under **Regulations**, then select **Next**.
5. Name the policy `PoC - Copilot Confidential Content`, so it is clearly distinguished from the email policy in 7.3. Add a description so that anyone who picks up the tenant later understands its purpose. Select **Next**.

7.2.2 Scope the policy to Copilot

Performed by **Data Security AI Admins**.

1. On **Assign admin units**, keep **Full directory** – the **Microsoft 365 Copilot and Copilot Chat** location doesn't support admin units – then select **Next**.
2. On the locations screen, turn on only **Microsoft 365 Copilot and Copilot Chat**. Selecting this location disables all other locations for the policy, so the restriction applies only to what this PoC is meant to show.
3. Select **Next**.

7.2.3 Build the advanced rule

Performed by **Data Security AI Admins**.

1. On the policy-settings screen, select **Create or customize advanced DLP rules**, then **Create rule**.
2. Name the rule (the wizard labels this field as a policy name, but it names the rule inside the policy). Record the rule name.
3. Select **Add condition** > **Content contains** > **Sensitivity labels**, and add the **Confidential** label.
4. Check the condition in the live rule and make sure that only the intended label or sublabels are present.
5. Complete and save the rule before advancing in the wizard. Do not continue with an empty rules list.

7.2.4 Add the Copilot restriction action

Performed by **Data Security AI Admins**.

1. Under **Actions**, select **Add an action** > **Restrict Copilot from processing content**.
2. Within that action, select **Accessing knowledge sources**.
3. Treat this as the specific restriction this PoC configures. It does not automatically cover every Copilot operation or every agent channel; if the PoC needs a broader restriction, configure it explicitly.

7.2.5 Review, enable, and submit

Performed by **Data Security AI Admins**.

1. Save the rule, confirm it shows as enabled in the rules list, and select **Next**.
2. If an administrator-alert action is shown in the summary, review its recipients.
3. Select **Turn on policy immediately**. Use immediate enforcement only within an approved test scope.
4. Select **Next**, review the final summary, and select **Submit**.

Check result

- The policy name and status are recorded.
- The policy has one enabled rule with the **Confidential** label condition and the **Restrict Copilot from processing content** > **Accessing knowledge sources** action.
- Policy updates can take up to four hours to reach Microsoft 365 Copilot and Copilot Chat. The restriction is validated with controlled content-access tests in 7.9.3, not assumed from the configuration alone.

7.3 DLP policy for sensitive email content

Documentation: Create and deploy a DLP policy · DLP policy reference · Credit card number entity definition · ABA routing number entity definition

Separate from what Copilot can read, organizations need to stop regulated payment data from leaving the tenant by email. This section builds a second, independent DLP policy scoped to Exchange email. Where 7.2 stops Copilot from processing labeled content, this policy prevents delivery of email that contains matching sensitive information – it detects content by pattern (sensitive information types) rather than by label.

7.3.1 Create the email DLP policy

Performed by **Information Protection Admins**.

1. Return to **Data Loss Prevention** > **Policies** and select **Create policy**. Keep this configuration entirely separate from the Copilot policy so the two can be tested and removed independently.
2. Select **Enterprise applications and devices**, then **Custom** under both **Categories** and **Regulations**, and select **Next**.
3. Name the policy `PoC - Email Payment Data`. Select **Next**.

7.3.2 Scope the policy to Exchange email

Performed by **Information Protection Admins**.

1. On **Assign admin units**, retain the displayed settings after confirming that the scope matches your approved test environment, then select **Next**.
2. On the locations screen, deselect every other location and keep only **Exchange email** selected.
3. Review the included and excluded users and groups. A stray exclusion can make a PoC test message silently skip the policy.
4. Select **Next**.

7.3.3 Build the rule to detect payment data

Performed by **Information Protection Admins**.

1. On the policy-settings screen, select **Create or customize advanced DLP rules**, then **Create rule**.
2. Name the rule and record the name.
3. Select **Add condition** > **Content contains** > **Sensitive info types**.
4. Select **Credit Card Number** and **ABA Routing Number**, then select **Add**.
5. Record the AND/OR grouping, confidence level, and instance count that the rule uses for the two detectors. These settings determine how the detectors combine and are tested separately and together in 7.9.4.

7.3.4 Add the blocking action

Performed by **Information Protection Admins**.

1. Under **Actions**, select **Add an action** > **Restrict access or encrypt the content in Microsoft 365 locations**. Selecting this action does not by itself enable encryption; the next selection defines the restriction.
2. Select **Block users from receiving email, or accessing shared SharePoint, OneDrive, Teams files, and Power BI items**, then **Block everyone**. This blocks delivery regardless of whether the recipient is internal or external.
3. Get explicit approval for the **Block everyone** scope, and test only with controlled recipients.

7.3.5 Review, enable, and submit

Performed by **Information Protection Admins**.

1. Save the rule, verify its condition and blocking action, and confirm it is enabled.

2. Check that no unintended exception changes the test, then select **Next**.
3. Select **Turn on policy immediately**, then **Next**.
4. Review the final summary and select **Submit**.

Check result

- The policy name, location (**Exchange email** only), included and excluded users, detectors and their settings, action, and mode are recorded.
- The policy is validated in 7.9.4 with a benign control message and approved synthetic matching content before blocking is treated as proven. Test within a limited scope before any production rollout.

7.4 SharePoint communication site and PoC files

Documentation: Manage site creation in SharePoint · Manage sensitivity labels in Office apps · Enable sensitivity labels for Office files in SharePoint and OneDrive

Both agents need a realistic, properly labeled knowledge source to answer questions from and to trigger the DLP policies above. This section creates a communication site with synthetic invoice data in Word and Excel, labels it, and shares the site for the agents. Use synthetic PoC content only – nothing in this section should touch real invoices, payment details, or customer names.

7.4.1 Start a new communication site

Performed by the **PoC maker account**.

1. Open your approved SharePoint tenant's home page and confirm you are authorized to create a site there.
2. Select **Create site**, then **Communication site** (not **Team site**). A communication site is built to share information broadly rather than to collaborate privately.
3. Select **Standard communication site** as the template, then **Use template**.

7.4.2 Name and create the site

Performed by the **PoC maker account**.

1. On **Give your site a name**, enter a site name and address that follow your organization's naming standards, then select **Next**.
2. Record the resulting site URL. You paste it into both agents' knowledge sources in 7.5 and 7.6.
3. On **Set language and other options**, review the language and other displayed options, then select **Create site**.

Check result

- The new site and its **Documents** library open successfully.

7.4.3 Add the Word sample and label it

Performed by the **PoC maker account**.

1. In **Documents**, select **New** > **Word document**.

2. Paste synthetic invoice content into the document. Keep the `TEST – Not Real` wording so no one mistakes it for a live record. Use this entry for one company:

```
Contoso Ltd.
Invoice ID: CT-INV-1001
Amount Due: $12,450.00
Payment Information (TEST – Not Real)
Credit Card Number: 4111 1111 1111 1111
ABA Routing Number: 021000021
Bank Account Number: 000123456789
```

3. On the **Home** ribbon, select **Sensitivity** > **Confidential**. This aligns the document with the label condition of the Copilot DLP policy in 7.2.
4. Close the document, open the saved document again, and check that the applied label shows **Confidential**. Rely on the saved document, not on the instruction text.

Check result

- The saved Word document shows the **Confidential** label.

7.4.4 Add the Excel sample and resolve its label

Performed by the **PoC maker account**.

1. In **Documents**, select **New** > **Excel workbook**.
2. Add the columns `Company`, `InvoiceID`, `CreditCard`, `Routing`, `Account`, and `Amount`.
3. Format the **Routing** and **Account** columns as **Text** before entering values. With Excel's default **General** format, numeric routing and account numbers can lose a leading zero or be rewritten in scientific notation.
4. Populate one or two rows with synthetic values in the same shape as the Word sample, for example:

Company	InvoiceID	CreditCard	Routing	Account	Amount
Contoso Ltd.	CT-INV-1001	4111 1111 1111 1111	021000021	000123456789	$12,450.00

5. Decide whether this workbook is a comparison source (labeled **General**) or another **Confidential**-labeled test source, and apply the label you chose. The knowledge baseline in 7.9.2 needs an approved comparison source on the site; if you label the workbook **Confidential**, add another approved **General**-labeled file with a harmless fact for that test.
6. Record the label the workbook carries and use that decision consistently in the tests in 7.9.

Check result

- The workbook shows the label you chose, and the **Routing** and **Account** values keep their leading zeros.

7.4.5 Verify the files and open Site access

Performed by the **PoC maker account**.

1. In the **Documents** library, confirm that the files appear with the labels you intended.
2. Optionally create a PDF copy of the Word document for later use. If you do, check its content and protection separately rather than assuming they match the Word source.
3. Select **Site access** to move to the sharing step.

7.4.6 Share the site

Performed by the **PoC maker account**.

1. In the sharing search box, type `Everyone` and select **Everyone except external users**. This grants broad internal access across the tenant, which keeps both agents' knowledge-source setup simple.
2. Review the permission level before confirming. Use a limited test group instead of tenant-wide sharing unless broad internal access has explicit approval.
3. Select **Share**.

Check result

- The site URL, the file names, each file's label, and the sharing scope and permission level are recorded.

7.5 Microsoft 365 Copilot knowledge agent

Documentation: Build agents in Agent Builder · Add knowledge sources to an agent in Agent Builder · Share and manage agents built in Agent Builder

This section shows how quickly a business user – not only IT – can build a knowledge agent in Microsoft 365 Copilot's agent builder, scoped to the SharePoint site from 7.4. It is a separate authoring path from the Copilot Studio agent in 7.6.

7.5.1 Create the agent and its details

Performed by the **PoC maker account**.

1. Open Microsoft 365 Copilot (`https://m365.cloud.microsoft`) and confirm you are signed in to the intended PoC tenant.
2. Select **New agent**, then open the **Configure** tab.
3. Enter a **Name**, **Description**, and **Instructions** for the agent.
4. Record the exact wording you use. The instructions are a deliberate decision for the PoC, not a setting to copy.

Check result

- The **Configure** tab shows the name, description, and instructions you recorded.

7.5.2 Add the SharePoint knowledge source

Performed by the **PoC maker account**.

1. Under **Knowledge**, paste the SharePoint site URL recorded in 7.4.2.
2. Turn on **Only use specified sources**. The agent then prioritizes the knowledge source shown in the **Knowledge** panel; this setting doesn't block the agent's general AI knowledge.
3. Treat this toggle as a scoping control, not as an access control: it controls what the agent searches, not who is allowed to see the results. Source citations and permissions are tested separately in 7.9.2, 7.9.3, and 7.9.7.

7.5.3 Share and publish the agent

Performed by the **PoC maker account**.

1. Select **Share** and verify the agent's name before changing access.

2. Select **Anyone in your organization** > **Apply**. Use a restricted pilot audience instead unless organization-wide sharing has explicit approval.
3. Select **Update** and confirm the success message.

Check result

- The agent name, instructions, knowledge source, **Only use specified sources** setting, and sharing audience are recorded.
- An account other than the one that created the agent can open and use it (tested in 7.9.6).

7.6 Copilot Studio email agent and publication

Documentation: Add SharePoint as a knowledge source · Add tools to custom agents · Office 365 Outlook connector · Key concepts – Publish and deploy your agent · Connect and configure an agent for Teams and Microsoft 365 · Manage agent requests in Microsoft 365 admin center · Agent overview in Microsoft 365 admin center

A more capable, IT-governed agent needs an action – not just knowledge retrieval – and a controlled path to publication. This section builds a second agent in Copilot Studio with the same SharePoint knowledge source plus an Outlook email tool, then takes it through the full publish lifecycle: add a tool, connect an identity, publish, enable a channel, submit to the org catalog, and have an administrator approve and publish it to the store. Track this agent's configuration independently from the agent in 7.5.

7.6.1 Create a blank agent and set its details

Performed by the **PoC maker account**.

1. Open Copilot Studio (`https://copilotstudio.microsoft.com`), select **Agents**, and confirm the intended tenant and environment before making changes.
2. Select **Create blank agent**.
3. Select **Edit details** and enter a **Name** and **Description** that clearly distinguish this email-capable agent from the knowledge-only agent in 7.5.
4. Select **Edit instructions** and define the agent's behavior, including how it handles recipients and whether it confirms before sending. Record the instructions; the behavior is verified in 7.9.5 once the email tool is wired up.

7.6.2 Add the SharePoint knowledge source

Performed by the **PoC maker account**.

1. Select **Add knowledge source** > **SharePoint**.
2. Paste the SharePoint site URL recorded in 7.4.2 and select **Add**, then **Add to agent**.
3. Confirm the source appears without an error before moving on to the tool.

7.6.3 Add the Outlook email tool

Performed by the **PoC maker account**.

1. Select **Add tool** and choose the **Office 365 Outlook** connector.
2. Select the operation **Send an email (V2)**. Check the operation name carefully – Outlook exposes more than one similarly named email action.

7.6.4 Create the connection and configure the tool

Performed by the **PoC maker account**.

1. Under **Connect**, select **Create new connection**.
2. Decide which approved test identity authenticates the connection, then select **Create** in the Outlook connection dialog to start the authentication flow.
3. Select the approved account and review the requested permissions.
4. Select **Add and configure** to attach the operation.
5. On the tool's configuration page, review and record:
    - The **Authentication** setting: **End user** (the default – the tool uses the credentials of the user chatting with the agent) or **Maker-provided** (the tool uses the maker's credentials, so email is sent from the connection account).
    - The **Ask the end user before running** setting, which controls whether the agent asks for confirmation before sending.
    - How the recipient (**To**), **Subject**, and **Body** inputs are filled.

    Don't assume email is always sent as the person chatting with the agent; the sending identity is verified in 7.9.5.

Check result

- **Send an email (V2)** is listed on the agent's **Tools** page with the settings you recorded.

7.6.5 Publish the agent

Performed by the **PoC maker account**.

1. Select **Publish**.
2. Before continuing to channel configuration, run a controlled test of both knowledge retrieval and the email action, sending only to the controlled recipient.
3. When prompted, read the dialog's explanation, select **Force newest version**, and complete publication.

Check result

- Publication completes, and the controlled test returns an answer from the knowledge source and delivers a benign email to the controlled recipient.

7.6.6 Enable the Microsoft 365 Copilot and Teams channel

Performed by the **PoC maker account**.

1. Go to **Channels** for the published agent.
2. Under **Microsoft channels**, select **Microsoft 365 Copilot and Teams**.
3. Select the **Microsoft 365 Copilot** option and choose **Add channel**.
4. Confirm the channel is added successfully. Channel creation alone does not establish administrator approval or guarantee end-user access; both are checked later in this section and in 7.9.6.

Check result

- **Microsoft 365 Copilot and Teams** shows as added on the **Channels** page.

7.6.7 Make the agent available and submit for review

Performed by the **PoC maker account**.

1. Select **Availability options** > **Show to everyone in my org**. Obtain approval before making a test agent widely available in a real tenant.
2. Select **Submit to org catalog for review**.
3. Read the confirmation dialog text and select **Yes**.
4. Record the submission status. Submission and administrator approval are separate stages; approval is in 7.6.9.

Check result

- Copilot Studio shows the submission status for the agent, and the status is recorded.

7.6.8 Republish after the channel change

Performed by the **PoC maker account**.

1. Select **Publish** again so that the channel and availability changes take effect.
2. Select **Force newest version**, then **Publish**.
3. Confirm success and record the version or timestamp if one is shown.

7.6.9 Administrator review and publish to store

Performed by **AI Administrator**.

1. In the Microsoft 365 admin center (`https://admin.cloud.microsoft`), go to **Agents** > **All agents** > **Requests**.
2. Select the submitted agent. Confirm it is the agent that was actually reviewed and tested.
3. Select **Publish to store**.
4. Select **All users can install the agent**, then **Next**. Review installation eligibility separately from the document permissions and email-tool access configured earlier – they are independent controls.
5. At **Apply security template**, review the displayed settings and select **Next**. Record the actual template selection and obtain the security owner's sign-off on it.
6. Review the permissions screen and select **Next** only after that approval. Treat this as an actual consent review.
7. Review the final summary and select **Publish**.

Check result

- The submission status, the security template applied, and the security owner's sign-off are recorded.
- Discovery and installation are tested with an intended, non-owner account in 7.9.6, followed by separate source-access and email tests.

7.7 Communication Compliance policy

Documentation: Configure a Communication Compliance policy to detect generative AI interactions · Create and manage Communication Compliance policies · Get started with Communication Compliance

Once agents can send email and chat with users, someone needs to review what they actually say. A Communication Compliance policy does not block anything by itself – it supervises agent interactions and routes matches to a human reviewer, a complementary control to the DLP policies in 7.2 and 7.3.

7.7.1 Open Communication Compliance and start the policy

Performed by **Communication Compliance Administrators**.

1. In Microsoft Purview, select **Solutions** > **Communication Compliance**. Confirm the operator account has approved access to this workflow.
2. Under **Policies**, check for existing policies to avoid creating an unnecessary duplicate.
3. Select **Create policy**.

7.7.2 Choose the template and set sources and reviewer

Performed by **Communication Compliance Administrators**.

1. Select the template **Detect unethical interactions for AI agents**. Check that the template is available in your environment before continuing.
2. Under the agents to supervise, select **Copilot Studio** and **Azure AI Foundry**.
3. Add yourself or the approved reviewer as reviewer.
4. Select **Create policy**.

Check result

- The saved scope (Copilot Studio and Azure AI Foundry) and the authorized reviewer are recorded.
- Policy creation alone does not demonstrate event ingestion or review; that is checked in 7.9.8 once agent traffic exists.

7.8 Insider Risk Management verification

Documentation: Monitoring agents with Insider Risk Management · Create and manage Insider Risk Management policies

Agent risk detection only works if the underlying policy is actually turned on – a step that is easy to assume rather than confirm. This section verifies an existing policy rather than configuring a new one.

7.8.1 Confirm the default agent policy is enabled

Performed by **Insider Risk Management Admins**.

1. In Microsoft Purview, select **Solutions** > **Insider Risk Management**.
2. Under **Policies**, switch to the **Agent policies** tab. Agent policies are listed separately from the standard user-risk policies, so check you are on the correct tab before searching.
3. Find **Default policy for agents** and open it or inspect its status.
4. Confirm it is enabled with the scope you expect. Seeing the policy name in the list does not prove it is enabled – open it and check.
5. If the policy is absent or inactive, escalate it and do not mark this configuration complete.

Check result

- The actual status and coverage of **Default policy for agents** are recorded.

7.9 Acceptance checklist (tests)

The sections above configure the PoC; they do not by themselves prove that it works. Run every check below before calling the environment validated, and record the result of each. Policy updates can take up to four hours to reach Microsoft 365 Copilot and Copilot Chat, so run 7.9.3 only after that time has passed since 7.2.5. Start a new conversation with the agent for each test prompt.

7.9.1 Policy configuration

Performed by the **read-only reviewer** for the DLP policies (7.2, 7.3), **Communication Compliance Administrators** for 7.7, and **Insider Risk Management Admins** for 7.8.

1. For each policy (7.2, 7.3, 7.7, 7.8), record its name, scope, mode, conditions, and exceptions.

Expected result

- Each recorded configuration matches what was set up in its section.

7.9.2 Knowledge baseline

Performed by an **authorized test user**.

1. In each agent (7.5 and 7.6), ask for a harmless fact from the approved comparison source (see 7.4.4).
2. Record the response and its citation.

Expected result

- The authorized user retrieves the harmless fact from both agents.

7.9.3 Label restriction

Performed by an **authorized test user**.

1. In each intended agent (7.5 and 7.6) and channel, ask for a synthetic fact that is unique to a **Confidential**-labeled file – for example, the bank account number for invoice `CT-INV-1001`.
2. Record the response and any available policy evidence for each agent and channel.

Expected result

- The fact from the **Confidential**-labeled file is not returned in the response. The file can still appear in the response's citations, but its content isn't used.

7.9.4 Email restriction

Performed by an **authorized test user** with the Copilot Studio agent from 7.6.

1. Ask the agent to send a benign control message to the controlled recipient.
2. Ask the agent to send approved synthetic matching messages to the controlled recipient, testing the detectors separately (only the credit card number from 7.4.3; only the ABA routing number) and together (both). Include the synthetic values in your request, because the agent can't use the content of the **Confidential**-labeled invoice.
3. Check the controlled recipient's mailbox and record, for each message, whether it was delivered.

Expected result

- The benign control message is delivered; messages with matching payment data are not delivered, according to the detector settings recorded in 7.3.3.

7.9.5 Tool identity

Performed by the **PoC maker account**.

1. Verify the actual sending account of the email tool, the recipient mapping, and the confirmation behavior, using the messages received in 7.9.4.
2. Compare them with the settings recorded in 7.6.4 and the instructions recorded in 7.6.1.

Expected result

- The sending account, recipient, and confirmation behavior match the recorded settings and instructions.

7.9.6 Agent availability

Performed by an **intended non-owner test account**.

1. Discover the Copilot Studio agent published in 7.6.9 and install it.
2. Open the agent built in 7.5 from the same account.

Expected result

- The non-owner account can discover, install, and use the agents.

7.9.7 Permissions

Performed by an **authorized test user** and an **unauthorized test user**.

1. As the authorized test user, confirm the intended access to the site, the files, and both agents.
2. As the unauthorized test user, try to open the site and both agents.

Expected result

- The authorized test user has the intended access; the unauthorized test user does not.

7.9.8 Compliance review

Performed by the **Communication Compliance reviewer**.

1. Open the Communication Compliance policy created in 7.7.
2. Review the approved test evidence generated by the agent tests above.

Expected result

- The reviewer can access the policy and review the test evidence.

7.9.9 Default policy

Performed by **Insider Risk Management Admins**.

1. Record the actual status and coverage of the **Default policy for agents** (7.8.1).

Expected result

- The policy is enabled, and its status and coverage are recorded.

7.10 Evidence

- Policy name, status, rule name, condition, action, and mode of `PoC - Copilot Confidential Content` (7.2).
- Policy name, location, included and excluded users, detector grouping, confidence level, instance count, action, and mode of `PoC - Email Payment Data` (7.3).
- Site URL, file names, the label of each file (including the decision for the Excel workbook), and the sharing scope and permission level (7.4).
- Agent names, descriptions, exact instructions, knowledge source, **Only use specified sources** setting, and sharing audience for the Microsoft 365 Copilot agent (7.5).
- For the Copilot Studio agent: email tool operation, connection identity, recipient, subject, body, authentication and confirmation settings, publish versions or timestamps, channel, availability option, submission status, security template applied, and the security owner's sign-off (7.6).
- Communication Compliance template, agents supervised, and reviewer (7.7).
- Status and coverage of **Default policy for agents** (7.8).
- The recorded result of every acceptance check in 7.9, including responses, delivery results, and available policy evidence.

7.11 Troubleshooting

Symptom	Likely cause	Fix
A PoC test email is delivered although it contains payment data	An included or excluded user or group setting makes the message skip the policy, or the detector grouping, confidence level, or instance count does not match the test content	Review the included and excluded users in 7.3.2 and the recorded detector settings in 7.3.3, then retest with controlled recipients.
Copilot still uses a file you expected to be restricted	The saved file carries a different label than intended, the rule's label condition contains other labels or sublabels, or the policy update hasn't reached Copilot yet	Reopen the saved file and check the applied label (7.4.3); check that only the intended label or sublabels are in the rule (7.2.3); allow up to four hours after 7.2.5 before retesting.
Restriction appears to work in one agent or channel but not another	The **Accessing knowledge sources** action does not automatically cover every Copilot operation or agent channel	Test each intended agent and channel (7.9.3) and configure any broader restriction explicitly.
Routing or account numbers look wrong in the workbook	Columns left in Excel's **General** format dropped a leading zero or switched to scientific notation	Format **Routing** and **Account** as **Text** before entering values (7.4.4).
Email tool sends as an unexpected account	The tool's **Authentication** is set to **Maker-provided**, so email is sent from the connection account instead of the user chatting with the agent	Check the **Authentication** setting recorded in 7.6.4 and verify the sending account in 7.9.5.
Several similarly named Outlook actions are listed	Outlook exposes more than one email action	Select **Send an email (V2)** (7.6.3).
Users can't find or install the Copilot Studio agent	Channel added but agent not yet approved and published to the store, or channel changes not republished	Complete 7.6.8 and 7.6.9, then test with a non-owner account (7.9.6).
Communication Compliance policy exists but shows nothing to review	Policy creation alone does not demonstrate ingestion; no agent traffic yet	Generate the agent tests in 7.9, then check again as the reviewer (7.9.8).

Symptom	Likely cause	Fix
Default policy for agents appears in the list, but its status is unknown	The name in the list does not prove the policy is enabled	Open the policy and check its status; escalate if absent or inactive (7.8.1).

✅ What you proved

- [ ] A DLP policy scoped to Microsoft 365 Copilot and Copilot Chat restricts Copilot from processing content that carries the **Confidential** sensitivity label.
- [ ] A separate DLP policy scoped to Exchange email blocks delivery of email that contains credit card or ABA routing numbers.
- [ ] A Microsoft 365 Copilot agent and a Copilot Studio agent use the PoC SharePoint site as their knowledge source; an authorized user retrieves a harmless fact from an approved comparison source.
- [ ] A synthetic fact unique to a **Confidential**-labeled file is not returned by either agent in any intended channel, and the response and available policy evidence are recorded.
- [ ] Email that the Copilot Studio agent's email tool sends with approved synthetic payment data to a controlled recipient isn't delivered, while a benign control message is delivered.
- [ ] The Copilot Studio agent is approved and published to the store by an administrator, and an intended non-owner account can discover and install it.
- [ ] A Communication Compliance policy for AI agents is created and the authorized reviewer can access it and review test evidence.
- [ ] The **Default policy for agents** in Insider Risk Management is confirmed enabled, and its actual status and coverage are recorded.

References

- Microsoft Purview DLP for Microsoft 365 Copilot and Cowork
- Create and deploy a DLP policy
- DLP policy reference
- Credit card number entity definition
- ABA routing number entity definition
- Manage site creation in SharePoint
- Manage sensitivity labels in Office apps
- Enable sensitivity labels for Office files in SharePoint and OneDrive
- Build agents in Agent Builder
- Add knowledge sources to an agent in Agent Builder
- Share and manage agents built in Agent Builder
- Add SharePoint as a knowledge source
- Add tools to custom agents
- Office 365 Outlook connector
- Key concepts – Publish and deploy your agent
- Connect and configure an agent for Teams and Microsoft 365
- Manage agent requests in Microsoft 365 admin center
- Agent overview in Microsoft 365 admin center
- Configure a Communication Compliance policy to detect generative AI interactions
- Create and manage Communication Compliance policies
- Get started with Communication Compliance
- Monitoring agents with Insider Risk Management
- Create and manage Insider Risk Management policies

Runbook index · Previous: Chapter 6 – Conditional Access and least privilege · Next: Chapter 8 – Threat detection and runtime protection (Defender)

Prerequisites - Chapter 8: Threat detection and runtime protection (Defender)

> **Goal:** Prepare the accounts, access, and test assets needed for this chapter and its selected optional paths.

Apply the shared PoC preparation guidance and agree the chapter outcomes. Finish dependent build steps before their tests.

Table of contents

- Tenant readiness
- Prepare accounts and role assignments
- Audit recording
- Test assets and dependencies
- Security for AI preparation sequence
- Enable security for AI
- Connect the Microsoft 365 connector
- Connect Copilot Studio real-time protection
- Prepare real-time protection rules
- Prepare a published Foundry agent (optional)
- Prepare local runtime protection (optional)
- Prepare a custom detection rule (optional)
- Prepare test cases and baselines
- Setup troubleshooting
- Plan propagation and test timing
- References

Tenant readiness

Documentation: Get started with Microsoft Agent 365 · Agent management in the Microsoft 365 admin center · Frontier program

Confirm an Agent 365-eligible license and onboarding before enabling agent security. Use the setup permissions for preparation and 8.1 Required permissions for testing.

Follow the shared tenant readiness check using the chapter permissions and feature prerequisites linked above.

Prepare accounts and role assignments

Alongside the shared setup and reviewer accounts, prepare the following scenario accounts:

Account	Purpose
Standard test user	Runs the runtime-protection test prompts and verifies containment. No admin role.
Maker account	Prepares the published Copilot Studio agent and any optional Foundry test agent.
Controlled recipient mailbox	Receives any permitted test email from the Copilot Studio agent. Use only the controlled recipient prepared for the email-agent tests.

Assign the setup permissions and the test permissions using the shared model. Verify each account can access the pages needed for its tasks.

Setup permissions

Task	Least-privilege role
Enable Security for AI, connect Microsoft 365 and Copilot Studio, manage protection rules, and configure prompt evidence	Security Administrator
Register the provider authentication application and federated identity credential	User permitted to create application registrations in Microsoft Entra
Configure the Copilot Studio environment's threat detection provider	Power Platform Administrator
Add the synthetic test document and verify the Copilot Studio knowledge source, tools, and publication	PoC maker account from Chapter 7
(Optional) Publish the Foundry test agent to Microsoft 365 Copilot and Teams	Foundry User on the Foundry project; AI Administrator for approval if organizational publication is selected
(Optional) Configure local runtime protection on a test device	Local administrator for elevated PowerShell
(Optional) Deploy local runtime protection policies	Permission to manage endpoint security policies in Defender, or an Intune role such as Policy and Profile Manager
(Optional) Create an Advanced Hunting custom detection	Security Administrator

Security Operator can manage custom detections only when Defender for Endpoint RBAC is off. When it is on, the operator also needs **Manage Security Settings**. Use the documented setup roles rather than elevating the read-only reviewer.

Audit recording

Documentation: Turn auditing on or off · Audit logs for Copilot and AI applications

Performed by a member of **Audit Manager**. Follow the shared audit recording check before the activity to be reviewed.

Test assets and dependencies

Documentation: Enable security for AI agents using Microsoft Defender · Protect AI agents using Microsoft Defender · Enable security for AI agents using Microsoft Defender – Connect data sources · Enable external threat detection and protection for Copilot Studio custom agents · Protect AI agents in real time using Microsoft Defender

- Verify the required inventory using Chapter 1 – Agent discovery and inventory. Agent 365 must be onboarded in the tenant before you enable security for AI agents.
- **Copilot Studio test agent:** use the Copilot Studio email agent from 7.6 Copilot Studio email agent and publication. It has a SharePoint knowledge source and the **Send an email (V2)** tool, and is published to Microsoft 365 Copilot and Teams. It must use generative orchestration, because external threat detection is only called on agents that use generative orchestration. Complete the Chapter 7 tests before you run 8.5, because 8.5.6 blocks this agent.
- (Optional, Foundry in scope) Prepare a published agent from 1.2.2 Create and publish a Foundry test agent, then complete Foundry preparation.
- (Optional, local agents in scope) Prepare the Windows test device onboarded to Defender for Endpoint, as described in 9.2 Tenant and device prerequisites, with a supported local AI agent installed.
- Ownerless-agent governance (identify ownerless agents, reassign ownership, Agent management rules) is covered in 5.5 Govern ownerless agents. This chapter only links to it.
- Shadow AI and local agent discovery in the Microsoft 365 admin center are covered in Chapter 9 – Shadow AI and local agents. This chapter only covers the optional Defender for Endpoint runtime protection for local agents (8.6).
- Detection and investigation of AI agent threats in Microsoft Defender is in public preview. Copilot Studio external threat detection and real-time protection for Copilot Studio agents are in preview.

Check result

- Each asset needed for the selected paths is prepared or has a named build task that will finish before its dependent tests.

Security for AI preparation sequence

Documentation: Enable security for AI agents using Microsoft Defender · Enable external threat detection and protection for Copilot Studio custom agents · Protect AI agents in real time using Microsoft Defender · AI agent runtime protection in Microsoft Defender for Endpoint

Complete the shared cloud-agent connection, Copilot Studio integration, and rule preparation below before testing. Prepare Foundry or local runtime protection only when those paths are selected. Estimates cover hands-on work; service and approval waits are separate.

Preparation	Estimated hands-on time
Enable security for AI	~5-10 min
Connect Microsoft 365	~10-15 min
Connect Copilot Studio	~20-30 min
Prepare protection rules	~5-10 min
Prepare a published Foundry agent (optional)	~10-15 min if the agent is already built
Prepare local runtime protection (optional)	~15-25 min

Check result

- Each selected setup section's **Check result** is confirmed before its dependent tests.

Enable security for AI

Documentation: Enable security for AI agents using Microsoft Defender · Protect AI agents using Microsoft Defender

When you onboard to Agent 365, security for AI agents is enabled automatically, including AI agent discovery, security posture assessment, and threat detection.

Review the Get started checklist

Performed by **Security Administrator**.

1. Sign in to the Microsoft Defender portal at `https://security.microsoft.com`.
2. Go to **Settings** > **Security for AI** > **Get started**.
3. Confirm that the **Enable** toggle is on. It is on by default. Switching it to **Off** stops collecting data for AI agents; leave it on for the PoC.
4. Review the setup checklist. It shows the status of the required and optional data sources:
   - **Agent 365**: provides unified visibility into AI agents and is marked **Done** automatically.
   - **Microsoft 365 connector**: complete Connect the Microsoft 365 connector.
   - **Copilot Studio**: complete Connect Copilot Studio real-time protection.
5. Note the names under **Identified Power Platform Admins**. One of them completes the Power Platform connection.

Check result

- The **Enable** toggle is on.
- **Agent 365** shows **Done**.

Connect the Microsoft 365 connector

Documentation: Enable security for AI agents using Microsoft Defender – Connect data sources · Detect and investigate threats to AI agents using Microsoft Defender

The Microsoft 365 connector provides investigation and advanced hunting capabilities for AI agent activity, and collects the Agent 365 observability data that near-real-time detections rely on.

Connect Microsoft 365 components

Performed by **Security Administrator**.

1. On **Settings** > **Security for AI** > **Get started**, select the **Microsoft 365 connector** step.
2. On the **Select Microsoft 365 components** step, select at least the two components required for AI agent monitoring:
   - **Microsoft Entra ID Management events**: audit admin activities performed in Microsoft Entra ID.
   - **Microsoft 365 activities**: audit activities performed by users in your Microsoft 365 apps.

   Microsoft Entra Users and groups is a prerequisite for all monitoring capabilities and is selected by default.
3. Select **Connect Microsoft 365** to complete the connector setup.

Check result

- The **Microsoft 365 connector** step shows **Connected** on the **Get started** page.

> For Copilot Studio agents, if the Microsoft 365 connector isn't connected, real-time protection continues to block suspicious activity during runtime, but alerts and incidents related to these actions don't appear in the Microsoft Defender portal. Connect it before you run the tests in 8.5.

Connect Copilot Studio real-time protection

Documentation: Enable security for AI agents using Microsoft Defender – Connect data sources · Enable external threat detection and protection for Copilot Studio custom agents · Protect AI agents in real time using Microsoft Defender

With this integration, every time a Copilot Studio agent's orchestrator considers invoking a tool, it sends relevant data about the proposed tool use to the threat detection endpoint, which returns a decision to allow or block the tool invocation. If a security issue is detected, the agent immediately stops processing and notifies the user that their message is blocked. If the operation is approved, the agent proceeds with no visible effect for the user.

Data shared with the threat detection provider includes the user's recent prompt and the latest chat history, outputs of previous tools, conversation metadata (agent, user, tenant, and trigger where applicable), and the tool the agent wants to invoke, including the agent-generated reasoning and the proposed inputs.

Coordinate the setup in this order:

Step	Who	Where	Output
Defender connection	Security Administrator	Defender portal	Real-time protection on; integration URL copied
Authentication application	User with permissions to create application registrations	Microsoft Entra	App ID of an application with a federated identity credential
Power Platform connection	Power Platform Administrator	Power Platform admin center	Threat detection configured for the test environment
App ID validation	Security Administrator	Defender portal	App ID saved; Copilot Studio step **Connected**

Important:

- External threat detection is only called on agents that use generative orchestration. It is skipped for classic agents.
- The external security provider is configured per environment. There is no global or tenant-wide setting. Turn on threat detection for new environments after you create them.

Turn on real-time protection in the Defender portal

Performed by **Security Administrator**.

1. On **Settings** > **Security for AI** > **Get started**, select the **Copilot Studio** step. The **Copilot Studio real-time protection** pane opens.
2. Toggle **Real-time protection** to on.
3. Under **Enable Power Platform Integration**, copy the URL provided. Use the same endpoint for the authentication application and the Power Platform connection.

4. Share the URL with your Power Platform administrator and with the person who registers the Microsoft Entra application.

Check result

- **Real-time protection** is on and you have the integration URL.

Register the Microsoft Entra application

Performed by a **user with permissions to create application registrations** in your Microsoft Entra tenant.

The agent uses Federated Identity Credentials (FIC) as a secure, secret-less authentication method with the threat detection system. Use one of the two options below. You need your tenant ID and the endpoint URL from the Defender portal.

Option A – PowerShell script (recommended)

1. Download the Create-CopilotWebhookApp.ps1 script.
2. Open Windows PowerShell 5.1 or later as an administrator and go to the directory that contains the script.
3. Run the script with your values. With the optional `-DryRun` flag, the script performs a validation run without creating resources.

```
.\Create-CopilotWebhookApp.ps1 `
-TenantId "<your tenant ID>" `
-Endpoint "<integration URL from the Defender portal>" `
-DisplayName "A365 PoC - Copilot Studio Defender integration" `
-FICName "A365PoCDefenderFIC"
```

4. Copy the App ID that the script outputs.

Option B – Manual configuration in the Azure portal

1. Sign in to the Azure portal and go to the **Microsoft Entra ID** page.
2. Under **App registrations**, select **New registration**.
3. Provide a name and select **Accounts in this organizational directory only (Single tenant)**.
4. Select **Register**, then copy the App ID.
5. In the app, select **Manage** > **Certificates & secrets** > **Federated credentials**, then select **Add credential**.
6. In the **Federated credentials scenario** drop-down, select **Other issuer**, and fill in:
   - **Issuer**: `https://login.microsoftonline.com/{tenantId}/v2.0`
   - **Type**: **Explicit subject identifier**
   - **Value**: `/eid1/c/pub/t/{base 64 encoded tenantId}/a/m1WPnYRZpEaQKq1Cceg--g/{base 64 encoded endpoint}`
   - **Name**: a descriptive name
7. Select **Add**.

To get the base64 encoding of your tenant ID and endpoint URL, use the script from the Copilot Studio article:

```
# Encoding tenant ID
$tenantId = [Guid]::Parse("11111111-2222-3333-4444-555555555555")
$base64EncodedTenantId = [Convert]::ToBase64String($tenantId.ToByteArray()).Replace('+','-
').Replace('/','_').TrimEnd('=')
Write-Output $base64EncodedTenantId

# Encoding the endpoint
$endpointURL = "https://provider.example.com/threat_detection/copilot"
$base64EncodedEndpointURL =
```

```
[Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes($endpointURL)).Replace('+','-
').Replace('/','_').TrimEnd('=')
Write-Output $base64EncodedEndpointURL
```

Replace the placeholder tenant ID and endpoint URL with your values.

Check result

- You have the App ID of a single-tenant application with a federated identity credential.

Configure threat detection in the Power Platform admin center

Performed by **Power Platform Administrator**.

1. Sign in to the Power Platform admin center at `https://admin.powerplatform.microsoft.com`.
2. On the side navigation, select **Security**, then select **Threat detection**.
3. Select **Additional threat detection**. A pane opens.
4. Select the environment that hosts the Copilot Studio test agent and select **Set up**.
5. Select **Allow Copilot Studio to share data with a threat detection provider**.
6. Under **Azure Entra App ID**, enter the App ID from the authentication application.
7. Enter the **Endpoint link**: the integration URL from the Defender portal, the same base URL used for the Microsoft Entra application.
8. Under **Set error behavior**, define the default behavior when the threat detection system doesn't respond in time or responds with an error. The default is **Allow the agent to respond**; you can choose **Block the query** to further reduce risk.
9. Select **Save**.
10. Share the App ID with the Security Administrator.

Check result

- The save succeeds. The save fails if the Microsoft Entra app isn't properly configured in Microsoft Entra or not properly authorized with the provider.

> If the agent doesn't receive a decision from the threat detection system within one second, by default it allows the tool to execute. **Set error behavior** changes this default.

Save the App ID in the Defender portal

Performed by **Security Administrator**.

1. Return to **Settings** > **Security for AI** > **Get started** and select the **Copilot Studio** step.
2. Paste the App ID from the Power Platform administrator into the **App ID** field. The Power Platform administrator must use the same App ID as the Microsoft Entra application.
3. Select **Save**.

Check result

- The App ID saves. If you recently changed the App ID in Power Platform, it can take up to one minute to propagate; if you get a validation error, wait a short time and try again.
- When the Power Platform administrator completes the onboarding steps, the **Copilot Studio** step shows **Connected**.

Prepare real-time protection rules

Documentation: Protect AI agents in real time using Microsoft Defender

Performed by **Security Administrator**.

For cloud agents, there are two types of real-time protection rules:

- **Default rule**: audits all agents, recording matching activity as a behavior without stopping the action.
- **Custom rules**: block matching actions before they execute and record the behavior.

When Defender audits or blocks an action, it records the event as a behavior in the `BehaviorInfo` table. Near-real-time detections continue to surface as alerts only in audit mode. When a blocking rule covers an agent, near-real-time alerts aren't generated for that agent.

1. Go to **Settings** > **Security for AI** > **Policies & rules** > **Real-time protection**.
2. Confirm that the built-in **Default** rule is listed.
3. Go to **Settings** > **Security for AI** > **Prompt evidence collection** and review the **Enabled** toggle. When enabled (the default), each alert includes as evidence only the portions of user prompts or agent responses identified as suspicious, with sensitive data and secrets redacted. Agree with the customer whether to keep it on for the PoC.
4. (Optional, for the blocking pass in 8.5.3) Create a custom rule for the test agent:
    1. Select **Create rule**.
    2. On the **Rule details** step, set the rule **Status** to off, enter the **Rule name** `A365 PoC - block test agent` and a **Rule description**, then select **Next**.
    3. Under **Scope**, choose specific agents and select the Copilot Studio test agent. Use a separate rule scoped to the Foundry test agent if that path is selected.
    4. Under **Detection types**, select the agreed scenarios from **Malicious content propagation**, **Secret exfiltration**, **Evasion techniques**, and **Unsafe email domain**, then select **Apply**. Only agents with a Microsoft Entra agent ID appear in the selection list.
    5. Select **Next**, confirm the rule name, description, action, scope, detection rules, and status, then select **Create**.

Check result

- The **Default** rule is listed, and prompt evidence collection is set as agreed.
- (Optional) The custom rule is listed, scoped to the test agent, and not enabled until 8.5.3.

Prepare a published Foundry agent (optional)

Documentation: Detect and investigate threats to AI agents using Microsoft Defender · Protect AI agents in real time using Microsoft Defender · Transition agent security capabilities to Agent 365 · Publish agents to Microsoft Copilot and Microsoft Teams

Defender for Cloud is not a prerequisite for these Foundry agent tests. Agent discovery, posture, and agent-specific threat detection use Agent 365 and its observability data in Microsoft Defender. The Defender for AI Services plan remains relevant to Foundry **models**, such as Azure OpenAI, and separate infrastructure protection. Enable that plan only for a separately agreed model or infrastructure scenario; see Enable threat protection for AI services.

Performed by **Foundry User** on the project (publication), **Security Administrator** (Defender configuration), and the **standard test user** (access check).

1. Complete Enable security for AI and Connect the Microsoft 365 connector. Foundry agents send observability data to Microsoft 365 by default; confirm activity arrives rather than assuming it does.
2. Select the tested active version of the Foundry agent and publish it to Microsoft 365 Copilot and Teams using the linked publishing guide. Threat detection doesn't support unpublished agents or playground-only sessions. Current direct publishing handles Teams bot registration; a separate Azure Bot Service resource isn't required.
3. Use a publication scope approved for the PoC. With **Just you**, sharing a link doesn't grant Foundry permissions: participants need the required project permissions. With **People in your organization**, obtain approval in the Microsoft 365 admin center and verify the applicable app access policies before testing.
4. Confirm the standard test user can open the agent in its published channel.
5. In Defender, verify the agent appears in **Assets** > **AI agents** and record its agent identifiers. Confirm a benign invocation appears in `CloudAppEvents` using 8.4.3.
6. If a blocking pass is selected, prepare a separate disabled rule scoped to the Foundry test agent as in Prepare real-time protection rules. Foundry coverage includes user requests, agent responses, tool invocations, and tool responses.

Check result

- The published agent is accessible to the test account, its inventory and activity are visible in Defender, and any optional block rule is scoped to it and disabled before the audit pass.

Prepare local runtime protection (optional)

Documentation: Set up AI agent runtime protection with Microsoft Defender for Endpoint · AI agent runtime protection with Microsoft Defender for Endpoint · AI agent runtime protection demonstration

Run this section only if local AI agents are in scope. Local agents are onboarded separately from cloud agents. Runtime protection inspects the user prompt, tool requests before execution, and tool responses after execution, detects prompt injection, and audits or blocks supported agent actions. Discovery of local agents and Shadow AI is covered in Chapter 9 – Shadow AI and local agents.

Prerequisites on each test device:

- The device is onboarded to Defender for Endpoint, and Microsoft Defender Antivirus is running in active mode with real-time protection enabled.
- The device runs a supported version of Windows, and Microsoft Defender Antivirus has the latest platform, engine, and security intelligence updates.
- One or more supported local AI agents are installed for the runtime protection approach you plan to enable.
- During public preview, the test device receives Microsoft Defender platform and engine updates from the **Beta Channel**.

Two runtime protection methods are available, each with the modes `Disabled`, `Audit`, and `Block`:

- `AiAgentProtection`: agent-native event inspection, for agents that expose vendor-supported agent event interfaces (such as Claude Code, Codex CLI, and GitHub Copilot CLI).
- `AiAgentNetworkInspection`: network inspection, for agents that don't expose vendor-supported agent event interfaces. Network inspection doesn't support agents that use certificate pinning or HTTP/3.

Microsoft recommends a phased rollout: **Test** in audit mode on a small set of devices, **Review** alerts for one to two weeks, **Deploy** in audit mode to more device groups, then **Enforce** with block mode.

Turn on runtime protection on a single test device

Performed in an elevated PowerShell session (**Run as administrator**) on the test device.

1. Verify that `AntivirusSignatureVersion` is `1.451.224.0` or later:

```
Get-MpComputerStatus | Select-Object AntivirusSignatureVersion
```

2. Enable the method or methods you need in audit mode:

```
Set-MpPreference -AiAgentProtection Audit
```

```
Set-MpPreference -AiAgentNetworkInspection Audit
```

3. Verify the current settings:

```
Get-MpPreference | Select-Object AiAgentProtection, AiAgentNetworkInspection
```

4. Close the PowerShell window and any terminal windows used to run agents. Then open a new terminal window before starting the agent.

Check result

- `Get-MpPreference` returns `Audit` for each method you enabled.

Deploy runtime protection with a policy

Performed by an account with **permission to manage endpoint security policies** (Defender portal) or an Intune role such as **Policy and Profile Manager** (Intune).

From the Defender portal:

1. On the **Windows policies** tab of the **Endpoint security policies** page (`https://security.microsoft.com/policy-inventory?osPlatform=Windows`), select **Create new policy**.
2. For **Select platform**, select **Windows**. For **Select template**, select **Microsoft Defender AI agent runtime protection**.
3. On the **Configuration settings** page, set **Ai Agent Protection** to **Audit**.
4. Complete the wizard and assign the policy to a test device group. For devices managed through Defender for Endpoint security settings management that aren't enrolled in Intune, assign the policy to Microsoft Entra device groups.

From Intune:

1. In the Intune admin center at `https://intune.microsoft.com`, on the **Endpoint security | Overview** page, go to **Manage** > **Antivirus** and select **Create policy**.
2. For **Platform**, select **Windows**. For **Profile**, select **Microsoft Defender AI agent runtime protection**.
3. On the **Configuration settings** page, set **Ai Agent Protection** to **Audit**.
4. Complete the wizard, assign the policy to a test device group, and create the policy.

The **Microsoft Defender AI agent runtime protection** profile and template configure agent-native event inspection. To deploy network inspection with Intune, use a PowerShell platform script that runs `Set-MpPreference -AiAgentNetworkInspection Audit`, with **Run this script using the logged on credentials** set to **No**.

Check result

- In the Defender portal, open the device page, select the **Configuration management** tab, select **Effective settings**, find **Ai Agent Protection**, and confirm its effective value and configuration source.

Prepare a custom detection rule (optional)

Documentation: Create custom detection rules

Performed by **Security Administrator**.

This rule alerts on tool executions by the PoC test agent. It shows that agent activity can drive custom SOC detections.

1. In **Advanced hunting**, run the following query with the agent identifier recorded in 8.4.3. It returns `Timestamp` and `ReportId` from the same event, and `AccountObjectId` to map the impacted account.

   ```
   let pocAgentIds = dynamic(["<agent identifier from 8.4.3>"]);
   CloudAppEvents
   | where ActionType in ("ExecuteToolBySDK", "ExecuteToolByGateway", "ExecuteToolByMCPServer")
   | extend resData = parse_json(tostring(RawEventData))
   | extend AgentId = tostring(resData.AgentId)
   | where AgentId in (pocAgentIds)
   | project Timestamp, ReportId, AccountObjectId, ActionType, AgentId
   ```

2. Select **Create detection rule**.
3. Enter the alert details: **Detection name** `A365 PoC - test agent tool execution`, **Frequency Every hour**, **Alert title**, **Severity**, **Category**, **Description**, and **Recommended actions**.
4. Map the impacted asset to the `AccountObjectId` column.
5. Don't add response actions. Set the rule scope, review the rule, and select **Create**. The rule runs immediately, then again based on the configured frequency.
6. Go to **Hunting** > **Custom detection rules** and confirm that the rule is listed and its **Status** is on.

Check result

- The rule is listed and enabled, with no automated response actions. Validate its triggered alert during 8.4.5.

Prepare test cases and baselines

Performed by **Security Administrator** with the **PoC maker account**.

#	Precondition	How to check
1	Security for AI is enabled; Microsoft 365 connector and Copilot Studio show **Connected**	Security for AI checklist, Microsoft 365 connection, App ID validation
2	Threat detection is configured for the environment that hosts the test agent	Power Platform connection
3	The test agent uses generative orchestration	In Copilot Studio (`https://copilotstudio.microsoft.com`), open the agent's **Settings** page; in the **Generative AI** section, check **Orchestration**. New agents use generative orchestration by default.

#	Precondition	How to check
4	The test agent has a knowledge source and at least one tool: the SharePoint knowledge source and the **Send an email (V2)** tool of the 7.6 agent	Agent configuration in Copilot Studio (7.6.2, 7.6.3)
5	The test agent is published to Microsoft 365 Copilot and Teams, and the standard test user can use it there	Test user opens the agent (7.9.6)
6	The test agent appears in **Assets** > **AI agents** and in `AgentsInfo`	8.3.1, 8.4.1
7	The **Default** real-time protection rule is listed; the optional custom block rule exists and is not enabled	Protection rules
8	Baseline captured: risk level and risk indicators in Defender, **Risks** count in the registry	8.3.2, 8.2.1
9	For scenario 1, the PoC maker account has added an unlabeled test document with the injected content to the SharePoint site that the agent uses as knowledge (7.4)	The document is listed in the site library

> Custom agents created in Copilot Studio include built-in protection against user prompt-injection attacks (UPIA) and cross-domain prompt injection attacks (XPIA), and block these attacks at runtime. Record the agent's response for each scenario.

Before testing, the PoC team replaces every `<to be added>` prompt with an approved test case, records the expected detection or rule type, and prepares any synthetic knowledge document needed for it. Never use real credentials or uncontrolled recipients. For the audit pass, verify that no existing blocking rule covers the test agent; don't change unrelated rules to force a result.

Check result

- All applicable preconditions are verified, test inputs are agreed, and baseline evidence is captured. Record excluded optional paths before opening the test chapter.

Setup troubleshooting

Symptom	Likely cause	Fix
Settings > **Security for AI** setup isn't available	The tenant isn't onboarded to Agent 365	Onboard to Agent 365 (prerequisite), see Tenant readiness and Chapter 1.
Microsoft 365 connector doesn't show **Connected**	Setup wasn't completed, or the required components weren't selected	Reopen the connector step, select **Microsoft Entra ID Management events** and **Microsoft 365 activities**, and select **Connect Microsoft 365**.
Validation error when saving the App ID in Defender	The App ID was recently changed in Power Platform	Wait a short time (up to one minute) and try again. Use the same App ID as the Microsoft Entra application.

Symptom	Likely cause	Fix
Power Platform: "There was a problem connecting to the protection provider. Try checking the endpoint link. If that doesn't work, contact the protection provider for help."	A call to the provided endpoint failed	Check the endpoint link (the integration URL from the Defender portal).
Power Platform: "There was a problem with the configuration. Try checking the details you entered and the Microsoft Entra configuration. If the problem persists, contact your admin for help."	Token acquisition failed	Check the Microsoft Entra application configuration and the Federated Identity Credentials. Select **Copy error info** for details.
Power Platform: "To change a configuration, make sure you have Power Platform admin permissions."	The user isn't a Power Platform Administrator	Use a user with the Power Platform Administrator role.
Copilot Studio step doesn't show **Connected**	The Power Platform administrator hasn't completed the onboarding steps	Ask one of the **Identified Power Platform Admins** to complete the Power Platform connection.

Plan propagation and test timing

Documentation: Agent Registry

Configuration	Propagation window to plan for
Risk signals in the Agent Registry compared with the source security portals	Up to 1 hour

> ⚠️ **Watch out:** Allow for the documented propagation window and verify the actual configuration or signal before testing. Elapsed time alone is not proof that a change has applied.

References

- Get started with Microsoft Agent 365
- Agent management in the Microsoft 365 admin center
- Frontier program
- Turn auditing on or off
- Audit logs for Copilot and AI applications
- Enable security for AI agents using Microsoft Defender
- Protect AI agents using Microsoft Defender
- Enable security for AI agents using Microsoft Defender – Connect data sources
- Enable external threat detection and protection for Copilot Studio custom agents
- Protect AI agents in real time using Microsoft Defender
- Enable threat protection for AI services
- AI agent runtime protection in Microsoft Defender for Endpoint
- Detect and investigate threats to AI agents using Microsoft Defender
- Transition agent security capabilities to Agent 365
- Publish agents to Microsoft Copilot and Microsoft Teams

- AI agent runtime protection with Microsoft Defender for Endpoint
- AI agent runtime protection demonstration
- Create custom detection rules
- Agent Registry

Additional resources

- Create-CopilotWebhookApp.ps1

> **Ready?** Continue to the chapter. →

Chapter 8 - Threat detection and runtime protection (Defender)

Pillar: Secure **What it proves:** The SOC can investigate AI agent activity in Microsoft Defender, correlate a detected threat with its agent and incident, review the registry risk signal, and contain the agent. Optional blocking tests verify enforcement separately from detection.

Success criteria: Agree the selected outcomes in What you proved before testing.

Topics and time guide

Approximate hands-on estimates after the prerequisites are complete. Preparation estimates are on that page; service waiting time is separate.

Topic	Estimated hands-on time
8.1 Required permissions	~5 min
8.2 Review agent risk in the Agent 365 registry	~5-10 min
8.3 Review the AI agent inventory in Defender	~5-10 min
8.4 Hunt for agent activity with Advanced Hunting	~10-15 min
8.5 Test runtime protection end to end	~15-25 min
8.6 Test local runtime protection (optional)	~10-15 min
8.7 Evidence	~5-10 min
8.8 Troubleshooting	As needed

Outcome checklist · References

8.1 Required permissions

Use the shared account and role-assignment model with the task-specific permissions below.

Task	Least-privilege role	Section
View the Agent 365 registry, **Agents at risk**, and **Risk details**	AI Reader, Global Reader, or Security Reader	8.2, 8.5.5
Open the Microsoft Defender deep link from **Risk details**	Global Reader, Security Reader, Security Administrator, or AI Administrator	8.2.2
Block or unblock an agent in the registry	AI Administrator	8.2.3, 8.5.6
Manage alert status and classification	Security Operator	8.5.4, 8.5.6, 8.5.7
Enable or disable the prepared test-agent blocking rule during a test pass	Security Administrator	8.5.3, 8.5.7
Run the test scenarios	Standard test user	8.5.3, 8.5.7

Task	Least-privilege role	Section
Validation / read-only review: Advanced Hunting, alerts, Defender views opened from the registry	Security Reader	8.6, 8.3, 8.4, 8.5.4

Before you start:

- [ ] Complete the chapter prerequisites, including setup permissions, connections, protection rules, and approved test inputs. This page contains testing only.

8.2 Review agent risk in the Agent 365 registry

Documentation: Agent Registry in Microsoft 365 admin center – Agent risks · Governance and lifecycle actions for agents – Block or unblock agents · Agent management roles and permissions

Risk signals in the Agent Registry are a consolidated set of agent-related security detections from multiple security platforms. IT administrators can understand what is flagged without switching between portals or elevating their permissions to different security roles, and share the details with the security team for investigation and remediation. Risk signal counts might be up to an hour behind what the security portals show, and reflect active risk signals. The Registry baseline, including the **Agents at risk** tile, is recorded in 1.3.4 Review risk signals; this section focuses on investigating and containing a flagged agent.

8.2.1 Open Agents at risk

Performed by **AI Reader**, **Global Reader**, or **Security Reader**.

1. Sign in to the Microsoft 365 admin center at `https://admin.cloud.microsoft`.
2. Go to **Agents** > **Overview** and review the **Agents at risk** card. It lists the three agents with the highest aggregated risk counts. Select **View agents** to open **All agents** > **Registry**, filtered and sorted by risk level.
3. Alternatively, go to **Agents** > **All agents** > **Registry** and select the **Agents at risk** tile. It opens a prefiltered view of agents with one or more risk signals.
4. Review the **Risks** column. It shows the aggregated count of risk signals for each agent.

Check result

- You can see the **Agents at risk** tile and the **Risks** column. A count of zero means that no active signals are currently detected for that agent.

8.2.2 Review risk details for a flagged agent

Performed by **AI Reader**, **Global Reader**, or **Security Reader**.

1. In the registry, select the count in the **Risks** column for a flagged agent. The **Risk details** pane opens.
2. Review the risk signals, grouped into expandable high, medium, and low severity sections, with a description and an **Occurrences** count for each signal.
3. Review **Risk signals are sourced from**. It shows the Microsoft security platforms contributing risk signals for the agent, such as Microsoft Purview, Microsoft Entra, and Microsoft Defender. Only platforms with contributing signals are shown.
4. If the agent has multiple instances, drill down into the signals for the selected instance.
5. Select a platform name to follow the deep link into that portal. The Microsoft Defender deep link requires Global Administrator, Global Reader, Security Reader, Security Administrator, or AI Administrator.

Check result

- For one agent, you can show which platforms contributed its risk signals and open the corresponding portal.

> Purview risk signals come from the Insider Risk Management agent policy verified in 7.8 Insider Risk Management verification. Entra ID Protection for agents is covered in 3.9 Detect and respond with ID Protection. The Microsoft Purview IRM alerts deep link requires IRM Analyst or IRM Investigator; Global Administrator alone is insufficient.

8.2.3 Block a risky agent from the registry

Performed by **AI Administrator**.

Use this procedure for containment in 8.5.6.

1. Sign in to the Microsoft 365 admin center and select **Agents** > **All agents**.
2. Select the agent from the list.
3. In the agent details pane, immediately under the agent's name, select **Block**.
4. In the **Block agent** pane, select **Block agent**, then select **Save**.

To unblock, select **Unblock** in the agent details pane, then in the **Unblock agent** pane select **Unblock agent** and **Save**.

Blocking an agent restricts access to it across the organization, preventing any user from using it. For agents created with Microsoft Copilot Agent Builder and Microsoft Copilot Studio, blocking affects availability and functionality in Microsoft Copilot and in other host products, such as Outlook and Teams. Blocking an agent created with SharePoint or Microsoft Foundry only affects its availability in Microsoft Copilot Chat.

Check result

- The agent details pane now offers **Unblock** for the agent.

> Ownerless agents are handled in 5.5 Govern ownerless agents. Blocking and unblocking in the lifecycle context is covered in 5.3.1 Block or unblock an agent.

8.3 Review the AI agent inventory in Defender

Documentation: Discover AI agents and assess security posture using Microsoft Defender · AI agent posture risk in Microsoft Defender

Microsoft Defender provides a centralized inventory of AI agents and assesses their security posture. The inventory includes agents built with Microsoft Copilot Studio, Microsoft Foundry, Microsoft 365, and supported non-Microsoft platforms, and local AI agents discovered on endpoint devices.

8.3.1 Open the AI agents inventory

Performed by **Security Reader**.

1. Sign in to the Microsoft Defender portal at `https://security.microsoft.com`.

2. In the left navigation pane, select **Assets** > **AI agents**, then select the **Agents** tab.
3. Review the risk-related columns:
   - **Risk level**: the overall risk level calculated from the agent's active risk indicators: **High**, **Medium**, **Low**, **No known risk**, or **Not evaluated**.
   - **Risk indicators**: the conditions contributing to the agent's risk level. Examples include **Weak Instructions**, **High-usage Agent**, **Indirect Prompt Injection Exposure**, **Privileged Business-system Access**, and **Active Threat** (active security alerts are associated with the agent).
   - **Recommendations**: the number and severity of active security recommendations for the agent.
   - **Active alerts**: the number and severity of active alerts associated with the agent.
4. Use the filter bar to narrow the inventory by properties such as **Agent name**, **Platform**, **Publish status**, **Risk level**, or **Risk indicators**. Select **Customize columns** to add, remove, or reorder columns.
5. Filter on the Copilot Studio test agent and the other PoC agents.

Check result

- The PoC agents are listed in the inventory.

> Not all risk indicators apply to all agent platforms. Defender assesses an indicator only when the required platform support and data are available.

8.3.2 Review the agent details pane and agent page

Performed by **Security Reader**.

1. Select the Copilot Studio test agent to open its details pane. The pane displays the risk and configuration information available for the agent.
2. Select **Open Agent page**. The **Overview** tab shows the security and configuration information available for the agent, which can include risk level and active risk indicators, agent configuration details, identity and authentication information, endpoint and user context, attack-surface relationships, active alerts, security recommendations, tools, and MCP servers.
3. Record the agent's risk level and risk indicators as the baseline before the test in 8.5.

A risk indicator isn't always something that can be remediated; it might reflect the agent's intended purpose or design.

8.3.3 Review recommendations and incidents for the agent

Performed by **Security Reader**.

1. On the agent page, select **Security recommendations**. Select a recommendation to review its description and supporting evidence on the **Overview** tab, corrective actions on the **Remediation steps** tab, and affected agents on the **Exposed assets** tab.
2. Select **Incidents and alerts** to review security incidents and alerts associated with the agent. Select an incident to open its details pane, or select **Open incident page** for the full investigation experience.

The available tabs depend on the agent platform and the security information associated with the agent.

Check result

- You can open the **Security recommendations** and **Incidents and alerts** tabs for the test agent, where available.

> Recommendations are determined separately from the risk level. A high-risk agent might have no recommendations, and a lower-risk agent might receive one. Review risk level and recommendations together.

8.4 Hunt for agent activity with Advanced Hunting

Documentation: Detect and investigate threats to AI agents using Microsoft Defender · Advanced hunting overview · AgentsInfo table · CloudAppEvents table · BehaviorInfo table · AlertInfo table · AlertEvidence table · Troubleshoot Direct OTel Observability – Defender advanced-hunting query · Create custom detection rules · Manage existing custom detection rules

Table	Contents for AI agent investigation
`AgentsInfo`	Inventory and configuration details for AI agents, including agent identity, platform, ownership, and metadata (replaces `AIAgentsInfo`)
`CloudAppEvents`	Agent 365 observability data for AI agent activity, including agent actions, tool invocations, and data access events
`BehaviorInfo`	Behaviors that record real-time protection rule activity, including audit and block events
`AlertInfo`	Alert metadata, including alerts related to near-real-time detections
`AlertEvidence`	Entities and artifacts associated with alerts

Go to **Investigation & response** > **Hunting** > **Advanced hunting**. To use the prebuilt queries maintained by Microsoft, select the **Queries** tab, and then select **AI Agents**. Security Reader grants full read access to advanced hunting data.

8.4.1 Query the agent inventory

Performed by **Security Reader**.

The `AgentsInfo` table stores multiple snapshots of each agent over time. Use `arg_max(Timestamp, *)` to return the latest state of each agent.

```
AgentsInfo
| summarize arg_max(Timestamp, *) by AgentId
| where LifecycleStatus != "Deleted"
| project Timestamp, Name, Platform, PublishedStatus, LifecycleStatus, Availability,
          Owners, InstanceCount, Model, CreatedDateTime, EntraAgentID, SourceAgentId
| sort by CreatedDateTime desc
```

Expected result

- The PoC agents are listed. `PublishedStatus` is `Draft` or `Published`; `LifecycleStatus` is `Active`, `Blocked`, `Uninstalled`, or `Deleted`.
- Record the `AgentId`, `EntraAgentID`, and `SourceAgentId` of the Copilot Studio test agent.

8.4.2 Hunt for weak agent configuration

Performed by **Security Reader**.

This query lists active agents with no system prompt in `Instructions`, together with their guardrails, tools, and MCP servers for review.

```
AgentsInfo
| summarize arg_max(Timestamp, *) by AgentId
| where LifecycleStatus == "Active"
| where isempty(Instructions)
| project Name, Platform, PublishedStatus, Availability, Guardrails, DeclaredTools,
          McpServers, Owners, CreatedDateTime, EntraAgentID
```

Expected result

- A clean environment can return no rows. Review any returned agent with its owner, and compare with the **Weak Instructions** risk indicator in 8.3.1.

> To find and govern ownerless agents, use 5.5 Govern ownerless agents.

8.4.3 Trace agent activity in CloudAppEvents

Performed by **Security Reader**.

List the activity recorded for the standard test user after they used the agent (replace the object ID):

```
CloudAppEvents
| where Timestamp > ago(1d)
| where AccountObjectId == "<standard test user object ID>"
| summarize Events = count() by Application, ActionType
| sort by Events desc
```

Trace agent invocations, inference calls, and tool executions (action types and `RawEventData` fields as documented in the Agent 365 Defender advanced-hunting query):

```
CloudAppEvents
| where Timestamp > ago(1d)
| where ActionType in ("InvokeAgent", "InferenceCall", "ExecuteToolBySDK", "ExecuteToolByGateway",
"ExecuteToolByMCPServer")
| extend resData = parse_json(tostring(RawEventData))
| extend AgentId = tostring(resData.AgentId),
         TargetAgentId = tostring(resData.TargetAgentId),
         PlatformTargetAgentId = tostring(resData.PlatformTargetAgentId)
| project Timestamp, ActionType, AccountDisplayName, AccountObjectId, IPAddress,
          AgentId, TargetAgentId, PlatformTargetAgentId, ReportId
| sort by Timestamp desc
```

Expected result

- Rows for the test user's activity with the agent are returned. Record the agent identifier from `AgentId`, `TargetAgentId`, or `PlatformTargetAgentId` for the test agent; 8.4.5 uses it.
- No unexpected users or IP addresses appear.

8.4.4 Query alerts and real-time protection events

Performed by **Security Reader**.

Alerts in which the standard test user is an evidence entity:

```
let testUser = "<standard test user UPN>";
AlertEvidence
| where Timestamp > ago(1d)
| where AccountUpn =~ testUser
| distinct AlertId
| join kind=inner (AlertInfo | where Timestamp > ago(1d)) on AlertId
| project Timestamp, AlertId, Title, Category, Severity, ServiceSource, DetectionSource
| sort by Timestamp desc
```

Entities and artifacts associated with one alert:

```
AlertEvidence
| where AlertId == "<AlertId from the previous query>"
| project Timestamp, EntityType, EvidenceRole, AccountUpn, Application, OAuthApplicationId,
          CloudResource, AdditionalFields
```

Recent behaviors, including real-time protection audit and block events:

```
BehaviorInfo
| where Timestamp > ago(1d)
| project Timestamp, BehaviorId, ActionType, Title, Description, Categories,
          ServiceSource, DetectionSource, AccountUpn, AdditionalFields
| sort by Timestamp desc
```

Expected result

- After the audit pass in 8.5.3, the first query returns the alerts from the test.
- After the optional blocking pass, `BehaviorInfo` returns the block events.

8.4.5 Validate a custom detection (optional)

Performed by **standard test user** (activity) and **Security Reader** (review).

1. Confirm the custom detection prerequisite is complete.
2. Run an agreed benign prompt that invokes the selected tool on the PoC agent. Record the time and confirm the tool execution in 8.4.3.
3. Open **Hunting** > **Custom detection rules**, select the prepared rule, and review **Triggered alerts** after its scheduled run.

Expected result

- A triggered alert correlates with the test agent and tool execution. A successful benign tool call isn't itself evidence of a malicious action.

8.5 Test runtime protection end to end

Documentation: Detect and investigate threats to AI agents using Microsoft Defender · Protect AI agents in real time using Microsoft Defender · Enable external threat detection and protection for Copilot Studio custom agents – How it works · Orchestrate agent behavior with generative AI

Use the audit pass to validate detection, incident correlation, registry risk, and containment for the Copilot Studio test agent. Use the optional blocking pass to validate enforcement and behavior telemetry separately; a blocking rule suppresses near-real-time detection alerts for the covered agent.

Microsoft Defender detects threats such as jailbreak attempts, indirect prompt injection (XPIA) attempts, malicious content propagation, secret and credential leakage, evasion techniques, large language model (LLM) reconnaissance, and suspicious user or IP access.

8.5.1 Verify test readiness

Performed by **Security Reader** with the **standard test user**.

1. Review the completed test preparation and baselines.
2. Confirm the agreed test account can open the agent and that the selected audit or blocking mode matches the planned pass.

Expected result

- The test starts from the recorded baseline with approved prompts. Stop and resolve unmet prerequisites before proceeding.

8.5.2 Test scenarios

The PoC team provides the prompts. Use only the test agent, the standard test user, and test data.

#	Scenario	Prompt	Expected result
1	Indirect prompt injection via knowledge document	`<to be added>`	Audit: verify a matching detection and agent activity. Block: verify enforcement only if a tool invocation matches the selected rule
2	Direct prompt injection / jailbreak	`<to be added>`	Verify a matching detection if the attempt reaches Defender; a built-in agent refusal alone doesn't prove Defender enforcement
3	Data exfiltration via tool call	`<to be added>`	Audit: verify matching activity. Block: verify that a tool invocation matching the selected detection type is stopped
4	Benign control prompt that uses the same tool	`<to be added>`	The agent proceeds with no visible effect or interruption

Use the agreed prompts from the prerequisites; placeholders aren't runnable test cases. Map blocking tests to the supported detection types: **Malicious content propagation**, **Secret exfiltration**, **Evasion techniques**, or **Unsafe email domain**. For Copilot Studio, external protection evaluates tool invocations, not every user prompt. A refusal by built-in safeguards without corroborating Defender evidence doesn't satisfy the Defender test.

8.5.3 Run the scenarios as the standard test user

Performed by **standard test user** (and **Security Administrator** to enable or disable the custom rule).

Pass A – audit. Only the **Default** rule applies.

1. Sign in as the standard test user and open the Copilot Studio test agent in Microsoft 365 Copilot or Teams. Start a new conversation for each scenario.
2. Run scenario 4 (benign control) first.
3. Run scenarios 1 to 3, one per conversation. For each, record the time, the scenario number, and the agent's response (screenshot).

Pass B – blocking (optional). The Security Administrator selects the test-agent rule prepared in the prerequisites on the **Real-time protection** page and selects **Enable**.

1. Repeat scenarios 1 to 4 as the standard test user and record the same evidence.
2. After the pass, the Security Administrator selects the rule and selects **Disable**.

Expected result

- Pass A: matching activity is recorded as a behavior without stopping the action, and near-real-time detections surface as alerts.
- Pass B: matching actions are blocked before they execute and recorded as behaviors in `BehaviorInfo`. When the threat detection system blocks a tool invocation, the agent immediately stops processing and notifies the user that their message is blocked. Near-real-time alerts aren't generated for an agent covered by a blocking rule.
- Scenario 4: the agent proceeds with no visible effect or interruption.

8.5.4 Verify the alert and incident in Defender

Performed by **Security Reader** (review) or **Security Operator** (manage alerts).

1. In the Microsoft Defender portal, open the incidents queue and find the incident that contains the alert from Pass A.
2. Use the incident graph and investigation experience to review the relationships between the involved entities and the blast radius.
3. If prompt evidence collection is enabled, open the alert and review the prompt snippets included as evidence.
4. Go to **Assets** > **AI agents** > **Agents**, select the test agent, select **Open Agent page**, then select **Incidents and alerts** and confirm that the incident is listed.
5. On the **Agents** tab, review **Active alerts**, **Risk level**, and **Risk indicators** for the test agent.
6. Run the queries in 8.4.3 and 8.4.4 to correlate the alert with the agent's tool invocations and any block events.
7. As Security Operator, set the alert status and classification.

Expected result

- Defender correlates the AI agent alert into an incident.
- The incident is listed on the test agent's **Incidents and alerts** tab.
- The hunting queries return the alert and the related agent activity.

8.5.5 Verify the risk signal in the Agent 365 registry

Performed by **AI Reader**, **Global Reader**, or **Security Reader**.

1. Open **Agents** > **All agents** > **Registry** in the Microsoft 365 admin center. Risk signal counts might be up to an hour behind the security portals.
2. Select the **Agents at risk** tile and confirm that the test agent is listed.
3. Select its **Risks** count and confirm that **Microsoft Defender** appears under **Risk signals are sourced from**.
4. Select **Microsoft Defender** to follow the deep link into the Defender portal. The deep link requires Global Reader, Security Reader, Security Administrator, AI Administrator, or Global Administrator; AI Reader can't follow it.

Expected result

- IT sees what is flagged for the test agent from the Microsoft 365 admin center, without switching between portals or elevating to security roles.

8.5.6 Contain the agent and verify

Performed by **AI Administrator** (block), **standard test user** (verify), and **Security Operator** (alert status).

1. Block the test agent as described in 8.2.3. Record the time.
2. As the standard test user, try to use the test agent in Microsoft 365 Copilot and in Teams.
3. As Security Operator, set the alert status and classification, and record the containment time.

Expected result

- Blocking restricts access to the agent across the organization, preventing any user from using it; the test user can't use the agent in Microsoft Copilot or Teams.
- The time from the first alert to the block is the PoC containment metric.

> For identity-level containment of an agent with a Microsoft Entra agent identity, see 3.11.1 Disable and re-enable an individual agent identity.

8.5.7 Run the Foundry test agent scenarios (optional)

Documentation: Detect and investigate threats to AI agents using Microsoft Defender · Protect AI agents in real time using Microsoft Defender

Performed by **standard test user** (scenarios), **Security Reader** (review), **Security Administrator** (optional blocking pass), and **Security Operator** (alert status).

Confirm the Foundry prerequisites before testing. These are Agent 365 agent-security tests in the Microsoft Defender portal, not Defender for Cloud model-security tests.

Use the agreed prompts, the published Foundry test agent, and synthetic test data only.

#	Scenario	Prompt	Expected result
F1	Direct prompt injection / jailbreak	`<to be added>`	Audit: inspect the matching agent detection and related activity; distinguish a model or Prompt Shields refusal from Defender rule enforcement
F2	Secret leakage using approved synthetic test material	`<to be added>`	Audit: inspect the matching detection; optional block pass: verify an interaction matching **Secret exfiltration** is stopped and recorded as a behavior
F3	Indirect prompt injection via grounding data	`<to be added>`	Audit: inspect the matching agent detection; optional block pass: verify a match to a selected supported detection type
F4	Benign control prompt	`<to be added>`	The agent responds normally

1. With only the **Default** audit rule applying to the test agent, run F4 first, then F1 to F3 in separate conversations in its published channel. Record the time, scenario, agent identity, and response.
2. In the Microsoft Defender portal, find any matching agent alert and open its incident. On **Assets** > **AI agents** > **Agents**, open the Foundry agent page and review **Incidents and alerts**.
3. Use the queries in 8.4.3 and 8.4.4 to correlate agent activity, alerts, and behaviors by agent identity and test time. Record an absent detection as not demonstrated; don't substitute an unrelated model alert.
4. (Optional) The Security Administrator enables the prepared rule scoped to the Foundry test agent. Repeat F4 and the scenarios mapped to its detection types, then disable the rule. Record the blocked interaction and the corresponding `BehaviorInfo` event. Foundry protection can evaluate user requests, agent responses, tool invocations, and tool responses.
5. As Security Operator, set the status and classification of the audit-pass alert.

Expected result

- Audit: a matching agent detection is visible in Microsoft Defender, correlated into an incident, and associated with the published Foundry agent.
- Optional block: a matching interaction is stopped and recorded in `BehaviorInfo`; don't require a near-real-time alert for an agent covered by a blocking rule.
- F4 succeeds. If prompt evidence collection is enabled, inspect the suspicious snippets in the alert; otherwise, use the remaining alert and activity evidence.

8.6 Test local runtime protection (optional)

Documentation: AI agent runtime protection demonstration

Confirm local runtime preparation before this test. Performed on the test device; alert review by **Security Reader**.

1. Close any terminal windows used to run agents, and open a new terminal.
2. Follow AI agent runtime protection demonstration and submit the benign test prompt from that article in a new session of a supported local agent.
3. In the Microsoft Defender portal, review the alert.

Expected result

- Defender raises a **Suspicious AI prompt injection** alert. The alert appears on the device timeline, and related alerts are correlated into incidents.
- In audit mode, the alert is **Informational**. In block mode, the alert severity is **Critical**, **High**, **Medium**, or **Low** based on assessed risk.
- In block mode, the agent terminal displays a block message and, if Windows Security notifications are enabled, a Windows toast notification appears. The detection is also listed under **Windows Security** > **Virus & threat protection** > **Current threats** and in the **Protection history**.

8.7 Evidence

- Screenshot of **Settings** > **Security for AI** > **Policies & rules** > **Real-time protection** with the **Default** rule and (optional) the custom rule.
- Screenshot of **Assets** > **AI agents** > **Agents** with the PoC agents, and the test agent page before and after the test (risk level, risk indicators, **Active alerts**).
- Exported results of the queries in 8.4.1, 8.4.3, and 8.4.4.
- Per scenario in 8.5.2: time, scenario number, pass (A or B), and the agent's response.
- The incident page for the test incident and the **Incidents and alerts** tab of the test agent.

- Screenshot of the registry **Agents at risk** view and the **Risk details** pane for the test agent.
- Screenshot of the **Block agent** pane and the test user's attempt to use the blocked agent.
- (Optional) Custom detection rule details page with **Triggered alerts**.
- (Optional) **Suspicious AI prompt injection** alert from 8.6.3.
- Timeline: first alert time, block time.
- (Optional, Foundry in scope) Per scenario in 8.5.7: time, scenario number, agent response, the Agent 365 agent alert and incident in Defender, and any optional block event in `BehaviorInfo`.

8.8 Troubleshooting

Symptom	Likely cause	Fix
Copilot Studio activity is blocked but no alerts or incidents appear	Microsoft 365 connector not connected	Connect the connector (Microsoft 365 connector).
No Defender decision for the test agent	The agent uses classic orchestration, or the environment isn't configured	Use generative orchestration and configure threat detection for the agent's environment.
Tool runs although the threat detection system didn't answer	No decision within one second; default error behavior is **Allow the agent to respond**	Review **Set error behavior** (Power Platform connection).
Blocking pass produces no alert	Near-real-time alerts aren't generated for an agent covered by a blocking rule	Query `BehaviorInfo` (8.4.4) for the block events.
`CloudAppEvents` returns no agent rows	The **Microsoft 365 activities** component isn't selected, or the agent hasn't been used	Check Microsoft 365 connector, use the agent as the test user, and rerun 8.4.3.
Queries fail on `AIAgentsInfo`, `AgentName`, or `EntraAgentId`	Table and column names	Use `AgentsInfo`, `Name`, `EntraAgentID` and `EntraBlueprintID`. Run `AgentsInfo | getschema` to list the columns in your tenant.
Risks count in the registry is still zero after an alert	Risk signal counts might be up to an hour behind the security portals	Wait and refresh.
You can't block an agent	Signed in with a role that can view but not manage agents (Global Reader, AI Reader, Security Administrator, Security Reader)	Use AI Administrator.
Custom detection can't be created	Read-only role, or Security Operator without **Manage Security Settings** when Defender for Endpoint RBAC is on	Use Security Administrator, or assign the permission.
No **Suspicious AI prompt injection** alert in 8.6	Device prerequisites not met, Beta Channel not configured, or the terminal wasn't reopened after enabling protection	Fix the prerequisite, close agent and terminal sessions, open a new terminal, and rerun the demonstration.
Alert has no prompt snippets	Prompt evidence collection may be disabled	Check the agreed prompt evidence setting; don't change evidence collection without approval.
No threat detection for the Foundry test agent	The agent isn't published, or it's used only in a playground environment	Publish the Foundry agent and test it through its published channel.

✅ What you proved

- [ ] The Copilot Studio test agent appears in **Assets** > **AI agents** > **Agents**, and its agent page opens with **Open Agent page**.

- [ ] The `AgentsInfo` inventory query in Advanced Hunting returns the PoC agents.
- [ ] After the standard test user runs the audit pass in 8.5.3, an alert for the test agent is listed on the agent's **Incidents and alerts** tab.
- [ ] (Optional) In the blocking pass, the risky tool invocation doesn't run, the agent tells the test user that the message is blocked, and the block event is recorded in `BehaviorInfo`.
- [ ] The test agent appears under **Agents at risk** in the Agent 365 registry, and its **Risk details** pane lists **Microsoft Defender** as a source.
- [ ] An AI Administrator blocks the test agent from the registry, and the test user can no longer use it.
- [ ] (Optional) A custom detection rule built on agent activity runs and lists a triggered alert.
- [ ] (Optional, local agents in scope) The Microsoft demonstration prompt in 8.6 raises a **Suspicious AI prompt injection** alert in the Defender portal.
- [ ] (Optional, Foundry in scope) The published Foundry agent's audit test produces a matching agent alert and incident in Microsoft Defender; any selected blocking test is verified separately in `BehaviorInfo`.

References

- Agent Registry in Microsoft 365 admin center – Agent risks
- Governance and lifecycle actions for agents – Block or unblock agents
- Agent management roles and permissions
- Discover AI agents and assess security posture using Microsoft Defender
- AI agent posture risk in Microsoft Defender
- Detect and investigate threats to AI agents using Microsoft Defender
- Advanced hunting overview
- AgentsInfo table
- CloudAppEvents table
- BehaviorInfo table
- AlertInfo table
- AlertEvidence table
- Troubleshoot Direct OTel Observability – Defender advanced-hunting query
- Create custom detection rules
- Manage existing custom detection rules
- Protect AI agents in real time using Microsoft Defender
- Enable external threat detection and protection for Copilot Studio custom agents – How it works
- Orchestrate agent behavior with generative AI
- AI agent runtime protection demonstration

Runbook index · Previous: Chapter 7 – Sensitive data protection (Purview) · Next: Chapter 9 – Shadow AI and local agents

Prerequisites - Chapter 9: Shadow AI and local agents

> **Goal:** Prepare the accounts, access, and test assets needed for this chapter and its selected optional paths.

Apply the shared PoC preparation guidance and agree the chapter outcomes. Finish dependent build steps before their tests.

Table of contents

- Tenant readiness
- Prepare accounts and role assignments
- Test assets and dependencies
- Plan propagation and test timing
- References

Tenant readiness

Documentation: Get started with Microsoft Agent 365 · Agent management in the Microsoft 365 admin center · Frontier program

Use the task-specific roles in 9.1 Required permissions. Confirm availability rather than assuming the tenant is ready.

Follow the shared tenant readiness check using the chapter permissions and feature prerequisites linked above.

For Frontier opt-in and device readiness, follow 9.2 Tenant and device prerequisites. Verify each **Check result** before detection or blocking tests.

Prepare accounts and role assignments

Alongside the shared setup and reviewer accounts, prepare the following scenario accounts:

Account	Purpose
Standard test user	Signs in to the managed test device and runs approved test tools. No admin role.

Assign the roles in 9.1 Required permissions using the shared model. Verify each account can access the pages needed for its tasks.

Test assets and dependencies

Documentation: Understand Shadow AI in Microsoft 365 admin center · Understand Local Agents in Microsoft 365 admin center · Preview Agent 365 features through the Frontier program · Discover local AI agents with Microsoft Defender for Endpoint · Device action: Sync · Tutorial: Enable Internet Access traffic forwarding · onboarded to Microsoft Defender for Endpoint

- Chapter 1 – Agent discovery and inventory covers the overall agent inventory in the Microsoft 365 admin center.
- Chapter 8 – Threat detection and runtime protection (Defender) covers the optional Defender for Endpoint runtime protection for local agents: complete its preparation, then run 8.6 Test local runtime protection.
- Prepare at least one dedicated Windows test device that's enrolled in Microsoft Intune and onboarded to Microsoft Defender for Endpoint, and one standard test user who signs in to it.
- Get written approval from the customer's security team for the exact list of AI tools that you may install on the test device (see 9.5.1 Agree the test scope and prepare the test device). Don't install any tool that isn't on the approved list.
- Agree with the customer, before you test blocking, that a block created from the Microsoft 365 admin center creates a Microsoft Intune policy that automatically propagates to all managed Windows devices enrolled in Intune (see 9.3.4, 9.4.4 and 9.4.6).

Build guidance: 9.2 Tenant and device prerequisites

Check result

- Each asset needed for the selected paths is prepared or has a named build task that will finish before its dependent tests.

Plan propagation and test timing

Documentation: Understand Shadow AI in the Microsoft 365 admin center

Configuration	Documented propagation window
Intune block policy from the Shadow AI or Local Agents page to a device	15 minutes up to 8 hours

> ⚠️ **Watch out:** Allow for the documented propagation window and verify the actual configuration or signal before testing. Elapsed time alone is not proof that a change has applied.

References

- Get started with Microsoft Agent 365
- Agent management in the Microsoft 365 admin center
- Frontier program
- Understand Shadow AI in Microsoft 365 admin center
- Understand Local Agents in Microsoft 365 admin center
- Discover local AI agents with Microsoft Defender for Endpoint
- Device action: Sync

- Tutorial: Enable Internet Access traffic forwarding
- onboarded to Microsoft Defender for Endpoint

> **Ready?** Continue to the chapter. →

Chapter 9 - Shadow AI and local agents

Pillar: Secure **What it proves:** Unapproved AI apps and developer agent tools that run on managed Windows devices are visible in the Microsoft 365 admin center (which agent, on which devices, how many users, and how much network traffic), and administrators can block supported agents centrally through a Microsoft Intune policy.

Success criteria: Agree the selected outcomes in What you proved before testing.

Topics and time guide

Approximate hands-on estimates for the selected scope, with prerequisites prepared. Service waiting time is separate.

Topic	Estimated hands-on time
9.1 Required permissions	~5 min
9.2 Tenant and device prerequisites	~10-20 min
9.3 Shadow AI	~10-15 min
9.4 Local agents	~10-15 min
9.5 Test and validation	~15-20 min
9.6 Evidence	~5-10 min
9.7 Troubleshooting	As needed

Outcome checklist · References

Optional Global Secure Access setup in 9.2 may need additional time.

9.1 Required permissions

Use the shared account and role-assignment model with the task-specific permissions below.

Task	Least-privilege role	Section
Enable Agent 365 Frontier for the tenant	Global Administrator	9.2.1
Confirm the Microsoft Defender for Endpoint discovery prerequisites on the test device	The customer's Defender for Endpoint administrator	9.2.2
Review Intune-enrolled Windows test devices	Intune Administrator	9.2.3
Enable Internet Access traffic forwarding and assign test users (optional)	Global Secure Access Administrator	9.2.4
View the **Shadow AI (Frontier)** and **Local Agents (Frontier)** pages, and block agents from these pages	Any one of: Security Administrator, AI Administrator, Global Reader, Security Reader, Security Operator, Reports Reader, User Experience Success Manager, Intune Administrator	9.3, 9.4, 9.5.4

Task	Least-privilege role	Section
Review the block policy in Intune	Intune Administrator	9.3.5, 9.5.5
Import the VS Code ADMX template and create the VS Code extension policy profile	Policy and Profile Manager (Intune built-in role)	9.4.5, 9.5.5
Sync the test device from Intune	Help Desk Operator or Endpoint Security Manager (Intune built-in roles)	9.5.6
Install and use the approved test tools on the test device	Standard test user (no admin role)	9.5.2, 9.5.6
Validation / read-only review	Security Reader or Reports Reader	9.5.3

Before you start:

- [ ] Confirm the chapter prerequisites for the selected topics.

9.2 Tenant and device prerequisites

Documentation: Understand Shadow AI in Microsoft 365 admin center · Understand Local Agents in Microsoft 365 admin center · Preview Agent 365 features through the Frontier program · Discover local AI agents with Microsoft Defender for Endpoint · Device action: Sync · Tutorial: Enable Internet Access traffic forwarding

Shadow AI and local agents at a glance

	Shadow AI	Local agents
What it is	Consumer-facing AI applications and standalone agents deployed across the organization without IT visibility or approval, which can operate autonomously on user devices	Developer-controlled tools and extensions (IDE plugins, CLIs, code editors) intentionally integrated into development workflows with explicit configuration
Microsoft 365 admin center page	**Agents** > **Shadow AI**	**All Agents** > **Local Agents (Frontier)**
Blocking mechanism	Microsoft Intune policy created from the Microsoft 365 admin center	Microsoft Intune policy created from the Microsoft 365 admin center; VS Code extension policy for agents that run as Visual Studio Code extensions
Device scope	Managed Windows devices enrolled in Microsoft Intune	Managed Windows devices enrolled in Microsoft Intune

Local agents are intentionally chosen developer tools managed within known environments, while Shadow AI consists of unmanaged, autonomous applications deployed without organizational awareness. Both experiences are part of the Frontier preview program and are in public preview. Features, supported agents and behaviors might change before general availability.

This section covers configuration only. Both pages require the Frontier preview opt-in and Microsoft Intune enrollment for managed Windows devices. Shadow AI also requires Microsoft Defender for Endpoint on devices for shadow agent detection. Global Secure Access is optional and adds metadata about how detected agents are being used.

9.2.1 Opt in to the Frontier preview

Skip this task if Frontier opt-in has already been verified for the PoC tenant.

Performed by **Global Administrator**. Only a Global Administrator can enable Agent 365 Frontier.

1. Sign in to the Microsoft 365 admin center at `https://admin.cloud.microsoft`.
2. Go to **Agents** > **Overview**.
3. In the banner, select **Try now**, and then select **I agree** to accept the Agent 365 Terms of Service.

Check result

- The Agent 365 Terms of Service are accepted. This is how you check that the tenant is enrolled in Agent 365 Frontier.

9.2.2 Confirm Microsoft Defender for Endpoint on the test devices

Performed by the administrator who manages Microsoft Defender for Endpoint onboarding in the customer's organization.

Shadow AI requires Microsoft Defender for Endpoint on devices for shadow agent detection. The Defender local AI agent discovery prerequisites are:

- The environment is in the commercial cloud. Sovereign and national clouds aren't supported.
- The devices are onboarded to Microsoft Defender for Endpoint.
- The devices run a supported version of Windows, and Microsoft Defender Antivirus is updated with current monthly platform and engine updates.
- Microsoft Defender Antivirus runs in active mode on the devices, with real-time protection enabled.

No other deployment, configuration or scripts are needed beyond device onboarding. When a device meets all the prerequisites, agent discovery starts automatically.

Check result

- The test device meets every prerequisite in the list.

9.2.3 Confirm Intune enrollment of the Windows test devices

Performed by **Intune Administrator**.

Detection and blocking on both pages apply only to managed Windows devices enrolled in Microsoft Intune.

1. Sign in to the Microsoft Intune admin center at `https://intune.microsoft.com`.
2. Go to **Devices** > **All devices**.
3. Select the test device from the devices list.

Check result

- The Windows test device is listed in Intune.

9.2.4 Enable Global Secure Access for traffic metadata (optional)

Performed by **Global Secure Access Administrator** (steps 1–6). Steps 7–9 are performed on the test device.

Enable Global Secure Access for the tenant and the enrolled devices to view more metadata about how detected agents are being used. The fields that need Global Secure Access are listed in 9.3.2 and 9.4.2.

1. Sign in to the Microsoft Entra admin center at `https://entra.microsoft.com`.
2. Go to **Global Secure Access** > **Connect** > **Traffic forwarding**.
3. Select the **Internet access profile** checkbox to enable it.
4. In the **Internet access profile** section, under **User and group assignments**, select **View**.
5. Under **Assigned**, select **0 users, 0 groups assigned**, and then select **Add user/group**.
6. Search for and select the PoC test user (or a PoC test group), and then select **Assign**.
7. On the test device, download the Global Secure Access client for Windows 11 from `https://aka.ms/GlobalSecureAccess-Windows` (or `https://aka.ms/GlobalSecureAccess-WindowsOnArm` for Arm-based devices), and complete the installation wizard.
8. Confirm that the Global Secure Access client icon appears in the Windows system tray.
9. Right-click the icon, select **Advanced Diagnostics** > **Forwarding profile**, and confirm that the Internet Access rules are present.

Check result

- The Internet Access traffic forwarding profile is enabled and assigned to the PoC test users or group.
- The client on the test device shows the Internet Access rules. The client checks for forwarding profile changes every five minutes. If the rules aren't shown yet, wait five minutes and then select **Refresh**.

9.3 Shadow AI

Documentation: Understand Shadow AI in Microsoft 365 admin center · Assign device profiles in Microsoft Intune · Understand Local Agents in Microsoft 365 admin center · Settings list for the Local AI Agent Baseline - OpenClaw security baseline in Intune

Shadow AI refers to consumer-facing AI applications and standalone agents deployed across the organization without IT visibility or approval. Unmanaged usage introduces risks such as data leakage, compliance violations, security vulnerabilities, and lack of auditability and governance. The **Shadow AI (Frontier)** page is a dedicated experience, separate from the **All agents** page, that focuses on unmanaged AI agents that require detection and governance.

9.3.1 Open the Shadow AI (Frontier) page

Performed by any viewing role from 9.1 (for example **Security Reader**).

1. Sign in to the Microsoft 365 admin center at `https://admin.cloud.microsoft`.
2. Select **Agents** > **Shadow AI**.
3. Review the list of known Shadow AI agents that can be detected in your environment.

During public preview, the Shadow AI experience provides the following capabilities:

Agent	Detection	Blocking
OpenClaw	Available	Available
ChatGPT Desktop	Available	Not available
Ollama Desktop	Available	Not available
Poe Desktop	Available	Not available
Claw/ZeroClaw	Available	Not available
OpenCode	Available	Not available

Agent	Detection	Blocking
Claude Desktop	Available	Not available

Check result

- The **Shadow AI (Frontier)** page opens and lists the known Shadow AI agents.

9.3.2 Review Shadow AI agent details

Performed by any viewing role from 9.1 (for example **Security Reader**).

1. On the **Shadow AI (Frontier)** page, select the Shadow AI agent from the list. The details pane opens.
2. Confirm that **Details** is selected, and review the agent information:
   - **Details** tile: **First accessed** (date the agent was first accessed), **Most recent activity** (date the agent was last used), **Last scanned** (date the agent was last detected).
   - **Detections** tile: **Devices** (count of devices that this agent is detected running on), **Users** (unique count of users that are detected using this agent).
   - **Security policies** tile: displays whether a blocking Intune policy has been applied to prevent this agent from running.
   - **Total traffic** tile: **Unique endpoints** (number of unique FQDNs the agent accessed), **Requests** (number of network requests made by the agent), **Data sent**, **Data received**.

The following fields and tiles are populated only if Global Secure Access is enabled (9.2.4):

- **Total traffic** tile
- **Users** field
- **Most recent activity** field
- **Last scanned** field

Check result

- The details pane opens for the selected agent, and **Devices** shows the count of devices that the agent is detected running on.

9.3.3 Review detected devices for a Shadow AI agent

Performed by any viewing role from 9.1 (for example **Security Reader**).

1. In the Shadow AI agent details pane, select the **Detected devices** tab.
2. Search for the test device name.
3. Review the device data: **Device name**, **Model** (Desktop, Virtual Machine, Server, Laptop and so on), **Operating system**, and **Last Seen** (the last time Microsoft Defender detected the agent on the device).

Check result

- The test device is listed with a **Last Seen** value.

9.3.4 Block a Shadow AI agent

Performed by any role from 9.1 that can view the page (for example **Intune Administrator**).

After the Shadow AI agent is identified in your environment, you can block it to prevent execution on managed devices. When you block a Shadow AI agent, such as OpenClaw, it blocks common ways of running it by creating a new Microsoft Intune policy that automatically propagates to **all managed Windows devices enrolled in Intune**.

> Caution: depending on how Intune is configured in your organization, the Intune policy update can take anywhere from 15 minutes up to 8 hours to apply. Show the policy being applied in a live demo, and check the device in a later session.

> Caution: OpenClaw is a Node.js-based agent. Node.js-based agents share a single run type, so blocking one blocks them all (see 9.4.6).

1. Open the details pane of a Shadow AI agent for which blocking is available (see 9.3.1).
2. Select **Security policies**.
3. Under **Security policies**, select **Block** > **Apply Policies**.
4. Record the date and time of the block.

Check result

- The **Security policies** tile displays that a blocking Intune policy has been applied to prevent this agent from running.

9.3.5 Review the block policy in Intune

Performed by **Intune Administrator**.

Full policy details, including when the Intune policy applies, can be found in Intune. Policies can also be edited in Intune to add more controls. For the Node.js block, see Settings list for the Local AI Agent Baseline - OpenClaw security baseline in Intune. That baseline limits the use of unauthorized local AI agents such as OpenClaw by configuring device settings that disrupt commonly used execution paths, including firewall rules that restrict outbound network communication from Node.js. Its settings might not fully block all agent execution paths, and it might also block other processes in addition to OpenClaw.

1. Sign in to the Microsoft Intune admin center at `https://intune.microsoft.com`.
2. Search for the policy name **A365 - Block OpenClaw**.
3. Review the policy details.

Check result

- The **A365 - Block OpenClaw** policy is found in Intune, and its details are recorded.

9.4 Local agents

Documentation: Understand Local Agents in Microsoft 365 admin center · Assign device profiles in Microsoft Intune · Settings list for the Local AI Agent Baseline - OpenClaw security baseline in Intune · Import custom and third-party partner ADMX templates in Microsoft Intune · Manage extensions in enterprise environments (Visual Studio Code) · Centrally manage VS Code settings with policies

Local agents are developer-controlled tools and extensions, such as IDE plugins, CLIs and code editors, that are intentionally integrated into development workflows with explicit configuration. They include extensions for VS Code and similar developer-focused applications. The **Local Agents (Frontier)** page is a dedicated experience, separate from the **All agents** page.

9.4.1 Open the Local Agents (Frontier) page

Performed by any viewing role from 9.1 (for example **Security Reader**).

1. Sign in to the Microsoft 365 admin center at `https://admin.cloud.microsoft`.
2. Select **All Agents** > **Local Agents (Frontier)**.
3. Review the list of known local agents that you can detect in your environment.

During public preview, the Local Agents experience can detect the following local agents:

Name	Detection	Blocking
Antigravity CLI	Available	Not Available
Antigravity Desktop	Available	Not Available
Antigravity IDE	Available	Not Available
Claude Code	Available	Not Available
Codex CLI	Available	Not Available
Codex Desktop	Available	Not Available
Cursor	Available	Not Available
Devin Desktop	Available	Not Available
GitHub Copilot App	Available	Not Available
GitHub Copilot CLI	Available	Not Available
Junie CLI	Available	Not Available
Kiro CLI	Available	Not Available
Kiro IDE	Available	Not Available
Microsoft Scout	Available	Not Available
Warp	Available	Not Available
Windsurf	Available	Not Available
Gemini CLI	Available	Available
VSCode Claude Code Extension	Available	Available
VSCode Cline Extension	Available	Available
VSCode Codex Extension	Available	Available
VSCode Gemini Code Assist Extension	Available	Available
VSCode GitHub Copilot Extension	Available	Available
VSCode Roo Code Extension	Available	Available

Check result

- The **Local Agents (Frontier)** page opens and lists the known local agents.

9.4.2 Review local agent details

Performed by any viewing role from 9.1 (for example **Security Reader**).

1. Select the agent from the **Local Agents** list on the **Local Agents (Frontier)** page. The **Details** pane opens.
2. Select the **Details** tab and review the agent information:
   - **Details** tile: **First accessed**, **Most recent activity**, **Last scanned**.
   - **Detections** tile: **Devices** (count of devices that this agent is detected running on), **Users** (unique count of users that are detected using this agent).
   - **Total traffic** tile: **Unique endpoints** (number of unique FQDNs the agent accessed), **Requests**, **Data sent**, **Data received**.

The following fields are populated only if Global Secure Access is enabled (9.2.4): **Users**, **Most recent activity**, **Last scanned**, **Unique endpoints**, **Requests**, **Data sent** and **Data received**.

Check result

- The **Details** pane opens for the selected agent, and **Devices** shows the count of devices that the agent is detected running on.

9.4.3 Review detected devices for a local agent

Performed by any viewing role from 9.1 (for example **Security Reader**).

1. Select the agent from the **Local Agents** list on the **Local Agents (Frontier)** page.
2. Select the **Detected devices** tab.
3. View the devices in the device table, or search for the test device name.

Device information	Description
Device name	Name of the device
Model	Type of device, such as Desktop, Virtual Machine, Server or Laptop
Operating system	Operating system installed on the device
Last Seen	The last time Microsoft Defender detected the agent on the device

Check result

- The test device is listed with a **Last Seen** value.

9.4.4 Block a local agent with an Intune policy

Performed by any role from 9.1 that can view the page (for example **Intune Administrator**).

After you identify the local AI agent in your environment, you can block it to prevent execution on managed devices. When you block a local AI agent, it blocks common ways of running it by creating a new Microsoft Intune policy that automatically propagates to **all managed Windows devices enrolled in Intune**. Depending on how Intune is configured in your organization, the policy update can take anywhere from 15 minutes up to 8 hours to apply. To view the policy details, search Intune for the policy name **A365 - Block OpenClaw** (9.3.5).

> Caution: Gemini CLI is a Node.js-based agent. Blocking it blocks all Node.js-based agents (see 9.4.6).

1. Open the details pane of a local agent for which blocking is available (see 9.4.1).
2. Select **Security policies**.
3. Under **Security policies**, select **Block agent** > **Apply Policies**.
4. Record the date and time of the block.

Check result

- **Block agent** > **Apply Policies** completes for the selected agent, and the Intune policy is found in Intune (9.3.5).

9.4.5 Block VS Code extension agents with a VS Code extension policy

Performed by **Policy and Profile Manager**.

Agents that run as Visual Studio Code extensions (the **VSCode … Extension** entries in 9.4.1) are blocked by using a VS Code extension policy. The policy is deployed to managed devices and enforced by Visual Studio Code. You can view blocked VS Code extension agents in Visual Studio Code under the allowed extension list.

The VS Code extension policy is the `AllowedExtensions` policy, which manages the `extensions.allowed` setting (supported from VS Code version 1.96). Use the full extension ID (`<publisher>.<extension>`) to allow or block a specific extension. The more specific selector takes precedence, and `"*"` is the only supported wildcard. If you block an extension that's already installed, the extension is disabled. For cloud-managed devices, VS Code documents Microsoft Intune as an MDM solution for configuring the imported VS Code administrative template policies.

1. Get the `vscode.admx` file and its ADML file from the `policies` folder of an existing VS Code installation (version 1.69 or later) or of the extracted VS Code zip archive.
2. Sign in to the Microsoft Intune admin center at `https://intune.microsoft.com`.
3. Select **Devices** > **Manage devices** > **Configuration** > **Import ADMX** tab > **Import**.
4. Upload the ADMX file and the ADML file for the default language, select **Next**, and in **Review + Create** select **Create**. Select **Refresh** to see the upload **Status**.
5. Select **Devices** > **Manage devices** > **Configuration** > **Create** > **New policy**.
6. For **Platform**, select **Windows 10 and later**. For **Profile type**, select **Templates** > **Imported Administrative templates (Preview)**, and then select **Create**.
7. In **Basics**, enter a name such as `PoC - VS Code - Block <extension>`, and then select **Next**.
8. In **Configuration settings**, configure the `AllowedExtensions` policy with a value that blocks the test extension while allowing others, and then select **Next**:

   ```
   {
     "*": true,
     "<publisher>.<extension>": false
   }
   ```

9. In **Assignments**, select the PoC test group that contains the test device or test user, and then select **Next**.
10. In **Review + create**, select **Create**.

If there's a syntax error in the policy value, the `extensions.allowed` setting isn't applied. To check for errors, open the Command Palette in VS Code and run **Show Window Log**.

Check result

- The VS Code ADMX template is imported, and the policy profile is created, shown in the profiles list and assigned to the PoC test group.
- After the test device receives the policy (9.5.6), the blocked extension is disabled in VS Code.

9.4.6 Understand the Node.js shared run type

Node.js-based agents share a single run type, so blocking one blocks them all. When you block any of the following agents, you block all of them:

- OpenClaw
- QClaw
- Claw/Nanobot
- Gemini CLI

> Caution: blocking a Node.js-based agent blocks all agents and processes of that run type on managed devices. Make sure that no legitimate business processes depend on these agents before you apply the policy. Removing a block on any Node.js-based agent removes the Node.js block for all other Node.js-based agents, not just the one you selected.

For the settings of this Intune policy, see Settings list for the Local AI Agent Baseline - OpenClaw security baseline in Intune.

9.5 Test and validation

Run these tests on a dedicated test device. Plan the block check (9.5.6) for a block created from the Microsoft 365 admin center at least 15 minutes and up to 8 hours after the block, because of the documented Intune policy update window.

9.5.1 Agree the test scope and prepare the test device

Performed by the **PoC lead** together with the customer's security team.

1. Choose the test tools from the lists in 9.3.1 and 9.4.1, and get the customer's written approval for each one. A minimal set:
   - **Detection only:** one Shadow AI desktop app (for example, Claude Desktop or ChatGPT Desktop) and one local developer tool (for example, GitHub Copilot CLI), if the customer approves them.
   - **Detection and blocking:** one VS Code extension agent that the customer approves (for example, the VSCode GitHub Copilot Extension). Test Gemini CLI or OpenClaw only if the customer explicitly accepts the Node.js block described in 9.4.6 for all managed Windows devices enrolled in Intune.
2. Confirm that the test device meets 9.2.2 and 9.2.3 (and 9.2.4 if Global Secure Access metadata is in scope).
3. Record the test device name, test user, approved tools and versions, and the agreed block window.

Expected result

- A signed-off list of approved test tools and a test device that meets all prerequisites.

9.5.2 Install the approved test tools and generate activity

Performed by the **standard test user** on the test device.

1. Install each approved tool from its official source.
2. Use each tool for a short task that contains no company data.
3. Record the time of the activity.

Expected result

- Each approved tool is installed and has been used on the test device.

9.5.3 Confirm detection in the Microsoft 365 admin center

Performed by **Security Reader** or **Reports Reader**.

1. For each Shadow AI test tool, complete 9.3.1 to 9.3.3.
2. For each local agent test tool, complete 9.4.1 to 9.4.3.

Expected result

- For each approved test tool, **Devices** shows the count of devices that the agent is detected running on, and the test device is listed on **Detected devices** with a **Last Seen** value.
- If Global Secure Access is enabled, **Users**, **Most recent activity**, **Last scanned** and the **Total traffic** values are populated.

9.5.4 Apply the block

Performed by any viewing role from 9.1 (for example **Intune Administrator**) for 9.3.4 and 9.4.4, and by **Policy and Profile Manager** for 9.4.5.

1. Confirm that the customer accepts the scope of the block: all managed Windows devices enrolled in Intune, and the Node.js shared run type if a Node.js-based agent is in scope.
2. For a Shadow AI test tool for which blocking is available, complete 9.3.4. For a local agent test tool for which blocking is available, complete 9.4.4. For a VS Code extension agent, complete 9.4.5.
3. Record the date and time of the block.

Expected result

- For a Shadow AI agent, the **Security policies** tile displays that a blocking Intune policy has been applied.
- For a local agent, **Block agent** > **Apply Policies** completes.
- For a VS Code extension agent, the VS Code policy profile is created and assigned to the PoC test group.

9.5.5 Confirm the policy in Intune

Performed by **Intune Administrator** (block policy) or **Policy and Profile Manager** (VS Code policy profile).

1. For a block applied from the Microsoft 365 admin center, complete 9.3.5.
2. For a VS Code extension policy, go to **Devices** > **Manage devices** > **Configuration** and confirm that the profile from 9.4.5 is in the profiles list.

Expected result

- The block policy is found in Intune, and the VS Code policy profile (if used) is in the profiles list.

9.5.6 Confirm the block on the test device

Performed by **Help Desk Operator** or **Endpoint Security Manager** (device sync) and the **standard test user** (device test).

1. In the Microsoft Intune admin center, select **Devices** > **All devices**, and select the test device.
2. At the top of the device overview pane, select **Sync**, and then select **Yes** to confirm. To track progress, select the **Device sync status** tab (this tab requires the **Preview new device view** toggle at the top right of the Intune admin center to be on).
3. On the test device, try to run the blocked tool again in the same way as in 9.5.2.
4. For a VS Code extension block, open Visual Studio Code and review the allowed extension list.

Expected result

- For a block created from the Microsoft 365 admin center, after the Intune policy applies (15 minutes up to 8 hours, depending on Intune configuration), the common ways of running the blocked agent are blocked on the test device.
- For a VS Code extension block, the blocked extension is disabled and the blocked VS Code extension agent is shown under the allowed extension list.

9.6 Evidence

- Screenshot of the **Shadow AI (Frontier)** page and the **Local Agents (Frontier)** page with the list of known agents.
- Screenshot of the details pane (**Details**, **Detections**, **Security policies**, **Total traffic**) for each detected test tool.
- Screenshot of the **Detected devices** tab with the test device and **Last Seen**.
- Screenshot of the **Security policies** tile after **Block** > **Apply Policies** (Shadow AI), or of the completed **Block agent** > **Apply Policies** action (local agents), with the date and time of the block.
- Screenshot of the block policy details in Intune, and of the VS Code policy profile if used.
- Screenshot of the Intune **Device sync status** tab for the test device.
- Screenshot from the test device that shows the blocked tool or the disabled VS Code extension.
- The signed-off list of approved test tools (9.5.1).

9.7 Troubleshooting

Symptom	Likely cause	Fix
The **Shadow AI** or **Local Agents (Frontier)** page isn't available	The tenant hasn't opted in to the Frontier preview, or the account doesn't hold one of the required roles	Complete 9.2.1. Assign one of the roles listed in 9.1.
The test tool is installed but not detected on the Shadow AI page	Microsoft Defender for Endpoint discovery prerequisites aren't met, or the device isn't enrolled in Intune	Recheck 9.2.2 and 9.2.3.
The test tool isn't listed on either page	The tool isn't in the list of agents supported during public preview, or the device isn't a managed Windows device enrolled in Intune	Compare with the tables in 9.3.1 and 9.4.1, and recheck 9.2.3.
Users, **Most recent activity**, **Last scanned** or **Total traffic** are empty	Global Secure Access isn't enabled for the tenant and enrolled devices	Complete 9.2.4.
Blocking isn't available for an agent	Blocking isn't available for that agent during public preview	Check the **Blocking** column in 9.3.1 and 9.4.1, and choose a test tool for which blocking is available.

Symptom	Likely cause	Fix
The policy is applied but the agent still runs on the device	The Intune policy update hasn't applied yet (15 minutes up to 8 hours, depending on Intune configuration)	Sync the device (9.5.6), and check again within the documented window.
Other Node.js-based agents or processes stopped running on managed devices	Node.js-based agents share a single run type, and blocking one blocks all agents and processes of that run type	Expected shared run-type behavior (9.4.6). Review the agreed blocking scope before applying this policy.
The VS Code extension isn't blocked	Syntax error in the policy value, or VS Code version earlier than 1.96	Run **Show Window Log** in VS Code to check for errors, and use VS Code 1.96 or later.

✅ What you proved

- [ ] The **Shadow AI (Frontier)** and **Local Agents (Frontier)** pages open for the PoC reviewer and list the agents that can be detected.
- [ ] At least one approved test tool installed on a managed Windows test device appears on the matching page, and the test device is listed on the agent's **Detected devices** tab with a **Last Seen** value.
- [ ] If Global Secure Access is enabled, the **Users**, **Most recent activity**, **Last scanned** and **Total traffic** fields are populated for the detected agent.
- [ ] After **Block** > **Apply Policies** on a Shadow AI agent, the **Security policies** tile shows that a blocking Intune policy is applied, and the policy (for OpenClaw, **A365 - Block OpenClaw**) is found in Microsoft Intune.
- [ ] After the policy applies to the test device, the common ways of running the blocked agent are blocked on that device.

References

- Understand Shadow AI in Microsoft 365 admin center
- Understand Local Agents in Microsoft 365 admin center
- Preview Agent 365 features through the Frontier program
- Discover local AI agents with Microsoft Defender for Endpoint
- Device action: Sync
- Tutorial: Enable Internet Access traffic forwarding
- onboarded to Microsoft Defender for Endpoint
- Assign device profiles in Microsoft Intune
- Settings list for the Local AI Agent Baseline - OpenClaw security baseline in Intune
- Import custom and third-party partner ADMX templates in Microsoft Intune

Additional resources

- Manage extensions in enterprise environments (Visual Studio Code)
- Centrally manage VS Code settings with policies

Runbook index · Previous: Chapter 8 – Threat detection and runtime protection (Defender)